

BECKMAN COULTER LIFE SCIENCES

Biomek JSON Authoring Guide

Method Foundations, JSON Structure & Per-Step Reference

JSON format version 1.0

86 documents

Companion volumes: Biomek Instrument & Project Reference (in *Biomek Instrument & Project Reference*) · Biomek JSON Add-on Steps (in *Biomek JSON Add-on Steps*)

Table of Contents

Overview

Biomek JSON Authoring Guide

Method Foundations

Introduction for LLMs
Introduction to Biomek and Liquid Handling Method Writing
Method JSON Structure
Introduction to Pipetting Steps
Pipetting Techniques, Templates, and Auto-Selection

Quick Reference

Biomek Method JSON — Quick-Reference Cheatsheet
Worked Example: Simple i3 Fixed-8 Method

Walkthrough

Worked Example: Full i5 Span-8 Method

Step Reference

Step Documentation Catalog
Base Keys & Serialization Conventions
Start
Finish
Instrument Setup
Move Labware
Cleanup
Move Pod
Hold Labware
End
Device Action
INHECO Peltier
Transfer
Combine
Span-8 Transfer From File
Span-8 Serial Dilution
Span-8 Aspirate
Span-8 Dispense
Span-8 Load Tips
Span-8 Unload Tips
Span-8 Wash Tips
Multichannel Aspirate
Multichannel Dispense
Multichannel Load Tips
Multichannel Unload Tips
Multichannel Mix
Multichannel Wash Tips
Multichannel Select Tips
Multichannel Select Tips End
Multichannel Select Tips Serial Dilution

Multichannel Select Tips Aspirate
Multichannel Select Tips Dispense
Multichannel Select Tips Mix
Multichannel Select Tips Load
Multichannel Select Tips Unload
Multichannel Select Tips Advanced Load
Multichannel Select Tips Advanced Unload
Fixed-8 Aspirate
Fixed-8 Dispense
Fixed-8 Mix
Fixed-8 Load Tips
Fixed-8 Unload Tips
Create Data Set
Data Set Management
Data Set Processing
Data Set Reporting
View Data Sets
Fly-By Read
Fly-By Log
Break
Comment
Define Pattern
Group
If
Just In Time
Worklist
Let
Loop
Named Container
Next Item
Pause
Run Method
Define Procedure
Run Procedure
Scripted Let
Script
Set Global
Run Program
SILAS
Create Labware Group
Next Labware

Appendix A: Concept Guides

Concept Guides
Concept Guide: Expressions
Concept Guide: Labware & Position Resolution
Concept Guide: Probe & Mandrel Selection
Concept Guide: The Variable Environment ('let' / 'weak' / 'prompt')
Concept Guide: Well Volume Tracking

Biomek JSON Authoring Guide

Reference documentation for authoring Biomek methods in the JSON format.

The intended audience of the Concept guides is LLMs that are generating Biomek methods. Humans who are familiar with Biomek Software and Biomek instrument capabilities will want to start with Method JSON Structure and Introduction to Pipetting Steps, and make use of the Step Reference.

When using an LLM to author a method, provide the LLM with this documentation, the JSON-formatted export of the instrument settings for your Biomek instrument, and the following reference manuals:

- D24406 Biomek i3 Instructions for Use
- B54473 Biomek i5 and i7 Instructions for Use
- B56358 Biomek Software Reference Manual

You will also need to provide your LLM with the list of labware, pipetting techniques, liquid types, and tip types available on your system, and be sure to instruct it not to invent additional items that are not on those lists.

LLMs consuming this documentation should begin with Introduction for LLMs.

[†] Table of contents

Introduction for LLMs

Reference documentation for authoring Biomek methods in the JSON format.

For general information about Biomek systems, refer to the Biomek i3 Instructions for Use and Biomek i5 and i7 Instructions for Use.

Read order:

- 1. Introduction to Biomek Method JSON — what a Biomek is, what a method is, pod/step vocabulary.
- 2. Method JSON Structure — the JSON envelope, `_biomekType` discriminators, the step catalog, structural validation, and how to handle add-on / plugin steps.
- 3. Introduction to Pipetting Steps - low-level versus high-level pipetting steps, common keys, and tip handling.
- 4. Pipetting Techniques and Templates — techniques, template-vs-technique split, technique-family prefixes, auto-select, liquid types, volume types, and height override. Read when authoring any pipetting-family step.
- 5. Concept guides — cross-cutting models (expressions, labware/position resolution, probe/mandrel selection, variable environment, well-volume tracking) that the per-step docs assume you are familiar with.
- 6. Per-step reference under step-documentation/ — browse the grouped step index.

The envelope's `formatVersion` is the **method** JSON format version — the JSON format, not the software release — and is versioned independently of the Instrument Settings export, which carries its own `formatVersion`. See Method JSON Structure for the current value. Author exactly the current version: import accepts only that version and rejects anything else — an older value as *too old*, and any higher or longer value (for example a three-part `"x.y.z"`, which reads as higher) as *too new*.

Quick reference

Resource	What it's for
Cheatsheet	One-page quick reference — envelope, step-node shape, anchor/terminator rules, the well-selection idioms, and the gotchas that bite most often.
Worked Example: Simple i3 Fixed-8 Method	The smallest complete method: a single-column plate-to-plate transfer on i3/Fixed-8. Start here if you want a minimal working template to modify.
Worked Example: Full i5 Span-8 Method	A full-scale method on i5/Span-8 — loops over columns, reservoir draws, tip washing, and timed pauses. Read this once the minimal template is no longer enough.
Step documentation index	Grouped catalog of every step — by pod family, by category, and by number.

Foundations

File	Description
Introduction to Biomek Method JSON	Instrument family, pod types, method tree structure, and the vocabulary the rest of the docs use
Method JSON Structure	How to construct a valid Biomek method in the JSON format Biomek reads and writes
Pipetting Techniques and Templates	Biomek pipetting technique system, template selection, height overrides, and <code>C_</code> context variables

Concept guides

Cross-cutting Biomek Software concepts that the per-step reference assumes you are already familiar with. The step docs are self-contained — these guides can be removed without breaking them; they simply explain the underlying concepts a step doc expects you to know.

#	Guide
—	Concept guides index — table of contents

#	Guide
01	Expressions
02	Labware & Position Resolution
03	Probe & Mandrel Selection
04	The Variable Environment
05	Well Volume Tracking

Instrument/pod model, technique auto-selection, liquid & volume types, and base-key conventions are covered in the Foundations documents above (Introduction to Biomek Method JSON, Pipetting Techniques and Templates, and Base keys and conventions). Well-numbering / `firstWell` math lives in Introduction to Pipetting Steps §"Well Numbering".

Instrument & project-item reference

The per-step docs assume you are familiar with your own instrument's configuration. Before authoring, export your instrument's settings from the Biomek software (**File** ▶ **Export** ▶ **Instrument** ▶ **Save As JSON**) and examine them using this guide and the sample/reference material here to interpret what you see. A method that targets a configuration your instrument does not actually have will likely fail to enqueue or run.

That material is a companion volume with its own index — see Instrument & Project-Item Reference (in *Biomek Instrument & Project Reference*) for the full listing, including the seven default-instrument exports and the resolved deck views that accompany them. The entry points:

Resource	What it's for
Pod Settings Format Specification (in <i>Biomek Instrument & Project Reference</i>)	The pod-settings portion of an Instrument Settings export — envelope, per-pod-class key sets, <code>\$id</code> / <code>\$ref</code> refs.
Deck Layouts Format Specification (in <i>Biomek Instrument & Project Reference</i>)	How to read a deck out of <code>deckLayouts.<deckKey></code> — per-position keys, the <code>device</code> block, and the reference model.
Reachability and Access (in <i>Biomek Instrument & Project Reference</i>)	How deck position reachability and access constrain which positions a pod can address.
Pod Reach Envelopes (in <i>Biomek Instrument & Project Reference</i>)	Per-pod X travel envelopes, with an i7 reach table.
ALP Catalog (in <i>Biomek Instrument & Project Reference</i>)	The common default ALP (Automated Labware Positioner) types that can appear in a deck.
Project Items (in <i>Biomek Instrument & Project Reference</i>)	Valid values a method references — labware classes and patterns, liquid types, tip classes, techniques, and templates.

step-documentation/

Per-step reference docs, numbered sequentially. Every step covered by this documentation set has its own reference file under `step-documentation/` ; browse the catalog, grouped by pod family and area, in the step documentation index.

add-on-steps/

Reference docs for steps contributed by add-on modules and device integrations, rather than by the core step registry. See the add-on steps index (in *Biomek JSON Add-on Steps*).

[†] Table of contents

Introduction to Biomek and Liquid Handling Method Writing

This document introduces the Biomek instrument family, its pod types, the tree structure of a method, and the vocabulary the rest of the docs use. Read it before writing or editing a method.

Coverage note: The step documentation index is the complete list of steps this documentation set covers, and `add-on-steps/` (in *Biomek JSON Add-on Steps*) covers the documented add-on steps. A step type outside those two lists may carry structural or parameter requirements that are not described here — obtain its parameter reference rather than inferring one.

The Biomek Instrument

A **Biomek** is an automated liquid handling workstation. It is a robotic platform that moves precise volumes of liquid between containers (wells, tubes, reservoirs) on a work surface called a **deck**.

Biomek instruments are used in life science laboratories to automate repetitive pipetting tasks — serial dilutions, plate replication, normalization, PCR setup, ELISA preparation, sample reformatting, and more.

Instrument Variants

Model	Chassis / Deck	Pod Configuration	Gripper
i3	One arm, 15 deck positions (default layout)	Fixed-8 pod (integrated)	Integrated with pod
i5	One arm, up to 25 deck positions	One pod: Span-8 or Multichannel	Rotating gripper
i7	One or two arms (one per pod), up to 40 deck positions	One or two pods: any combination of Span-8 and/or Multichannel ("dual-pod" when two)	Rotating gripper

On an i7 each pod has its own arm, so a two-pod ("dual-pod") i7 is a genuine two-arm system. Default i3 decks often leave a column open — the i3's leftmost column is typically reserved for add-ons and a trash/storage area — so a default layout can show fewer positions than the deck can physically hold.

Pod naming. In the `pod` field of any step, use the pod's **configured name**, which comes from the instrument's pod settings rather than from the pod's type. Do **not** invent names like `"MPod"`, `"MC"`, or `"Span8"` — the instrument cannot substitute a made-up pod name, and enqueue fails.

The name follows one rule: **PodN is a 1-based index of the pods, counted left→right**, so the leftmost pod is `"Pod1"`, and the one to its right is `"Pod2"`. It is a *position* index, not a pod type — a single-pod instrument's only pod is always `"Pod1"`, and on a two-pod i7 the left pod is `"Pod1"` and the right pod is `"Pod2"`. Which type sits at which index is defined in the instrument's pod settings:

Instrument	"Pod1"	"Pod2"
i3	Fixed-8	—
i5 Multichannel	Multichannel	—
i5 Span-8	Span-8	—
i7 Multichannel	Multichannel	—
i7 Span-8	Span-8	—
i7 Hybrid	Multichannel	Span-8
i7 Dual-Multichannel	Multichannel	Multichannel

Some methods parameterize the pod name through a `let /global` expression (`"=podID"` , `"=thisSpanPod.name"`) instead of a literal; this is legitimate whenever the enclosing scope defines the referenced variable.

One caution: **do not treat "Pod2" as the canonical Span-8 pod**. Span-8 examples using `"Pod2"` reflect where the Span-8 sits on an i7 Hybrid — not a rule. On a standalone Span-8 the runnable value is `"Pod1"` . Always match the target instrument's actual configuration by reading the pod name from the instrument's pod settings.

Pod Types

- **Fixed-8**: 8 pipetting channels in a column, all move together on shared X, Y, Z, and D (syringe) axes. Integrated gripper for moving labware. Uses disposable tips. Fixed-8 uses only manually specified pipetting techniques — auto-select is not supported for Fixed-8 aspirate/dispense/mix steps.
- **Span-8**: 8 independent probes with variable spacing. Each probe has independent Z and syringe (D) axes. Supports liquid-level sensing (LLS). Rotating gripper. Tip type is configured **per group of four** probes — configurations include fixed tips (always attached) or disposable tips (loaded from boxes).
- **Multichannel (MC)**: Interchangeable heads that pipette many wells at once in an SBS-aligned grid. Available densities are 96-channel and 384-channel. All channels share one syringe axis. Rotating gripper. Uses disposable tips loaded from a tip-loader ALP (a permanent deck fixture designed for MC tip loading). A 96-channel head on a 384-well plate accesses one quadrant of 96 wells at a time; use the Multichannel Select Tips container family for partial-mandrel operations.

The Multichannel pod supports three types of heads, which cannot be changed at runtime: - 300 µL MC-96 Head - 1200 µL MC-96 Head - 60 µL MC-384 Head

96-channel heads can access 384-well labware one quadrant at a time.

Pipetting cannot exceed the volume capacity of the pod (Fixed-8), head (MC), syringe (Span-8), or tip, whichever is smaller. Refer to the Biomek i5 and Biomek i7 Instructions for Use and the Biomek i3 Instructions for Use for information about tip capacities (B54473 and D24406).

Choosing a pod for a given layout

- **Multichannel (MC)** moves all channels together on a fixed grid. Use it when you address **the whole plate**. To touch a *subset* of wells (a single row, an arbitrary set, fewer than a full column) you must add a **Multichannel Select Tips** step first — an MC Aspirate/Dispense on its own cannot pick an arbitrary well pattern.
- **Span-8** has 8 independently addressable probes. Use it for **arbitrary or partial** well sets (e.g. 12 samples that don't fill full columns), for per-well volume differences, and for reagent/reservoir work.
- **Fixed-8** behaves like a fixed 8-probe column; see the Fixed-8 step docs.
- A `Transfer / Combine` step runs on whichever pod its resolved `pod` names — so the same layout question decides the `pod` value.
- Only choose a pod that is present on the given instrument configuration.

Core Concepts

Deck

The deck is the physical work surface of the instrument. It is divided into **positions** — defined slots where labware sits. Each position has a known location in the instrument's coordinate system. Do not guess at deck position names — ask the user or read the corresponding local deck file if available.

Labware

Labware is any container placed on the deck: microplates (96-well, 384-well, 1536-well), reservoirs, tube racks, tip boxes, waste containers. Each labware item has a **labware class** that defines its geometry (well shape, spacing, depth, per-well capacity). Do not guess at available labware — ask the user or read the corresponding local project file if available.

Tips

Biomek instruments pipette using disposable tips (or fixed tips on some Span-8 configurations). Tips are loaded from tip boxes on the deck (or from a tip-loader ALP for Multichannel). The tip type determines the volume range and some physical properties of pipetting. For

a pod that uses disposable tips, tips must be loaded before any pipetting is performed. At the end of a method, it is best practice to unload any disposable tips still loaded. Do not guess at available tip types — ask the user or read the corresponding local project file if available.

Certain operations require certain tip types. LLS and Clot Detection require conductive tips or fixed tips. Piercing requires piercing fixed tips.

Techniques and Templates

A **pipetting template** is a project entity that defines the physical procedure the instrument executes for an aspirate, dispense, or mix operation — as an ordered sequence of actions (aspirate, dispense, tip touch, air gap, etc.).

A **pipetting technique** is a separate project entity that stores parameter values (heights, speeds, delays, blowout, tip touch, prewet, LLS, clot detection, etc.) and is *associated* with a template. Together they control how any aspirate, dispense, mix, pod height, pod speed, or tip touch operation is performed.

Some pods (Span-8, Multichannel) can auto-select a technique from context (tip type, labware class, volume range, liquid type). Fixed-8 requires a technique named explicitly.

For detail, see Pipetting Techniques, Templates, and Auto-Selection.

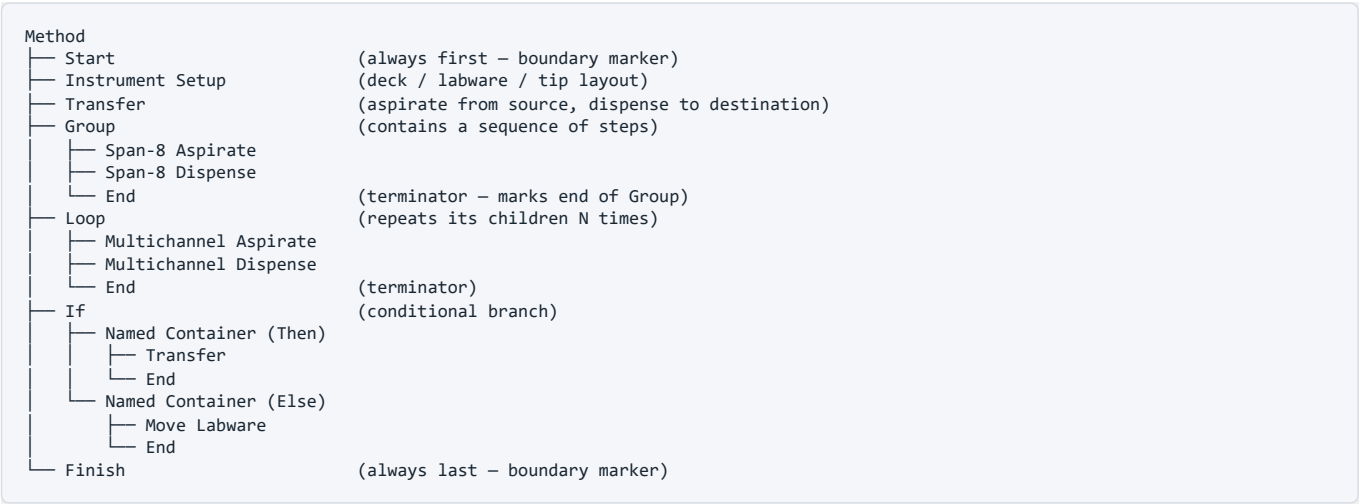
Methods

A **method** is a program that tells the Biomek what to do. It is structured as a **tree of steps** — a hierarchy where some steps contain other steps as children.

When the user presses "Run," the Biomek software walks through this tree top-to-bottom, executing each step in sequence. Container steps (groups, loops, conditionals) control flow by holding child steps that execute under their rules.

Method Tree Structure

Think of a method like an outline. Note that low-level pipetting steps are **pod-prefixed** — there is no bare "Aspirate", "Dispense", or "Mix" at method level. `Transfer` and `Combine` resolve their pod family at run time from the `pod` field.



`Named Container` is a step type. `If` always has exactly two of them as children, representing the Then and Else branches.

Common steps

The steps you'll reach for most often are **Start/Finish** (method boundaries), **Instrument Setup**, the transfer-family steps **Transfer/Combine**, the per-pod pipetting families (**Fixed-8**, **Span-8**, **Multichannel** aspirate/dispense/mix/load/unload), and the control-flow containers **Group/Loop/If/Let**. For the complete catalog and structural rules, see the step documentation index.

Add-on steps. Beyond this core set, add-on modules and device integrations register their own step types — sometimes under a COM ProgID rather than a friendly name. These are documented separately in `add-on-steps/` (in *Biomek JSON Add-on Steps*); see also *Beyond the Built-in Steps* in Method JSON Structure.

How Users Build Methods in the Biomek Editor

In the Biomek Editor (the desktop GUI application), a new method already contains `Start` and `Finish`. Users drag step types from a toolbox and drop them between those two anchors, then configure each one through a step-specific dialog — source and destination labware, volumes, tip handling. Steps that belong together go inside containers (groups, loops, if-branches) for control flow.

The editor enforces structural rules automatically: terminators are added when a container is created, anchors cannot be moved or crossed, and `If` always gets its Then/Else branches on creation. Hand-authored JSON must satisfy the same rules.

Key Terminology Quick Reference

Terms used across these documents.

Term	Meaning
Method	A program (tree of steps) that automates liquid handling on a Biomek
Step	A single node in the method tree — either a leaf (action) or container (control flow)
Deck	The work surface with defined positions for labware
Labware	Physical containers (plates, reservoirs, tip boxes) placed on deck positions
Position	A defined slot on the deck where labware can be placed
ALP	Automated Labware Positioner — a bolted deck fixture (tip loader, wash station, trash, static holder)
Tip	Disposable (or fixed) pipette tip used for liquid transfer
Template	A project entity: an ordered sequence of actions defining the physical procedure of an aspirate/dispense/mix
Technique	A project entity: parameter values (heights, speeds, delays) associated with a template
Pod	The pipetting head assembly (Fixed-8, Span-8, or Multichannel)
Instrument Setup	Configuration step that defines which labware/tips are on which positions
Anchor	Start or Finish — immovable boundary markers at method root; other top/bottom anchors exist for some containers
Terminator	The immovable final child marking a container's end — <code>End</code> , <code>Multichannel Select Tips End</code> , or a container-specific terminator registered by an add-on step
Named Container	A step type used as a labeled branch inside <code>If</code>
Enqueue	The phase you enter by pressing Run or Simulate: the method tree is compiled into the actions the instrument will perform, and most configuration errors surface here, before anything moves
Execute	The phase after enqueue, in which the compiled actions drive the physical hardware

What "Writing a Method" Means

When you write or generate a Biomek method, you are constructing the **step tree** — the hierarchical structure of steps with their parent-child relationships and (optionally) their configuration parameters.

At minimum, a valid method must have:

1. A root node representing the method.
2. `Start` as the first child of the root.
3. `Finish` as the last child of the root.
4. Any operational steps inserted between the anchors — typically starting with `Instrument Setup` to define what labware and tips are on the deck.
5. All containers properly terminated.
6. All structural rules satisfied (see Method JSON Structure for the full rule set).

The **step configuration** (which wells, which volumes, which labware, which technique) lives in each step's dictionary — that is a separate concern from the tree structure. See per-step reference in [step-documentation/](#).

Common Method Patterns

Simple Transfer

Aspirate from a source plate, dispense to a destination plate. One `Transfer` step between the anchors.

Serial Dilution

Prefer the dedicated `Span-8 Serial Dilution` or `Multichannel Select Tips Serial Dilution` steps when they fit the pod. If you need finer control, fall back to a `Loop` containing a `Transfer` that increments the source/destination column each iteration.

Conditional Processing

An `If` where the `Then` branch (a `Named Container`) processes samples one way and the `Else` branch (a `Named Container`) handles the alternative path (e.g., skip low-concentration samples).

Multi-Step Protocol

A `Group` (or `Let` for scoped variables) containing multiple sequential operations — aspirate reagent, mix, incubate with a `Pause`, transfer product.

Plate Replication

`Transfer` configured to copy all wells from a source plate to one or more destination plates.

Partial-Plate Multichannel

Wrap a `Multichannel Select Tips` container around `Multichannel Select Tips Load` / `Multichannel Select Tips Aspirate` / `Multichannel Select Tips Dispense` / `Multichannel Select Tips Unload` children when only some mandrels of an MC head should be active.

Method JSON Structure

Scope: Structure and value-encoding rules only — up to but NOT including how to populate step-specific parameter values. For per-step parameter reference, see step-documentation/.

Prerequisite: Introduction to Biomek and Liquid Handling Method Writing.

Step documentation scope: Step-specific parameter documentation is available for **Documented** steps only. When generating methods, **use only steps marked "Documented" in the step catalog below**. Other step types are not covered by this documentation set and may have unpublished structural or parameter requirements.

JSON Envelope

Every method file is a JSON object with three required top-level properties:

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "author": "",
  "description": "",
  "steps": [ ... ]
}
```

Property	Required	Type	Description
format	Yes	string	Must equal "Biomek Method" (case-insensitive comparison).
formatVersion	Yes	string	The Biomek Method JSON format version. Format should be major.minor (e.g. "1.0").
author	No	string	Free-text method author. Round-trips to/from the method's Author dictionary key. Missing or null becomes an empty string on import; always written on export. A non-string JSON value (e.g. a number or object) is a deserialization error.
description	No	string	Free-text method description. Round-trips to/from the method's Description dictionary key. Missing or null becomes an empty string on import; always written on export. A non-string JSON value (e.g. a number or object) is a deserialization error.
steps	Yes	array	Ordered list of root-level step nodes.

format , formatVersion , and steps are required — omitting any fails import with a hard error. author and description are optional. format and formatVersion are also verified in a first-pass check before schema-level parsing, so their errors surface first; a version mismatch is reported as too-old or too-new for the running software (see the formatVersion row above for the exact string rules).

The steps array holds the direct children of the method root: the top level of the method tree.

Step Node Structure

Each element in steps (and in any nested subSteps array) is a **step node**:

```
{
  "stepType": "Transfer",
  "disabled": false,
  "parameters": { },
  "subSteps": [ ... ]
}
```


Property	Required	Type	Description
<code>stepType</code>	Yes	string	Step type name — see step catalog. Matched case-insensitively against the step registry or the step's ProgID / CLSID string.
<code>disabled</code>	No	boolean	Whether the step is commented out (disabled) in the editor. Defaults to <code>false</code> and is omitted from export when <code>false</code> . A <code>disabled</code> value other than <code>true</code> or <code>false</code> fails deserialization. When <code>true</code> , the step is skipped at run time but preserved in the tree. Author disabled state only via this step-node property; do not author an equivalent flag inside <code>parameters</code> .
<code>parameters</code>	Yes	object	Case-insensitive key-value map of step configuration. May be <code>{}</code> but the property must be present.
<code>subSteps</code>	No	array	Child step nodes. Omit (or set to <code>null</code>) for leaf steps. An empty array is also accepted. <code>subSteps</code> is written only when the exporter has one to emit.

Property Naming Convention

Envelope and step-node property names (`format`, `formatVersion`, `steps`, `stepType`, `parameters`, `subSteps`) use **camelCase**. Parameter keys follow the casing rule documented in Base Keys & Serialization Conventions §1, which includes the full worked-example table. On read, envelope and step-node property names are matched case-insensitively; parameter keys are matched case-insensitively too — **except within SILAS message payloads, which are case-sensitive**.

JSON Parsing Behavior

Biomek's JSON reader/writer uses the options below — they affect what counts as valid JSON on import:

Behavior	Setting	Effect
Duplicate JSON keys	Rejected	Duplicate keys in the same JSON object cause the parser to fail.
Trailing commas	Allowed	A trailing comma at the end of an array or object is accepted.
Property-name matching	Case-insensitive	Envelope and step-node field names are matched case-insensitively on read.
Naming on write	camelCase	Envelope fields are written in camelCase. Parameter keys are converted by the same rule: the leading run of uppercase letters is lowercased except the last one (when a lowercase follows), and the rest is preserved verbatim (see Base Keys & Serialization Conventions §1).
Comments	Allowed and ignored	<code>//</code> line comments and <code>/* */</code> block comments are accepted and stripped on read.
Null values	Rejected for non-nullable properties	A JSON <code>null</code> for a property that is not declared nullable is rejected on read. Property <i>presence</i> is enforced separately, on <code>format</code> / <code>formatVersion</code> / <code>steps</code> / <code>stepType</code> / <code>parameters</code> .
Output formatting	Pretty-printed	Written JSON is indented for readability.

`double` values are written by a custom converter that forces a decimal point on integral values (`3` → `3.0`) so a round-trip preserves double-vs-integer typing. Numeric reading is locale-invariant.

Parameter Value Encoding

The `parameters` object is a JSON representation of the step's parameter map — a case-insensitive key/value collection. Most values are ordinary JSON primitives (`string`, `number`, `boolean`, `null`, or nested `object` / `array`), but some Biomek types require a **`_biomekType discriminator`** — a reserved property on the JSON object that identifies its Biomek type at deserialization time.

`_biomekType` Discriminator

Any JSON object inside `parameters` may carry a `_biomekType` string property (matched case-insensitively) that selects one of the following providers:

<code>_biomekType</code> value	Represents	Notes
<i>(absent or empty string)</i>	Plain nested dictionary	The default. A JSON object without <code>_biomekType</code> is treated as a nested dictionary. Keys are case-insensitive.
<code>"dateTime"</code>	Date/time value	Serialized as <code>{ "_biomekType": "dateTime", "value": "2024-01-31T13:45:30.000000Z" }</code> — ISO 8601 round-trip (<code>"0"</code>) format, locale-invariant.
<code>"empty"</code>	Absent / unset value (distinct from <code>null</code>)	Distinct from <code>null</code> in COM semantics.
<code>"missing"</code>	Parameter not supplied	Rarely authored by hand.
<code>"comArray"</code>	Typed array	See "comArray" section below.
<code>"nonFiniteNumber"</code>	A non-finite <code>double</code> / <code>float</code> (<code>NaN</code> , <code>Infinity</code> , <code>-Infinity</code>)	JSON numbers can't express these, so they serialize as <code>{ "_biomekType": "nonFiniteNumber", "value": "NaN" }</code> . <code>value</code> must be the string <code>"NaN"</code> , <code>"Infinity"</code> , or <code>"-Infinity"</code> (case-insensitive). Finite numbers use plain JSON numbers.
<code>"labware"</code>	Serialized labware reference	Used inside Instrument Setup and similar steps.
<code>"technique"</code>	Custom pipetting technique object	An inline technique definition, distinct from a technique referenced by name.
<code>"SILASMessage"</code>	SILAS device/consumer message payload	Case- sensitive keys inside — SILAS payloads preserve casing exactly.

Rule: **You must not use `_biomekType` as your own key** anywhere inside `parameters`. Attempting to serialize a step whose dictionary contains a `_biomekType` key raises a `JsonException` up front — the exporter refuses to hide the discriminator behind user data.

On import, a JSON object whose `_biomekType` value does not match any provider fails deserialization.

`comArray` required shape

Any typed Biomek COM array serializes as:

```
{
  "_biomekType": "comArray",
  "arraySubtype": "<subtype>",
  "values": [ ... ]
}
```

Legal `arraySubtype` values:

<code>arraySubtype</code>	Element type	Example use
<code>"boolean"</code>	<code>bool</code>	Well-selection masks, probe-active arrays
<code>"byte"</code>	<code>byte</code>	Binary payloads
<code>"dateTime"</code>	<code>DateTime</code>	Timestamp arrays
<code>"double"</code>	<code>double</code>	Per-well volumes
<code>"integer"</code>	<code>int</code> (32-bit)	Section indices
<code>"short"</code>	<code>short</code> (16-bit)	16-bit integer indices
<code>"single"</code>	<code>float</code>	Single-precision floats
<code>"string"</code>	<code>string</code>	Text arrays
<code>"variant"</code>	<code>object</code>	Mixed-type arrays

If you emit a plain JSON array (`[true, false, ...]`) where a `comArray` is required, deserialization will treat it as a plain list — a different Biomek type — and downstream step logic may reject it or interpret it incorrectly. When in doubt, follow the shape shown in the step's documented examples.

Complex (multidimensional / non-zero-based) comArrays

Most `comArray` values are one-dimensional and zero-based, needing only `arraySubtype` and `values` . Two optional properties preserve unusual COM array shapes:

Property	When present	Meaning
<code>dimensionCount</code>	Written only when the array has more than one dimension	Number of dimensions (2–32). When present, <code>values</code> is a nested array matching that rank.
<code>lowerBound</code>	Written only when at least one dimension does not start at index 0	Either a single integer (all dimensions share the lower bound) or an array of integers (one per dimension).

You will rarely author these by hand — no built-in step is known to use them — but they appear when round-tripping methods authored by custom step plug-ins. Preserve them verbatim if you see them. A multidimensional array may have zero length only in its final dimension.

Plain nested dictionary

A JSON object with no `_biomekType` is a plain nested dictionary. Keys are case-insensitive; on write, each key is converted by the camelCase rules described in Base Keys & Serialization Conventions §1. Nesting is supported to a depth of 500, far deeper than any real method.

Locale, `null` , and non-finite numbers

- Numeric formatting is locale-invariant (`.` decimal separator always). Finite `double` values are always written with a decimal point (`10` → `10.0`) so a round-trip preserves double-vs-integer typing.
- JSON `null` represents a null value; `_biomekType: "empty"` represents an absent/unset value; both are distinct from each other and from a missing key.
- Non-finite doubles (`NaN` , `Infinity` , `-Infinity`) serialize as a `_biomekType: "nonFiniteNumber"` object (see the discriminator table above) — plain JSON does not support them as numbers.

Step Types and Structural Kinds

The core step registry defines the built-in step types. `stepType` matching is case-insensitive; prefer the canonical spelling. Steps contributed by add-on modules and device integrations are documented separately in `add-on-steps/` (in *Biomek JSON Add-on Steps*).

Steps fall into a few **structural kinds**:

- **Leaf** — no child steps.
- **Free-form container** — a user-arranged list of children, closed by a **terminator** (the fixed trailing anchor child, usually `"End"`).
- **Exact-children container** — a fixed set of specific children in a defined order.

Structure classifications are derived from the step's runtime editor-options flags plus its factory-default children. The catalog reflects the current implementation; the running software is authoritative.

For the full list of step types and per-step docs, see the step documentation index.

Unknown `stepType` names — Unrecognized names fall through to a CLSID-string parse and then a ProgID lookup; if nothing resolves, import fails. Structural validation gives such unknown types "permissive" rules (no additional constraints), so the meaningful error comes from the deserializer rather than the validator.

Beyond the Built-in Steps (Add-on / Plugin Steps)

The step documentation index covers the core step registry, and `add-on-steps/` (in *Biomek JSON Add-on Steps*) covers the documented add-on steps. Add-on modules, device integrations, and site plug-ins register their own step types beyond those two sets — handle those as follows:

- **An add-on step's `stepType` may be a friendly name or a COM ProgID, depending on the add-on.** A step registered by a plug-in that has not published a JSON type name carries its registration identifier (a ProgID) as `stepType`. Import resolves the name against the step registry first, then a CLSID-string parse, then a ProgID lookup (see the *Unknown `stepType` names* note above), so a ProgID string is a legal `stepType`.
- **Do not guess parameters for a step type, device, or integration this documentation does not cover.** A `stepType` outside the step and add-on indexes — or a device or integration not described here — may carry unpublished structural or parameter requirements. Authoring one from a guess risks a method that fails import, or one that imports and then runs incorrectly.
- **Escalate for the authoring spec.** When a method must use an add-on step outside this set, obtain its parameter reference from Beckman Coulter — or, for a third-party module, the add-on's vendor — rather than inferring it.

Structural Rules

Every step falls into one of three structural shapes, which determine what its `subSteps` may contain:

- **Leaf:** takes no child steps. Omit `subSteps` (or provide an empty array `[]`).
- **Fixed-children container:** requires an exact set of children — a specific count and type, in order — and `subSteps` must match that set exactly. Example: `"If"` requires exactly 2 `"Named Container"` children.
- **Free-form container:** accepts any legal steps in the middle, but its fixed **anchor** steps must stay put — any leading anchors come first and any trailing anchors come last, in their defined order.

Key Rules Summary

1. **Method root** is a free-form container with a leading `"Start"` (top anchor) and a trailing `"Finish"` (bottom anchor): root children must have `"Start"` first and `"Finish"` last.
2. **A free-form container's terminator is whatever step type that container defines** as its trailing bottom-anchor child — it is not always `"End"` : - Most built-in containers (`"Group"` , `"Loop"` , `"Let"` , `"Scripted Let"` , `"Hold Labware"` , `"Named Container"` , `"Just In Time"` , `"Worklist"` , `"Define Procedure"`) use `"End"` . The editor caption varies (`"End Group"` , `"End Loop"` , `"End Just In Time"` , ...) but that is display text only — the step registry registers a single `"End"` for the whole group, and serializing `"stepType": "End Group"` fails import. - The `"Multichannel Select Tips"` family ends with `"Multichannel Select Tips End"` . - **Add-on / plugin-registered steps may register their own terminator types.** Match the terminator to what the specific container populates rather than assuming a fixed set.
3. **"If"** is an exact-children container with exactly 2 `"Named Container"` children (Then, Else). Each Named Container is itself a free-form container ending in `"End"` .
4. **Leaf steps** must not have `subSteps` with elements. Omitting or providing `[]` is equivalent.
5. **Anchors and terminators** (`"Start"` , `"Finish"` , and each container's own terminator such as `"End"` or `"Multichannel Select Tips End"`) must appear only in their designated positions — never as free user-inserted steps mid-body.

Minimal Valid Method

The simplest possible valid method:

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "steps": [
    { "stepType": "Start", "parameters": { "bitmap": "OStepUI.ocx,START", "let": {}, "weak": {}, "prompt": {} } },
    { "stepType": "Finish", "parameters": { "bitmap": "OStepUI.ocx,FINISH" } }
  ]
}
```

```
]
}
```

This is an empty method — it has only the required anchors and does nothing. Author `Start` with `let / weak / prompt` dictionaries (empty `{}` is fine) and `bitmap: "OStepUI.ocx,START"`; author `Finish` with `bitmap: "OStepUI.ocx,FINISH"`. All four are required because JSON import replaces the step's parameters wholesale, discarding the built-in defaults — an omitted key hard-fails at Run (for `let / weak / prompt`) or leaves the anchor icon broken (for `bitmap`). See Base Keys & Serialization Conventions §4 (environment-extension keys) and §2 (`bitmap`); on every other step `bitmap` is auto.

Building Up a Method

Adding a Simple Step

Insert operational steps between `Start` and `Finish`:

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "steps": [
    { "stepType": "Start", "parameters": { "bitmap": "OStepUI.ocx,START", "let": {}, "weak": {}, "prompt": {} } },
    { "stepType": "Transfer", "parameters": { ... } },
    { "stepType": "Finish", "parameters": { "bitmap": "OStepUI.ocx,FINISH" } }
  ]
}
```

Adding a Free-form Container (Group)

The `Group` container uses `subSteps` and ends with an `"End"` step:

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "steps": [
    { "stepType": "Start", "parameters": { "bitmap": "OStepUI.ocx,START", "let": {}, "weak": {}, "prompt": {} } },
    {
      "stepType": "Group",
      "parameters": {},
      "subSteps": [
        { "stepType": "Span-8 Aspirate", "parameters": { ... } },
        { "stepType": "Span-8 Dispense", "parameters": { ... } },
        { "stepType": "End", "parameters": {} }
      ]
    },
    { "stepType": "Finish", "parameters": { "bitmap": "OStepUI.ocx,FINISH" } }
  ]
}
```

Adding a Loop

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "steps": [
    { "stepType": "Start", "parameters": { "bitmap": "OStepUI.ocx,START", "let": {}, "weak": {}, "prompt": {} } },
    {
      "stepType": "Loop",
      "parameters": {},
      "subSteps": [
        { "stepType": "Span-8 Aspirate", "parameters": { ... } },
        { "stepType": "Span-8 Dispense", "parameters": { ... } },
        { "stepType": "End", "parameters": {} }
      ]
    },
    { "stepType": "Finish", "parameters": { "bitmap": "OStepUI.ocx,FINISH" } }
  ]
}
```

Adding a Conditional (If)

Exact-children container — exactly 2 "Named Container" children:

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "steps": [
    { "stepType": "Start", "parameters": { "bitmap": "OStepUI.ocx,START", "let": {}, "weak": {}, "prompt": {} } },
    {
      "stepType": "If",
      "parameters": {},
      "subSteps": [
        {
          "stepType": "Named Container",
          "parameters": {},
          "subSteps": [
            { "stepType": "Transfer", "parameters": { ... } },
            { "stepType": "End", "parameters": {} }
          ]
        },
        {
          "stepType": "Named Container",
          "parameters": {},
          "subSteps": [
            { "stepType": "Pause", "parameters": { ... } },
            { "stepType": "End", "parameters": {} }
          ]
        }
      ]
    }
  ],
  { "stepType": "Finish", "parameters": { "bitmap": "OStepUI.ocx,FINISH" } }
}
```

Nesting Containers

Containers can be nested arbitrarily deep:

```
{
  "stepType": "Loop",
  "parameters": {},
  "subSteps": [
    {
      "stepType": "If",
      "parameters": {},
      "subSteps": [
        {
          "stepType": "Named Container",
          "parameters": {},
          "subSteps": [
            {
              "stepType": "Group",
              "parameters": {},
              "subSteps": [
                { "stepType": "Transfer", "parameters": { ... } },
                { "stepType": "End", "parameters": {} }
              ]
            },
            { "stepType": "End", "parameters": {} }
          ]
        },
        {
          "stepType": "Named Container",
          "parameters": {},
          "subSteps": [
            { "stepType": "End", "parameters": {} }
          ]
        }
      ]
    },
    { "stepType": "End", "parameters": {} }
  ]
}
```

Structural Validation Errors

When structural validation fails, the importer reports every violation and raises a single error covering all of them. Each violation is annotated with a breadcrumb `location` identifying the offending step in the tree, and a message describing the specific rule broken. Categories of validation checked:

- Free-form container leading/trailing sequences (anchor position and count).
- Leaf and exact-children container child counts and types.
- Per-step rules, including the anchor-cannot-be-disabled rule (a step carrying `disabled: true` that is a structural anchor — `Start`, `Finish`, or a container's `End` terminator — is rejected).

Envelope errors (missing `format` / `formatVersion`, malformed version string, wrong `format` value, or a `formatVersion` that is not the current one — rejected as too old or too new) are raised before structural validation, so a bad envelope surfaces first.

Composition Checklist

When constructing a method, verify:

1. Envelope has `format = "Biomek Method"`, the current `formatVersion`, and a `steps` array.
2. Root `steps` starts with `"Start"` and ends with `"Finish"`.
3. No `"Start"` or `"Finish"` inside any container — they belong only at the method root. Author the required anchor keys on each (see the *Minimal Valid Method* section above).
4. Every `"If"` has exactly 2 `"Named Container"` children.
5. Every free-form container's `subSteps` ends with the terminator that container defines (`"End"` for most, `"Multichannel Select Tips End"` for the Select-Tips family, or an add-on step's own terminator type).
6. No leaf step has `subSteps` with elements.
7. Every `parameters` object with COM-array or COM-object values uses the correct `_biomekType` discriminator and shape.
8. No parameter dictionary contains a `_biomekType` key of your own.
9. Only Documented steps from the catalog are used.

If all checks pass, the method structure is valid.

Next: populating step parameters

This document covers method *structure* — the tree of steps. Once that structure is valid, each step's `parameters` still need the correct keys and values for its step type. For those, see the per-step reference in `step-documentation/` and the Pipetting Techniques, Templates, and Auto-Selection reference.

Introduction to Pipetting Steps

Reference information common to pipetting steps.

See Pipetting Techniques and Templates for more details, or refer to individual step documentation.

Low-Level Pipetting Steps

Low-level pipetting steps consist of individual pipetting operations, such as aspirate, dispense, or mix. Each pod type has its own set of low-level steps, prefixed with the pod type.

Additionally, the Multichannel pod supports low-level pipetting steps which use the whole head's worth of tips, as well as low-level pipetting tips with a subset of tips loaded via the **Select Tips** steps.

The following table shows the step names for each low-level operation for each pod type.

Pod Type	Aspirate	Dispense	Mix	Wash Tips
Fixed-8	Fixed-8 Aspirate	Fixed-8 Dispense	Fixed-8 Mix	(not supported)
Span-8	Span-8 Aspirate	Span-8 Dispense	(not supported)	Span-8 Wash Tips
Multichannel	Multichannel Aspirate	Multichannel Dispense	Multichannel Mix	Multichannel Wash Tips
Multichannel Select Tips	Select Tips Aspirate	Select Tips Dispense	Select Tips Mix	(not supported)

Note that Multichannel Select Tips steps must be contained in a **Select Tips** group step. See Multichannel Select Tips.

High-Level Pipetting Steps

High-level pipetting steps put together sequences of multiple pipetting operations, including tip handling between operations.

The following table shows the pod types supported by each high-level pipetting step.

Step Name	Description	Supported Pod Types
Transfer	Transfer from the selected wells in a single source to the selected wells in multiple destinations.	Fixed-8, Span-8, Multichannel
Combine	Transfer from the selected wells of multiple sources to the selected wells of a single destination.	Span-8, Multichannel
Transfer From File	Use an input file to determine which wells to transfer.	Span-8
Serial Dilution	Perform multiple dilutions of a sample on a single microplate.	Span-8
Select Tips Serial Dilution	Perform multiple dilutions of a sample on a single microplate.	Multichannel, when using Select Tips

Common Pipetting JSON Keys

The following JSON keys are common to most pipetting steps, either on the step itself for low-level pipetting or each operation in a step for high-level pipetting. Refer to the individual step's reference documentation to confirm.

Key	Type	Description
prototype	string or expression	The string name of the pipetting technique to use
autoSelectPrototype	boolean	Whether the software picks the best technique automatically (not available on Fixed-8 pods)
liquidType	string or expression	The type of liquid being operated on. May be <code>Well Contents</code> for aspirates or mixes, <code>Tip Contents</code> for dispenses, or a named liquid type.
overrideHeight	boolean	Whether or not this step will use a custom height instead of the height configured in the technique
customHeight	boolean	True if <code>height</code> will be specified with an expression (only used when <code>overrideHeight</code> is <code>true</code>)
height	number or expression	Height offset value (only used when <code>overrideHeight</code> is <code>true</code>)
heightFrom	integer or expression	Reference point for the height (only used when <code>overrideHeight</code> is <code>true</code>)

Well Numbering

Pipetting steps address wells by a **1-based, row-major** well number: numbering runs across the top row first, then down. For labware with `WellsX` columns and `WellsY` rows, well `n` is at row $\lfloor n / \text{WellsX} \rfloor$ and column $(n - 1) \bmod \text{WellsX} + 1$.

For a standard 96-well plate (`WellsX` = 12, `WellsY` = 8):

- Row A = wells 1–12, row B = 13–24, ..., row H = 85–96.

A multi-probe pod that spans a full **column** (Span-8, Fixed-8, a 96-head column) places its first tip at `firstWell` and steps the remaining tips **down the column by** `+WellsX` — i.e. wells `firstWell`, `firstWell+WellsX`, `firstWell+2*WellsX`, If that list runs past the last row of the plate, the step fails at enqueue because the pod cannot place all its tips there.

Consequence for looping over columns. To visit plate columns `1..12` with a column-spanning pod, the first-well expression is the **column index itself**:

```
// Loop variable Col = 1..12, dispensing a full 8-well column each pass:
"useWellExpression": true,
"firstWellExpression": "=Col"      // wells Col, Col+12, ..., Col+84 on a 12-column plate
```

Do **not** write `"=1+(Col-1)*8"` — stepping by 8 assumes column-major numbering (8 wells per column). On a row-major plate that lands the first tip mid-plate (e.g. `Col=3` → well 17 = row B), leaving fewer than 8 rows below it, so the pod cannot fit tip 8 and the step fails at enqueue. Multiply by a stride only when iterating rows — and the correct row stride is `WellsX` (12 on a 96-well plate), not the pod's probe count.

Reservoirs and multi-section troughs

Reservoir and trough labware is **not** addressed by the row-major well rule above. Its "wells" are **sections**, and `WellsX` / `WellsY` do not apply. A section is addressed by its **1-based position in that labware type's section list** (valid range `1 ... Sections.Count`), and a pipetting step's `selectionInfo`, `pattern`, and `sectionExpression` use those same 1-based section indices.

The section number is **not geometric**. When a labware type is built in the Labware Type Editor, the author chooses the section order — "Right-Then-Down", "Down-Then-Right", or "No Sorting" with each section's number assigned by hand — so section 1 is **not** guaranteed to be the top-left section, and the numbering need not follow any physical progression.

Because the mapping is defined by the labware type, the only reliable source of truth is that type's own section definition — inspect it in the Labware Type Editor, which pairs each section's number with its physical rectangle, or read the corresponding local project file if you have it. Do not assume a fixed top-to-bottom or left-to-right order. When you author a starting section, the step fills its remaining tips from that section onward in list order.

Tip Handling

Each pod defines its own Load Tips and Unload Tips steps, prefixed with the pod type. Additionally, Select Tips groups define Select Tips Load Tips, Select Tips Unload Tips, Advanced Load Tips, and Advanced Unload Tips.

Some low-level pipetting steps support a **Refresh Tips** option to load tips prior to an operation, but generally speaking, tip handling occurs before and after low-level pipetting steps.

High-level pipetting steps provide options for whether and how to change tips between operations.

Splitting a draw that will not fit

On a Transfer, Combine, or Transfer From File step, `splitVolume: true` breaks an oversized draw into several **equally sized** cycles rather than one large one. The number of cycles is driven by the smaller of two *usable* capacities — the tip's capacity after blowout and air gaps, and the syringe or plunger capacity after air gaps and volume calibration — so neither raw tip capacity nor `max.d` predicts it directly.

Splitting engages only when the aspirate serves a single destination, and repeat-by-volume mode (`repeatsByVolume: true`) disables it as well. Raising `repeats` above 1 does not make the cycles coarser — it turns splitting **off** and leaves the oversized draw to be attempted whole, on top of grouping more destinations onto that one draw. Pair `splitVolume: true` with `repeats: "1"` and `repeatsByVolume: false` . See Transfer Cross-Field Rule 17.

Pipetting Techniques, Templates, and Auto-Selection

Applies to any pipetting-family step (Span-8 Aspirate/Dispense, Multichannel Aspirate/Dispense/Mix, Transfer, Combine, etc.).

Not a Biomek export — these are the defaults. Pipetting techniques and templates are a kind of **project item**, and Biomek has **no JSON export for project items** today. Everything described here therefore documents the **defaults included with Biomek**, not an exported file. In practice most users work with **their own** techniques and templates (and other project items) that they have created or tuned for their assays, rather than the default set — so treat the technique names and values shown here as illustrative examples, and confirm the actual set installed on your target instrument. For everyday authoring you only need to reference an existing technique **by name** (one already defined in your project), or leave auto-selection on. Hand-authoring the technique *configuration* in JSON — the height overrides and the `c__` pipetting context variables covered below — is an **advanced** capability most authors never need; techniques are normally created and tuned in the Biomek software, not written by hand. Pipetting **templates** cannot be authored in JSON at all.

Terminology

The JSON key names and the Biomek UI use different terms for the same concepts:

JSON key	UI / user-facing term	What it is
<code>Prototype</code> (step dictionary key)	Technique name	The string name of the selected technique
<code>AutoSelectPrototype</code>	Auto-Select technique	Whether the software picks the best technique automatically

Templates vs. Techniques

A **pipetting template** is the parameterized *recipe*: an ordered program of atomic physical operations (aspirate, dispense, mix, air-gaps, tip-touch, ...) whose values are left open as `=c__` variable placeholders. It is pod-family-specific — a template such as `F8 Pipetting`, `S8 Pipetting`, or `MC Pipetting` runs only on its own pod family — and a single template is not tip- or volume-specific, so one template serves many techniques across that family's regimes. Each family also has several templates for procedures that differ physically (base pipetting plus variants such as septa-piercing, clot-detection, active-wash, and low-volume).

A **technique** is a *named binding* on top of a template: it picks one template and fills in concrete values for that template's variables, plus the LLS / clot / piercing / liquid-type / calibration settings, scoped to a specific pod family + tip / volume / liquid regime (min/max volume, rank).

Think of the template as a **function definition with parameters** and the technique as a **named partial application** of it — which is why many techniques can bind the same template (several `MC ...` techniques in the default project drive `MC Pipetting` with different values). A technique without a template, or a template without a technique, does nothing.

The one rule for JSON authors: method JSON references **techniques**, never **templates**. There is **no JSON representation for a pipetting template** — you cannot create one, edit its operation tree, or point a step at one in authored JSON. Templates live only inside the project and are reached *indirectly*, as the thing a technique selects. The JSON Support Boundary below states this precisely.

Where to work with these in Biomek

You do not author techniques or templates in JSON — you configure them in the Biomek software. If you need to:

- **see which techniques a project provides** (and the pod family, tips, and volume range each targets), open the **Technique Browser** (Utilities ▸ Technique Browser);
- **view or adjust a technique's settings** (and see which template it uses), open it in the **Technique Editor**;

- **inspect or edit a template's operation tree**, open the **Pipetting Template Editor** (Utilities ▸ Pipetting Template Editor).

JSON Support Boundary

Method JSON references **techniques**, never **templates**. A step can name a technique, auto-select one, or embed an inline technique object — the three modes are detailed under Technique Selection below.

Templates sit entirely outside that boundary: no step and no `_biomekType` value refers to a template, so you cannot author one, edit its operation tree, or even change *which* template a technique uses from JSON. To change the physical procedure, edit the technique's template in the Pipetting Template Editor, or pick a different technique whose template already does what you need.

How Techniques Work

Layers

1. **Pipetting step** (e.g., Multichannel Aspirate) — configured in the method tree. Contains keys like `pod` , `location` , `volume` , `autoSelectPrototype` , `prototype` , `overrideHeight` , etc.
2. **Technique** — stores all the detailed pipetting parameters (aspirate speed, dispense speed, heights, blowout settings, tip touch, prewet, LLS, clot detection, etc.). Selected either automatically or by name.
3. **Pipetting template** — an ordered sequence of physical actions (aspirate, dispense, tip touch, mix, etc.) that the instrument physically executes. Each action reads its parameters from `C__` context variables populated by the technique.

Execution Flow

1. The step resolves a technique (auto-select or by name).
2. The step injects `C__` context variables into the runtime scope — some from step properties, some from the technique.
3. The step calls the technique's template operation (Aspirate, Dispense, or Mix).
4. The template's actions execute in order, reading `C__` variables for their parameters.

Technique Selection

Auto-Select Mode (`autoSelectPrototype: true`)

When auto-select is enabled, the step builds a context from the current state and finds the best-matching technique.

Inputs to auto-selection:

Input	Source
Pod type	the pod's configured type
Head type	the pod's head type
Tip class	the class of the tips loaded on the pod
Labware class	From the step's <code>labwareClass</code> property
Liquid type	Resolved from the step's <code>liquidType</code> — may come from well contents, tip contents, or the literal value
Volume	From the step's <code>volume</code> — or computed from tip contents when emptying tips

If no technique matches, enqueue fails with a technique-not-found error.

Named Technique Mode (`autoSelectPrototype: false`)

When auto-select is disabled:

1. If `customPrototype` is present (an inline `_biomekType:"technique"` object), it is used directly and overrides `prototype` . It is a valid — if uncommon — thing to author, and must be preserved when round-tripping an existing method; the editor usually creates it.

- Otherwise, `prototype` is looked up via the technique lookup (by name). The lookup is attempted **twice**: first against the raw (unevaluated) string, then, if that fails, against the expression-evaluated string. So `prototype` may either be a literal technique name or an expression that resolves to one at runtime.
- If both lookups fail, enqueue fails with a technique-not-found error.

Which Mode to Use

Scenario	Recommendation
General-purpose method, user has techniques configured	<code>autoSelectPrototype: true</code> — let the system pick the best match
Method requires a specific technique	<code>autoSelectPrototype: false</code> , set <code>prototype</code> to the exact technique name
Unknown technique availability	<code>autoSelectPrototype: true</code> — adapts to what's installed

The default for `autoSelectPrototype` is `false`. This means if you don't set it, the step will look for a named technique in `prototype` — which will fail if `prototype` is empty. Prefer to set `autoSelectPrototype` explicitly.

Fixed-8 exception: The Fixed-8 pipetting steps (`Fixed-8 Aspirate`, `Fixed-8 Dispense`, `Fixed-8 Mix`) **do not support auto-selection at all** — this is a permanent, intentional design choice. Setting `autoSelectPrototype: true` on a Fixed-8 step fails at enqueue. For Fixed-8, always use `autoSelectPrototype: false` with an explicit `prototype` (or a `customPrototype` object). Auto-select applies only to Span-8 and Multichannel steps.

Notes On Choosing Techniques

Techniques that are intended to do LLS or clot detection are only compatible with conductive tips or fixed tips on the Span-8 pod.

Techniques with "wash" in the name are typically intended for tip washing, not for regular liquid transfers.

Techniques sometimes include the intended volume in the name (e.g. "low" or "high" or "medium"). For transfers of low volumes, choose a low volume technique when available. For transfers of high volumes, choose a high volume technique when available. For all others, choose a medium technique.

Inline Technique Object (`customPrototype`)

A step may also carry an inline technique object under the `customPrototype` key — a JSON object with `_biomekType: "technique"` embedded directly in the step's parameters rather than a named reference resolved from the project. When present, `customPrototype` is used directly and **overrides** `prototype` at enqueue.

This mode is uncommon and not recommended in hand-authored JSON — the Biomek editor is what usually creates a `customPrototype`, typically when the user tunes technique settings on a specific step without saving them back as a named project technique. It is nonetheless a valid, authorable JSON parameter, and it **must be preserved when round-tripping** an existing method. Do not strip it on the assumption that it is UI-only state.

Set `autoSelectPrototype: false` alongside it. On Span-8 and Multichannel steps the inline object is only consulted when auto-selection is off; leaving `autoSelectPrototype: true` there means the step auto-selects a named technique and your inline object is ignored. On Fixed-8 auto-selection is rejected outright, so `false` is required anyway.

An inline technique must be complete. It is not a patch over a named technique: nothing is merged in and no defaults are filled: the object you author becomes the whole technique. Omitting a block the step needs fails at enqueue — a missing `general` block, for instance, fails with `The technique does not implement the General settings`. The reliable way to author one is to copy a full technique payload for your pod family from Catalog: Techniques (in *Biomek Instrument & Project Reference*) and change only the keys you want, leaving the rest intact.

The keys worth knowing when tuning one:

What you want to change	Key	Where it nests
Aspirate / dispense speed	<code>c__Speed</code>	in the <code>aspirate</code> and <code>dispense</code> blocks respectively — this is the one to edit on every pod family

What you want to change	Key	Where it nests
Aspirate / dispense speed (Fixed-8 only, additional)	<code>c__aspiratespeed</code> , <code>c__dispensespeed</code>	the <code>general</code> block. Fixed-8 techniques carry these as well; Multichannel and Span-8 techniques do not.
Mix after dispensing	<code>c__Mix</code> (on/off), <code>c__MixCount</code> , <code>c__MixVolume</code> , <code>c__MixAspirateHeight</code> / <code>c__MixAspirateFrom</code> / <code>c__MixAspirateSpeed</code> , and the matching <code>c__MixDispense*</code> trio	the <code>dispense</code> block
Mixing as the step's own operation	the same <code>c__Mix*</code> keys minus <code>c__MixCount</code> and <code>c__MixVolume</code>	the <code>mix</code> block. A Mix step supplies count and volume itself from its <code>mixCount</code> and <code>volume</code> parameters, overriding whatever the technique carries.

The editor writes `prototype: "[Custom]"` next to a `customPrototype` it created, matching the inline object's `name` . That is harmless — the inline object wins — and should be preserved on round-trip.

Technique-Family Rule (F8 / S8 / MC prefixes)

Techniques are **pod-family specific**. In practice, technique names follow a prefix convention that maps to the pod they were configured for:

Prefix	Pod family	Examples
F8 ...	Fixed-8	F8 High , F8 Low , F8 Medium , F8 MultiDispense , F8 Medium TopDispense
S8 ...	Span-8	S8 250 , S8 1000 High , S8 Active Wash , S8 Clot Detection , S8 SeptaFluted
MC ...	Multichannel	MC P60 , MC P300 High , MC P1200 High , MC MultiDispense , MC Active Wash

Naming a technique from the wrong family on a step is a common porting failure — e.g. leaving `prototype: "F8 Medium"` on a `Span-8 Aspirate` after converting a Fixed-8 method to Span-8. The `_autoSelect_` category machinery further scopes each technique to its `Pod.Type` , so a cross-family reference is either rejected at auto-select time or, when named explicitly, mis-renders the step in the editor even when the underlying JSON is otherwise valid.

When you change pod or the pod family of any pipetting step, review every prototype (and any inline customPrototype) and re-point them at same-family techniques as needed. Prefer confirming the actual technique names available on the target instrument, rather than guessing.

The prefix is a naming convention — the *authoritative* pod-family binding lives inside the technique's `Pod` and `_autoSelect_` sub-dictionaries.

Technique names are project-specific. The names in the table above (e.g. `F8 Medium` , `MC P60` , `S8 1000 Medium`) are the defaults in Beckman's example projects (`Biomek_i3Project` , etc.). A given customer project may define entirely different technique names, or may not include some of those defaults. A `prototype` value must name a technique that **actually exists in your project** — do not assume a default catalog name is present.

To find the valid technique names for your project, open the **Technique Browser** in the Biomek software (`Utilities` ▶ `Technique Browser`). It lists exactly the techniques your project defines. If you do not have that list, obtain it from whoever owns the project before authoring method steps.

Because **Fixed-8 never auto-selects**, every Fixed-8 Aspirate / Dispense / Mix step **must** name a `prototype` that exists in the project (or supply an inline `customPrototype`) — otherwise enqueue fails with: No technique is defined with the name '<name>' . Multichannel and Span-8 steps can instead set `autoSelectPrototype: true` .

If you cannot rely on a specific named technique being present, use `customPrototype` — an inline technique object (`_biomekType: "technique"`) embedded directly in the step — as the project-independent alternative. See [Inline Technique Object](#) above for details.

Liquid Types

Every aspirate/dispense/mix operation has a `liquidType`. It feeds technique auto-selection (as one of the match criteria) and records what ends up in the tips and wells. There are two special tokens plus any number of named liquids:

liquidType value	Meaning	Valid on
"Well Contents"	Auto-detect the liquid from the labware's tracked contents at run time.	Aspirate / Mix (source) only
"Tip Contents"	Use whatever the tips are currently known to hold.	Dispense (destination) only
A named liquid (e.g. "Water", "Serum")	Use this specific liquid type explicitly, overriding auto-detection.	Any operation

Rules to watch:

- "Tip Contents" on an **aspirate or mix** step fails at run time — you cannot aspirate "the contents of the tips." Use "Well Contents" or a named liquid.
- "Well Contents" on a **dispense** step fails at run time (symmetric to the rule above) — a dispense is defined by what the tips hold, not the destination well's contents. Use "Tip Contents" or a named liquid.
- A **named** liquid must be one the project knows about; it participates in auto-select as one of the match criteria (see §"Auto-Select Mode" above).

In a Transfer/Combine step the `liquidType` lives on each *item*: source items typically carry "Well Contents", destination items "Tip Contents".

Liquid Types are project-specific. The default liquid types provided by Biomek Software are "Serum" and "Water" only. A given customer project may define entirely different liquid types, or may not include some of these defaults.

To find the valid liquid types for your project, open the **Liquid Type Editor** in the Biomek software.

Volume types and locatable surfaces

When labware is placed on the deck (in **Instrument Setup**), each piece can declare how well its per-well volumes are known, via `volumeType`:

volumeType	Meaning	Effect at pipetting time
"Unknown"	Volumes are not known. (default if omitted)	A liquid-relative move needs the surface located at run time. On Span-8 , an LLS technique can sense the surface live even when the volume is unknown. Fixed-8 and Multichannel (96- or 384-channel) pods do not have LLS, so on those pods an "Unknown" well fails a liquid-relative move — declare at least "Nominal".
"Nominal"	Volumes are approximate/estimated.	Surface is <i>locatable</i> from the estimate, but LLS (if available) still refines it.
"Known"	Volumes are definitively known.	The surface is computed from the tracked volume; LLS is skipped.

Pod-family carve-out. The "LLS rescues an "Unknown" well" behavior is **Span-8 only**. On Fixed-8 and Multichannel (96- or 384-channel) pods, a liquid-relative move against an "Unknown" well always fails at run time regardless of the technique — LLS does not lift the restriction there. The minimum you can declare is "Nominal".

The single `volumeType` string is authoritative. Matching is **case-insensitive** on import — "Unknown", "unknown", "NOMINAL", "known", etc. all parse to the same enum values. Any other **present** value — an unrecognized string, a numeric string like "1", or a non-string JSON value — throws a `JsonException` on import. **Omitting** `volumeType` entirely is allowed and defaults to `Unknown`.

Pair "Known" / "Nominal" with a fill

Declaring a volume type is not the same as *stating the volume*. To actually track contents you also supply, on the labware in Instrument Setup:

- `evalAmounts` — the per-well volumes (µL), and
- `evalLiquids` — the per-well liquid types.

Setting `volumeType: "Known"` **without** an `evalAmounts` fill leaves the tracked well volume at 0 — an aspirate then fails with `Cannot pipette <v> uL; the well only has 0.000 uL in it`. Always pair `"Known"` (or `"Nominal"`) with `evalAmounts` (and `evalLiquids` for the liquid identity). The tracked volume is a running ledger maintained across the whole method, so this error can also mean an earlier step drained the well rather than that the seed is missing — see Well Volume Tracking.

Height Override

The step can override the technique's pipetting height on a per-step basis.

Keys

Key	Type	Default	Description
<code>overrideHeight</code>	boolean	<code>false</code>	Whether to override the technique's height for this step
<code>height</code>	number or expression	<code>0</code>	Height offset value (only used when <code>overrideHeight</code> is <code>true</code>)
<code>heightFrom</code>	integer or string	<code>0</code>	Reference point for the height (only used when <code>overrideHeight</code> is <code>true</code>)

`heightFrom` Values

Value	String alias	Meaning
<code>0</code>	<code>"Liquid"</code>	Height measured from the liquid surface
<code>1</code>	<code>"Bottom"</code>	Height measured from the bottom of the labware
<code>2</code>	<code>"Top"</code>	Height measured from the top of the labware

Sign Conventions

- From **liquid** (`0`): negative values go below the surface (common for aspirating submerged).
- From **bottom** (`1`): positive values go above the bottom (common for avoiding dead volume).
- From **top** (`2`): negative values go below the top rim (common for dispensing into a well).

Liquid-relative heights need a locatable liquid surface

When a step positions relative to the **liquid surface** — `heightFrom = "Liquid"` (`0`), or a technique with follow-liquid enabled (`C__FollowLiquid = True`, the default for the stock "follow" techniques) — the target well's surface must be **locatable at runtime**, via one of:

- a **known/nominal volume**: the labware is placed in Instrument Setup with `volumeType: "Known"` (or `"Nominal"`) **and** an `evalAmounts` fill (µL per well) — plus `evalLiquids` for the liquid type; or
- **liquid-level sensing (LLS)** — Span-8 only: LLS-capable tips + an LLS technique, so the level is sensed at runtime. **Fixed-8 and Multichannel (96- or 384-channel)** pods do **not** have LLS, so this alternative does not apply on those pods — an `"Unknown"` well there still needs `"Nominal"` or `"Known"`.

If neither holds — an `"Unknown"` well with no locatable surface (and, on Fixed-8/96, any `"Unknown"` well, because those pods lack LLS entirely) — the step fails at **runtime** (not enqueue) with a "liquid amounts not defined" error.

`heightFrom = "Bottom" / "Top"` never triggers this — overriding to one of those avoids the error but changes where the tip goes.

The `volumeType` string values are `"Unknown"`, `"Nominal"`, and `"Known"`. See \$Volume types above for the per-value semantics and the Fixed-8/96 carve-out.

How Override Works at Runtime

When `overrideHeight` is `true`:

- `C__Height` is set from the step's `height` property (instead of inheriting from the technique).
- `C__HeightFrom` is set from the step's `heightFrom` property.
- The template's actions use these overridden values for their pipetting height.

When `overrideHeight` is `false`:

- `C__Height` and `C__HeightFrom` are NOT set by the step.
- The template's actions inherit these values from the technique's own height settings.

Height Encoding Keys

Key	Description
<code>customHeight</code>	Encoding discriminator for <code>height</code> / <code>heightFrom</code> : when <code>false</code> , <code>height</code> is a numeric offset and <code>heightFrom</code> the numeric enum; when <code>true</code> , <code>height</code> is instead a text string (a typed/custom height, which may be an expression), and <code>heightFrom</code> stays the numeric enum for <code>Liquid</code> / <code>Bottom</code> / <code>Top</code> but is written as text for anything else. <code>true</code> implies <code>overrideHeight</code> is also <code>true</code> . For authored JSON with numeric heights, set <code>customHeight: false</code> .

`customHeight` means the same thing on every step that carries `height` / `heightFrom`. It is this one encoding discriminator — not a per-step flag that means something different from step to step, and not a record of how the value was entered in the editor. What varies by step is only whether the key has to be present:

- **Required** in each `aspirateProperties` / `dispenseProperties` / `diluentAspirateProperties` / `diluentDispenseProperties` sub-dictionary of `Span-8 Serial Dilution`, on the same terms as `height`, `heightFrom`, and `overrideHeight` there.
- **Optional everywhere else**, defaulting to `false`. Preserve it on round-trip when present.

No step branches on `customHeight` beyond the encoding it selects, so with numeric heights `false` is always the right value.

C__ Pipetting Context Variables

These variables are injected into the runtime `Let` dictionary during pipetting step execution. The template's actions read them via expressions like `=C__Volume`.

Advanced — authoring these directly is not recommended. The `c__` variables are an internal mechanism that the technique and template system manages for you. This section is reference material for understanding runtime behavior; you should not normally set `C__` variables by hand in method JSON. Configure pipetting behavior by selecting an appropriate **technique** (or adjusting one in the Technique Editor) instead — see the note at the top of this document.

Step-Injected Variables

These are set by the pipetting step itself from its dictionary properties and the resolved runtime state:

Variable	Type	Source	Description
<code>C__Pod</code>	string	<code>pod</code> property	Pod name
<code>C__Position</code>	string	Resolved <code>location</code>	Deck position string
<code>C__Volume</code>	number	<code>volume</code> property	Pipetting volume in μL
<code>C__RepeatCount</code>	number	Hard-coded <code>1.0</code>	Repeat count (always 1 for single-operation steps)
<code>C__LiquidType</code>	string	Resolved liquid type	Liquid type name
<code>C__RealWellX</code>	number	Pod X position	Physical X coordinate of the well

Variable	Type	Source	Description
C__RealWellY	number	Pod Y position	Physical Y coordinate of the well
C__RealWellZ	number	Calculated Z	Physical Z coordinate for pipetting
C__LabwareWellsArray	array	Well indices	Array of well indices being targeted
C__LabwareWellDups	boolean	Hard-coded <code>false</code>	Whether wells are duplicated
C__Operation	string	Operation name	"Aspirate", "Dispense", or "Mix"
C__Height	number	<code>height</code> property	Only when <code>overrideHeight</code> is <code>true</code>
C__HeightFrom	integer	<code>heightFrom</code> property	Only when <code>overrideHeight</code> is <code>true</code>
C__ConditioningExcess	number	Read from technique General dict, injected by step	Conditioning excess volume (may be % or μL)

Technique-Provided Variables

These come from the technique's stored parameters (not from the step's parameters). They are set by the template system, not by the step:

Category	Variables
Speed	C__Speed, C__AspirateSpeed, C__DispenseSpeed
Height	C__Height, C__HeightFrom, C__MinimumHeight (when not overridden by step)
Blowout	C__Blowout, C__BlowoutVolume, C__BlowoutDelay
Tip Touch	C__TipTouch, C__TipTouchDelay, C__TipTouchSpeed, C__TipTouchHeight, C__TipTouchHeightFrom, C__TipTouchTheta
Prewet	C__Prewet, C__PrewetOverage, C__PrewetDelay
Mix	C__Mix, C__MixCount, C__MixVolume, C__MixAspirateHeight, C__MixAspirateFrom, C__MixAspirateSpeed, C__MixDispenseHeight, C__MixDispenseFrom, C__MixDispenseSpeed
Follow Liquid	C__FollowLiquid
Trailing Air Gap	C__TrailingAirGap, C__TrailingAirGapVolume
Delays	C__AspirateDelay, C__DispenseDelay
Calibration	C__CalibrationSlope, C__CalibrationOffset, C__CutoffVelocity
Conditioning	C__ConditioningSectionVolume, C__ConditioningSectionCount
Liquid Level Sensing (LLS)	C__DetectionMode, C__Conductivity, C__ConductivityInherit, C__DetectionSpeed, C__DoubleDistance, C__ReservoirSafeDepth, C__LiquidLevel, C__LLSInitial, C__LLSInitialHeight, C__LLSInitialHeightFrom, C__LLSRepeat, C__LLSAccept, C__LLSFail, C__LLSAcceptValue, C__LLSFailValue, C__LLEveryTime, C__LLSErrorAutoRetry, C__LLSErrorAutoRecover
Clot Detection	C__ClotDetectionSpeed, C__ClotLimit, C__CDHeight, C__CDHeightFrom, C__CDRepeat, C__CDEveryTime, C__CDErrorAutoRetry, C__CDErrorAutoRecover, C__CDSpeed, C__CDConductivity, C__CDConductivityInherit
Piercing	C__PiercingInitialHeight, C__PiercingFinalHeight, C__PiercingExitHeight, C__PiercingFinalSpeed, C__PiercingExitSpeed, C__PiercedSpeedLimit

Using C__ Variables in Expressions

Any expression-capable string field in a step's parameters can reference `C__` variables with the `=` prefix. For example:

- "volume": "`=C__Volume * 0.5`" — half the context volume
- "height": "`=C__Height + 1.0`" — 1mm above the context height

This is primarily used within template actions, not in the method-level step parameters. Method-level steps set `C__` variables; they don't typically read them.

Advanced: The `let` Override Mechanism

If a step's dictionary contains a bound `let` key (an object), all key-value pairs from it are copied into the runtime `Let` dictionary, potentially overriding any `C__` variable. This is an advanced automation feature — do not include in normal authored JSON.

What Authored Steps Need to Know

For **technique selection and height override**, a pipetting step's `parameters` use these keys:

1. `autoSelectPrototype` — always set explicitly (`true` or `false`).
2. `prototype` — required when `autoSelectPrototype` is `false` ; must be the exact name of an installed technique.
3. `overrideHeight` — set to `true` only when you need a specific pipetting height.
4. `height` and `heightFrom` — required when `overrideHeight` is `true` .
5. `customHeight` — include as `false` for numeric heights. When `true` , `height` is interpreted as text (a typed/custom height, possibly an expression) instead of a numeric offset.

These are **only** the technique-and-height keys. A pipetting step still needs its own core parameters — the pod, the labware/location, the volume, the well selection, the liquid type, and so on — which are documented per step in `step-documentation/`. The blocks below show just the technique/height keys.

The **auto-select** technique/height keys:

```
{
  "autoSelectPrototype": true,
  "prototype": "",
  "overrideHeight": false,
  "height": 0,
  "heightFrom": 0,
  "customHeight": false
}
```

The **named-technique** technique/height keys (the required form for Fixed-8, which never auto-selects):

```
{
  "autoSelectPrototype": false,
  "prototype": "F8 Medium",
  "overrideHeight": false,
  "height": 0,
  "heightFrom": 0,
  "customHeight": false
}
```

You do NOT need to:

- Know or set any `C__` variables — these are populated automatically at runtime.
- Know the template structure — this is internal to the technique.
- Configure individual template-action parameters — these come from the technique.

Note on `customPrototype` : an inline technique object; you rarely hand-author one, but it is valid JSON, overrides `prototype` , and must be preserved on round-trip.

Biomek Method JSON — Quick-Reference Cheatsheet

One-page summary of the rules that bite most often when authoring method JSON by hand. Each section links to the full reference for depth.

Want a working template? See [Worked Example: Simple i3 Fixed-8 Method](#) for a complete, copy-ready i3/Fixed-8 method you can modify.

Method Envelope

(Method JSON Structure)

```
{
  "format": "Biomek Method",           // required, must equal "Biomek Method" (compared case-insensitively)
  "formatVersion": "1.0",              // required; the Biomek Method JSON format version, currently "1.0"
  "author": "",                        // optional
  "description": "",                   // optional
  "steps": [ ... ]                    // required, the method tree
}
```

Step Node Shape

(Method JSON Structure)

```
{
  "stepType": "Transfer",              // required -- case-insensitive match
  "disabled": false,                  // optional (omitted when false)
  "parameters": { ... },              // required (may be {})
  "subSteps": [ ... ]                 // optional -- containers only
}
```

disabled: Author `"disabled": true` on the step node to disable it without removing it from the method. (Base Keys & Serialization Conventions §3)

Anchor & Terminator Rules

(Method JSON Structure)

Rule	Detail
Start/Finish required at root	Every method root: <code>[Start, ...body..., Finish]</code> .
Free-form containers need a terminator	Last child must be <code>"End"</code> (most containers), <code>"Multichannel Select Tips End"</code> (Select Tips family), or the terminator an add-on container registers for itself.
Anchors can never be disabled	<code>disabled: true</code> on Start, Finish, or any terminator (End, Multichannel Select Tips End, etc.) is rejected at validation.
Exact-children: If	Exactly 2 <code>"Named Container"</code> children (Then / Else), each ending with <code>"End"</code> .

Common `_biomekType` Discriminators

(Method JSON Structure)

<code>_biomekType</code>	Minimal shape
<code>"comArray"</code>	<code>{ "_biomekType": "comArray", "arraySubtype": "<type>", "values": [...] }</code>
<code>"labware"</code>	<code>{ "_biomekType": "labware", ... }</code> (inside Instrument Setup)
<code>"dateTime"</code>	<code>{ "_biomekType": "dateTime", "value": "2024-01-31T13:45:30.000000Z" }</code>
<code>"technique"</code>	<code>{ "_biomekType": "technique", ... }</code> (inline custom technique — usually editor-written, but valid in JSON; overrides <code>prototype</code>)
(<i>absent</i>)	Plain nested dictionary (the default)

Never use `_biomekType` as your own key inside `parameters` .

`comArray` **One-Liner**

```
{ "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, ...] }
```

Legal `arraySubtype` values: `boolean` , `byte` , `dateTime` , `double` , `integer` , `short` , `single` , `string` , `variant` .

A plain JSON array (`[true, false]`) without the wrapper is deserialized as a different Biomek type and will break downstream logic.

Pod-Family --> Step-Family

(Introduction to Biomek and Liquid Handling Method Writing)

Pod	Steps
Fixed-8 (i3)	<code>Fixed-8 Aspirate</code> , <code>Fixed-8 Dispense</code> , <code>Fixed-8 Mix</code> , <code>Fixed-8 Load/Unload Tips</code>
Span-8	<code>Span-8 Aspirate/Dispense/Load/Unload/Wash Tips</code> , <code>Span-8 Serial Dilution</code> , <code>Span-8 Transfer From File</code>
Multichannel	<code>Multichannel Aspirate/Dispense/Mix/Load/Unload/Wash Tips</code> , <code>Multichannel Select Tips *</code>
Any	<code>Transfer</code> (pod family resolved from the <code>pod</code> field at runtime)
Span-8 or Multichannel	<code>Combine</code> (not available on a Fixed-8 pod)

Low-level pipetting steps are pod-prefixed (`Fixed-8` , `Span-8` , `Multichannel`); `Transfer` and `Combine` are not — they pick their pod family at run time from the `pod` field.

Expression = Prefix

(Concept Guide: Expressions)

A string whose first character is `=` is evaluated at run time using VBScript syntax; without it the value is literal.

```
"amount": "10"           // literal 10 uL
"amount": "=dose * 2"    // expression, evaluated at run time
```

Write numeric values destined for expression-capable fields as **quoted strings** (`"10"` , not `10`). Some fields require an explicit flag (`useWellExpression` , `useExpression`) before the expression key is read — setting the expression without flipping the flag has no effect; the engine reads the literal key instead.

Exception: condition fields (e.g. `If.condition`) auto-prefix the `=` , so a bare string there is still evaluated as an expression. (Concept Guide: Expressions §1)

Well-Selection Idioms

(Introduction to Pipetting Steps §"Well Numbering"; Concept Guide: Probe & Mandrel Selection)

Idiom	Where	Shape
firstWell + stride	Span-8 / Fixed-8 low-level steps	"firstWell": 1 (1-based, row-major). Pod adds +wellsx per tip down the column for 96-format plates.
pattern comArray	Transfer / Combine items, Span-8 Serial Dilution	"pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, ...] } — one entry per addressable position, full-length: 96 booleans for a 96-well plate, but one per section on a reservoir ([true] for a one-section reservoir). Which of pattern and selectionInfo is the operative well selector depends on pod family — Fixed-8 and Multichannel read selectionInfo ; Span-8 reads pattern . See Rule 12 in Transfer. Keep both keys present and consistent with each other on Transfer/Combine items.
selectionInfo sections	Multichannel Aspirate / Dispense / Mix	"selectionInfo": { "_biomekType": "comArray", "arraySubtype": "integer", "values": [1] } — 1-based section/quadrant index. 96-well + 96-head = section [1] only; 384-well + 96-head = quadrants 1 - 4 .

The =Col vs =1+(Col-1)*8 Stride Trap

To sweep plate columns 1..12 with a column-spanning pod, the first-well expression is "=Col" (the column index itself). Do **not** write "=1+(Col-1)*8" — that assumes column-major numbering, lands the first tip mid-plate on a row-major layout, and fails at enqueue because the pod cannot fit its remaining tips.

Fixed-8 Never Auto-Selects a Technique

(Pipetting Techniques, Templates, and Auto-Selection)

Setting autoSelectPrototype: true on a Fixed-8 Aspirate / Dispense / Mix step throws at enqueue. This is permanent and by design. Always use autoSelectPrototype: false with an explicit prototype (e.g. "F8 Medium") on Fixed-8 steps.

Start & Finish Steps Special Import Requirements

(Start, Finish)

Step	Must author
Start	"bitmap": "OStepUI.ocx,START", plus "let": {}, "weak": {}, "prompt": {} (empty is fine). Omitting any of these fails at enqueue or breaks the icon.
Finish	"bitmap": "OStepUI.ocx,FINISH" . Omission breaks the icon.

Authored parameters replace defaults wholesale — nothing is merged in.

Key Casing

(Base Keys & Serialization Conventions §1)

Keys are **case-insensitive on import**, but author the **serialized (camelCase-first-word) form** for round-trip fidelity. The rule: only the first PascalCase word is lowercased; everything else (spaces, underscores, trailing ?) is preserved verbatim.

Source constant	Serialized JSON key
LiquidType	liquidType

Source constant	Serialized JSON key
Use Shake	use Shake
HTML_Dialog	html_Dialog
Dynamic?	dynamic?

Key Gotchas

Gotcha	Fix
Span-8 has no standalone Mix step	Mixing in place is a technique setting — configure it on the technique. An item-level <code>mixCount</code> on a Transfer item is inert on every pod family. (Transfer, Pipetting Techniques, Templates, and Auto-Selection)
MC partial-plate needs Select Tips	A <code>Multichannel Aspirate / Dispense</code> alone cannot pick an arbitrary well subset. Wrap in <code>Multichannel Select Tips</code> container (terminator: <code>Multichannel Select Tips End</code>). (Introduction to Biomek and Liquid Handling Method Writing)
A technique is built for one pod family	A technique built for one pod family won't apply to another — referencing it from the wrong pod fails or mis-renders. The default techniques signal their family in their names (<code>F8 ...</code> = Fixed-8, <code>S8 ...</code> = Span-8, <code>MC ...</code> = Multichannel), but a technique you create can be named anything, so match by the pod family a technique was built for, not by its name. (Pipetting Techniques, Templates, and Auto-Selection)
<code>LabwareClasses\\</code> prefix only in Instrument Setup	In <code>deckItems</code> , the class field takes the path-prefixed form (e.g. <code>"class": "LabwareClasses\\BCF1at96"</code>). In pipetting steps' <code>what / labwareClass</code> fields, use the short name only (<code>"BCF1at96"</code>). Using the prefixed form in a pipetting step causes an enqueue failure.
Budget tip boxes to tip usage	Too many Load-Tips cycles for the placed boxes fails enqueue (<code>Unable to find and retrieve tips...</code>); add boxes, or keep the same tips loaded across consecutive steps.
Not every deck position is reachable by every pod	Pipetting at an unreachable position fails enqueue (<code>Unable to find a path ...</code>). If the message continues <code>destination <axis> ... outside of travel range</code> , it is a hard axis limit — move the labware; clearing neighbors will not help. (Reachability and Access (in <i>Biomek Instrument & Project Reference</i>))
A short <code>evalAmounts</code> does not zero the rest	Wells past the end of the array take element 0's value, not <code>0.0</code> . Supply a full-length array with explicit zeros. An array <i>longer</i> than the well count is a hard error; a reservoir is section-length. (Instrument Setup)
<code>Cannot pipette ...; the well only has 0.000 uL in it.</code>	The enqueue-time volume ledger thinks the well is empty. Seed it with <code>volumeType + evalAmounts</code> if the liquid arrives off-deck — or account for earlier draws, which are cumulative. (Concept Guide: Well Volume Tracking)
<code><where they came from> is not a <code>DiscardTipsLocation</code> value</code>	It belongs to Fixed-8 Unload <code>tipDestination / Transfer unloadLocation</code> . For tip-box routing use <code>DiscardTipsLocation: "<TipBox>"</code> . It fails silently on the wrong key. (Instrument Setup)
A Select Tips rearrange box must be declared <i>and</i> empty	Declare a same-class tip box with <code>volumeType: "Known"</code> and a full-length <code>evalAmounts</code> of <code>0.0</code> — a tip box's inventory is its amounts array. Omitting the box is not the same as an empty box. (Multichannel Select Tips)
One MC Load Tips uses a whole box	Multichannel skips partially-used boxes, so on a hybrid, a Span-8 draw from a shared box retires it from the MC pool. The Multichannel head can only reload the same tips multiple times if the tips are configured for reuse in the instrument setup step, or if using Select Tips steps. (Multichannel Load Tips)

Position / Class Key by Family

Family	Position field	Class field
Fixed-8	<code>where</code>	<code>what</code>
Span-8	<code>where</code>	<code>what</code>
Multichannel	<code>location</code>	<code>labwareClass</code>
Transfer/Combine item	<code>position</code>	<code>labwareClass</code>

Which positions exist on each deck — the sample decks under `biomek-file-formats/deck-layouts/samples/` (in *Biomek Instrument & Project Reference*) list every position name (and its ALP) on each instrument's default decks. However, an individual instrument may have a different deck, as defined in its instrument settings. Use them to look up valid `where / location / position / source / target` values before authoring.

Worked Example: Simple i3 Fixed-8 Method

A complete, copy-ready Biomek method JSON that transfers liquid from one plate to another on an i3 instrument with a Fixed-8 pod. Every position name, labware class, and tip type used here is valid for the i3 default deck and project.

What each step does

1. **Start** -- initializes the method. The `let`, `weak`, and `prompt` dictionaries are empty (no variables), and the required `bitmap` icon reference is included.
2. **Instrument Setup** -- selects the `"i3"` deck layout and places three items: a BC230 tip box at P7, a BCFlat96 source plate at P3 pre-filled with 200 uL of Water in every well (`volumeType`: `"Known"`, `evalAmounts` / `evalLiquids`), and an empty BCFlat96 destination plate at P9 with defined-but-zero volumes (`volumeType`: `"Known"`, `evalAmounts` all 0.0, `evalLiquids` all `"Water"`). The destination is `"Known"` (not `"Unknown"`) because the default `"F8 Medium"` technique moves relative to the liquid level, which Fixed-8 cannot sense at run time -- see Pipetting Techniques and Templates "Liquid types". All other positions are declared empty, and `podSetup` confirms no tips are pre-loaded.
3. **Fixed-8 Load Tips** -- picks up 8 disposable tips from the tip box at P7.
4. **Fixed-8 Aspirate** -- aspirates 100 uL from column 1 of the source plate at P3 (wells A1 through H1, i.e. `firstWell`: `1`). Uses the `"F8 Medium"` technique.
5. **Fixed-8 Dispense** -- dispenses 100 uL into column 1 of the destination plate at P9. Uses the same technique.
6. **Fixed-8 Unload Tips** -- returns the tips to the box they came from.
7. **Finish** -- parks the pod, clears the deck, and ends the method.

Complete method JSON

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "author": "",
  "description": "Simple column-1 plate-to-plate transfer on i3 Fixed-8",
  "steps": [
    {
      "stepType": "Start",
      "parameters": {
        "bitmap": "OStepUI.ocx,START",
        "let": {},
        "weak": {},
        "prompt": {}
      }
    },
    {
      "stepType": "Instrument Setup",
      "parameters": {
        "barcodeInput?": false,
        "deckItems": {
          "p1": [],
          "p2": [],
          "p3": [
            {
              "_biomekType": "labware",
              "class": "LabwareClasses\\BCFlat96",
              "volumeType": "Known",
              "properties": {
                "name": "SourcePlate"
              }
            }
          ],
          "evalAmounts": {
            "_biomekType": "comArray",
            "arraySubtype": "variant",
            "values": [
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,
              200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0
            ]
          }
        }
      }
    }
  ]
}
```

```

    },
    "evalLiquids": {
      "_biomekType": "comArray",
      "arraySubtype": "variant",
      "values": [
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water"
      ]
    }
  },
  "p4": [],
  "p5": [],
  "p6": [],
  "p7": [
    {
      "_biomekType": "labware",
      "class": "LabwareClasses\\BC230",
      "volumeType": "Unknown",
      "properties": {},
      "tipType": "TipClasses\\T230"
    }
  ],
  "p8": [],
  "p9": [
    {
      "_biomekType": "labware",
      "class": "LabwareClasses\\BCFlat96",
      "volumeType": "Known",
      "properties": {
        "name": "DestPlate"
      },
      "evalAmounts": {
        "_biomekType": "comArray",
        "arraySubtype": "variant",
        "values": [
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
        ]
      }
    }
  ],
  "evalLiquids": {
    "_biomekType": "comArray",
    "arraySubtype": "variant",
    "values": [
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
      "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water"
    ]
  }
},
"p10": [],
"p11": [],
"p12": [],
"s1": [],
"s2": [],
"s3": []
},
"layout": "i3",
"pause?": true,
"podSetup": {
  "leftHasTips": false,
  "leftTipType": "",
  "rightHasTips": false,
  "rightTipType": ""
},
"verifyPodSetup?": true
}

```

```

    },
    {
      "stepType": "Fixed-8 Load Tips",
      "parameters": {
        "tips": "P7",
        "mode": "NumberOfTips",
        "numberOfTips": "8"
      }
    },
    {
      "stepType": "Fixed-8 Aspirate",
      "parameters": {
        "pod": "Pod1",
        "where": "SourcePlate",
        "what": "BCFlat96",
        "operation": "Aspirate",
        "amount": "100",
        "firstWell": 1,
        "useWellExpression": false,
        "liquidType": "Well Contents",
        "autoSelectPrototype": false,
        "prototype": "F8 Medium"
      }
    },
    {
      "stepType": "Fixed-8 Dispense",
      "parameters": {
        "pod": "Pod1",
        "where": "DestPlate",
        "what": "BCFlat96",
        "operation": "Dispense",
        "amount": "100",
        "firstWell": 1,
        "useWellExpression": false,
        "liquidType": "Tip Contents",
        "autoSelectPrototype": false,
        "prototype": "F8 Medium"
      }
    },
    {
      "stepType": "Fixed-8 Unload Tips",
      "parameters": {
        "tipDestination": "<where they came from>"
      }
    },
    {
      "stepType": "Finish",
      "parameters": {
        "bitmap": "OStepUI.ocx,FINISH"
      }
    }
  ]
}

```

Common adjustments

- Swap to a different instrument.** Replace `"layout": "i3"` with the target deck (e.g. `"Span8"` for an i5 Span-8, `"Multichannel"` for an i5 MC), swap all Fixed-8 step types for the matching family (`Span-8 Load Tips` / `Span-8 Aspirate` / `Span-8 Dispense` / `Span-8 Unload Tips` , or `Multichannel Load Tips` / `Multichannel Aspirate` / `Multichannel Dispense` / `Multichannel Unload Tips`), and replace the technique name with one built for the target pod family (`"S8 1000 Medium"` for Span-8, `"MC P60"` for Multichannel). Move the tip box to a valid tip-load position for the target deck (e.g. a `TL*` position on the MC deck).
- Change volumes.** Edit `"amount"` in both the Aspirate and Dispense steps. If the source plate's tracked volumes change, update the `evalAmounts` array to match.
- Transfer multiple columns.** Wrap the Aspirate and Dispense steps inside a Loop with `"variable": "Col"` , `"start": "1"` , `"end": "12"` , `"increment": "1"` and set `useWellExpression: true` with `firstWellExpression: "=Col"` on each pipetting step so the pod sweeps columns 1 through 12. To get fresh tips per column, move the Fixed-8 Load Tips and Fixed-8 Unload Tips steps *inside* the loop.
- Use a Transfer step instead.** A single Transfer step can replace the Load Tips + Aspirate + Dispense + Unload Tips sequence. Transfer handles tip loading, aspiration, dispensing, and tip disposal in one step -- see the Transfer step reference for the full parameter set.

Related

Each step above is documented under [step-documentation/](#). For the Instrument Setup labware/position model, see [Labware and Position Resolution](#); for technique naming conventions ([F8 Medium](#) , etc.), see [Pipetting Techniques and Templates](#).

[↑ Table of contents](#)

Worked Example: Full i5 Span-8 Method

A whole method, start to finish, in the order it is built: a plate-absorbance QC run on an i5 Span-8. It is longer than the i3 Fixed-8 worked example and is here for a different reason — that page shows the smallest thing that works, this one shows what a real method looks like once a deck, a control read, a column sweep, and a device that has no step of its own are all in the same file.

Everything below is one method. The full JSON is at the end of the page in a single block; the sections before it quote pieces of that same block and explain why each key holds the value it does.

1. What the method does

Quantify nucleic acid concentration across a 96-well sample plate by absorbance, then normalize every sample to a common target concentration in a fresh output plate.

- 1. **Blank.** 30 µL of blank buffer goes from a reservoir into column 1 of a dedicated reader plate. The reader measures it and records the baseline optical density that is subtracted from every later reading.
- 2. **Sample load.** 25 µL from each of the 96 sample wells goes into the matching well of a second reader plate, one plate column per pass, fresh tips each pass. The reader measures concentration and the A260/A280 and A260/A230 purity ratios.
- 3. **Normalize.** 30 µL of each sample plus 30 µL of diluent go into the output plate, again one column per pass, with a tip change between the sample leg and the diluent leg.

The absorbance reader is a third-party device. A real method drives it with that device's own add-on step; this example has no such step available, so Section 5 shows the placeholder pattern used in its place — see the caveat there.

Two choices below drive several others:

For this baseline read, each optically-read well should hold a single known liquid so the measurement isn't confounded. The blank and the samples therefore go into two separate plates. A 96-well plate has no spare well once 96 samples occupy it, so there is nowhere on the sample plate to put a blank without overwriting a well that will be read.

The output plate is the one place two liquids are deliberately combined — sample plus diluent. It is never read, so that is a mix, not a contamination.

2. Instrument and deck

Target: **i5 Span-8**, pod `Pod1`, disposable LLS-capable tips (`BC230_LLS` boxes holding `T230_LLS` tips). Deck `Span8`.

Position	Labware class	Instance name	volumeType	Seeded contents
P1, P2, P16, P17	<code>BC230_LLS</code>	—	<code>Unknown</code>	tip boxes, 96 tips each
P3	<code>BCFullReservoir</code>	<code>BlankReservoir</code>	<code>Known</code>	1 section, 50 000 µL Water
P4	<code>BCFullReservoir</code>	<code>DiluentReservoir</code>	<code>Known</code>	1 section, 50 000 µL Water
P6	<code>BCFlat96</code>	<code>SamplePlate</code>	<code>Known</code>	96 wells, 200 µL Water each
P7	<code>BCFlat96</code>	<code>CuvettePlate</code>	<code>Known</code>	96 wells, 0.0 µL
P8	<code>BCFlat96</code>	<code>BlankCuvettePlate</code>	<code>Known</code>	96 wells, 0.0 µL
P11	<code>BCFlat96</code>	<code>DestPlate</code>	<code>Known</code>	96 wells, 0.0 µL
all others	—	—	—	declared empty (<code>[]</code>)

Four tip boxes are on the deck because the method runs 37 tip-load cycles of 8 probes — 296 tips — and one box holds 96. Run out of tips and the load fails at run time with a "unable to find and retrieve tips" error, so count them before you author.

`layout`: "Span8" names a **deck defined on that instrument**, not a pod family, and is the one value on this page not to copy — deck names are per-instrument, and "Span8" may be absent on yours. Read the name from your own settings export, or omit `layout` and take the instrument's default deck.

3. The envelope and Start

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "author": "",
  "description": "Absorbance concentration check with normalization on i5 Span-8 (disposable tips)",
  "steps": [ ... ]
}
```

`format` and `formatVersion` are matched exactly on import — author the version shown above (the method JSON format version, not a software release); import rejects an older value, and a longer form such as "1.0.0" is rejected as newer. `author` and `description` are free text and may be empty. `steps` is the method tree — a flat array here, except where a Loop carries its children in `subSteps`.

```
{
  "stepType": "Start",
  "parameters": {
    "bitmap": "OStepUI.ocx,START",
    "let": {},
    "weak": {},
    "prompt": {}
  }
}
```

`let`, `weak`, and `prompt` are the three variable dictionaries a method can declare at Start. All three are empty here: the only variable this method uses is the Loop counter `co1`, and a Loop exposes its counter to its own children — it does not have to be declared at Start. Emit all three keys even when their objects are empty. See The Variable Environment.

4. Instrument Setup: declaring the deck

Instrument Setup is where this method places its labware, and every pipetting step afterwards references labware placed here — a pipetting step never places a plate, so set the deck up first. (A later Instrument Setup step can place more labware, and some add-on steps place labware too, but keep to a single Instrument Setup where you can.)

A tip box:

```
"p1": [
  {
    "_biomekType": "labware",
    "class": "LabwareClasses\\BC230_LLS",
    "volumeType": "Unknown",
    "properties": {},
    "tipType": "TipClasses\\T230_LLS"
  }
],
```

- The `deckItems` **key** is the position name camelCased — `p1`, not `P1`. Pipetting steps later use the display form ("P8"). A key that matches no position on the selected deck is silently ignored, and the labware is simply never placed, so a typo here surfaces much later as a "position does not exist" or class-mismatch failure.
- `class` is **path-prefixed** here (`LabwareClasses\\BC230_LLS`). Every pipetting step below uses the **short** name (`"BCF1at96"` , `"BCFullReservoir"`). Swapping the two forms is a hard enqueue failure in both directions.
- A tip box holds no liquid, so `volumeType` is `Unknown` and there are no `evalAmounts`.
- `tipType` names the tip class the box dispenses. The box class and tip class pair by suffix: `BC230_LLS` → `T230_LLS`. Name a tip class the box does not actually hold and tip loading fails.

A reservoir:

```
"p3": [
  {
    "_biomekType": "labware",
    "class": "LabwareClasses\\BCFullReservoir",
    "volumeType": "Known",
    "properties": {
      "name": "BlankReservoir",
      "liquidtype": "Water"
    },
    "evalAmounts": {
      "_biomekType": "comArray",
      "arraySubtype": "variant",
      "values": [50000.0]
    },
    "evalLiquids": {
      "_biomekType": "comArray",
      "arraySubtype": "variant",
      "values": ["Water"]
    }
  }
],
```

- `evalAmounts` and `evalLiquids` must be **exactly as long as the labware's well count** — or, for a reservoir, its **section** count. `BCFullReservoir` is a single 200 000 µL section, so both arrays hold one element. Writing 96 entries here because other labware on the deck have 96 wells is wrong and fails — a reservoir's section count is not a plate's well count.
- `properties.name` gives the labware an instance name. Downstream position fields resolve a deck position, a labware instance name, or a labware class name interchangeably, which is why the aspirate steps below can say `"where": "BlankReservoir"` instead of `"p3"`. Names survive a deck rearrangement; positions do not.

A plate seeded with sample, and an empty destination plate (abridged — the real arrays hold 96 entries each):

```
"p6": [
  {
    "_biomekType": "labware",
    "class": "LabwareClasses\\BCFlat96",
    "volumeType": "Known",
    "properties": { "name": "SamplePlate" },
    "evalAmounts": { "_biomekType": "comArray", "arraySubtype": "variant",
      "values": [200.0, 200.0, "... 96 entries ..."] },
    "evalLiquids": { "_biomekType": "comArray", "arraySubtype": "variant",
      "values": ["Water", "Water", "... 96 entries ..."] }
  }
],
```

The three destination plates (P7, P8, P11) are declared the **same way**, with `Known` and 96 amounts of `0.0`. That looks redundant for an empty plate and is the single most common reason a first method fails: the default techniques position the tip relative to the liquid surface, and a destination declared `Unknown` has no locatable surface before anything has been dispensed into it. The first dispense then fails with *"Cannot move relative to liquid level if liquid amounts are not defined for specified labware."* Declare empty destinations `Known` (or `Nominal`) with an all-zero `evalAmounts` and a full-length `evalLiquids`.

Positions the method does not use are still listed, as `[]`, which clears them.

The step-level keys:

```
"layout": "Span8",
"pause?": true,
"podSetup": {
  "leftHasTips": false,
  "leftTipType": "",
  "rightHasTips": false,
  "rightTipType": ""
},
"verifyPodSetup?": true
```

`pause?: true` shows the setup-confirmation dialog so the operator can check the deck against what the method declared.

`verifyPodSetup?` plus `podSetup` assert that neither pod side starts with tips on it — this sets the enqueue-time model, it does not physically move tips. If the pod really does start loaded and the method says otherwise, the first Load Tips step discards whatever is there.

5. The blank leg

Load tips, aspirate 30 μ L of blank from the reservoir, dispense it into column 1 of `BlankCuvettePlate`, hand the plate to the reader, unload tips.

```
{
  "stepType": "Span-8 Load Tips",
  "parameters": {
    "pod": "Pod1",
    "tips": "BC230_LLS",
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "useExpression": false
  }
}
```

`tips` is a multi-purpose locator: a tip **class** (as here), a box **instance name**, or a **deck position**. Because no position is named, the step finds any reachable box of that class — which is why four interchangeable boxes on P1/P2/P16/P17 work with no further bookkeeping. `useExpression: false` makes `useProbes` authoritative; set it `true` and probe selection comes from `mandrelExpression` instead and `useProbes` is ignored.

```
{
  "stepType": "Span-8 Aspirate",
  "parameters": {
    "amount": "30",
    "aspirate": true,
    "autoSelectPrototype": true,
    "prototype": "",
    "firstWell": 1,
    "firstWellExpression": "",
    "useWellExpression": false,
    "spacing": "",
    "height": 2.0,
    "heightFrom": 1,
    "overrideHeight": true,
    "customHeight": false,
    "liquidtype": "Well Contents",
    "operation": "Aspirate",
    "pod": "Pod1",
    "what": "BCFullReservoir",
    "where": "BlankReservoir",
    "useProbes": { "_biomekType": "comArray", "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true] },
    "discardExcess": false,
    "emptyTips": false,
    "mandrelExpression": "",
    "refreshTips": false,
    "tipLabwareClass": "",
    "useExpression": false
  }
}
```

- `where` names the reservoir by its instance name; `what` is the **short** class name. Both must agree with what Instrument Setup placed at that position, or enqueue fails with a class mismatch.
- `spacing: ""` — a probe-to-probe stride is meaningless on a one-section reservoir. Empty makes the step substitute the labware's own minimum interval. All eight probes draw from the single section.
- `overrideHeight: true` with `height: 2.0` and `heightFrom: 1` puts the tips **2 mm off the section bottom**. `heightFrom` is `1` = bottom (`0` = liquid surface, `2` = well top). A fixed bottom reference is deliberate here: a deep trough is a poor target for a liquid-relative move. On the plates further down, `overrideHeight` is `false` and the technique's own height governs — where `overrideHeight` is `false`, the `height` and `heightFrom` values sitting next to it are inert.
- `liquidtype: "Well Contents"` takes the liquid type from what the labware is tracked as holding. "`Tip Contents`" is **rejected** on an aspirate.
- `autoSelectPrototype: true` lets the engine choose a technique from liquid type, labware, tip type and volume, which is why `prototype` is empty. Set `autoSelectPrototype: false` and `prototype` must name a technique that exists in the project.
- `aspirate: true` is what distinguishes this step from Span-8 Dispense — they share one implementation.

The matching dispense into the blank plate:

```
{
  "stepType": "Span-8 Dispense",
  "parameters": {
    "amount": "30",
    "aspirate": false,
    "operation": "Dispense",
    "liquidtype": "Tip Contents",
    "firstWell": 1,
    "useWellExpression": false,
    "spacing": "1",
    "overrideHeight": false,
    "height": 0.0,
    "heightFrom": 1,
    "what": "BCFlat96",
    "where": "BlankCuvettePlate",
    "pod": "Pod1",
    "...": "remaining keys as above"
  }
}
```

spacing: "1" on a 96-well plate: consecutive probes step one well apart. Combined with firstWell: 1 and useWellExpression: false, probes 1–8 land on A1...H1 — column 1. Well numbering is 1-based and **row-major**, so A1 = 1, A12 = 12, B1 = 13; successive probes step down a column by +12 wells each. liquidtype: "Tip Contents" dispenses what was picked up.

Placeholder for a device you have no step for

To actually operate a device, use its add-on step. A reader, thermocycler, shaker, magnet, Peltier, and so on are each driven by that device's own **add-on step**, authored like any other step — consult the device's documentation for its stepType and parameters (see add-on-steps/ (in *Biomek JSON Add-on Steps*)). Do not use the pattern below to control a device.

The Comment + Pause pattern here performs **no** device action. It is only a placeholder for a device you have no authorable step for: it records the intended operation and reserves the plate's position while an operator performs the read by hand. The absorbance reader in this example has no step available, so the method marks the read with a Comment that names the operation, followed by a Pause that reserves the plate's position:

```
{
  "stepType": "Pause",
  "parameters": {
    "mode": "TimedResource",
    "message": "Paused",
    "time": "60",
    "location": "p8"
  }
}
```

mode: "TimedResource" pauses **one resource** — here deck position P8 — for time seconds, leaving the rest of the system free to work in parallel. message is not read in this mode, and location is not read in "PromptedGlobal" mode, but **all four keys are emitted anyway**: a JSON import replaces the step's parameter dictionary wholesale, so a key you leave out is simply absent, and omitting one the chosen mode does read fails at run time. mode is compared literally and is never expression-evaluated — "=someVar" always fails.

Then the tips come off:

```
{
  "stepType": "Span-8 Unload Tips",
  "parameters": {
    "pod": "Pod1",
    "useProbes": { "_biomekType": "comArray", "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true] },
    "useExpression": false
  }
}
```

There is no destination key, and that is not an omission — a Span-8 pod has no return-to-box path for disposable tips. The pod resolves the destination itself from the box's routing keys or, failing that, the nearest reachable trash.

6. The sample sweep: a Loop over plate columns

Twelve passes, four steps each, fresh tips every pass:

```
{
  "stepType": "Loop",
  "parameters": {
    "variable": "Col",
    "start": "1",
    "end": "12",
    "increment": "1"
  },
  "subSteps": [
    { "stepType": "Span-8 Load Tips", "parameters": { "...": "..." } },
    { "stepType": "Span-8 Aspirate", "parameters": { "...": "..." } },
    { "stepType": "Span-8 Dispense", "parameters": { "...": "..." } },
    { "stepType": "Span-8 Unload Tips", "parameters": { "...": "..." } },
    { "stepType": "End", "parameters": {} }
  ]
}
```

- `start`, `end` and `increment` are **strings**, and all three are required. They are read with no fallback, so an omitted key imports fine and then fails at run time; the editor's own default of `"1"` does not survive a JSON import.
- The last child **must** be `{ "stepType": "End", "parameters": {} }`. Loop is a free-form container and this is its terminator. Do not confuse the `end` parameter (the counter bound) with the `"End"` step (the structural terminator) — they are unrelated.
- Load Tips and Unload Tips sit **inside** the loop. Move them outside and one set of tips visits all 96 sample wells, which is exactly the carryover this assay cannot tolerate.

The loop variable reaches the pipetting steps through the well expression:

```
{
  "stepType": "Span-8 Aspirate",
  "parameters": {
    "amount": "25",
    "useWellExpression": true,
    "firstWellExpression": "=Col",
    "firstWell": 1,
    "spacing": "1",
    "what": "BCFlat96",
    "where": "SamplePlate",
    "liquidtype": "Well Contents",
    "overrideHeight": false,
    "...": "remaining keys as in the blank aspirate"
  }
}
```

`useWellExpression: true` makes `firstWellExpression` govern and `firstWell` inert. The expression is the loop variable **on its own** — `"=Col"`. Because well numbering is row-major, the top well of column `c` is simply well `c`, so `Col` is already the correct first well. The tempting `"=1+(Col-1)*8"` assumes column-major numbering: it lands the first probe mid-plate, addresses the wrong wells, and fails outright as soon as fewer than eight rows remain below the starting well.

Note also that `useWellExpression` (which well) and `useExpression` (which probes) are independent flags with confusingly similar names. This method sets the first `true` and the second `false` throughout.

7. Normalization: two legs per column

The second loop runs the same 1...12 sweep but does two transfers per pass, with a tip change between them:

1. Load tips → aspirate 30 µL of sample from `SamplePlate` column `Col` → dispense into `DestPlate` column `Col` → unload.
2. Load tips → aspirate 30 µL of diluent from `DiluentReservoir` → dispense into `DestPlate` column `Col` → unload.

The tip change between the legs is the point: reusing the sample tips to draw diluent would put sample into a shared reservoir that every remaining column still has to draw from.

The two aspirates are addressed differently, and the difference is instructive. The sample aspirate sweeps columns (`useWellExpression: true`, `firstWellExpression: "=Col"`, `spacing: "1"`). The diluent aspirate does **not** — `useWellExpression: false`, `firstWell: 1`,

spacing: "", plus the same bottom-referenced 2 mm height as the blank reservoir — because a one-section reservoir has no columns to sweep. Its matching dispense *does* use "=Col", because the destination is a plate. Source and destination addressing are independent; do not copy the well expression across a step boundary just because the two steps are adjacent.

8. Finish

```
{
  "stepType": "Finish",
  "parameters": {
    "bitmap": "OStepUI.ocx,FINISH"
  }
}
```

The minimum. Finish also accepts `clearDeck`, `clearDevices` and `cleanupPods` if you want the run to tear down the deck state.

9. The volume ledger

Once labware is placed `Known` or `Nominal`, the engine keeps a running per-well volume across every pipetting step in the method. Two things then fail at enqueue rather than on the bench: a dispense that would push a well past its class capacity, and an aspirate that would draw a well below zero. Both errors name the well and the numbers, which makes them easy to fix but only if the seed you declared in Instrument Setup actually matches what the method moves.

Labware	Seed	Draws	Adds	Final	Capacity
SamplePlate, each well	200	-25 (read) -30 (normalize)	—	145	362.76
CuvettePlate, each well	0	—	+25	25	362.76
BlankCuvettePlate, column 1	0	—	+30	30	362.76
DestPlate, each well	0	—	+30 sample, +30 diluent	60	362.76
BlankReservoir section	50 000	-240	—	49 760	200 000
DiluentReservoir section	50 000	-2 880	—	47 120	200 000

Every volume this method moves is $\geq 25\ \mu\text{L}$, which keeps it inside the working range of the 1000 μL Span-8 syringe techniques and lets `autoSelectPrototype: true` find a match. Drop a volume below a technique's minimum and auto-selection has nothing to pick.

Worth checking alongside the arithmetic: **what each well ends up holding**, not just how much. `CuvettePlate` receives only from `SamplePlate` and `BlankCuvettePlate` only from `BlankReservoir` — one liquid each, because both are read optically. `DestPlate` receives from two sources, which is the intended dilution. A volume ledger that balances can still describe a plate that is chemically wrong.

10. Complete method JSON

Copy-ready. This is the whole method; the excerpts above are quotations from it.

```
{
  "format": "Biomek Method",
  "formatVersion": "1.0",
  "author": "",
  "description": "Absorbance concentration check with normalization on i5 Span-8 (disposable tips)",
  "steps": [
    {
      "stepType": "Start",
      "parameters": {
        "bitmap": "OStepUI.ocx,START",
        "let": {},
        "weak": {},
        "prompt": {}
      }
    },
    {
      "stepType": "Instrument Setup",
      "parameters": {
```

```
barcodeInput?": false,  
"deckItems": {  
  "p1": [  
    {  
      "_biomekType": "labware",  
      "class": "LabwareClasses\\BC230_LLS",  
      "volumeType": "Unknown",  
      "properties": {},  
      "tipType": "TipClasses\\T230_LLS"  
    }  
  ],  
  "p2": [  
    {  
      "_biomekType": "labware",  
      "class": "LabwareClasses\\BC230_LLS",  
      "volumeType": "Unknown",  
      "properties": {},  
      "tipType": "TipClasses\\T230_LLS"  
    }  
  ],  
  "p3": [  
    {  
      "_biomekType": "labware",  
      "class": "LabwareClasses\\BCFullReservoir",  
      "volumeType": "Known",  
      "properties": {  
        "name": "BlankReservoir",  
        "liquidtype": "Water"  
      },  
      "evalAmounts": {  
        "_biomekType": "comArray",  
        "arraySubtype": "variant",  
        "values": [50000.0]  
      },  
      "evalLiquids": {  
        "_biomekType": "comArray",  
        "arraySubtype": "variant",  
        "values": ["Water"]  
      }  
    }  
  ],  
  "p4": [  
    {  
      "_biomekType": "labware",  
      "class": "LabwareClasses\\BCFullReservoir",  
      "volumeType": "Known",  
      "properties": {  
        "name": "DiluentReservoir",  
        "liquidtype": "Water"  
      },  
      "evalAmounts": {  
        "_biomekType": "comArray",  
        "arraySubtype": "variant",  
        "values": [50000.0]  
      },  
      "evalLiquids": {  
        "_biomekType": "comArray",  
        "arraySubtype": "variant",  
        "values": ["Water"]  
      }  
    }  
  ],  
  "p5": [],  
  "p6": [  
    {  
      "_biomekType": "labware",  
      "class": "LabwareClasses\\BCFlat96",  
      "volumeType": "Known",  
      "properties": {  
        "name": "SamplePlate"  
      },  
      "evalAmounts": {  
        "_biomekType": "comArray",  
        "arraySubtype": "variant",  
        "values": [  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0,  
          200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0, 200.0  
        ]  
      }  
    }  
  ],  
}
```

[illegible]

```

        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
        "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water"
    ]
}
}
],
"p9": [],
"p10": [],
"p11": [
{
    "_biomekType": "labware",
    "class": "LabwareClasses\\BCFlat96",
    "volumeType": "Known",
    "properties": {
        "name": "DestPlate"
    },
    "evalAmounts": {
        "_biomekType": "comArray",
        "arraySubtype": "variant",
        "values": [
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
        ]
    },
    "evalLiquids": {
        "_biomekType": "comArray",
        "arraySubtype": "variant",
        "values": [
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water",
            "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water", "Water"
        ]
    }
}
],
"p12": [],
"p13": [],
"p14": [],
"p15": [],
"p16": [
{
    "_biomekType": "labware",
    "class": "LabwareClasses\\BC230_LLS",
    "volumeType": "Unknown",
    "properties": {},
    "tipType": "TipClasses\\T230_LLS"
}
],
"p17": [
{
    "_biomekType": "labware",
    "class": "LabwareClasses\\BC230_LLS",
    "volumeType": "Unknown",
    "properties": {},
    "tipType": "TipClasses\\T230_LLS"
}
],
"p18": [],
"p19": [],
"p20": []
},
"layout": "Span8",
"pause?": true,
"podSetup": {
    "leftHasTips": false,
    "leftTipType": "",
    "rightHasTips": false,
    "rightTipType": ""
},
"verifyPodSetup?": true
}
},
{

```

```

    "stepType": "Comment",
    "parameters": {
      "description": "Reader cartridges are represented by two 96-well plates",
      "comment": "The reader's cuvette cartridge is not a catalog labware class, so BCFlat96 stands in for it. Two plates are placed"
    }
  },
  {
    "stepType": "Comment",
    "parameters": {
      "description": "Review measurement parameters before the run",
      "comment": "Operator checkpoint: confirm wavelengths, target concentration, and the active sample set before any liquid moves"
    }
  },
  {
    "stepType": "Span-8 Load Tips",
    "parameters": {
      "pod": "Pod1",
      "tips": "BC230_LLS",
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useExpression": false
    }
  },
  {
    "stepType": "Span-8 Aspirate",
    "parameters": {
      "amount": "30",
      "aspirate": true,
      "autoSelectPrototype": true,
      "customHeight": false,
      "discardExcess": false,
      "emptyTips": false,
      "firstWell": 1,
      "firstWellExpression": "",
      "height": 2.0,
      "heightFrom": 1,
      "liquidtype": "Well Contents",
      "mandrelExpression": "",
      "operation": "Aspirate",
      "overrideHeight": true,
      "pod": "Pod1",
      "prototype": "",
      "refreshTips": false,
      "spacing": "",
      "tipLabwareClass": "",
      "useExpression": false,
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useWellExpression": false,
      "what": "BCFullReservoir",
      "where": "BlankReservoir"
    }
  },
  {
    "stepType": "Span-8 Dispense",
    "parameters": {
      "amount": "30",
      "aspirate": false,
      "autoSelectPrototype": true,
      "customHeight": false,
      "discardExcess": false,
      "emptyTips": false,
      "firstWell": 1,
      "firstWellExpression": "",
      "height": 0.0,
      "heightFrom": 1,
      "liquidtype": "Tip Contents",
      "mandrelExpression": "",
      "operation": "Dispense",
      "overrideHeight": false,
      "pod": "Pod1",
      "prototype": "",
      "spacing": "1",
      "useExpression": false,
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
    }
  }
}

```

```

        "useWellExpression": false,
        "what": "BCFlat96",
        "where": "BlankCuvettePlate"
    }
},
{
    "stepType": "Comment",
    "parameters": {
        "description": "Blank read",
        "comment": "The reader measures BlankCuvettePlate at P8 at 260, 280 and 230 nm and records the baseline optical density that"
    }
},
{
    "stepType": "Pause",
    "parameters": {
        "mode": "TimedResource",
        "message": "Paused",
        "time": "60",
        "location": "P8"
    }
},
{
    "stepType": "Span-8 Unload Tips",
    "parameters": {
        "pod": "Pod1",
        "useProbes": {
            "_biomekType": "comArray",
            "arraySubtype": "boolean",
            "values": [true, true, true, true, true, true, true, true]
        },
        "useExpression": false
    }
},
{
    "stepType": "Comment",
    "parameters": {
        "description": "Sample loading: one column per pass",
        "comment": "Twelve passes sweep the sample plate column by column. Fresh tips on every pass eliminate carryover between columns"
    }
},
{
    "stepType": "Loop",
    "parameters": {
        "variable": "Col",
        "start": "1",
        "end": "12",
        "increment": "1"
    },
    "subSteps": [
        {
            "stepType": "Span-8 Load Tips",
            "parameters": {
                "pod": "Pod1",
                "tips": "BC230_LLS",
                "useProbes": {
                    "_biomekType": "comArray",
                    "arraySubtype": "boolean",
                    "values": [true, true, true, true, true, true, true, true]
                },
                "useExpression": false
            }
        }
    ],
    {
        "stepType": "Span-8 Aspirate",
        "parameters": {
            "amount": "25",
            "aspirate": true,
            "autoSelectPrototype": true,
            "customHeight": false,
            "discardExcess": false,
            "emptyTips": false,
            "firstWell": 1,
            "firstWellExpression": "=Col",
            "height": 0.0,
            "heightFrom": 0,
            "liquidType": "Well Contents",
            "mandrelExpression": "",
            "operation": "Aspirate",
            "overrideHeight": false,
            "pod": "Pod1",
            "prototype": "",
            "refreshTips": false,
            "spacing": "1",
            "tipLabwareClass": "",
            "useExpression": false,

```



```

        "useProbes": {
            "_biomekType": "comArray",
            "arraySubtype": "boolean",
            "values": [true, true, true, true, true, true, true, true]
        },
        "useWellExpression": true,
        "what": "BCFlat96",
        "where": "SamplePlate"
    }
},
{
    "stepType": "Span-8 Dispense",
    "parameters": {
        "amount": "25",
        "aspirate": false,
        "autoSelectPrototype": true,
        "customHeight": false,
        "discardExcess": false,
        "emptyTips": false,
        "firstWell": 1,
        "firstWellExpression": "=Col",
        "height": 0.0,
        "heightFrom": 1,
        "liquidtype": "Tip Contents",
        "mandrelExpression": "",
        "operation": "Dispense",
        "overrideHeight": false,
        "pod": "Pod1",
        "prototype": "",
        "spacing": "1",
        "useExpression": false,
        "useProbes": {
            "_biomekType": "comArray",
            "arraySubtype": "boolean",
            "values": [true, true, true, true, true, true, true, true]
        },
        "useWellExpression": true,
        "what": "BCFlat96",
        "where": "CuvettePlate"
    }
},
{
    "stepType": "Span-8 Unload Tips",
    "parameters": {
        "pod": "Pod1",
        "useProbes": {
            "_biomekType": "comArray",
            "arraySubtype": "boolean",
            "values": [true, true, true, true, true, true, true, true]
        },
        "useExpression": false
    }
},
{
    "stepType": "End",
    "parameters": {}
}
],
{
    "stepType": "Comment",
    "parameters": {
        "description": "Sample read",
        "comment": "The reader measures CuvettePlate at P7 at 260, 280 and 230 nm and records per-well concentration and the A260/A280 ratio"
    }
},
{
    "stepType": "Pause",
    "parameters": {
        "mode": "TimedResource",
        "message": "Paused",
        "time": "120",
        "location": "p7"
    }
},
{
    "stepType": "Comment",
    "parameters": {
        "description": "Normalization volumes",
        "comment": "Per-well sample and diluent volumes are computed from the recorded concentrations and the operator's target concentration"
    }
},
{
    "stepType": "Loop",
    "parameters": {

```

```

"variable": "Col",
"start": "1",
"end": "12",
"increment": "1"
},
"subSteps": [
{
"stepType": "Span-8 Load Tips",
"parameters": {
"pod": "Pod1",
"tips": "BC230_LLS",
"useProbes": {
"_biomekType": "comArray",
"arraySubtype": "boolean",
"values": [true, true, true, true, true, true, true, true]
},
"useExpression": false
}
},
{
"stepType": "Span-8 Aspirate",
"parameters": {
"amount": "30",
"aspirate": true,
"autoSelectPrototype": true,
"customHeight": false,
"discardExcess": false,
"emptyTips": false,
"firstWell": 1,
"firstWellExpression": "=Col",
"height": 0.0,
"heightFrom": 0,
"liquidtype": "Well Contents",
"mandrelExpression": "",
"operation": "Aspirate",
"overrideHeight": false,
"pod": "Pod1",
"prototype": "",
"refreshTips": false,
"spacing": "1",
"tipLabwareClass": "",
"useExpression": false,
"useProbes": {
"_biomekType": "comArray",
"arraySubtype": "boolean",
"values": [true, true, true, true, true, true, true, true]
},
"useWellExpression": true,
"what": "BCFlat96",
"where": "SamplePlate"
}
},
{
"stepType": "Span-8 Dispense",
"parameters": {
"amount": "30",
"aspirate": false,
"autoSelectPrototype": true,
"customHeight": false,
"discardExcess": false,
"emptyTips": false,
"firstWell": 1,
"firstWellExpression": "=Col",
"height": 0.0,
"heightFrom": 1,
"liquidtype": "Tip Contents",
"mandrelExpression": "",
"operation": "Dispense",
"overrideHeight": false,
"pod": "Pod1",
"prototype": "",
"spacing": "1",
"useExpression": false,
"useProbes": {
"_biomekType": "comArray",
"arraySubtype": "boolean",
"values": [true, true, true, true, true, true, true, true]
},
"useWellExpression": true,
"what": "BCFlat96",
"where": "DestPlate"
}
},
{
"stepType": "Span-8 Unload Tips",

```

```

    "parameters": {
      "pod": "Pod1",
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useExpression": false
    }
  },
  {
    "stepType": "Span-8 Load Tips",
    "parameters": {
      "pod": "Pod1",
      "tips": "BC230_LLS",
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useExpression": false
    }
  },
  {
    "stepType": "Span-8 Aspirate",
    "parameters": {
      "amount": "30",
      "aspirate": true,
      "autoSelectPrototype": true,
      "customHeight": false,
      "discardExcess": false,
      "emptyTips": false,
      "firstWell": 1,
      "firstWellExpression": "",
      "height": 2.0,
      "heightFrom": 1,
      "liquidtype": "Well Contents",
      "mandrelExpression": "",
      "operation": "Aspirate",
      "overrideHeight": true,
      "pod": "Pod1",
      "prototype": "",
      "refreshTips": false,
      "spacing": "",
      "tipLabwareClass": "",
      "useExpression": false,
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useWellExpression": false,
      "what": "BCFullReservoir",
      "where": "DiluentReservoir"
    }
  },
  {
    "stepType": "Span-8 Dispense",
    "parameters": {
      "amount": "30",
      "aspirate": false,
      "autoSelectPrototype": true,
      "customHeight": false,
      "discardExcess": false,
      "emptyTips": false,
      "firstWell": 1,
      "firstWellExpression": "=Col",
      "height": 0.0,
      "heightFrom": 1,
      "liquidtype": "Tip Contents",
      "mandrelExpression": "",
      "operation": "Dispense",
      "overrideHeight": false,
      "pod": "Pod1",
      "prototype": "",
      "spacing": "1",
      "useExpression": false,
      "useProbes": {
        "_biomekType": "comArray",
        "arraySubtype": "boolean",
        "values": [true, true, true, true, true, true, true, true]
      },
      "useWellExpression": true,
      "what": "BCFlat96",
      "where": "DestPlate"
    }
  }
}

```

```

    },
    {
      "stepType": "Span-8 Unload Tips",
      "parameters": {
        "pod": "Pod1",
        "useProbes": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [true, true, true, true, true, true, true, true]
        },
        "useExpression": false
      }
    },
    {
      "stepType": "End",
      "parameters": {}
    }
  ]
},
{
  "stepType": "Comment",
  "parameters": {
    "description": "Final data review",
    "comment": "Normalized volumes, concentrations and purity ratios are reviewed and archived before the method ends."
  }
},
{
  "stepType": "Finish",
  "parameters": {
    "bitmap": "OStepUI.ocx,FINISH"
  }
}
]
}

```

11. Adapting it

- **Fewer than 12 columns.** Change the two Loops' `end` . If the count is a run-time input, declare it in Start's `let` and write `"end": "=NumCols"` .
- **Per-well normalization volumes.** The transfers here move a fixed 30 μL of sample and 30 μL of diluent. `amount` is expression-capable on both Span-8 Aspirate and Span-8 Dispense, so a computed volume goes in as `"amount": "=SampleVolume"` once the value is in scope.
- **Collapse a leg into one step.** Load Tips + Aspirate + Dispense + Unload Tips is a Transfer written out longhand. A single Transfer step handles tip loading, the aspirate/dispense pair, and tip disposal, and takes source and destination as entries in its `items` array (`position` / `labwareClass` , not `where` / `what`). The longhand form is used here because the two loops need a tip change at a specific point inside the pass and because the blank leg has a device read wedged between its dispense and its unload.
- **Move to a Multichannel pod.** Swap every `Span-8` * step type for its `Multichannel` * equivalent and rename the position and class keys: Multichannel steps use `location` and `labwareClass` where Span-8 uses `where` and `what` . Reservoir aspiration and bottom-referenced heights carry over; per-probe selection does not, and neither does liquid-level sensing — Multichannel and Fixed-8 pods cannot sense a surface, so on those pods a well pipetted with a **liquid-relative** technique (one that references off the liquid surface) must be declared at least `Nominal` . A from-bottom or fixed-height technique references off the well itself and has no such requirement.

12. Further reading

Background for the choices made above:

- Labware & Position Resolution — position vs. class, the per-family key spellings, name resolution, reachability.
- Well Volume Tracking — the running per-well ledger the table in §9 replays.
- Expressions and The Variable Environment — `=Col` , `let` , and what is in scope where.
- Pipetting Techniques and Templates — `autoSelectPrototype` , technique naming, height override, liquid types and volume types.
- Catalog: Labware Classes (in *Biomek Instrument & Project Reference*) and Catalog: Tip Classes (in *Biomek Instrument & Project Reference*) — well counts, capacities, and the box-to-tip-class pairing used in §2.

Step Documentation Catalog

The Step Reference has one file per step. Each per-step file has a header table (`stepType` , `Category` , `Compatible Hardware`), a Behavior Summary, a Parameters Reference, enumerated values, cross-field rules, and worked JSON examples.

Refer to the Biomek Software Reference Manual (B56358) for more information about steps and their usage.

This index is a **navigation aid** — a single catalog grouped by pod family / area, with one row per step showing its compatible hardware and a one-line summary. For definitive detail on any step, open its own file. For the structural rules (Leaf / container / terminator / anchor), see Method JSON Structure.

Only author the steps listed here. Other step types exist internally (framework- or add-on-only) but are not intended for hand-authoring in the JSON format. Steps contributed by add-on modules and device integrations are documented separately — see `../add-on-steps/` (in *Biomek JSON Add-on Steps*).

Shared conventions (universal base keys, expression syntax, labware/position resolution, probe/mandrel selection, the variable environment) live in Base Keys & Serialization Conventions and in `../concept-guides/`. Every per-step doc assumes you have read those.

Step catalog (by pod family / area)

Grouped by what the step operates on — pod hardware family, deck/setup activity, or method-structure concern. A step that participates in more than one grouping (e.g., a pod pipetting step also relevant to a broader category) is listed under its primary use.

Any-pod pipetting

Consolidated pipetting steps that decide the pod family internally.

#	Step	Compatible Hardware	What it does
11	Transfer	i5, i7, i3	Moves liquid from a single source location to one or more destinations, including tip handling.
12	Combine	i5, i7	Mirror image of Transfer: moves liquid from multiple sources into a single destination, including tip handling.

Fixed-8 pipetting (i3)

#	Step	Compatible Hardware	What it does
36	Fixed-8 Aspirate	i3	Aspirates from a labware position using the Fixed-8 pod; technique must be specified (no auto-select).
37	Fixed-8 Dispense	i3	Delivers liquid to a labware position using the Fixed-8 pod; technique must be specified (no auto-select).
38	Fixed-8 Mix	i3	Mixes liquid in place at a labware position using the Fixed-8 pod, aspirating and dispensing a set volume per cycle.
39	Fixed-8 Load Tips	i3	Picks up disposable tips from a 96-density tip box and attaches them to the Fixed-8 pod.
40	Fixed-8 Unload Tips	i3	Removes tips from the Fixed-8 pod — return to box, discard to trash, or place at a specific location.

Span-8 pipetting

#	Step	Compatible Hardware	What it does
13	Span-8 Transfer From File	Span-8 pod (i5, i7)	Reads a comma-delimited pick-list file and builds one well-to-well aspirate/dispense per row on a Span-8 pod.
14	Span-8 Serial Dilution	Span-8 pod (i5, i7)	Performs a serial dilution across a contiguous run of wells on a Span-8 pod, expanding internally into load/aspirate/dispense/transfer/wash operations.
15	Span-8 Aspirate	Span-8 pod (i5, i7)	Aspirates from source labware using one or more selected probes on a Span-8 pod.
16	Span-8 Dispense	Span-8 pod (i5, i7)	Delivers liquid from Span-8 tips into destination labware.
17	Span-8 Load Tips	Span-8 pod (i5, i7)	Loads disposable tips onto the selected Span-8 probes.
18	Span-8 Unload Tips	Span-8 pod (i5, i7)	Discards disposable tips from the selected Span-8 probes into the trash.
19	Span-8 Wash Tips	Span-8 pod (i5, i7)	Washes Span-8 tips (or fixed mandrels) at a wash station, in either passive or active mode.

Multichannel (fixed head) pipetting

#	Step	Compatible Hardware	What it does
20	Multichannel Aspirate	i5 or i7 with a Multichannel pod	Aspirates from source labware using a fixed multichannel head (96- or 384-channel).
21	Multichannel Dispense	i5 or i7 with a Multichannel pod	Dispenses into destination labware using a fixed multichannel head.
22	Multichannel Load Tips	i5 or i7 with a Multichannel pod	Loads disposable tips onto the multichannel head from a matching tip box on the deck.
23	Multichannel Unload Tips	i5 or i7 with a Multichannel pod	Removes disposable tips from the multichannel head; no-op if none are loaded.
24	Multichannel Mix	i5 or i7 with a Multichannel pod	Aspirates and re-dispenses in place a set number of times to mix well contents on the multichannel head.
25	Multichannel Wash Tips	i5 or i7 with a Multichannel pod	Washes the tips on a multichannel head by mixing them in a wash-station reservoir.

Multichannel Select-Tips family

A container plus a specialized terminator plus a suite of leaf steps that operate on the currently-loaded partial tip pattern.

#	Step	Compatible Hardware	What it does
26	Multichannel Select Tips	Multichannel pod only	Container that groups partial-tip-pattern operations (aspirate, dispense, mix, load, unload, serial dilution, advanced load, advanced unload) on a multichannel pod.
27	Multichannel Select Tips End	Multichannel pod only	Mandatory terminator for the Select Tips container; validates that no tips remain on the pod.
28	Multichannel Select Tips Serial Dilution	Multichannel pod only	Serial-dilution protocol using the multichannel pod's currently loaded tip pattern of equally spaced rows or columns.
29	Multichannel Select Tips Aspirate	Multichannel pod only	Aspirates at the target labware using the multichannel pod's currently loaded tip pattern.
30	Multichannel Select Tips Dispense	Multichannel pod only	Dispenses at the target labware using the multichannel pod's currently loaded tip pattern; supports empty-tips mode.
31	Multichannel Select Tips Mix	Multichannel pod only	Performs repeated aspirate/dispense cycles at one position to mix well contents, using the current tip pattern.

#	Step	Compatible Hardware	What it does
32	Multichannel Select Tips Load	Multichannel pod only	Picks up tips from a tip box in a specific pattern (single, rows, or columns), with an optional backup location.
33	Multichannel Select Tips Unload	Multichannel pod only	Unloads tips from the multichannel pod to a tip box (auto-fit) or discards to trash.
34	Multichannel Select Tips Advanced Load	Multichannel pod only	Loads tips from a specific row/column in a tip box for precise placement control.
35	Multichannel Select Tips Advanced Unload	Multichannel pod only	Unloads tips to a specific row/column in a tip box for precise placement control.

Method boundaries and deck setup

Steps that mark start/finish, configure the deck, or move labware and pods around the deck.

#	Step	Compatible Hardware	What it does
01	Start	All	Mandatory first step in every method; defines method-level variables, weak-bound defaults, and runtime prompts, and performs hardware initialization at run time.
02	Finish	All	Mandatory last step in every method; performs post-run cleanup (tips off, pods parked, deck state cleared) under independent boolean flags.
03	Instrument Setup	All	Selects a named deck layout and populates positions with the labware in <code>deckItems</code> ; optionally verifies the setup with the operator.
04	Move Labware	All	Moves labware from one deck position to another using a pod-mounted gripper, with control over stack depth and grip side.
05	Cleanup	i5, i7	Disposes of tips and optionally returns tip boxes to their configured destinations.
06	Move Pod	All	Moves a pod to a specified deck position without pipetting — used to clear the pod before manual operations or a Pause.
07	Hold Labware	i5, i7	Picks labware up with a pod gripper, holds it while child steps run, then places it back at the original source.

Control flow

Containers and leaf steps that shape the order, condition, or timing of what runs.

#	Step	Compatible Hardware	What it does
08	End	All	Structural marker that closes a container step; the required last child of most container steps.
49	Break	All	Terminates execution of one or more enclosing loops; must be inside a Loop to have effect.
50	Comment	All	Runtime no-op that carries description/comment text shown in the editor and in method reports.
52	Group	All	Free-form container that enqueues its child steps in order — used to organize the tree.
53	If	All	Evaluates a script expression and executes exactly one of two branches (Then / Else Named Container children).
54	Just In Time	All	Free-form container whose child actions are scheduled as late as the schedule allows, rather than as early as possible.
55	Worklist	All	Reads a CSV worklist file and repeats child steps once per row, binding the row's columns as scoped variables.

#	Step	Compatible Hardware	What it does
58	Loop	All	Repeats child steps a computed number of times from start/end/increment; optionally binds an iteration variable.
59	Named Container	All	Free-form container that groups child steps under a caption and adds no control flow.
62	Pause	All	Halts method execution until the operator acknowledges a prompt, or for a timed duration on a specific resource.

Data sets

#	Step	Compatible Hardware	What it does
41	Create Data Set	All (i3, i5, i7)	Reads a text file or SQL Server table and attaches the values as data sets on destination labware, one value per well.
42	Data Set Management	All (i3, i5, i7)	Manipulates existing data sets on a piece of labware — copy, rename, remove, or change flags.
43	Data Set Processing	All (i3, i5, i7)	Applies a transformation expression to a source data set to build a new data set on the same labware.
44	Data Set Reporting	All (i3, i5, i7)	Generates a report of every reportable data set on every piece of labware known to the world.
45	View Data Sets	All (i3, i5, i7)	Design-time inspection for viewing data sets, labware properties, and global variables; runtime no-op.

Device integration

External and framework devices controlled from a method.

#	Step	Compatible Hardware	What it does
09	Device Action	i5, i7	Sends a command to a digital I/O device or generic script device attached to the instrument.
10	INHECO Peltier	i5, i7	Controls INHECO thermal and shaking devices on the deck (temperature incubation, shaking, initialization).
70	Run Program	All	Launches an external program during method execution with a specified command line, working directory, window mode, and reserved resource.
71	SILAS	All (requires configured SILAS modules)	Sends a command to an external SILAS-controlled device module (plate reader, incubator, centrifuge, barcode reader, etc.).

Scope, variables, and scripting

Steps that define patterns/variables, run scripts, or emit log lines.

#	Step	Compatible Hardware	What it does
51	Define Pattern	All	Creates a named well-selection pattern (fixed, file-driven, or operator-prompted) usable by other steps.
56	Let	All	Container that binds scoped variables (strong, weak, or prompted) for its child steps.
67	Scripted Let	All	Runs a script block, then enqueues child steps in a scope containing the variables the script bound.
68	Script	All	Runs a script block at enqueue (during Run / Simulate); a leaf with no child steps.
69	Set Global	All	Defines or updates a global variable available to any step enqueued after it, at any nesting depth.

Procedures and methods

Sub-methods and reusable procedures called from a top-level method.

#	Step	Compatible Hardware	What it does
64	Run Method	All	Runs another method from the current project inline as part of the current method, with optional variable overrides.
65	Define Procedure	All	Declares a named, reusable procedure made up of its own child steps; a later Run Procedure inlines the body.
66	Run Procedure	All	Invokes a procedure previously declared by Define Procedure, applying its own <code>let</code> values over the procedure's defaults.

Labware groups and iteration

Named labware groups and list-driven iteration.

#	Step	Compatible Hardware	What it does
60	Next Item	All	Advances a global variable to the next value in a list; end-of-list behavior is chosen by <code>onDone</code> .
72	Create Labware Group	i5, i7	Tags a set of labware as a named, ordered group so a later Next Labware step can cycle through them.
73	Next Labware	i5, i7	Cycles a named labware group to its next member — returns the current piece, brings the next unused one to the working location.

Fly-By Bar Code Reader

#	Step	Compatible Hardware	What it does
46	Fly-By Read	i5, i7	Uses a gripper-equipped pod to fly a stack of labware past a Fly-By Bar Code Reader, reading each piece.
47	Fly-By Log	i5, i7	Writes accumulated Fly-By Bar Code Reader read results to a CSV log file.

[†] Table of contents

Base Keys & Serialization Conventions

Applies to **every** step. The per-step docs assume this file and do **not** re-document the universal base keys, the casing rules, or the shared value-encoding references. Read this once; each step doc covers only its step-specific keys.

The JSON envelope, step-node structure, `_biomekType` / `comArray`, and tree validation are in **Method JSON Structure**; the shared pipetting/technique keys are in **Pipetting Techniques, Templates, and Auto-Selection**. This file covers the base-step and serializer keys and the key-name casing rules.

0. What "Required" means in the parameters table

Per-step docs mark each key `Yes` / `Conditional` / `No` in the Required column. That column describes what the step needs to **run** (Run or Simulate), not what import checks. Import is lenient: a method can import cleanly and still be missing a required key — the error surfaces only when you run or simulate it.

- **Required = Yes** — you must author this key. If it is absent, Run/Simulate fails with a missing-required-key error.
- **Required = Conditional** — needed only in some configurations, because the step reads it only when another key has a particular value (e.g. Move Labware's `depth` vs `leaveBottomLabware`, or an offset-gripper-only field).
- **Required = No** — safe to omit. When absent, the engine applies the default shown in that key's Default column.

What this means for authoring

Author every **Required = Yes** key explicitly, plus any **Conditional** key your configuration triggers. You never need to add a **Required = No** key just to be safe — omit it and the engine supplies the default. When you author a method from scratch, the software uses only the keys you provide; it does not re-add the editor's other defaults for you, so a missing Required = Yes key is never filled in automatically.

1. Key-name casing & spelling (global rule)

Biomek step dictionaries are **case-insensitive** at runtime. Casing only matters for *matching what real exports look like*, which is what authored methods should emit.

Biomek applies a camelCase rule to each parameter key on export. The conversion lowercases only the **first PascalCase word**: a single leading uppercase letter is lowercased; a multi-letter leading uppercase run has all but its last letter lowercased. Everything after the first word boundary — including internal spaces, underscores, digits, and trailing `?` — is **preserved verbatim**:

Source constant	Serialized JSON key	Note
<code>LiquidType</code>	<code>liquidType</code>	first word <code>L</code> → <code>l</code>
<code>HelpContextID</code>	<code>helpContextID</code>	first word <code>H</code> → <code>h</code> ; second word <code>Context</code> and third word <code>ID</code> unchanged
<code>HTML_Dialog</code>	<code>html_Dialog</code>	first word <code>HTML</code> : all but last lowered → <code>html</code> ; <code>_Dialog</code> unchanged
<code>StepUI</code>	<code>stepUI</code>	first word <code>S</code> → <code>s</code> ; second word <code>UI</code> unchanged
<code>Use Shake</code>	<code>use Shake</code>	internal space preserved ; <code>U</code> → <code>u</code> ; <code>Shake</code> unchanged
<code>Dynamic?</code>	<code>dynamic?</code>	trailing <code>?</code> preserved
<code>WS1</code>	<code>wS1</code>	first word <code>WS</code> : all but last lowered → <code>wS</code> ; digit <code>1</code> unchanged

Author in this serialized form (first-letter-lowercase, spaces/ `?` intact). Because matching is case-insensitive, a PascalCase key still *imports*, but it will not string-match other authored methods or the docs — prefer the serialized spelling everywhere.

Note: matching is fully case-insensitive, so `liquidtype`, `liquidType`, and `LiquidType` all resolve identically. If step examples or exports use an all-lowercase form, it works the same as the camelCase form.

Exception: keys **inside a SILAS message payload** are case-sensitive (see the SILAS step doc and the `_biomekType: "SILASMessage"` note in the JSON-structure reference).

2. Universal base keys (present on every step; not repeated in the per-step docs)

These keys can appear on essentially any step. They are step **metadata / serializer bookkeeping**, populated by the framework from the step's registration or editor state. A method author normally **never sets them**; omit them and the step still imports (the software supplies its own). Preserve them verbatim when round-tripping an existing method.

JSON key	Type	Author sets?	Meaning
<code>caption</code>	string	Only to override the label	User caption override shown in the editor
<code>defaultCaption</code>	string	No (auto)	Built-in/auto-generated caption
<code>helpContextID</code>	integer	No	Help topic id
<code>editorOptionsMask</code>	integer	No	Editor options bitmask (drives structure rules)
<code>bitmap</code>	string	Yes for Start / Finish , otherwise No	Step icon reference (e.g., <code>"0StepUI.ocx,START"</code>). For Start and Finish , JSON import replaces the step's parameters wholesale and discards any default value, so the icon breaks unless the key is authored — see the per-step docs. For every other step, omit and the framework supplies its own.
<code>tooltip</code>	string	No	Tooltip override
<code>orderWeight</code>	integer	No	Palette sort order (design-time)
<code>palette</code>	string	No	Editor palette group (design-time)
<code>stepUI</code>	string (CLSID)	No	Editor UI CLSID override
<code>isPreconfigured</code>	boolean	No	Step came from a preconfigured template
<code>dynamic?</code>	boolean	Usually no — see Exceptions	Error-handling flag. <code>false</code> (the default): the step handles its own enqueue errors. <code>true</code> : enqueue errors propagate to the framework instead. Framework-set for most steps — do not author. Exceptions: (1) Run Program and Run Procedure carry <code>dynamic?</code> explicitly and it must be authored to match the step's own rule (see Run Program and Run Procedure); (2) the Fixed-8 pipetting family (Aspirate/Dispense/Mix) writes <code>dynamic?: true</code> at construction, so round-trip exports of those steps carry <code>dynamic?: true</code> ; (3) SILAS reuses the same key name for a step-functional flag with different semantics — blocks until device action completes and enables RDD/DataSet features (see SILAS).

3. Disabling a step (disabled)

Whether the step is disabled (commented out) in the editor. Set the **step-node** boolean `disabled: true` — see the JSON-structure reference for where it sits on the node. Defaults to `false` and is omitted from export when `false`. When `true`, the step is skipped at run time but kept in the method tree. A `disabled` value other than `true` / `false` fails deserialization. The node-level flag is the entire authoring interface: you never write anything inside `parameters` to disable a step.

Anchor steps cannot be disabled. Start, Finish, and any container terminator (`End`, `Multichannel Select Tips End`, or an add-on container's own terminator) are structural bookends; `disabled: true` on those is rejected at import.

4. Keys that look shared but are genuinely per-step

`let`, `weak`, and `prompt` appear on many steps but are **not** base keys — they are real scope keys with step-specific meaning, documented in the relevant per-step docs. What matters for authoring:

- **Author them on the scope steps that use them** — `Start`, `Let`, `Run Method`, `Define Procedure`, and `Run Procedure` each carry all three (empty `{}` is fine); author them the same way.
- **Do NOT author them on a `Worklist` step** — its per-row scope is built internally, so `let` / `weak` / `prompt` do not belong on a `Worklist` node.
- `let` alone also drives an advanced runtime-override mechanism on `Script` and similar steps — see [Pipetting Techniques, Templates, and Auto-Selection §"The `let` Override Mechanism"](#).

5. Inert / trap keys policy

Many step dictionaries carry keys that are **UI panel state** (`collapsed`, `showTransferDetails`, `splitterPosition`, `PanelState*`, ...) or **vestigial/dead** (`podsToMaxZ`, `wizard`, ...) — persisted but never read at runtime, or read only for editor cosmetics. Per-step docs classify each such key explicitly as **inert (UI-state)** or **vestigial (no runtime effect)** so an author neither relies on them nor treats their presence in exports as meaningful. Do **not** invent behavior for a key Biomek does not read.

Steps may carry other keys that are not defined in the step's documented parameters. **If you encounter a key that isn't documented here**, treat it as unsupported: preserve it when round-tripping an existing method, but do not author it or rely on it.

[†] Table of contents

Start

Property	Value
stepType	"Start"
Category	Boundary
Terminator	N/A
Compatible Hardware	All (no hardware restriction — inherits default)

Behavior Summary

The Start step is the mandatory first step in every Biomek method. It serves two purposes: (1) it defines method-level variables (`let`), weak-bound defaults (`weak`), and runtime prompts (`prompt`) that are available to all subsequent steps in the method; and (2) at runtime, when this Start is confirmed to be the first substep of the method root, it performs hardware initialization — notifying all pipettor devices, reserving the universal resource to ensure safe initialization, resetting piercing-tip state on Span-8 pods, and coercing any out-of-limit pod axes back within their physical bounds.

If the Start step is not the first substep (e.g., in a non-root context), it does nothing at enqueue time.

The Start step also supports SILAS module initialization and method extension enqueueing. Users cannot delete, move, copy, or insert steps above the Start step in the editor.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>let</code>	<code>object</code>	—	Yes	<code>{}</code> (empty dictionary)	Dictionary of method-level variable definitions. Keys are variable names (alphanumeric + underscore, starting with a letter, max 255 chars); values are the variable's default value. Author as a JSON string (e.g., <code>"25"</code>); JSON numbers are accepted at runtime but the editor normalizes them to strings, so string form preserves round-trip fidelity. These variables are accessible from any step in the method via <code>=variableName</code> .
<code>weak</code>	<code>object</code>	—	Yes	<code>{}</code> (empty dictionary)	Dictionary of weak-bound variable flags. A variable listed here is "overridable" — its value in <code>let</code> can be replaced by a same-named variable from a calling method (via Run Method step). Keys are variable names matching entries in <code>let</code> ; values should be <code>null</code> . Presence of the key is the flag.
<code>prompt</code>	<code>object</code>	—	Yes	<code>{}</code> (empty dictionary)	Dictionary of runtime-prompt variable flags. A variable listed here causes a dialog prompt at method start allowing the operator to confirm or change its value. Keys are variable names matching entries in <code>let</code> ; values should be <code>null</code> . Presence of the key is the flag.
<code>silasModules</code>	<code>object</code>	—	No	Not present	SILAS module and consumer initialization data. Keys are device/consumer identifiers; values are either <code>true</code> (for SILAS devices) or serialized SILAS message objects (for SILAS consumers). Treat SILAS consumer payloads as opaque unless specifically editing known SILAS content. This key is the serialized form of an internal extension entry, renamed during export.
<code>extensions</code>	<code>object</code>	—	No	Not present	Container for registered method-extension data, keyed by extension class ID. Each extension receives its sub-dictionary during enqueue. Typically not authored manually.

Key	Type	Units	Required	Default	Description
bitmap	string	—	Yes	"0StepUI.ocx,START"	Step-icon reference. Defaults to "0StepUI.ocx,START", but JSON import replaces the step's parameters wholesale and discards defaults — if omitted from authored JSON the Start icon breaks on import. Author it exactly as shown.

Other universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

The Start step has no parameters with fixed enumerated value sets. The `let` , `weak` , and `prompt` dictionaries accept arbitrary user-defined variable names as sub-keys with arbitrary string/numeric values (for `let`) or `null` flags (for `weak` / `prompt`).

Cross-Field Validation Rules

- Every key in `weak` **must** have a corresponding key in `let` . A weak flag without a defined variable value has no effect and indicates an authoring error.
- Every key in `prompt` **must** have a corresponding key in `let` . A prompt flag without a defined variable value will prompt for an undefined variable.
- The `silasModules` key should only be present if SILAS hardware is configured on the instrument. Including it on a non-SILAS system will not cause a failure but has no effect.
- The `extensions` key should not be hand-authored — it is populated by the system based on registered method extensions. Including unknown extension CLSIDs will be silently ignored at enqueue.

Structural Context

The Start step is a **Boundary** step with the following structural constraints:

- Must be at index 0** of the method root `steps` array. The method is always structured as `[Start, ...body steps..., Finish]` .
- Exactly one Start step** per method. Duplicates are invalid.
- Root-only**: The Start step must never appear inside a container step's `subSteps` .
- Cannot be deleted, moved, copied, or have steps inserted above it.**
- Does not have `subSteps`** — the Start step is not a container. Never include a `subSteps` array on it.

Canonical Examples

Minimal Start (empty parameters — the most common case)

```
{
  "stepType": "Start",
  "parameters": {
    "bitmap": "0StepUI.ocx,START",
    "let": {},
    "weak": {},
    "prompt": {}
  }
}
```

Start with method-level variables and runtime prompts

A serial dilution method where the operator is prompted for volume and plate count at runtime, and the plate count can be overridden by a calling method:

```
{
  "stepType": "Start",
  "parameters": {
    "bitmap": "OStepUI.ocx,START",
    "let": {
      "aspirateVolume": "25",
      "numberOfPlates": "4",
      "tipType": "BC190F",
      "rows": "\"1,2,3,4,5,6,7,8\""
    },
    "weak": {
      "numberOfPlates": null
    },
    "prompt": {
      "aspirateVolume": null,
      "numberOfPlates": null
    }
  }
}
```

Start for a sub-method (weak defaults only, no prompts)

A utility method intended to be called via Run Method, where all variables are overridable by the caller:

```
{
  "stepType": "Start",
  "parameters": {
    "bitmap": "OStepUI.ocx,START",
    "let": {
      "dilutionFactor": "2",
      "mixCycles": "3",
      "sourcePosition": "P1"
    },
    "weak": {
      "dilutionFactor": null,
      "mixCycles": null,
      "sourcePosition": null
    },
    "prompt": {}
  }
}
```

Start with SILAS module initialization

```
{
  "stepType": "Start",
  "parameters": {
    "bitmap": "OStepUI.ocx,START",
    "let": {
      "sampleCount": "96"
    },
    "weak": {},
    "prompt": {
      "sampleCount": null
    },
    "silasModules": {
      "orbitShaker1": true,
      "plateReader1": true
    }
  }
}
```

Common Mistakes

- **Omitting `bitmap`** — Import replaces the step's parameters wholesale and discards the default icon value; if `bitmap` is missing the Start icon breaks on the next open. Set it exactly to `"OStepUI.ocx,START"`.
- **Numeric `let` defaults as JSON numbers instead of strings** — Both are accepted at runtime (values are expression-capable), but the editor stores them as strings; using strings preserves round-trip fidelity.

- **Confusing `let` with `weak`** — `let` holds the actual value; `weak` is a parallel flag dictionary (values are `null`) marking which `let` entries a caller may override.
- **Forgetting escaped quotes on multi-value row/column strings** — When a variable feeds a Select Tips row/column list, the value itself must include the double-quotes: `"\"1,2,3\""`. A bare `"1,2,3"` will not parse.

† Table of contents

Finish

Property	Value
stepType	"Finish"
Category	Boundary
Terminator	N/A
Compatible Hardware	All (no hardware restriction — inherits default)

Behavior Summary

The Finish step is the mandatory last step in every Biomek method. It marks the end of a method and performs post-run cleanup: discarding disposable tips from all pods (and washing fixed tips on Span-8 pods), parking pods and grippers at their safe home positions, clearing the software deck state of all labware, clearing external device (SILAS and pipettor) labware state, and clearing global variables. Each cleanup action is controlled by an independent boolean flag — any combination may be selected.

If the `report` flag is enabled, the Finish step enqueues a Reporting substep that writes labware data-set output to a text file, per-plate HTML/text files, or SQL Server. The Finish step also invokes registered method extensions.

Only the last Finish step in the method root actually executes its cleanup logic; if a Finish step is not the final substep of the method, the step returns immediately at enqueue. Users cannot delete, move, copy, or insert steps below the Finish step in the editor.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>clearDeck</code>	<code>boolean</code>	—	No	<code>true</code>	Clear the software state of all labware from the instrument deck at method completion. Ensures the next method starts with a clean deck. Always author this key.
<code>clearDevices</code>	<code>boolean</code>	—	No	<code>true</code>	Clear the labware state from all SILAS and pipettor external devices. Ensures devices are empty for the next method. Always author this key.
<code>cleanupPods</code>	<code>boolean</code>	—	No	<code>true</code>	Unload disposable tips from all pods. For Span-8 pods, also performs a post-run wash of fixed tips. Always author this key.
<code>podsToMaxZ</code>	<code>boolean</code>	—	No	<code>true</code>	Inactive — not read at runtime. Defaults to <code>true</code> but is not read at enqueue — the pod parking behavior is actually controlled by <code>parkPods</code> . Preserve for round-trip; do not rely on it.
<code>parkPods</code>	<code>boolean</code>	—	No	<code>true</code>	Move all pods and grippers to their park (home) locations after method completion. Should be disabled if the method is used as a sub-method. Engine tolerates omission by falling back to <code>true</code> .
<code>clearGlobals</code>	<code>boolean</code>	—	No	<code>true</code>	Clear all global variables defined in the Start or Set Global steps. Prevents variable carry-over into subsequent methods. Engine tolerates omission.
<code>report</code>	<code>boolean</code>	—	No	<code>false</code>	Enable data-set reporting at method completion. When <code>true</code> , a Reporting substep is enqueued using the reporting

Key	Type	Units	Required	Default	Description
					keys below. Engine tolerates omission (default <code>false</code> means no reporting).
<code>path</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	File path or folder location for report output. Meaning depends on <code>type</code> : file path for "Text File", folder for "Per-Plate HTML Files" / "Per-Plate Text Files". For "SQL Server" type, path holds the server hostname (the Data Source component of the built connection string). Omission throws a "could not find Path" system error only when <code>report=true</code> . When <code>report=false</code> , may be omitted safely.
<code>type</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	Report style. See Enumerated Values section for allowed values. Gated on <code>report=true</code> .
<code>server</code>	<code>string</code>	—	No	<code>""</code>	Display-only. Saved to the step dictionary by the editor and shown in the print view, but has no effect on the actual SQL Server connection. The effective server hostname goes into <code>path</code> . Preserve verbatim for round-trip fidelity if present in an existing method; do not rely on it to configure the connection.
<code>catalog</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	SQL Server database (catalog) name. Gated on <code>report=true</code> .
<code>tableName</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	SQL Server table name for report data. The engine appends a timestamp suffix (<code>mm_dd_yyyy_hh_nn_ss</code>) at run time to avoid collisions across runs; the table actually created is <code><tableName><mm_dd_yyyy_hh_nn_ss></code> . Gated on <code>report=true</code> .
<code>userName</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	Database user name for SQL Server authentication. Gated on <code>report=true</code> .
<code>password</code>	<code>string</code>	—	Conditional (when <code>report=true</code>)	<code>""</code>	Database password for SQL Server authentication. Gated on <code>report=true</code> .
<code>authent</code>	<code>boolean</code>	—	Conditional (when <code>report=true</code>)	<code>false</code>	Authentication mode flag for SQL Server. Gated on <code>report=true</code> .
<code>connectionString</code>	<code>string</code>	—	Conditional (when <code>report=true</code> and <code>type="SQL Server"</code>)	<code>""</code>	OleDb connection string for SQL Server. For manually authored JSON, this field must be provided — the engine uses this string to open the database connection; <code>userName</code> and <code>password</code> are used only when the Biomek editor builds the string interactively. An empty or missing string causes a database-connection validation failure. Example: <code>"Provider=SQLOLEDB.1;User ID=biomek_user;Password=****;Initial Catalog=BiomekData;Data Source=LABSERVER01"</code> .
<code>collapsed</code>	<code>boolean</code>	—	No	<code>true</code>	UI state flag controlling whether the reporting section is collapsed in the editor. Editor-written; not read at runtime. Not meaningful for method authoring.
<code>clearTool</code>	<code>boolean</code>	—	No	Not present	Inactive — not read at runtime. Do not author. Preserve verbatim if present in an existing method for round-trip fidelity.

Key	Type	Units	Required	Default	Description
extensions	object	—	No	Not present	Container for registered method-extension data, keyed by extension class ID. Each extension receives its sub-dictionary during enqueue. Typically not authored manually.
bitmap	string	—	Yes	"0StepUI.ocx,FINISH"	Step-icon reference. Defaults to "0StepUI.ocx,FINISH", but JSON import replaces the step's parameters wholesale and discards defaults — if omitted from authored JSON the Finish icon breaks on import. Author it exactly as shown.

Other universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Casing: step dictionary keys are case-insensitive at runtime (see *00-Base-Keys §1*). Exports mix camelCase and all-lowercase spellings of the cleanup flags (e.g. `cleanupPods` / `cleanupPods` , `clearDeck` / `cleardeck` , `clearDevices` / `cleardevices`); both import identically. Author the camelCase form shown in this table.

Enumerated / Constrained Values

type (Report Style)

Value	Description
"Text File"	Single text file report. <code>path</code> specifies the full file path including file name.
"Per-Plate HTML Files"	One HTML file per plate. <code>path</code> specifies the output folder.
"Per-Plate Text Files"	One text file per plate. <code>path</code> specifies the output folder.
"SQL Server"	Report data is written to a SQL Server database. Requires <code>path</code> (server hostname), <code>catalog</code> , and <code>tableName</code> ; requires <code>connectionString</code> when authoring JSON manually (the engine connects using only this string). <code>userName</code> , <code>password</code> , and <code>authent</code> are used by the editor to build <code>connectionString</code> interactively.

These four are exactly the values offered by the Finish step's report-type combo. A fifth constant, `"Excel Spreadsheet"` , is defined in source but **is not exposed in the combo** — do not author it.

All other parameters use unconstrained boolean or string values.

Cross-Field Validation Rules

- All reporting keys (`path` , `type` , `server` , `catalog` , `tableName` , `userName` , `password` , `authent` , `connectionString`) are **only meaningful when report is true** . When `report` is `false` , they are ignored at runtime.
- When `report` is `true` , `type` **must** be set to one of the four enumerated report styles. An empty or unrecognized `type` fails validation.
- When `type` is "SQL Server" , `path` (server hostname), `catalog` , and `tableName` must be populated; `connectionString` must contain the full OLEDB connection string (the engine connects using this directly). An empty or unreachable connection string fails the connection check. A missing table name fails a distinct check. `userName` and `password` are used by the editor to build `connectionString` interactively but do not drive the connection themselves.
- When `type` is "Text File" , "Per-Plate HTML Files" , or "Per-Plate Text Files" , `path` must point to a valid file/folder. For "Text File" : if the directory component of `path` does not exist, path validation fails; if the directory exists but `path` names an existing directory rather than a file, filename validation fails. For "Per-Plate HTML Files" : a non-existent directory fails path validation. For "Per-Plate Text Files" : a non-existent directory fails path validation. SQL-specific keys are ignored.

5. The five functional cleanup flags (`clearDeck` , `clearDevices` , `cleanupPods` , `parkPods` , `clearGlobals`) are **independent** — any combination may be selected.
6. If the method will be invoked as a sub-method via Run Method, set `parkPods` to `false` to avoid unnecessary pod movement between sub-method and parent method.
7. `cleanupPods=true` **requires a reachable trash ALP for every pod that has tips at Finish time**. If a pod is holding tips and no position with the `Can Discard Tips` characteristic is present (or reachable) for that pod, enqueue fails. Fix by placing/keeping a trash ALP the pod can reach, or by clearing the pod's tips before Finish.
8. `parkPods=true` **invokes the path planner and can fail when tall labware blocks the park path**. If a stack of labware makes the pod-park destination reachable only through a colliding configuration, enqueue fails with a path-planner collision error. Fix by moving the tall labware, choosing a different park configuration, or clearing tips so the gripper can retract before parking.

Structural Context

The Finish step is a **boundary/anchor** step with the following structural constraints:

- **Must be the last element** of the method root `steps` array. The method is always structured as `[Start, ...body steps..., Finish]`.
- **Exactly one Finish step** per method. Duplicates are invalid.
- **Root-only**: The Finish step must never appear inside a container step's `subSteps`.
- **Cannot be deleted, moved, copied, or have steps inserted below it**.
- **Does not have** `subSteps` — the Finish step is not a container. Never include a `subSteps` array on it.
- **Auto-seeded**: When a new method is created, the method's substep-population routine automatically inserts a Start step at index 0 and a Finish step at index 1 with `bitmap: "OStepUI.ocx,FINISH"`.
- **Only executes if truly last**: At runtime, the Finish step checks whether this instance is the last substep of the method. If it is not the last substep, it returns immediately without performing any cleanup.

Canonical Examples

Minimal Finish (all cleanup defaults — the most common case)

All cleanup actions are enabled by default. Author `bitmap` explicitly (import replaces the constructor-seeded icon), and always author `clearDeck` , `clearDevices` , and `cleanupPods` .

```
{
  "stepType": "Finish",
  "parameters": {
    "bitmap": "OStepUI.ocx,FINISH",
    "clearDeck": true,
    "clearDevices": true,
    "cleanupPods": true
  }
}
```

Finish with selective cleanup (sub-method usage)

A Finish step configured for a method that will be called as a sub-method. Pod parking and global clearing are disabled to preserve state for the calling method:

```
{
  "stepType": "Finish",
  "parameters": {
    "bitmap": "OStepUI.ocx,FINISH",
    "clearDeck": true,
    "clearDevices": true,
    "cleanupPods": true,
    "parkPods": false,
    "clearGlobals": false
  }
}
```

```
}  
}
```

Finish with text-file reporting

All cleanup enabled, plus a text-file report on labware data:

```
{  
  "stepType": "Finish",  
  "parameters": {  
    "bitmap": "OStepUI.ocx,FINISH",  
    "clearDeck": true,  
    "clearDevices": true,  
    "cleanupPods": true,  
    "parkPods": true,  
    "clearGlobals": true,  
    "report": true,  
    "type": "Text File",  
    "path": "C:\\Reports\\MethodReport.txt"  
  }  
}
```

Finish with SQL Server reporting

Reporting to a SQL Server database with SQL authentication. `path` holds the server hostname (used by the editor to build `connectionString`); `connectionString` must be provided explicitly in manually authored JSON — the engine connects using only this field. The actual table created will be `<tableName><mm_dd_yyyy_hh_nn_ss>` (timestamp appended at run time):

```
{  
  "stepType": "Finish",  
  "parameters": {  
    "bitmap": "OStepUI.ocx,FINISH",  
    "clearDeck": true,  
    "clearDevices": true,  
    "cleanupPods": true,  
    "parkPods": true,  
    "clearGlobals": true,  
    "report": true,  
    "type": "SQL Server",  
    "path": "LABSERVER01",  
    "server": "LABSERVER01",  
    "catalog": "BiomekData",  
    "tableName": "RunResults",  
    "userName": "biomek_user",  
    "password": "****",  
    "authnt": false,  
    "connectionString": "Provider=SQLOLEDB.1;User ID=biomek_user;Password=****;Initial Catalog=BiomekData;Data Source=LABSERVER01"  
  }  
}
```

Common Mistakes

- **Omitting `bitmap`** — Import replaces the step's parameters wholesale and discards the default icon value; without an explicit `"OStepUI.ocx,FINISH"` the Finish icon breaks on the next open.
- **Leaving `parkPods=true` in a sub-method** — When invoked via Run Method, parking between sub-method and parent wastes time and can perturb positioning.
- **Setting `report=true` without a `type`** — Fails validation. An empty `type` is not treated as "no report".
- **Confusing `clearDeck` with `clearDevices`** — `clearDeck` clears the software deck's labware state; `clearDevices` clears external device (SILAS, pipettor) labware state. They are independent, not redundant.
- **Trusting `podsToMaxZ`** — It's inactive: defaults to `true` but is not read at runtime. Preserve verbatim for round-trip fidelity, but do not expect it to control parking (that's `parkPods`).
- **Omitting `connectionString` from manually authored SQL Server JSON** — The engine connects using only `connectionString`; individual credential fields (`userName` , `password`) are used only when the Biomek editor builds the string interactively. Without an explicit `connectionString` , the step fails the database-connection check at validation.

- **Using `server` instead of `path` as the server hostname** — `server` is display-only and has no effect on the database connection. The server hostname must go in `path` ; the editor reads `path` as the `Data Source` when building `connectionString` .

[↑ Table of contents](#)

Instrument Setup

Property	Value
stepType	"Instrument Setup"
Category	Leaf
Terminator	N/A
Compatible Hardware	Not restricted (default applies to all: i3, i5, i7)

Behavior Summary

The Instrument Setup step configures the method's runtime representation of the instrument deck and the pods' tip status.

Multiple Instrument Setup steps may appear in a single method to change the deck configuration mid-run.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>layout</code>	string	—	No (valid to omit)	instrument's default deck	Which of the instrument's decks this step configures. Recommended to set explicitly rather than omit: omitting falls back to the instrument's default deck, so a method later run on an instrument whose default deck differs from the one it was written for can behave unexpectedly — and silently, because omission never fails at enqueue. Setting <code>layout</code> makes a wrong deck either fail loudly or run correctly. When present, the value must name a deck the target instrument actually defines; deck names are per-instrument configuration, not a fixed vocabulary, so read the name from that instrument's settings export rather than copying a literal out of an example. An unrecognized name fails with "Deck \<name> is not defined," and a resolved empty value fails with "No deck selected." Matching is case-insensitive. See § <code>layout</code> below.
<code>deckItems</code>	object (dictionary)	—	No	Empty dictionary	Dictionary keyed by position name (e.g., <code>"p1"</code> , <code>"tL1"</code> , <code>"wS1"</code>). Each value defines what labware to place at that position. See §Enumerated / Constrained Values for value types. If missing, uses an empty dictionary — engine tolerates omission (no labware placed).
<code>pause?</code>	boolean	—	No	<code>true</code>	When true, inserts a runtime pause with a confirmation dialog showing the deck layout. Omission is tolerated and defaults to <code>true</code> .
<code>verifyPodSetup?</code>	boolean	—	No	<code>true</code>	When true, applies pod tip configuration from <code>podSetup</code> at enqueue time and includes pod status in the confirmation dialog. Omission is tolerated and defaults to <code>true</code> .
<code>barcodeInput?</code>	boolean	—	No	<code>false</code>	When true, displays a barcode input dialog at runtime for labware identification. In simulation mode, the barcode form is constructed but not displayed; the visible dialog only appears during a real instrument run. Engine tolerates omission.
<code>podSetup</code>	object (dictionary)	—	No	Empty dictionary	Contains pod tip configuration. In enqueue the pod-setup confirmation runs only when <code>verifyPodSetup?=true</code> and

Key	Type	Units	Required	Default	Description
					<code>podSetup</code> is present in the step's parameters; the presence check makes omission always safe (the confirmation is skipped). Not required even when <code>verifyPodSetup?</code> is true.
<code>splitterPosition</code>	integer	pixels	No	Half the UI panel height	UI-only. Persists the vertical splitter position in the step editor. Editor-written; not read at runtime.
<code>verifyDeckOnly</code>	boolean	—	No	<code>false</code>	When true, the step only verifies/pauses without modifying the deck or pod state. Skips labware placement and pod setup entirely.
<code>deckSetupLabwareTypeFilter</code>	string	—	No	"<Any>"	UI filter for the labware type combo box in the step editor. Editor-written; not read at runtime.
<code>definesInitialConfiguration?</code>	boolean	—	No	Not present	Marks this Instrument Setup as the method's initial-configuration step (used by the editor to determine which Setup step defines the starting deck). Editor-written; not read at runtime. Author as a JSON boolean (<code>true</code>), e.g. <code>"definesInitialConfiguration?": true</code> .

Universal base keys (`caption` , `dynamic?` , `stepUI` , `isPreconfigured` , `bitmap` , ...) and the node-level `disabled` property apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

PodSetup Sub-Keys

Key	Type	Units	Required	Default	Description
<code>leftHasTips</code>	boolean or null	—	No	<code>false</code>	Tri-state: <code>true</code> = set the left pod's model state to have tips of the specified type; <code>false</code> = set the left pod's model state to no tips loaded; <code>null</code> = leave left pod unchanged (but still report current state in confirmation).
<code>leftTipType</code>	string	—	No	""	Tip box class name for the left pod, given as the bare tip-box class name with no prefix (e.g., <code>"BC230"</code>) — the same form a Select Tips step uses, NOT a <code>LabwareClasses\\ / TipClasses\\</code> path. Only meaningful when <code>leftHasTips</code> is <code>true</code> .
<code>rightHasTips</code>	boolean or null	—	No	<code>false</code>	Tri-state: <code>true</code> = set the right pod's model state to have tips of the specified type; <code>false</code> = set the right pod's model state to no tips loaded; <code>null</code> = leave right pod unchanged.
<code>rightTipType</code>	string	—	No	""	Tip box class name for the right pod, given as the bare tip-box class name with no prefix (e.g., <code>"BC230"</code>), NOT a <code>LabwareClasses\\ / TipClasses\\</code> path. Only meaningful when <code>rightHasTips</code> is <code>true</code> .

Single-pod vs dual-pod instruments: On single-pod instruments (i3, i5 Span-8, i5 Multichannel, or a one-pod i7), `leftHasTips` / `leftTipType` configure Pod1 (the only pod); `rightHasTips` / `rightTipType` are silently ignored when Pod2 is absent. On a dual-pod instrument (an i7 carrying two pods), left = Pod1, right = Pod2. See Introduction to Biomek and Liquid Handling Method Writing §"Instrument Variants" / "Pod Types" for the instrument/pod matrix.

DeckItems Value Structure

Each key in `deckItems` is a position name from the selected layout. To see the exact position names a given default deck defines (e.g. `P1 – P12` , `TL1` , `WS1`), read that deck's sample under `deck-layouts/samples/` (in *Biomek Instrument & Project Reference*) — there is one `biomek-<model>-<deck>.deck.md` per deck. The value may be one of:

Value Form	Meaning
<code>[]</code> (empty array)	Position is empty — no labware placed.

Value Form	Meaning
Array of labware objects	Labware stack placed at the position (bottom-up order).
String (e.g., "Water")	For permanent-labware positions (wash stations): sets the liquid type on the existing labware. Does not replace labware.

Position-name casing. `deckItems` keys are the position name **camelCased** — only the first letter is lowercased; internal capitals are kept: `P1` → `p1`, `TR1` → `tr1`, `WS1` → `ws1`, `TL1` → `tl1`. Pipetting-step position VALUES, by contrast, use the display name (`P1` , `TR1`). Position matching is **case-insensitive**, so either form resolves — follow each step's own examples.

Labware Object Structure

Each labware item in a `deckItems` array is a dictionary with:

Key	Type	Required	Description
<code>_biomekType</code>	string	Yes	Always "labware". Type discriminator read on import.
<code>class</code>	string	Yes	Labware class path (e.g., "LabwareClasses\BCFlat96", "LabwareClasses\BC230").
<code>volumeType</code>	string	No	Volume tracking mode for the labware. One of "Unknown", "Known", or "Nominal". See \$Enumerated / Constrained Values for meanings. Defaults to "Unknown" if omitted.
<code>properties</code>	object	No	Labware properties dictionary (liquid types, volumes, etc.).
<code>tipType</code>	string	No	For tip boxes only: the tip class the box holds, as a <code>TipClasses\</code> -prefixed path (e.g., " <code>TipClasses\T230</code> ") — the same path form <code>class</code> uses, and it must name a published tip class. This is not the same value as a Select Tips Load step's <code>tipType</code> , which takes a bare tip-box class name; see Tip-class grammar below.
<code>evalAmounts</code>	comArray object	No	Pre-evaluated well volumes or tip presence/usage information. See \$ <code>evalAmounts</code> / <code>evalLiquids</code> and \$ <code>evalAmounts</code> for tip boxes below.
<code>evalLiquids</code>	comArray object	No	Pre-evaluated liquid types per well. See \$ <code>evalAmounts</code> / <code>evalLiquids</code> below.

Tip-class grammar

`tipType` appears in two places in method JSON and takes a **different value shape in each**:

Where	What it names	Form	Example
A labware object in <code>deckItems</code> (this step)	The tip class inside the box	<code>TipClasses\</code> path	" <code>TipClasses\T230</code> "
A Multichannel Select Tips Load / Advanced Load step	The tip box labware class	Bare class name, no prefix	" <code>BC230</code> "

Both must name a class that exists in the published catalogs — see Catalog: Labware Classes (in *Biomek Instrument & Project Reference*) and Catalog: Tip Classes (in *Biomek Instrument & Project Reference*). A free-text description such as "230 uL tips" raises an unknown-labware-class error, and so does putting the wrong one of the two forms in either place.

Tip-box to tip-class mapping. A tip box's `tipType` is the tip class that corresponds to the box's labware class. The naming convention is that the tip class name matches the box's tip-size suffix: labware class `LabwareClasses\BC230` (a 230 uL tip box) pairs with tip class `TipClasses\T230`, and so on (`BC` prefix becomes `T`). For the complete box-to-tip-class pairing see the tip-box table in Catalog: Labware Classes (in *Biomek Instrument & Project Reference*); for what the `F`, `_LLS`, `_WB`, and `_384` suffixes mean see Naming scheme (in *Biomek Instrument & Project Reference*).

Labware properties sub-keys

The labware object's `properties` dictionary recognizes exactly these keys:

Key	Type	Notes
name	string	Labware INSTANCE name (a user label; may be a Biomek expression).
liquidtype	string	A liquid-type NAME from the project (case-insensitive; may be an expression). Permanent labware (reservoirs / wash stations) typically set ONLY this.
barcode	string	Barcode text.
senseEveryTime	boolean	Advanced; only meaningful for liquid-level-sensing labware. Omit to default to first-time-only.
device	string	Advanced; only for labware whose class requires an associated device.

name , liquidtype , and barcode are the common ones; senseEveryTime and device are advanced and rarely set. There is **no** sealed / lid key — lids are their own labware objects stacked in the deckItems array.

Tip-box routing keys are siblings of properties , not inside it. On a tip-box labware object the routing keys WhenDone , DiscardTips , and DiscardTipsLocation are set at the labware object's TOP LEVEL (alongside class , volumeType , properties , tipType) — NOT inside properties : - WhenDone — string: <Anywhere> (default), <Home> , <Any Trash> , or a deck-position name. Not used on i3. - DiscardTips — boolean. The read default DIFFERS by pod (Span-8 defaults true , Multichannel defaults false) , so set it explicitly when the behavior matters. - DiscardTipsLocation — string: <TipBox> , <Any Trash> , or a specific trash position (e.g. TR1). Apply to tip boxes. <TipBox> implies DiscardTips: false . **What it does depends on the pod family:** - **Multichannel and Fixed-8** — tips are returned to their own box, so <TipBox> is the value to use on a deck with no reachable trash. - **Span-8** — there is no return-to-box path. <TipBox> and DiscardTips: false do **not** keep the tips on the box; they route to a trash search exactly as <Any Trash> does, and fail identically when no trash position is both present and reachable by that pod. A Span-8 deck that unloads disposable tips needs a reachable trash position; no value of these keys substitutes for one.

These three keys are a **separate vocabulary from the unload steps**. "<where they came from>" is a Fixed-8 Unload Tips tipDestination / Transfer unloadLocation value and is **not** accepted by WhenDone or DiscardTipsLocation — on those keys it is treated as a position name, resolves to nothing, and fails later. The DiscardTipsLocation equivalent is <TipBox> . See the tip-discard entry under Common Mistakes.

Budget tip boxes to the method's tip usage. Each Load-Tips step consumes a fresh set of tips from a tip box; once a box is emptied it is marked used and won't be re-picked. If a method's total Load-Tips cycles exceed the tips available across all placed tip boxes, enqueue fails with Unable to find and retrieve tips of type <class> for <pod> (naming the boxes already used). To run many tip-change cycles either (a) place additional tip boxes of the same class on free deck positions — but the deck has a limited number of positions, and a crowded deck of tall tip boxes can block a pod's motion path — or (b) keep the same tips loaded across consecutive steps that permit it, which is the simplest tip-saving pattern. Returning tips to their box with Unload Tips does **not** save tips by default: a tip-box well carries a use count of 1, so a tip that has pipetted is recorded as spent and a later Load Tips draws fresh tips instead. Re-picking used tips is possible with a higher use count, but treat it as advanced method programming — the software marks a tip used, and does not track which samples it has already touched, so keeping a reused tip away from unrelated samples is your accounting to do.

Labware instance names and how position fields resolve. A labware INSTANCE name is set via the labware object's properties.name . Downstream, a pipetting step's position field (where / location / position , per step family) accepts ANY of: a literal deck-position name (e.g. P5), a labware instance name (the properties.name you assigned here), or a labware class name — all matched case-insensitively. If a name maps to several positions, the first is used. So e.g. "location": "SourcePlate" resolves when a labware placed here has properties.name set to "SourcePlate" . See Concept Guide: Labware & Position Resolution §1.

Waste position (for discarding bulk liquid). Biomek has no "liquid waste" labware characteristic — the Trash characteristic (used by <Any Trash> and the tip-box WhenDone / DiscardTipsLocation routing keys) applies only to LABWARE and TIP discard, not to liquid, and the Wash Station characteristic serves only wash cycles. To dispose of bulk liquid, place an empty reservoir (or plate) on a free deck position here and name it (e.g. "properties": {"name": "WastePlate"}) — pipetting steps then discard by dispensing to that position as an ordinary destination. Give it capacity for the method's total discard volume; the per-well volume tracker will reject a dispense that exceeds the labware's MaxVolume . See the "Dispensing to waste" section in Transfer for the authoring pattern and the pitfalls (do NOT dispense back into the source).

evalAmounts / evalLiquids

Both `evalAmounts` and `evalLiquids` are `comArray` objects with `arraySubtype: "variant"`. They pre-fill per-well state on the labware object — they are the starting balance of the enqueue-time volume ledger, not passive annotation. For what that ledger does downstream, see Concept Guide: Well Volume Tracking.

- `evalAmounts` values are per-well volumes in μL , as numbers (or expression strings).
- `evalLiquids` values are per-well liquid-type NAME strings.

The array length must equal the labware's well count (96, 384, ...). The array is **row-major**: index 0 = well A1, 1 = A2, ... 11 = A12, 12 = B1, ... 95 = H12 (for a 96-well plate). For a **reservoir** the addressable unit is the section, not the well, so a one-section reservoir takes a **one-element** array — passing 96 elements to it is the most common form of this mistake.

`evalAmounts` and `evalLiquids` react to a length mismatch **differently**, and the difference is a silent-corruption trap:

- `evalAmounts` **too long** fails at enqueue: `Position <Pn>: SetAllAmounts passed an array that was not the same size as the well count.` But **too short does not fail** — it silently pads the remaining wells with the array's **first** element's value (not `0.0`), so a partial array quietly seeds the wrong volumes with no error.
- `evalLiquids` rejects **any** length mismatch, too long or too short: `Position <Pn>: When using SetAll, the input array size must match the number of wells on the labware.`

Because the two are authored together, the only safe rule is a **full-length array for both**. There is no replication shortcut — a one-element array is correct only when the labware itself has one addressable unit (e.g., a one-section reservoir). Always supply a full-length array with explicit values for every well — use `0.0` for empty wells and `" "` for wells with no assigned liquid.

Each value is also range-checked as it is seeded: a value above the well's (or reservoir section's) maximum fails with `Position <Pn>: Well volume out of allowed range: <value>.` Seed against the capacity in the per-type tables in Catalog: Labware Classes (in *Biomek Instrument & Project Reference*), not against the nominal fill you have in mind.

They are consulted only when `volumeType` is `"Known"` or `"Nominal"`. If `volumeType` is `"Known"` but `evalAmounts` is omitted entirely, every well defaults to volume 0 ("known empty").

```
"evalAmounts": { "_biomekType": "comArray", "arraySubtype": "variant", "values": [200.0, 200.0, /* ...96 total, row-major A1..H12... */ ]
"evalLiquids": { "_biomekType": "comArray", "arraySubtype": "variant", "values": ["Water", "Water", /* ...96 total... */ ] }
```

Seeding for downstream liquid-relative heights. This step is where the tracked-volume surface for later liquid-relative moves comes from. On Fixed-8 and Multichannel (96- or 384-channel head) pods, declare the labware here with `volumeType` `"Nominal"` or `"Known"` and pair it with an `evalAmounts` fill (plus `evalLiquids`) — see the `volumeType` enum below and Pipetting Techniques, Templates, and Auto-Selection §"Technique Selection" and §"Height Override" for the pod-family rules. Without `evalAmounts` the tracked volume stays 0 and the aspirate fails on available volume.

evalAmounts for tip boxes

For tip box labware classes, `evalAmounts` indicates tip presence and the number of remaining uses of the tips. A value of `1` indicates that a tip is present and may be used one time. A value of `0` indicates that no tip is present at that location in the tip box.

The default behavior for a tip box is to set all values in `evalAmounts` to `1`, indicating a full box of tips that can be used one time. To create a tip box that is empty, set all values to `0`. To create a tip box that is partially empty, set the pattern matching the well layout of the box to have `0` for empty slots and `1` for occupied slots (ordering matches well ordering in microplates).

If the same tips are intended to be used repeatedly and contamination is not a concern, for example to use the same tips to repeatedly mix a reservoir, `evalAmounts` can be set to a higher number. It is recommended to name the tip box in such situations so that the tips are **only** used for the expected purpose. Higher numbers do not impact the Span-8 pod, as the Span-8 pod cannot unload tips to a tip box.

Tip tracking is handled differently in Select Tips steps. Refer to the individual steps for details.

Note that for Fixed-8 pods, tip usage is only decremented when tips contact liquid. Loading and unloading without any liquid handling in between does not decrement the usage. For Multichannel pods, tip usage is decremented each time tips are loaded, regardless of if they are used.

Enumerated / Constrained Values

volumeType

Value	Meaning
"Unknown"	Well volumes are not known, so a liquid-relative move has to locate the surface at run time. On Span-8 , an LLS technique can sense the surface live even without a known volume. On Fixed-8 and Multichannel , an "Unknown" well fails a liquid-relative move — those pods do not support liquid-level sensing (LLS); only Span-8 does. Declare at least "Nominal" there. This is the default if volumeType is omitted. See Pipetting Techniques, Templates, and Auto-Selection §"Liquid-relative heights need a locatable liquid surface".
"Known"	Well volumes are definitively known; pipetting steps skip LLS and calculate aspirate/dispense heights from tracked volumes.
"Nominal"	Well volumes are estimates. The estimate makes the surface locatable, so liquid-relative moves work the same as for "Known". The difference is trust: a "Known" volume is taken as verified and used directly, while a "Nominal" well is refined by one live LLS sense before the move on a pod that has LLS (Span-8) using a technique that senses — after which that reading is reused for the rest of that well's operation. On Fixed-8 and Multichannel there is no LLS, so "Nominal" uses the estimate directly, just like "Known". Volume tracking keeps the uncertainty: an aspirate is refused only when it would drop the well below the low end of the estimate (... has no more than <possible-max>.), whereas a "Known" well is checked against its exact tracked amount.

In JSON the single volumeType string is authoritative. The parser is case-insensitive: "Known", "KNOWN", and "known" are all accepted. Numeric strings (e.g. "3") are explicitly rejected even if they correspond to the enum's underlying integer value; any other non-matching value also fails to import.

Destination volume auto-tracking. The engine automatically updates tracked well volumes and liquid types on destination labware after a dispense, regardless of that labware's initial volumeType. A plate placed here as "Unknown" will have per-well volumes tracked once liquid has been dispensed into it, so a downstream Aspirate or Mix step using "Well Contents" volume on that plate works correctly.

Destination labware must have DEFINED amounts before the first liquid-relative dispense. This is distinct from the auto-tracking note above, which updates volumes *after* a dispense completes. Most default pipetting techniques (including "F8 Medium", "S8 1000 Medium", "MC P60") position the tips relative to the liquid level, and that move requires a locatable volume surface even at the destination -- before any liquid has been dispensed into it. If a destination plate has volumeType: "Unknown", the first Dispense step targeting it fails with: Cannot move relative to liquid level if liquid amounts are not defined for specified labware. To avoid this, declare the destination "Known" or "Nominal" with an evalAmounts array (all 0.0 for an initially empty plate) and an evalLiquids array. This applies to Fixed-8, Multichannel, and Span-8 destinations alike. See the seeding format under evalAmounts / evalLiquids and the Worked Example for a complete method with a correctly seeded destination.

Volume tracking (running per-well ledger). Once a labware is placed with "Known" or "Nominal" (or has been dispensed into and auto-tracked as noted above), the enqueue engine maintains a running per-well volume across every downstream pipetting step. Two hard-fail cases follow:

- Dispense exceeds the well's capacity.** If a dispense would push the well past the labware class's max well volume, enqueue fails with Cannot dispense <amt> because the <well> already has <current> and can only hold <max>. The <max> is the class's per-well capacity — see the max-volume column in the per-type tables under Catalog: Labware Classes (in Biomek Instrument & Project Reference) (per-class; on a reservoir it is the per-section max volume).
- Aspirate exceeds the well's tracked volume.** If a well is tracked as "Known" (or an aspirate would drop a "Nominal" well below zero considering the low end of its uncertainty), enqueue fails with Cannot aspirate <amt> because the <well> has only <current>. (or ... has no more than <possible-max>. for "Nominal").

Keep the seed here (volumeType + evalAmounts) consistent with the volumes each downstream aspirate/dispense will actually move — the tracker propagates through Transfers, Combines, Mix, Serial Dilution, etc.

leftHasTips / rightHasTips (tri-state)

JSON Value	Behavior
true	Pod is configured with tips of the type named in the corresponding *TipType key.
false	Pod is set to have no tips loaded.
null	Pod tip status is not changed; confirmation dialog reports current state.

deckItems position value sentinel

JSON Value	Behavior at Enqueue
[] (empty array)	Position cleared of labware (nothing placed).
String value	Permanent-labware position: sets LiquidType property and Liquid dataset on existing labware.

layout

layout chooses which of the instrument's decks the step configures. Deck names are per-instrument configuration — the deck one instrument calls hybrid may not exist on another — so **no literal is safe to copy from an example**.

Set layout explicitly — do not omit it. Omitting the key falls back to the instrument's default deck, so a method later run on an instrument whose default deck differs from the one it was written for can behave unexpectedly — and silently, because omission never fails at enqueue. Pinning the deck makes a wrong deck either fail loudly (an unknown name is rejected) or run correctly (the right name resolves). How to do it:

- **Set it to a deck name read from that instrument's settings export.** Matching is case-insensitive. Secondary caveat: a method that pins a deck is not portable to an instrument configured differently — but that portability cost buys the loud-failure protection above.
- **Do not guess a name.** An unknown deck fails at enqueue and the message names the deck rather than the mistake.

Omitting layout is a discouraged fallback. The step then uses the instrument's default deck and cannot name a deck that does not exist — but the choice is not neutral: it goes to whichever deck is flagged default, or, if none is, to an arbitrary one. That unpredictability across instruments is exactly why setting layout explicitly is recommended.

If the deck you name is not the one currently loaded, the step proceeds and warns:

WARNING: This method was configured with <layout> but <current deck> is the current deck.

Deck names are set at instrument creation and can be renamed, added, or deleted in the Deck Editor. To see the deck names an instrument defines and its default, read its Instrument Settings JSON export (see Deck Layouts Format Specification (in *Biomek Instrument & Project Reference*) §"Finding and using the instrument's default deck"); produce one from the Biomek software if you don't have it.

layout **names a deck, not a pod family.** "Fixed8" looks like a podType token but is not a deck name on any factory-configured instrument — the Fixed-8 (i3) deck is exported as "i3". "layout": "Fixed8" fails the enqueue check. Some factory deck names do coincide with pod-type tokens ("Span8" and "Multichannel" are real decks on the instruments configured with them), which is what makes the mistake easy.

Cross-Field Validation Rules

1. When layout is present, it must name a deck defined on the instrument; otherwise enqueue fails. When layout is absent (the normal form), the instrument's default deck is used. If the resolved value is an empty string (no default configured and no key provided), enqueue fails.
2. Each key in deckItems should correspond to a position in the selected layout. Keys that do not match a layout position are silently ignored (not placed).

3. When `leftHasTips` / `rightHasTips` is `true`, the corresponding `*TipType` must name a valid tip box class in the instrument's labware-class library. An invalid name causes a tip-box-not-found failure.
4. Any labware `class` referenced in `deckItems` must exist in the instrument's labware-class library; otherwise that labware is not placed and a mapping error is shown.
5. A string value in `deckItems` (liquid-type setting) only takes effect if the position already has permanent labware. If the position has no labware, the liquid type is not applied.
6. When `verifyDeckOnly` is `true`, labware placement and pod setup are skipped entirely — only pause/barcode behavior runs.
7. This step is where a deck position gets its labware `class`, its `volumeType`, and any `evalAmounts` / `evalLiquids` fill; pipetting steps do not create labware and inherit that seeding. If a downstream step uses a liquid-relative height, seed it here — see the note under `$Labware Object Structure, Concept Guide: Labware & Position Resolution` §4, and `Pipetting Techniques, Templates, and Auto-Selection` §"Technique Selection" and §"Height Override".

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Minimal Instrument Setup (empty deck, no pause)

```
{
  "stepType": "Instrument Setup",
  "parameters": {
    "barcodeInput?": false,
    "deckItems": {
      "p1": [],
      "p2": [],
      "p3": [],
      "p4": [],
      "p5": [],
      "p6": [],
      "p7": [],
      "p8": [],
      "p9": [],
      "p10": [],
      "p11": [],
      "p12": []
    },
    "layout": "i3",
    "pause?": false,
    "podSetup": {},
    "verifyPodSetup?": false
  }
}
```

Example 2: Typical setup with tip box and plate, pod verification enabled

```
{
  "stepType": "Instrument Setup",
  "parameters": {
    "barcodeInput?": false,
    "deckItems": {
      "p1": [],
      "p2": [],
      "p3": [],
      "p4": [],
      "p5": [],
      "p6": [],
      "p7": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BC230",
          "volumeType": "Unknown",
          "properties": {},
          "tipType": "TipClasses\\T230"
        }
      ],
      "p8": [],
      "p9": [
        {
          "_biomekType": "labware",
```

```

    "class": "LabwareClasses\\BCFlat96",
    "volumeType": "Unknown",
    "properties": {}
  }
],
"p10": [],
"p11": [],
"p12": []
},
"layout": "i3",
"pause?": true,
"podSetup": {
  "leftHasTips": false,
  "leftTipType": "",
  "rightHasTips": false,
  "rightTipType": ""
},
"verifyPodSetup?": true
}
}

```

The p9 plate here is placed "Unknown" (no tracked volumes). To seed known well volumes instead, set `volumeType` to "Known" (or "Nominal") and add matching `evalAmounts` / `evalLiquids` arrays on that labware object — see § `evalAmounts` / `evalLiquids`. A "Known" plate without `evalAmounts` treats every well as volume 0 (per Cross-Field Validation rule 7 and the seeding note).

Example 3: Setup with pod tip configuration and wash station liquid type

```

{
  "stepType": "Instrument Setup",
  "parameters": {
    "barcodeInput?": false,
    "deckItems": {
      "p1": [],
      "p2": [],
      "p3": [],
      "p4": [],
      "p5": [],
      "p6": [],
      "p7": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BC230",
          "volumeType": "Unknown",
          "properties": {},
          "tipType": "TipClasses\\T230"
        }
      ],
      "p8": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BCFlat96",
          "volumeType": "Known",
          "properties": {}
        }
      ],
      "p9": [],
      "p10": [],
      "p11": [],
      "p12": [],
      "tL1": [],
      "tL2": [],
      "tR1": [],
      "wS1": "Water"
    },
    "layout": "Multichannel",
    "pause?": true,
    "podSetup": {
      "leftHasTips": true,
      "leftTipType": "BC230",
      "rightHasTips": null,
      "rightTipType": ""
    },
    "verifyPodSetup?": true
  }
}

```

Example 4: i5 Span-8 deck with tip box and plate

```

{
  "stepType": "Instrument Setup",
  "parameters": {
    "barcodeInput?": false,
    "deckItems": {
      "p1": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BC230",
          "volumeType": "Unknown",
          "properties": {},
          "tipType": "TipClasses\\T230"
        }
      ],
      "p2": [],
      "p3": [],
      "p4": [],
      "p5": [],
      "p6": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BCFlat96",
          "volumeType": "Unknown",
          "properties": {
            "name": "SamplePlate"
          }
        }
      ],
      "p7": [],
      "p8": [],
      "p9": [],
      "p10": [],
      "p11": [],
      "p12": [],
      "p13": [],
      "p14": [],
      "p15": [],
      "p16": [],
      "p17": [],
      "p18": [],
      "p19": [],
      "p20": []
    },
    "layout": "Span8",
    "pause?": true,
    "podSetup": {
      "leftHasTips": false,
      "leftTipType": "",
      "rightHasTips": false,
      "rightTipType": ""
    },
    "verifyPodSetup?": true
  }
}

```

Example 5: i5 Multichannel deck with tip-loader, plates, and wash station

```

{
  "stepType": "Instrument Setup",
  "parameters": {
    "barcodeInput?": false,
    "deckItems": {
      "tL1": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BC230",
          "volumeType": "Unknown",
          "properties": {},
          "tipType": "TipClasses\\T230"
        }
      ],
      "tL2": [],
      "tL3": [],
      "tL4": [],
      "tL5": [],
      "p1": [
        {
          "_biomekType": "labware",
          "class": "LabwareClasses\\BCFlat96",
          "volumeType": "Known",
          "properties": {
            "name": "SourcePlate"
          }
        }
      ]
    }
  }
}

```



```

    }
  },
  "p2": [],
  "p3": [],
  "p4": [],
  "p5": [],
  "p6": [
    {
      "_biomekType": "labware",
      "class": "LabwareClasses\\BCFlat96",
      "volumeType": "Unknown",
      "properties": {
        "name": "DestPlate"
      }
    }
  ],
  "p7": [],
  "p8": [],
  "p9": [],
  "p10": [],
  "p11": [],
  "p12": [],
  "p13": [],
  "p14": [],
  "p15": [],
  "wS1": "water"
},
"layout": "Multichannel",
"pause?": true,
"podSetup": {
  "leftHasTips": false,
  "leftTipType": "",
  "rightHasTips": false,
  "rightTipType": ""
},
"verifyPodSetup?": true
}
}

```

Source-plate volume type on Multichannel. The source plate at `p1` uses `volumeType: "Known"` because Multichannel pods cannot perform a liquid-relative move on an `"Unknown"` well -- Multichannel pods do not support liquid-level sensing (LLS); only Span-8 does (see `$ volumeType`). For a source you aspirate from with a liquid-following technique, use `"Known"` or `"Nominal"` and pair it with `evalAmounts` to seed per-well volumes -- see `$ evalAmounts / evalLiquids`. A `"Known"` plate without `evalAmounts` treats every well as volume 0.

Common Mistakes

- **Bare labware class name without the `LabwareClasses\\` prefix:** Instrument Setup's `deckItems.*[].class` needs the fully-qualified path (e.g. `"LabwareClasses\\BCFlat96"`) — a bare `"BCFlat96"` fails at enqueue with a load-labware error at that position. The prefix requirement is specific to this step: other steps (Transfer/Combine/MC Select Tips, etc.) reference the same labware class by its short name (`BCFlat96`).
- **Changing a labware class without propagating it:** A class edit here must be mirrored in **every** downstream step referencing that position or class — the `labwareClass` / `what` / `tip-box` / `plate-class` fields in Transfer, Combine, Aspirate/Dispense/Mix, Move Labware, Hold Labware, etc. A stale reference fails at enqueue with a class-mismatch error. Treat a setup-class change as a method-wide find-and-update.
- **Reversed labware stacking order:** Items in a `deckItems` array are bottom-up — the *last* array element is the topmost (accessible) labware. Getting the order wrong at a lidded/nested position places the wrong labware on top.
- **Confusing tip type with tip class:** `leftTipType` / `rightTipType` name a *tip box* labware class as the **bare** class name (e.g., `"BC230"`) — the same form a Select Tips step uses — NOT a `LabwareClasses\\`-prefixed path and NOT a raw tip class like `"TipClasses\\T230"`. A prefixed value like `"LabwareClasses\\BC230"` is unbound and fails at enqueue with `the tip box type LabwareClasses\\BC230 not found`.
- **String value on a non-permanent position:** A string in `deckItems` sets liquid type on existing permanent labware (wash stations only). At a regular deck position it is silently ignored — the position stays empty and no error surfaces.
- **Omitting `_biomekType` on a labware object:** Without `"_biomekType": "labware"` Biomek will not recognize the item as labware on import — it drops out of the deck silently.

- **Multichannel tip box on a p* position instead of tL* :** On a Multichannel deck, a tip box used for MC tip loading must be placed on a tip-loader position (tL1 -- tL5 , which carry the "multichannel Can Load Tips" characteristic). Placing it on a p* position causes a runtime "unable to find/retrieve tips" failure. Span-8 tip loading similarly requires positions with the "span Can Load Tips" characteristic. Example 5 shows the correct Multichannel placement (tip box on tL1).
- **"<Any Trash>" on a deck with no reachable trash:** "<Any Trash>" resolves only to a deck position carrying the tip-discard characteristic, and "present on the deck" is not the same as "reachable by the pod that must use it" — on a two-pod instrument each pod may reach only some of the trash positions, or none. A deck that defines no trash position at all fails outright. The replacement depends on **which key** you are setting, and the two families do not share a vocabulary:
- Fixed-8 Unload Tips tipDestination , and Transfer / Combine unloadLocation : use "<where they came from>" (return tips to their originating box).
- The tip-box routing keys WhenDone and DiscardTipsLocation : "<where they came from>" is **not** a valid value for either — set DiscardTipsLocation to "<TipBox>" (which also means DiscardTips is false), and leave WhenDone at "<Anywhere>" . **This remedy is Multichannel/Fixed-8 only.** A Span-8 pod has no return-to-box path, so "<TipBox>" still routes to a trash search and still fails on a deck with no reachable trash — see the DiscardTipsLocation entry above. On Span-8 the only fix is a trash position the pod can actually reach.

Using "<where they came from>" on DiscardTipsLocation does not fail validation; it is read as an ordinary position name, fails to resolve, and surfaces later as a tip-discard-location error that names the pod rather than the key.

- **Destination plate left "Unknown" when the technique follows liquid level:** Most default pipetting techniques move relative to the liquid surface. If a destination plate is placed with volumeType: "Unknown" (the default when volumeType is omitted), the first Dispense step targeting it fails with Cannot move relative to liquid level if liquid amounts are not defined for specified labware. This is NOT rescued by auto-tracking (auto-tracking only runs after a dispense succeeds). Fix: set the destination to volumeType: "Known" (or "Nominal") with evalAmounts all 0.0 and evalLiquids filled. This applies to Fixed-8, Multichannel, and Span-8 destinations.

Move Labware

Property	Value
stepType	"Move Labware"
Category	Leaf
Terminator	N/A
Compatible Hardware	Any instrument with at least one pod that has a gripper (rotating offset gripper or Fixed-8 integrated gripper).

Behavior Summary

The Move Labware step moves labware from one deck position to another using a pod-mounted gripper. At enqueue time the step resolves the named pod (`pod`) and verifies that it has a gripper; if it does not, the step raises an error. It then resolves the `source` and `target` deck position names against the instrument's deck. The step supports three stacking modes: move the entire stack (default when `depth` is unbound and `leaveBottomLabware` is false), move all but the bottom piece (`leaveBottomLabware` = true), or move a specific number of pieces from the top (`depth` bound to a numeric value). A `gripSide` parameter controls whether the gripper fingers point toward "A1 near" (right) or "A1 away from" (left); for Fixed-8 pods the grip-side value is accepted but silently ignored, defaulting to A1-near. The step supports gripper offset overrides (`xoffset` , `yoffset` , `zoffset` , `squeeze` , `unsqueeze`) at three precedence levels: step-parameter keys override position/labware-class gripper-info overrides, which override instrument defaults (unset). Position-level overrides are resolved via `GripperInfoOverride` keys on the source and destination positions paired with a `GripperInfoOverrides` dictionary on the labware class.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	string	—	Yes	""	Name of the pod to use for the move (e.g., "Pod1"). The pod must have an attached gripper.
<code>source</code>	string	—	Yes	""	Name of the deck position that contains the labware to move (e.g., "P13"). Supports expressions. Valid position names come from the selected deck — see the sample decks under <code>deck-layouts/samples/</code> (in <i>Biomek Instrument & Project Reference</i>).
<code>target</code>	string	—	Yes	""	Name of the deck position to move the labware to (e.g., "P23"). The target position should be empty or capable of receiving additional labware; the gripper will raise an error if the move cannot be completed. Supports expressions.
<code>gripSide</code>	string	—	Yes	(none)	Side of the gripper to use: "A1 near" or "A1 away from" . Key must be present for every pod , including Fixed-8. An absent key raises a "could not find GripSide" system error. For Fixed-8 pods, an empty or invalid <i>value</i> is silently coerced to A1-near; for offset grippers an empty value raises a "missing grip side" error. See the Enumerated / Constrained Values section.
<code>leaveBottomLabware</code>	boolean	—	Conditional (when <code>depth</code> is unbound)	(none — required when <code>depth</code> is unbound)	When <code>true</code> and <code>depth</code> is unbound, moves all labware except the bottom piece. Source must contain more than one labware item. Omission raises a "could not find LeaveBottomLabware" system error. If <code>depth</code> is bound, <code>leaveBottomLabware</code> is never read and may be omitted freely.

Key	Type	Units	Required	Default	Description
depth	numeric	count	No	(unbound)	Number of labware pieces to move from the top of the stack. When bound, overrides leaveBottomLabware . Not bound by default — leaving it unbound moves the entire stack (subject to leaveBottomLabware).
xoffset	numeric	cm	No	(unbound)	Gripper X-axis offset override. When bound, takes precedence over position/labware-class gripper-info overrides.
yoffset	numeric	cm	No	(unbound)	Gripper Y-axis offset override.
zoffset	numeric	cm	No	(unbound)	Gripper Z-axis offset override. Used for moves to/from special positions like a Static Peltier ALP.
squeeze	numeric	cm	No	(unbound)	Gripper squeeze (inward clamp distance) override.
unsqueeze	numeric	cm	No	(unbound)	Gripper unsqueeze (outward release distance) override.

Note on offset keys: The five offset keys (xoffset – unsqueeze) are programmatic-only; they do not appear in the GUI and are set only when creating steps via scripting or JSON authoring. Values are in centimeters and override (replace) any position/labware-class gripper-info overrides — bind only the axes to displace, and omit (rather than set to 0) the ones to inherit.

Universal base keys (caption , dynamic? , stepUI , ...) apply to this step but are omitted here — see Base Keys & Serialization Conventions.

Enumerated / Constrained Values

gripSide

Value	Meaning
"A1 near"	Gripper fingers point to the right (toward A1 side).
"A1 away from"	Gripper fingers point to the left (away from A1 side).

For offset (rotating) grippers, gripSide must be set to one of the two values above; an empty or invalid value raises an error. For Fixed-8 pods (pod type "Fixed8"), any value including empty or invalid is silently coerced to A1-near — the parameter is accepted for compatibility but has no physical effect.

Step UI dynamically filters grip sides by physical reachability: the editor's "Holding labware with" dropdown lists only the grip side(s) that are physically reachable for the chosen source/destination. When both sides are reachable it lists both; when only one is, it lists only that one; when neither is reachable the dropdown is empty and the UI displays an error indicating the move cannot be carried out. This is editor validation — at enqueue an out-of-reach move fails with a "pipettor destination X/Y ... is outside of travel range" error (see Cross-Field Validation Rule 11).

Stacking mode (derived from depth and leaveBottomLabware)

depth bound?	leaveBottomLabware	Behavior
No	false	Moves the entire stack.
No	true	Moves all but the bottom piece. Requires stack depth > 1.
Yes	(ignored)	Moves exactly depth pieces from the top of the stack.

Cross-Field Validation Rules

1. **Pod must have a gripper:** The named `pod` must be installed on the instrument and must have a gripper. If the pod has no gripper or the gripper object is null, the step raises an "invalid pod type" error.
2. **Source must be specified and found:** `source` must be a non-empty string that resolves to a position on the deck. An empty string or unresolved name raises a source-not-specified or source-not-found error.
3. **Target must be specified and found:** Same rules apply to `target`.
4. **Source must not be empty:** The source position must contain at least one labware item. Otherwise the step raises a source-is-empty error.
5. **Grip side required for offset grippers:** When the pod is not Fixed-8, `gripSide` must be "A1 near" or "A1 away from". An empty value raises a "gripper cannot perform this move" error; any other invalid value raises an "unable to interpret grip side" error.
6. **Leave-bottom requires depth > 1:** When `leaveBottomLabware` is `true` and `depth` is unbound, the computed depth is the stack height minus one. If the stack has only one piece, this yields zero and the step raises a "cannot leave bottom labware" error.
7. **Depth must not exceed actual stack depth:** When `depth` is bound, it is used directly. If `depth` exceeds the actual stack depth, the step fails with a source-depth-vs-stack-height mismatch.
8. **Grip-info overrides must agree:** If both the source and destination positions have a `GripperInfoOverride` key, the values must be identical (case-insensitive). Mismatched values raise an incompatible-grip-override error.
9. **Labware class must contain named override:** If a `GripperInfoOverride` value is resolved, the labware class being gripped must have a matching key under its `GripperInfoOverrides` dictionary. If the key is not found, the step raises a bad-labware-class-grip-override error.
10. **Step-parameter offsets take final precedence:** If any of `xOffset`, `yOffset`, `zOffset`, `squeeze`, or `unsqueeze` is bound in the step's parameters, it overrides the corresponding value from position/labware-class gripper-info overrides.
11. **Resolved position must be within the pod's travel range:** The final gripper coordinate (deck position plus any offsets) must lie inside the pod's physical X/Y travel limits. A position outside the limit fails at enqueue with a "destination outside travel range" error. Discarding to a far-corner trash or applying large offsets are the common triggers.
12. **Labware class must carry Moveable:** The class of the labware being moved must carry the `Moveable` characteristic. A class that lacks it fails at enqueue with `Labware of type <class> cannot be moved.` See Catalog: Labware Classes (in *Biomek Instrument & Project Reference*) for which default classes lack it (notably `BCDeep96Square` and the wash stations among titer plates; most tube racks; four of the seven reservoirs).

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Simple move of a single plate

```
{
  "stepType": "Move Labware",
  "parameters": {
    "pod": "Pod1",
    "source": "P13",
    "target": "P23",
    "gripSide": "A1 near",
    "leaveBottomLabware": false
  }
}
```

Example 2: Move top 2 plates from a stack

```
{
  "stepType": "Move Labware",
  "parameters": {
    "pod": "Pod1",
    "source": "P5",
    "target": "P10",
    "gripSide": "A1 away from",
    "leaveBottomLabware": false,
    "depth": 2
  }
}
```

Example 3: Move stack leaving bottom labware in place

```
{
  "stepType": "Move Labware",
  "parameters": {
    "pod": "Pod1",
    "source": "P3",
    "target": "P8",
    "gripSide": "A1 near",
    "leaveBottomLabware": true
  }
}
```

Example 4: Move with gripper offset overrides (e.g., Static Peltier destination)

```
{
  "stepType": "Move Labware",
  "parameters": {
    "pod": "Pod1",
    "source": "P7",
    "target": "P15",
    "gripSide": "A1 near",
    "leaveBottomLabware": false,
    "zOffset": 0.5,
    "squeeze": 0.1,
    "unsqueeze": 0.1
  }
}
```

Common Mistakes

- **gripSide required even for Fixed-8:** The key is read at the top of enqueue before the pod-family branch, so omission throws a "could not find GripSide" system error on any pod. Only the *value* is ignored on Fixed-8.
- **Binding depth alongside leaveBottomLabware: true:** A bound `depth` silently wins — `leaveBottomLabware` is not consulted. Use one mode, not both.
- **Binding an offset to 0 instead of omitting it:** `0` is a bound override that displaces the position/labware-class default; omit the key to inherit.
- **Mismatched GripperInfoOverride on source vs. destination:** Both positions defining the same key with different values raises an incompatible-grip-override error. Match them, or define only one side.
- **Large offsets or corner discards past travel range:** Deck-corner trash or aggressive offsets can push the resolved coordinate outside the pod's X/Y limits, failing at enqueue with a "destination outside travel range" error.
- **Referencing a moved plate by its old position:** after this step relocates a plate, a downstream step that names the plate's *original* deck position finds it empty and fails. References by the plate's **instance name** (its `properties.name` from Instrument Setup) or class name auto-resolve to its current position and need no update — prefer naming labware over hard-coding positions across a Move.

Cleanup

Property	Value
stepType	"Cleanup"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7

Not available on i3 (Fixed-8). The Cleanup step is i5/i7 only. On an i3, unload disposable tips before Finish with a `Fixed-8 Unload Tips` step, or rely on Finish's `cleanupPods: true` (the default), which handles tip disposal at method end.

Behavior Summary

The Cleanup step directs the instrument to dispose of tips and optionally return tip boxes to their configured destinations. When executed, it iterates over both pods (Pod1 and Pod2) and removes any loaded tips. With no options selected (the default), the step only unloads tips to their configured unload location — tip boxes are left wherever they currently are. When `sendTipBoxes` is enabled, the step additionally sends all tip boxes that originated off-deck (and those configured to go to trash) to their final destinations. When `onDeckToHome` is also enabled, tip boxes that started on the deck are returned to their original on-deck positions as well. The Cleanup step resets the physical hardware state (tip and tip box positions) but does not reset software state — use it together with the Finish step to reset both hardware and software for repeated method runs. When used within a Select Tips group, the Cleanup step unloads tips to the configured unload location of the source tip box, or if that location is not available, discards tips to the trash.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>sendTipBoxes</code>	<code>boolean</code>	—	Yes	<code>false</code>	Send tip boxes to their configured final destinations. When <code>false</code> , tip boxes are left wherever they are after tip disposal. When <code>true</code> , all tip boxes that originated off-deck and those configured to go to trash are sent to those final destinations. Omission raises a "could not find SendTipBoxes" system error.
<code>onDeckToHome</code>	<code>boolean</code>	—	Cond.	<code>false</code>	Return tip boxes that started on the deck back to their original on-deck locations. Only <i>meaningful</i> when <code>sendTipBoxes</code> is <code>true</code> . Must be present when <code>sendTipBoxes</code> is <code>true</code> — omitting it then raises a "could not find OnDeckToHome" system error. When <code>sendTipBoxes</code> is <code>false</code> , the key is never read and omission is harmless, though authoring <code>false</code> explicitly is recommended for clarity.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

Both parameters are simple boolean flags (`true` / `false`). There are no enumerated or string-constrained values for this step.

Cross-Field Validation Rules

1. `sendTipBoxes` is always required at runtime — omitting it raises a "could not find SendTipBoxes" system error. `onDeckToHome` is required only when `sendTipBoxes` is `true` — omitting it then raises the analogous "could not find OnDeckToHome" system error; when `sendTipBoxes` is `false` the key is never read. The minimal safe authored form is `{ "sendTipBoxes": false, "onDeckToHome": false }` — authoring both keys explicitly is recommended for clarity.
2. `onDeckToHome` is **only meaningful when `sendTipBoxes` is `true`**. When `sendTipBoxes` is `false`, the value of `onDeckToHome` is ignored at runtime and omitting it is harmless. The UI enforces this by disabling the "Send boxes that started on deck back to their original locations" checkbox unless "Send tip boxes to their configured final destinations" is selected.
3. The two parameters are **independent of all other steps' parameters** — Cleanup does not reference or depend on any keys from Start, Finish, or other steps.

Cross-Step Context: per-tip-box routing keys drive Cleanup's destinations

Cleanup does not carry per-tip-box routing itself; it reads the **labware properties** that the Instrument Setup step attaches to each tip box and routes accordingly. Two drop-downs on the Labware Properties dialog write those keys:

- **"When empty, send to"** → per-tip-box key `whenDone`. Populated from the LabwareSetup UI with `Home`, `Trash` (`LOC_HOME`, `LOC_TRASH`) and any positions with the `Can Discard Labware` characteristic; the special sentinel `<Home>` appears as a menu item and stores literally that value.
- **"Unload Tips Into"** → per-tip-box keys `DiscardTips` (boolean) + `DiscardTipsLocation` (position; default `LOC_TRASH`).

When `sendTipBoxes: true`, Cleanup walks every tip box on the deck and routes each one based on where it started: `onDeckToHome: false` routes only tip boxes that started off-deck (leaving on-deck tip boxes in place), and `onDeckToHome: true` routes those plus on-deck tip boxes back to their `whenDone` destinations.

Storage Setup + Cytomat interplay. When a Storage Setup step supplies tip boxes from a Cytomat and Cleanup runs with `sendTipBoxes: true`, those tip boxes are returned via the storage device rather than the deck (tips retrieved from the Cytomat in the setup case are returned via that same path in the Cleanup+ `onDeckToHome` case). This is a property of the deck's storage-aware routing, not of a Cleanup-specific key.

Structural Context

The Cleanup step is a **leaf** step with the following structural constraints:

- **Does not support `subSteps`** — never include a `subSteps` array on a Cleanup step.
- **May appear anywhere in the method body** — between the Start and Finish boundary steps, at the method root level or nested inside container steps (Group, Loop, If, etc.).
- **May appear inside a Select Tips group** — when nested inside a Select Tips container, the step unloads tips to the configured unload location of the source tip box rather than the default disposal behavior.
- **Multiple Cleanup steps are permitted** in a single method if needed (e.g., cleanup between different pipetting phases).

Canonical Examples

Minimal Cleanup (tips only — the most common case)

Disposes of all tips from both pods. Tip boxes are left in their current positions.

```
{
  "stepType": "Cleanup",
  "parameters": {
    "sendTipBoxes": false,
    "onDeckToHome": false
  }
}
```


`sendTipBoxes` is required at enqueue — omitting it raises a "could not find SendTipBoxes" system error. `onDeckToHome` is required only when `sendTipBoxes: true`; omitting it when `sendTipBoxes: false` (as in this example) is harmless, though authoring it explicitly is recommended.

Cleanup with tip boxes sent to configured destinations

Disposes of tips and sends tip boxes that originated off-deck (or are configured to go to trash) to their final destinations. On-deck tip boxes remain in place.

```
{
  "stepType": "Cleanup",
  "parameters": {
    "sendTipBoxes": true,
    "onDeckToHome": false
  }
}
```

Full Cleanup — tips and all tip boxes returned

Disposes of tips, sends off-deck tip boxes to their destinations, and returns on-deck tip boxes to their original starting positions. Use this when the method will run repeatedly and the deck must be fully reset between runs.

```
{
  "stepType": "Cleanup",
  "parameters": {
    "sendTipBoxes": true,
    "onDeckToHome": true
  }
}
```

Common Mistakes

- **Omitting `sendTipBoxes`** always raises a "could not find SendTipBoxes" system error. **Omitting `onDeckToHome` when `sendTipBoxes: true`** raises the analogous "could not find OnDeckToHome" system error. Omitting `onDeckToHome` when `sendTipBoxes: false` is harmless but not recommended. Author both keys explicitly for clarity.
- **Confusing Cleanup with Finish** — Cleanup resets *hardware* (tips, tip boxes). Software state (deck labware, globals) needs Finish.
- **Expecting Cleanup to clear the deck or park pods** — It only touches tips and tip boxes; deck labware state and pod parking are Finish's job.
- **Omitting Cleanup in repeated-run methods** — Without it, tips and tip boxes carry over from the previous run's ending positions.

Move Pod

Property	Value
stepType	"Move Pod"
Category	Leaf
Terminator	N/A
Compatible Hardware	Any

Behavior Summary

The Move Pod step moves a pod to a specified deck position without performing any pipetting. This step may be used to move the pod out of the way before manual operations on the deck (e.g., adding or removing labware) or before a Pause step.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	""	Name of the pod to move (e.g., "Pod1", "Pod2"). Must resolve to a pod installed on the instrument.
location	string	—	Yes	""	Deck position or labware name to move the pod over (e.g., "P3", "P13"). Must resolve to a valid deck position on the instrument. Supports expressions.
xOffset	numeric	cm	Yes	0.0	Left to right offset in centimeters from the deck position origin. Added to the resolved labware origin X. Must be present (as 0.0 if not offsetting); enqueue reads via dictionary indexer and raises a "could not find XOffset" system error when omitted.
yOffset	numeric	cm	Yes	0.0	Front to back offset in centimeters from the deck position origin. Added to the resolved labware origin Y. Must be present (as 0.0 if not offsetting); enqueue reads via dictionary indexer and raises a "could not find YOffset" system error when omitted.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Enumerated / Constrained Values

pod

The `pod` value must name one of the pods installed on the instrument. Valid values are `"Pod1"` (default for single-pod instruments) and `"Pod2"` (second pod on dual-pod Biomek i7 systems).

location

The `location` value must name a valid deck position from the instrument's deck layout. Valid position names depend on the instrument model and deck configuration — consult the installed instrument's deck editor for the actual position names (they vary by chassis model and ALP configuration). This value may also be an expression that evaluates to a valid deck position name.

xOffset , yOffset

Unconstrained numeric values in **centimeters** (not millimeters). Zero is the default and positions the pod directly over the deck position's labware origin. Non-zero offsets shift the final position by the specified amount.

Cross-Field Validation Rules

1. **Pod must be valid:** The `pod` string must resolve to a pod installed on the instrument.
 2. **Location must be valid:** The `location` string must resolve to a valid deck position on the instrument. An unresolvable location causes an error at enqueue time.
 3. **Pod and Location are both required for execution:** If either `pod` or `location` is empty at print time (the method report and printed output), the step prints a not-configured placeholder. The design-caption is left blank in that case.
 4. **Offsets are independent:** `xOffset` and `yOffset` operate independently and do not influence each other. Both keys must be present at enqueue; author `0.0` explicitly when not offsetting (`0.0` is the conventional value, not a runtime default — see the Parameters table).
 5. **Labware alignment is automatic:** When labware is present at the target location, positional adjustment is computed internally from labware class geometry. The user-supplied offsets are added on top of this automatic alignment — they do not replace it.
 6. **Resolved destination must be within the pod's travel range:** The final X/Y (deck position origin plus `xOffset` / `yOffset` plus automatic alignment) must lie inside the pod's physical travel limits. A destination outside the limit fails at enqueue with a "destination outside travel range" error. Large offsets are the common trigger.
 7. **Pod moves are path-planned at runtime:** The approach is routed by the runtime path planner, which can fail even for an in-range destination when no collision-free path exists.
-

Structural Context

This step is a **leaf** and does not support `subSteps`. Never include a `subSteps` array on a Move Pod step.

The Move Pod step may appear anywhere in the body of a method (between Start and Finish).

Canonical Examples

Minimal Move Pod (defaults — most common case)

Move Pod1 to deck position P3 with no offset:

```
{
  "stepType": "Move Pod",
  "parameters": {
    "pod": "Pod1",
    "location": "P3",
    "xOffset": 0.0,
    "yOffset": 0.0
  }
}
```

Move Pod with offsets

Move Pod2 to deck position P13, shifted 1.5 cm in X and -0.5 cm in Y:

```
{
  "stepType": "Move Pod",
  "parameters": {
    "pod": "Pod2",
    "location": "P13",
    "xOffset": 1.5,
    "yOffset": -0.5
  }
}
```

Move Pod before a Pause step

A common pattern: move the pod out of the way, then pause for manual intervention:

```
[
  {
    "stepType": "Move Pod",
    "parameters": {
      "pod": "Pod1",
      "location": "P1",
      "xOffset": 0.0,
      "yOffset": 0.0
    }
  },
  {
    "stepType": "Pause",
    "parameters": {
      "location": "",
      "message": "Add reagent plate to P12, then click Resume.",
      "mode": "PromptedGlobal",
      "time": "0"
    }
  }
]
```

Common Mistakes

- **Omitting xOffset or yOffset** — Both are read via a throwing indexer; missing keys throw a "could not find XOffset/YOffset" system error at enqueue. Use `0.0` when not offsetting.
- **Confusing Move Pod with Move Labware** — Move Pod repositions the pod; no labware is touched. Move Labware physically transfers labware using a gripper.
- **Assuming offsets replace labware alignment** — `xOffset / yOffset` are *added on top of* the automatic labware-origin and well-alignment adjustments, not substituted for them.
- **Offsets that push the pod past its travel limit** — Large offsets or corner destinations fail at enqueue with a "destination outside travel range" error.
- **Forgetting Move Pod before a Pause that requires deck access** — Without it, the pod may block the area the user needs to reach.

Hold Labware

Property	Value
stepType	"Hold Labware"
Category	Free-form container
Terminator	"End" (caption "End Holding Labware" , helpContextID 141)
Compatible Hardware	i5, i7

Behavior Summary

The Hold Labware step picks up labware from a source deck position using a gripper attached to a named pod, holds the labware in the gripper while all child steps execute, then places the labware back at the original source position. The step supports three stacking modes: pick up a specific number of pieces from the top of the stack (`depth` bound), pick up all but the bottom piece (`moveEntireStack` false and `depth` unbound), or pick up the entire stack (`moveEntireStack` true). Gripper offset overrides (`gripperXOffset` , `gripperYOffset` , `gripperZOffset` , `squeeze` , `unsqueeze`) may be specified at the step level to override position- and labware-class-level gripper information. Typical use cases include de-lidding a plate to pipette to it and then replacing the lid, or lifting labware from a stack to access the bottom item.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	""	Name of the pod whose gripper will hold the labware (e.g., "Pod1"). The pod must have an attached gripper. Supports expressions.
source	string	—	Yes	""	Deck position name from which the labware is picked up and to which it is returned (e.g., "P5" , "TL3"). Must contain labware at runtime. Supports expressions.
gripSide	string	—	Yes	(unbound)	Side of the gripper to use: "A1 near" or "A1 away from" . Must be bound to a valid value; an empty or missing value raises an error. See the Enumerated / Constrained Values section. Supports expressions.
depth	string	count	No	"1"	Number of labware pieces to pick up from the top of the stack. Evaluated as an expression to an integer. Ignored when <code>moveEntireStack</code> is true . Engine tolerates omission (falls back to <code>stackDepth - 1</code>).
moveEntireStack	boolean	—	No	false	When true , picks up the entire stack regardless of <code>depth</code> . When false and <code>depth</code> is unbound, picks up all but the bottom piece (requires stack depth > 1). Engine tolerates omission (default false).
gripperXOffset	numeric	cm	No	(unbound)	Gripper X-axis offset override. Overrides position/labware-class grip offset when bound. Same unit convention as Move Labware and Move Pod.
gripperYOffset	numeric	cm	No	(unbound)	Gripper Y-axis offset override.
gripperZOffset	numeric	cm	No	(unbound)	Gripper Z-axis offset override.
squeeze	numeric	cm	No	(unbound)	Gripper squeeze (inward clamp distance) override.
unsqueeze	numeric	cm	No	(unbound)	Gripper unsqueeze (outward release distance) override.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

`gripSide`

Value	Meaning
"A1 near"	The gripper approaches at the deck position's nominal rotation; the labware's A1 corner faces the near (reference) side.
"A1 away from"	The gripper approaches rotated 180° (about the vertical axis) from "A1 near" , so the A1 corner faces the opposite side. Same grip geometry — only the orientation flips.

Both values are *normal* grips: the gripper fingers close across the labware's **Y span**. (The wide-grip orientations, which close across the X span at a 90°/270° rotation, exist in the underlying enum but are **not** offered by Hold Labware — only these two values are valid here.)

The comparison is case-insensitive, so "a1 near" and "A1 Near" are accepted. A bound non-empty value that matches neither "A1 near" nor "A1 away from" raises an "invalid grip side" error. Both an unbound (missing) value and a bound empty string raise a "missing grip side" error; the step cannot distinguish the two cases.

Stacking mode (derived from `moveEntireStack` and `depth`)

<code>moveEntireStack</code>	<code>depth</code> bound?	Behavior
true	(ignored)	Picks up the entire stack.
false	Yes	Picks up exactly <code>depth</code> pieces from the top of the stack.
false	No (unbound)	Picks up all but the bottom piece (top stack-count minus one). Requires stack depth > 1.

Cross-Field Validation Rules

- Pod must have a gripper:** The named `pod` must be installed on the instrument and must have a gripper. An empty pod name raises a "pod must be specified" error; a non-existent pod raises a "pod not found" error. A pod that exists but has no gripper attached surfaces as a generic COM automation error from the gripper pick call.
- Source must be specified and contain labware:** `source` must be a non-empty string that resolves to a deck position containing at least one labware item. Empty string, non-existent position, and empty position each raise a distinct error.
- Grip side is mandatory:** `gripSide` must be bound and evaluate to "A1 near" or "A1 away from" . A missing value raises a "missing grip side" error; an unrecognized value raises an "invalid grip side" error.
- Leave-bottom requires stack depth > 1:** When `moveEntireStack` is false and `depth` is unbound, the computed depth is the stack height minus one. If the source has only one piece of labware, this yields zero and raises a "cannot leave bottom labware" error.
- moveEntireStack overrides depth:** When `moveEntireStack` is true , the `depth` key is entirely ignored and the full stack depth is used.
- Depth must be evaluable and not exceed the stack:** When `depth` is bound, its value is passed through expression evaluation. A value that cannot be evaluated to an integer is reported as a "bad labware depth" error. Requesting more than the available stack height is reported as a source-depth-vs-stack-height mismatch, and a resolved depth that reaches no labware raises an "unable to retrieve labware" error.
- Gripper-info override must exist on labware class:** If the source position has a `GripperInfoOverride` value, the labware class being gripped must have a matching key under its `GripperInfoOverrides` dictionary. A missing override raises a "missing movement information" error.

8. **Step-parameter offsets take final precedence:** If any of the five offset keys (`gripperXOffset` , `gripperYOffset` , `gripperZOffset` , `squeeze` , `unsqueeze`) is bound in the step's parameters, it overrides the corresponding value from position/labware-class gripper-info overrides.

Structural Context

The Hold Labware step is a **container** with the following structural rules:

- **Has `subSteps`:** The step must contain a `subSteps` array.
- **Terminator:** The last element of `subSteps` must be an "End" step.
- **Allowed children:** Any step type may appear between the container's own parameters and the terminating End step. Typical children include liquid-handling steps (Span-8 Aspirate/Dispense, Multichannel Aspirate/Dispense) or other container steps. The step imposes no restrictions on child types, but it is not possible to have a Move Labware child step that uses the pod that is holding labware, as labware is already in the gripper.
- **Auto-release on End:** The gripper `Place` call occurs after all child steps have been enqueued, not in the End step itself. The End step is a structural marker only; the Hold Labware container's `Enqueue` method handles both the `Pick` (before children) and `Place` (after children).
- **A structural validation rule:** Per the JSON Import Structural Validation Spec, a Hold Labware step that does not end with an End step as its last child must be rejected.

Canonical Examples

Example 1: Hold a lid while dispensing into the lidded labware (i5 Span-8)

```
{
  "stepType": "Hold Labware",
  "parameters": {
    "depth": "1",
    "gripSide": "A1 near",
    "moveEntireStack": false,
    "pod": "Pod1",
    "source": "P2"
  },
  "subSteps": [
    {
      "stepType": "Span-8 Dispense",
      "parameters": {
        "amount": "50",
        "aspirate": false,
        "autoSelectPrototype": true,
        "customHeight": false,
        "discardExcess": false,
        "dynamic?": true,
        "emptyTips": false,
        "firstWell": 1,
        "firstWellExpression": "",
        "height": 1.5,
        "heightFrom": 1,
        "liquidType": "Tip Contents",
        "mandrelExpression": "",
        "operation": "Dispense",
        "overrideHeight": false,
        "pod": "Pod1",
        "podType": "Span8",
        "prototype": "S8 1000 Medium",
        "spacing": "1",
        "useExpression": false,
        "useProbes": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [
            true,
            true,
            true,
            true,
            true,
            true
          ]
        }
      }
    }
  ]
}
```

```

        true,
        true
    ]
  },
  "useWellExpression": false,
  "what": "BCFlat96",
  "where": "P2"
}
},
{
  "stepType": "End",
  "parameters": {
    "caption": "End Holding Labware",
    "helpContextID": 141
  }
}
]
}

```

Example 3: Minimal — empty container (no child work steps)

```

{
  "stepType": "Hold Labware",
  "parameters": {
    "depth": "1",
    "gripSide": "A1 near",
    "moveEntireStack": false,
    "pod": "Pod1",
    "source": "P7"
  },
  "subSteps": [
    {
      "stepType": "End",
      "parameters": {
        "caption": "End Holding Labware",
        "helpContextID": 141
      }
    }
  ]
}

```

Common Mistakes

- **Expecting a target / destination parameter:** Hold Labware always returns labware to the same source . Use a separate Move Labware step for relocation.
- **depth as a numeric type instead of a string:** The default is "1" (string), and UI round-trip depends on the string form; the runtime tolerates a number but the method won't round-trip cleanly. Always author as a string (e.g., "1" , "2").
- **Setting both moveEntireStack: true and a bound depth:** moveEntireStack wins unconditionally — depth is not consulted. Use one mode.
- **Binding an offset key to 0 instead of omitting it:** 0 is a bound override that displaces position/labware-class defaults; omit the key to inherit.
- **Leave-bottom mode with a single-item stack:** moveEntireStack: false + unbound depth computes (stack height minus one) = 0, raising a "cannot leave bottom labware" error. Use moveEntireStack: true for a single item.

End

Property	Value
stepType	"End"
Category	Boundary (terminator)
Terminator	N/A — it IS the terminator
Compatible Hardware	All (no hardware restriction — inherits default)

Behavior Summary

The End step is a purely structural marker that terminates every standard container step in a Biomek method. It performs no action at runtime. Its sole purpose is to serve as the required last child of container steps such as Group, Loop, Let, Worklist, Scripted Let, Just In Time, Hold Labware, Named Container (used by If), and Define Procedure. The one container family that does **not** use End is Multichannel Select Tips, which instead uses the specialized "Multichannel Select Tips End" terminator.

Parameters Reference Table

The End step reads nothing from the step's parameters. However, parent containers write keys to the End step's parameters, and these keys appear in serialized JSON. They should be preserved for round-trip fidelity but never modified when authoring or editing a method by hand.

The End step has no step-specific parameters. Universal base keys written by the parent container are caption and helpContextID ; other base keys (dynamic? , stepUI , ...) are defined in Base Keys & Serialization Conventions but are not written to End step parameters.

Note: When constructing a method from scratch, either preserve whatever caption the parent container sets (see the Structural Context section table) or omit base keys entirely.

Enumerated / Constrained Values

None. The End step has no parameters and therefore no enumerated or constrained values.

Cross-Field Validation Rules

None. Because the End step carries no step-specific dictionary keys, there are no cross-field dependencies to validate.

Structural Context

The End step is the **default container terminator** in the Biomek step tree.

Containers That Require End as Their Last Child

Every container in this table must have an "End" step as the final element of its subSteps array (structural rule). The override caption shown is the display label set by each container when it creates the End step.

Container stepType	Override Caption
"Group"	"End Group"
"Loop"	"End Loop"
"Let"	"End Let"
"Worklist"	"End Worklist"
"Scripted Let"	"End Scripted Let"
"Just In Time"	"End Just In Time"
"Hold Labware"	"End Holding Labware"
"Define Procedure"	"End Procedure"
"Named Container"	Inherits default "End"

Named Container and If

The "If" step itself does not directly contain an End step. Instead, it contains exactly two "Named Container" children (Then and Else branches), and **each** of those Named Container children must have an "End" as its last child.

The Exception: Multichannel Select Tips

"Multichannel Select Tips" does **not** use the generic End step. It uses "Multichannel Select Tips End" as its terminator instead. This is the only container family with a specialized terminator.

Structural Rules

- A bottom-anchor step (End) must be at the last index in its sibling list.
- Every container in the fixed-terminator table must have its expected terminator as the last child.
- The End step is a leaf — it must never have a subSteps array with elements.

Canonical Examples

End as Last Child of a Group

```
{
  "stepType": "Group",
  "parameters": {},
  "subSteps": [
    { "stepType": "Span-8 Aspirate", "parameters": {} },
    { "stepType": "Span-8 Dispense", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Group" } }
  ]
}
```

End as Last Child of a Loop

```
{
  "stepType": "Loop",
  "parameters": {},
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Loop" } }
  ]
}
```

End in an If Step (Both Branches)

Each Named Container branch requires its own End step:

```
{
  "stepType": "If",
  "parameters": {},
  "subSteps": [
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Then" },
      "subSteps": [
        { "stepType": "Transfer", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    },
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Else" },
      "subSteps": [
        { "stepType": "Pause", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    }
  ]
}
```

End in a Nested Container (Loop Inside Group)

End is required at every level of nesting:

```
{
  "stepType": "Group",
  "parameters": {},
  "subSteps": [
    {
      "stepType": "Loop",
      "parameters": {},
      "subSteps": [
        { "stepType": "Transfer", "parameters": {} },
        { "stepType": "End", "parameters": { "caption": "End Loop" } }
      ]
    },
    { "stepType": "End", "parameters": { "caption": "End Group" } }
  ]
}
```

Empty Container (End as Only Child)

A newly created container with no body steps yet — the End step is the sole child:

```
{
  "stepType": "Let",
  "parameters": {},
  "subSteps": [
    { "stepType": "End", "parameters": { "caption": "End Let" } }
  ]
}
```

Common Mistakes

- **Using the override caption as the `stepType`** — the caption ("End Group" , "End Loop") is display metadata set by the container; `stepType` is always "End" .
- **Using generic "End" inside Multichannel Select Tips** — that container's terminator is "Multichannel Select Tips End" , not the standard End.
- **Forgetting End in one branch of an If** — Then and Else are independent Named Containers; each needs its own terminator.
- **Confusing End with Finish** — End terminates containers; the method root uses "Finish" . They are not interchangeable.
- **Placing body steps after End** — anything after End is outside the container as far as insert/move logic is concerned.

Device Action

Property	Value
stepType	"Device Action"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7

Behavior Summary

The Device Action step sends a command to a digital I/O device or generic script device attached to the pipettor. It looks up the named device in the pipettor's device collection, then dispatches the command string. For digital I/O devices it dispatches the command without parameters. For generic script devices it dispatches the command with a required parameter array (`parameters` key must be present in the step node). This is the general-purpose step for controlling non-SILAS peripheral devices (e.g., control boxes, orbital shakers, custom hardware).

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>device</code>	<code>string</code>	—	Yes	<code>""</code>	Device name. Must exist in the pipettor's device collection. See Deck Layouts Format (in <i>Biomek Instrument & Project Reference</i>) §"The <code>device</code> block" for where to find the valid names on your instrument.
<code>command</code>	<code>string</code>	—	Yes	<code>""</code>	Command to send to the device.
<code>parameters</code>	<code>array of strings</code>	—	Conditional (when device is a generic script device)	<i>(not bound)</i>	Parameter array passed to the device command. Omission raises a "could not find Parameters" system error when that branch runs. Not read for digital I/O devices.
<code>slack</code>	numeric	seconds	No	Not present	Inert on this step. <code>slack</code> on a Device Action node is stray persistence with no runtime effect — safe to omit.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

The Device Action step has **no enumerated or string-constrained parameters**. `device` , `command` , and (for generic script devices) `parameters` are free-form strings whose accepted values are entirely defined by the target device driver, not by this step. Consult the specific device's documentation for the command vocabulary and any parameter-array conventions it expects.

The two supported device categories are dispatched internally and are not authorable values:

Device category	Behavior
Digital I/O device	Command dispatched; <code>parameters</code> key is ignored.
Generic script device	Command dispatched with the parameter array; <code>parameters</code> key is required.

Cross-Field Validation Rules

1. `device` **must not** be empty.
2. `device` **must** exist in the pipettor's device collection.
3. `command` **must not** be empty.
4. The device must be either a digital I/O device or a generic script device. If it is neither, enqueue fails with a "device not supported" error.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Simple device command:

```
{
  "stepType": "Device Action",
  "parameters": {
    "device": "ControlBox",
    "command": "TurnOn"
  }
}
```

With parameters (generic script device):

```
{
  "stepType": "Device Action",
  "parameters": {
    "device": "OrbitalShaker",
    "command": "SetSpeed",
    "parameters": ["500", "60"]
  }
}
```

Note: The `parameters` key here is a step-specific key that sits *inside* the universal `parameters` envelope object — the same name appears at two different nesting levels in the JSON.

Common Mistakes

- **Using on i3:** This step is compatible only with i5 and i7.
- **Omitting `parameters` for a generic script device:** For generic script device targets the key is read unconditionally in that enqueue branch and throws when missing; conversely, for digital I/O device targets the key is never read (so leaving it in is harmless but stray).
- **Authoring `slack` on this step:** see `$Parameters` row `slack`. Omit it; the key is harmless but has no runtime effect on this step.

INHECO Peltier

Property	Value
stepType	"INHECO Peltier"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7

Behavior Summary

The INHECO Peltier step controls INHECO thermal and shaking devices on the Biomek deck. It sends commands to a specific deck position's associated INHECO device for temperature incubation, shaking, or initialization.

Devices can be static peltiers (temperature only) or shaking peltiers (temperature + orbital shaking). SingleTEC devices have different parameter ranges than older models.

Parameters Reference Table

Several keys on this step carry **literal internal spaces** — `use Temp`, `temp Set Point`, `shake RPM`, `continue Shaking`, `deLuxe Shake` — and must be written exactly that way, spaces included. The reflexive camelCase forms (`useTemp`, `tempSetPoint`, ...) are not recognized. See the key-name casing rule in *Base Keys & Serialization Conventions*.

Core Parameters

Key	Type	Units	Required	Default	Description
position	expression-capable string	—	Yes	""	Deck position name. Must have an associated INHECO device — see Deck Layouts Format (in <i>Biomek Instrument & Project Reference</i>) §"The device block" to identify which positions on your instrument have one.
command	expression-capable string	—	Yes	"Initialize"	Command to execute. See enumeration below.

Temperature Parameters (for "Incubate" command)

Key	Type	Units	Required	Default	Description
use Temp	boolean	—	No	false	If true, set temperature to target. Engine tolerates omission (default False).
temp Set Point	expression-capable string	°C	Conditional	""	Target temperature. Required when use Temp is true. Must be numeric with ≤1 decimal place, within device range.
total Time	expression-capable string	seconds	Conditional	""	Duration of incubation. Required when Command="Incubate"; ignored otherwise. Must be integer within device range.

Shake Parameters (for shaker devices with "Incubate" or "Start Shaking")

Key	Type	Units	Required	Default	Description
use Shake	boolean	—	No	true	If true, shake during incubation. Only applies to shaker devices. Engine tolerates omission (default True).

Key	Type	Units	Required	Default	Description
shake RPM	expression-capable string	RPM	Conditional	""	Shake speed. Required when shaking. Must be within device RPM range.
shake Style	expression-capable string	—	Conditional	""	Shake pattern. Required when shaking in single-style mode.
continue Shaking	boolean	—	Yes (when shaking)	true	If true, shaking continues after incubation until device is opened. Required whenever shaking is active (Command="Start Shaking", or Command="Incubate" with use Shake true).
deluxe Shake	boolean	—	No	false	If true, enable dual-phase shaking with two alternating styles. Engine tolerates omission.
shake Enabled	boolean	—	No	false	UI-only flag reflecting whether the device supports shaking. Engine tolerates omission. Preserve for round-trip fidelity.

Dual-Phase Shake Parameters (when deluxe Shake is true)

Key	Type	Units	Required	Default	Description
shake Style1	expression-capable string	—	Yes	""	First shake pattern.
shake Time1	expression-capable string	seconds	Yes	""	Duration of first shake phase.
shake Style2	expression-capable string	—	Yes	""	Second shake pattern.
shake Time2	expression-capable string	seconds	Yes	""	Duration of second shake phase.

Universal base keys (caption , dynamic? , stepUI , ...) apply to this step but are omitted here — see Base Keys & Serialization Conventions.

Enumerated / Constrained Values

Command Values

Value	Applies To	Description
"Initialize"	All	Initialize the device
"Incubate"	All	Temperature control with optional shaking
"Start Shaking"	Shaker only	Begin continuous shaking
"Stop Shaking"	Shaker only	Stop shaking

Shake Style Values

Value
"Orbital (counter-clockwise)"
"Orbital (clockwise)"
"Diagonal (NE to SW)"
"Diagonal (NW to SE)"
"Vertical"
"Horizontal"

Device Type Ranges

Parameter	Older Models	SingleTEC
Min RPM	100	configurable
Max RPM	2000	configurable
Min Shake Time	5 s	1 s
Max Shake Time	100 s	99999 s
Min Total Time	1 s	1 s
Max Total Time	999999 s	99999 s
Temp Range	device-specific	device-specific

Cross-Field Validation Rules

1. `position` **must** exist on the deck and have an associated INHECO device.
2. `command` **must** be valid for the device type (static vs. shaker).
3. For `"Incubate"`: `total Time` must be an integer within the device's time range.
4. When `use Temp` is `true`: `temp Set Point` must be numeric (≤ 1 decimal), within device temp range.
5. When shaking: `shake RPM` must be within [minRPM, maxRPM].
6. When shaking in single mode: `shake Style` must be a valid style string.
7. When `deluxe Shake` is `true`: all four dual-phase parameters required and within ranges.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Incubate at 37°C for 5 minutes:

```
{
  "stepType": "INHECO Peltier",
  "parameters": {
    "position": "P7",
    "command": "Incubate",
    "use Temp": true,
    "temp Set Point": "37.0",
    "total Time": "300"
  }
}
```

Incubate with shaking:

```
{
  "stepType": "INHECO Peltier",
  "parameters": {
    "position": "P7",
    "command": "Incubate",
    "use Temp": true,
    "temp Set Point": "25.0",
    "total Time": "600",
    "use Shake": true,
    "shake RPM": "500",
    "shake Style": "Orbital (counter-clockwise)"
  }
}
```

Initialize device:

```
{
  "stepType": "INHECO Peltier",
  "parameters": {
```



```
"position": "P7",
"command": "Initialize"
}
```

Common Mistakes

- **Authoring on an i3 method** — the step is registered for i5/i7 hardware and is not available on an i3 method.
- **Shake commands on a static peltier** — "Start Shaking" / "Stop Shaking" (and shake parameters under "Incubate") require a shaker device; ranges and validity flip with device model.
- **Assuming older-model ranges on a SingleTEC** — RPM and Total Time bounds differ substantially (see \$Device Type Ranges); validate against the actual device.
- **temp Set Point with more than one decimal place** — numeric validation limits it to ≤ 1 decimal (e.g., "37.05" is rejected).
- **Setting deluxe Shake without all four dual-phase keys** — shake Style1, shake Time1, shake Style2, shake Time2 are all required together.
- **Omitting continue Shaking whenever shaking is active** — with "Start Shaking", or "Incubate" while use Shake is true, this key must be present. Unlike use Temp / use Shake / deluxe Shake, omission is not tolerated: it errors at enqueue rather than defaulting to true.

Transfer

Property	Value
stepType	"Transfer"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7, i3

Behavior Summary

The Transfer step moves liquid from a single source location to one or more destination locations in a single consolidated operation. At enqueue time, it verifies that all source wells and destination wells are defined with proper selections, resolves both source and destination labware positions and classes, and builds one or more transfer groups optimized for efficient pipetting. The step combines tip loading, aspiration from the source(s), dispense to the destination(s), and tip unload/disposal into a single logical unit.

The `stop` condition (`"Sources"` , `"Destinations"` , or `"Either"`) determines how many sources must be exhausted versus destinations filled before the step completes. The step supports multiple repetitions (`repeats` , `repeatsByVolume`), replicates (multiple times through the same sources and destinations), and large-volume splitting (when volume exceeds tip capacity, the step automatically breaks it into multiple aspirate-dispense cycles).

Tip handling is configurable: tips can be loaded fresh, reused from a previous step (`useCurrentTips`), changed between source groups, changed between destination groups, or washed between operations.

Note that the Transfer step and the Combine step share logic and keys. This document describes the necessary keys and their values for a Transfer step (one source, multiple destinations). To author a Combine step (multiple sources, one destination), see Combine.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>type</code>	string	—	No	"Transfer"	Do not set this key on a Transfer step. Optional; if present, must be <code>"Transfer"</code> .
<code>pod</code>	string	—	Yes	""	Name of the pipetting pod (e.g., <code>"Pod1"</code>). Must resolve at runtime to a valid pod (Fixed-8, Span-8, or Multichannel).
<code>podType</code>	string	—	No	derived from <code>pod</code>	Pod-family label (<code>"Fixed8"</code> / <code>"Span8"</code> / <code>"MC"</code> — see <code>\$podType</code>). Keep it in sync with the resolved pod. Which pod family actually runs is resolved from the named <code>pod</code> , so <code>podType</code> does not select the family; but it must still be correct, because it selects the pod-family wording of the printed method (mandrel-vs-section labels, Span-8 lines, tip count, Combine/Transfer header) — a stale value prints a method that contradicts the pod.
<code>span8</code>	boolean	—	No	(absent)	Span-8 pod-family discriminator. Keep it in sync with the resolved pod. Set to <code>true</code> for a Span-8 pod and <code>false</code> for any other pod.

Key	Type	Units	Required	Default	Description
<code>items</code>	array (object[])	—	Yes	<code>[]</code>	Array of source and destination item dictionaries. Each item describes a labware location, well selection pattern, volume, and pipetting parameters. Position is what assigns the role: on a Transfer, the first item is the source and every subsequent item is a destination — the array must contain at least one of each (so at least two entries). The item-level <code>source</code> key is written positionally at enqueue and any authored value is overwritten (see the <code>source</code> row in Item-Level Keys). See Item-Level Keys subsection below.
<code>stop</code>	string	—	No	"Destinations" (Fixed-8) / "Sources" (Span-8/Multichannel)	Stopping condition: "Sources" (exhaust all sources), "Destinations" (fill all destinations), or "Either" (stop when either is exhausted). Controls iteration over wells in Transfer assignment. The fallback default differs by pod family. Author it explicitly.
<code>repeats</code>	string	—	No	"1"	This key is used for configuring multidispense behavior. The default is to allow a single dispense for each aspirate, represented by a value of 1 for this key. If <code>repeatsByVolume</code> is <code>true</code> , this is the maximum volume in microliters that can be aspirated at once for repeated dispensing (i.e. aspirate once, dispense multiple times). If <code>repeatsByVolume</code> is <code>false</code> , this is the maximum number of dispenses allowed for a single aspirate (minimum 1). Multidispensing is not a common usage and requires specialized pipetting techniques. This should typically be authored and set to "1".
<code>repeatsByVolume</code>	boolean	—	No	<code>false</code>	When <code>true</code> , <code>repeats</code> specifies the maximum volume to aspirate for multidispensing; when <code>false</code> , <code>repeats</code> specifies the maximum number of dispenses allowed per aspirate.
<code>replicates</code>	string	—	No	"1"	Number replicates to create for each source well. Expression-capable. Minimum 1. This key impacts the mapping from source wells to destination wells, ensuring that each source well is transferred to the given number of destination wells before moving on to the next source well.
<code>useJIT</code>	boolean	—	No	<code>true</code>	When <code>true</code> , aspirate and dispense operations are bundled into a single Just-In-Time block to ensure no other actions occur in between the aspirate and dispense.
<code>useFixedTips</code>	boolean	—	No	<code>false</code>	One of the Span-8 tip-mode keys. When <code>true</code> , only fixed-tip probes are used and <code>useProbes</code> is ignored (tip physicality is inferred from the selected probes). Do not also set <code>useDisposableTips</code> . Fixed-8 ignores this key.
<code>useDisposableTips</code>	boolean	—	No	<code>false</code>	One of the Span-8 tip-mode keys. When <code>true</code> , all non-fixed (disposable-tip) probes are used and <code>useProbes</code> is ignored. Do not also set <code>useFixedTips</code> . Fixed-8 ignores this key.
<code>useMandrelSelection</code>	boolean	—	No	<code>true</code>	Inert UI-state (engine does not read it; it persists the editor's selection mode). Execution branches on

Key	Type	Units	Required	Default	Description
					<code>mandrelExpression</code> / <code>useProbes</code> (<code>UseExpression</code>), <code>useFixedTips</code> , and <code>useDisposableTips</code> .
<code>useProbes</code>	<code>comArray</code> (<code>boolean[8]</code>)	—	Conditional	All <code>true</code>	Array of eight booleans indicating which probes (1–8) are active on Span-8 pods. Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]}</code> . Required (with at least one <code>true</code> element) when <code>useFixedTips</code> , <code>useDisposableTips</code> , and <code>useExpression</code> are all <code>false</code> — i.e., SELECTION mode sourced from <code>useProbes</code> . In FIXED (<code>useFixedTips: true</code>) or DISPOSABLE (<code>useDisposableTips: true</code>) mode, or when <code>useExpression</code> is <code>true</code> , <code>useProbes</code> is ignored. Fixed-8 ignores this key entirely — Fixed-8 tip count is controlled by <code>numberOfTips</code> . Multichannel also ignores this key — MC head geometry (96 or 384 channels) is not per-probe selectable at the step level.
<code>useExpression</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , probe selection is determined by <code>mandrelExpression</code> instead of <code>useProbes</code> .
<code>mandrelExpression</code>	<code>string</code>	—	Conditional	<code>""</code>	Expression that evaluates to the probe selection at runtime (e.g., <code>"=[1,2,3,4,5,6,7,8]"</code> for Span-8). Required when <code>useExpression</code> is <code>true</code> .
<code>numberOfTips</code>	<code>string</code>	—	No	<code>"8"</code>	For Fixed-8 pods: the number of tips to use (1–8 as a string, or an expression). Expression-capable. Not used on Span-8 or Multichannel pods, although the value if present must be between 1 and 8.
<code>useCurrentTips</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , the step reuses tips already loaded on the pod from a previous step instead of loading new tips.
<code>changeTipsBetweenSources</code>	<code>boolean</code>	—	No	<code>true</code> on Fixed-8/Multichannel, <code>false</code> on Span-8	For a Multichannel or Fixed-8 pod, when <code>true</code> , tips are changed (or washed) between aspirations from different source locations. For a Span-8 pod, whether or not tips are changed/washed is the result of a logical OR of <code>changeTipsBetweenSources</code> and <code>changeTipsBetweenDestinations</code> . When authoring a Span-8 Transfer step, leave this set to <code>false</code> and utilize <code>changeTipsBetweenDests</code> .
<code>changeTipsBetweenDests</code>	<code>boolean</code>	—	No	<code>false</code> on Fixed-8/Multichannel, <code>true</code> on Span-8	When <code>true</code> , tips are changed (or washed) between after dispensing. While this key may suggest it applies when destinations are different, it actually should be interpreted as "change (or wash) tips after dispensing".
<code>leaveTipsOn</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , tips are left on the pod after the step completes; when <code>false</code> , tips are unloaded.
<code>splitVolume</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , a transfer volume larger than the tip capacity is split into several aspirate-dispense cycles.
<code>splitVolumeCleaning</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> and <code>splitVolume</code> is <code>true</code> , tips are cleaned between each split cycle (if <code>changeTipsBetweenSources</code> or <code>changeTipsBetweenDests</code> is <code>true</code>).
<code>tipLocation</code>	<code>string</code>	—	Conditional	<code>""</code>	Tip source used when loading fresh tips — a tip-box labware name , a deck position (e.g. <code>"P1"</code>), or a tip

Key	Type	Units	Required	Default	Description
					labware class (e.g. "BC230"). Required when loading new disposable tips (i.e., when <code>useCurrentTips</code> is <code>false</code> and <code>useFixedTips</code> is <code>false</code>). Class-name resolution only matches a box whose instance name is empty or equal to the class name — when the tip box on deck carries a distinct <code>properties.name</code> (renamed instance), author the box's instance name or its deck position.
<code>unloadLocation</code>	string	—	No	"<where they came from>"	Where to send tips after the transfer (Fixed-8 only ; ignored when <code>leaveTipsOn</code> is <code>true</code>). "<where they came from>" (default) returns tips to their originating box; "<Any Trash>" discards them to a deck position carrying the "Can Discard Tips" characteristic; or give an explicit deck-position / labware name. This is the tip- <i>unload</i> vocabulary, distinct from a tip box's <code>DiscardTipsLocation</code> routing key — "<TipBox>" is not valid here. For Multichannel and Span-8 pods, this key is not used, and tips are unloaded to the location configured in the Instrument Setup step.
<code>wash</code>	boolean	—	No	<code>false</code>	For Multichannel and Span-8 pods: when <code>true</code> , an active wash (using specified solvent and wash technique) is performed between tip handling events. Mutually exclusive with <code>span8Wash</code> . Fixed-8 pods do not support washing; do not use this key for Fixed-8 pods.
<code>span8Wash</code>	boolean	—	No	<code>false</code>	For Span-8 pods: when <code>true</code> , a passive wash is performed (uses <code>span8WashVolume</code> and <code>span8WasteVolume</code>). Mutually exclusive with <code>wash</code> .
<code>washVolume</code>	string	μL or %	Conditional	""	Volume of solvent to use in active wash, or percentage of the previously used tip capacity (e.g., "110%"). Expression-capable. Required when <code>wash</code> is <code>true</code> .
<code>span8WashVolume</code>	string	mL	No	"2"	Volume of passive wash solvent for Span-8 pods (e.g., "2"), in mL . Expression-capable. Default 2 mL.
<code>span8WasteVolume</code>	string	mL	No	"1"	Volume of waste liquid discarded after passive wash for Span-8 pods (e.g., "1"), in mL . Expression-capable. Default 1 mL.
<code>washCycles</code>	string	—	Conditional	""	Number of wash cycles to perform (e.g., "3"). Expression-capable. Required when <code>wash</code> is <code>true</code> .
<code>solvent</code>	string	—	Conditional	"Water"	Name of the washing solvent (e.g., "Water" , "Ethanol"). Required when <code>wash</code> is <code>true</code> . When configuring an active wash, the <code>solvent</code> must match the liquid type configured for an active wash station ALP on the deck.
<code>autoSelectActiveWashTechnique</code>	boolean	—	No	<code>false</code>	When <code>true</code> , an active wash technique is auto-selected based on liquid type and pod; when <code>false</code> , use <code>activeWashTechnique</code> (if a string) or <code>customActiveWashTechnique</code> (if an object). Tolerant read at enqueue; omission does not throw.
<code>activeWashTechnique</code>	string	—	No	""	Name of a predefined wash technique (e.g., "Standard Wash"). Ignored if

Key	Type	Units	Required	Default	Description
					<code>autoSelectActiveWashTechnique</code> is <code>true</code> or <code>customActiveWashTechnique</code> is bound.
<code>customActiveWashTechnique</code>	object	—	No	Not present	Custom wash technique definition (inline technique object). Takes precedence over <code>activeWashTechnique</code> .
<code>showTipHandlingDetails</code>	boolean	—	No	<code>false</code>	UI flag: when <code>true</code> , the tip handling summary is displayed in the method editor.
<code>showTransferDetails</code>	boolean	—	No	<code>false</code>	UI flag: when <code>true</code> , the transfer mapping summary is displayed in the method editor.
<code>wizard</code>	boolean	—	No	<code>false</code>	Internal UI flag. Not read at runtime.
<code>groupTransfer</code>	boolean	—	No	<code>false</code>	Obsolete and unused key. Do not author.

The step-level `tip/split/wash/UI` keys above (`useFixedTips`, `useDisposableTips`, `useMandrelSelection`, `useExpression`, `splitVolume`, `splitVolumeCleaning`, `activeWashTechnique`, `showTipHandlingDetails`, `showTransferDetails`, `wizard`) are persisted by the editor but stay *Optional* — omission does not throw at enqueue. However, they should be authored so that a method imported into the Biomek editor displays the appropriate values in the UI.

Universal base keys (`caption`, `defaultCaption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Item-Level Keys

Each item in the `items` array is an object with the following keys:

Key	Type	Units	Required	Default	Description
<code>source</code>	boolean	—	No (auto-set)	<code>true</code> or <code>false</code> (auto-marked)	Set automatically at enqueue based on item position — for a Transfer step, item 0 is the source and every subsequent non-ghosted item is a destination (for Combine, all items but the last are sources). Any value you author here is OVERWRITTEN, so do not set it manually. (Multi-source is a Combine step, not Transfer.)
<code>position</code>	string	—	Yes	<code>""</code>	Deck position name (e.g., <code>"P9"</code>) or labware name. Resolves to the labware to aspirate from (if source) or dispense to (if destination).
<code>labwareClass</code>	string	—	Yes	<code>""</code>	Labware class name (e.g., <code>"BCFlat96"</code> , <code>"CostarFlat384Square"</code>). Must match the class of labware at <code>position</code> .
<code>volume</code>	string	μL	Conditional	<code>""</code>	Volume to aspirate (if source) or dispense per well (if destination). Expression-capable. For Transfer, set the volume on each destination.
<code>pattern</code>	comArray (boolean[...])	—	Conditional	Empty	Boolean array of well selections (one element per well in labware). Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]}</code> . Each <code>true</code> indicates a well to be included. One of three mutually-usable well-selection modes: <code>pattern</code> , <code>dataSetPattern: true</code> + condition, or <code>useExpression: true</code> + <code>sectionExpression</code> (see Rule 12). Requirement varies by pod family: on Span-8 <code>pattern</code> is the operative well selector whenever <code>localPattern</code> is <code>true</code> (the default) and is hard-required there — it must be present and correct even when <code>useExpression: true</code> , so supply a full-length placeholder

Key	Type	Units	Required	Default	Description
					(e.g. an all-false array of the labware's well count) in expression mode. On Multichannel the key must be present no matter which selection mode you use, and its value is superseded by the resolved section selection. On Fixed-8 the operative selector is <code>selectionInfo</code> instead (see the next row); <code>pattern</code> is not consulted. A whole-plate operation is expressed as an all- <code>true</code> values array of the labware's well count (e.g., 96 <code>true</code> s for a 96-well plate — pair with the parallel <code>selectionInfo</code> on Fixed-8/Multichannel).
<code>selectionInfo</code>	comArray (subtype varies by pod)	—	Conditional	Same as pattern	Item-level well selection, kept identical to <code>pattern</code> by the editor. Must be present and correct on Fixed-8 and Multichannel: on the default boolean-mask path (<code>localPattern: true</code> , <code>useExpression: false</code> , <code>dataSetPattern: false</code>) <code>selectionInfo</code> is the operative well selector for those pod families, and a missing/invalid value fails the step with <code>Bad selection array.</code> . On Span-8 the operative selector is <code>pattern</code> (previous row), but keep <code>selectionInfo</code> in sync there too — it selects the well-selection wording of the printed method. Do not read "per-item" as implying per-well addressing on that pod; see \$Multichannel: what a Transfer item can and cannot address.
<code>operation</code>	string	—	Conditional	"Aspirate" (source) / "Dispense" (dest)	Pipetting role of the item — "Aspirate" for source items, "Dispense" for destinations. Read at enqueue when the transfer splits a volume across multiple aspirates (<code>splitVolume: true</code>): the authored item's value drives technique and split-count selection, and omitting it there hard-fails enqueue. On a plain non-splitting transfer the role is assigned positionally and a literal operation is used, so the key is not consulted there. The editor always emits it — author it to match each item's role so a splitting method still enqueues.
<code>liquidType</code>	string	—	Yes	"Well Contents" (source) or "Tip Contents" (dest)	Liquid type for technique selection: "Well Contents" (aspiration will auto-detect liquid from the deck), "Tip Contents" (for dispense, uses actual tip contents), or a named liquid type (e.g., "Serum"). Required: omitting it raises at enqueue. Real exports and the included example emit this item-level key lowercase (<code>liquidtype</code>); matching is case-insensitive, so either spelling imports correctly. Authoring tools should not depend on the exact case.
<code>prototype</code>	string	—	Yes (key must be present; may be "")	""	Name of the pipetting technique (transfer prototype). The key must be present on every item — the enqueue path reads it through an unconditional throwing accessor, so omitting it hard-fails enqueue even when <code>autoSelectPrototype</code> is <code>true</code> . On Fixed-8 it must name a real technique (auto-select is invalid there — see <code>autoSelectPrototype</code>); on Span-8/Multichannel with <code>autoSelectPrototype: true</code> the value may be empty (""), but the key still has to appear.
<code>autoSelectPrototype</code>	boolean	—	No	false	When <code>true</code> , the technique is auto-selected based on liquid type, labware, tip type, and volume — valid only for Span-8/Multichannel pods. INVALID for Fixed-8 . When <code>false</code> , <code>prototype</code> must be specified.
<code>customPrototype</code>	object	—	No	Not present	Custom technique definition (inline technique object). Takes precedence over <code>prototype</code> when present.

Key	Type	Units	Required	Default	Description
<code>overrideHeight</code>	boolean	—	Yes	<code>false</code>	When <code>true</code> , tip height is overridden using <code>height</code> and <code>heightFrom</code> instead of the technique's defaults. Required: omitting it raises at enqueue.
<code>height</code>	number or expression-capable string	mm	Yes (key must be present)	<code>0.0</code>	Tip height offset from reference point when <code>overrideHeight</code> is <code>true</code> . Positive values move the tip above the reference point; negative values move it below. For <code>heightFrom</code> = "Liquid", a value of <code>-2.0</code> places the tip 2 mm below the liquid surface (i.e., into the liquid) — this is the standard aspirate offset used by the default techniques. Expression-capable (prefix with <code>=</code>). The key must be present on every item — the enqueue path reads it through an unconditional throwing accessor — so omitting it hard-fails enqueue even when <code>overrideHeight</code> is <code>false</code> ; the numeric value only takes effect when <code>overrideHeight</code> is <code>true</code> .
<code>heightFrom</code>	integer or string	—	Conditional	<code>0</code>	Reference point for height: <code>0</code> or "Liquid" = liquid surface, <code>1</code> or "Bottom" = well bottom, <code>2</code> or "Top" = well top. String aliases are case-insensitive. Also supports expressions.
<code>customHeight</code>	boolean	—	No	<code>false</code>	Encoding discriminator for <code>height</code> : <code>false</code> = <code>height</code> is a numeric offset; <code>true</code> = <code>height</code> is a text/custom height and <code>overrideHeight</code> is treated as <code>true</code> . Use <code>false</code> with numeric heights. See Pipetting Techniques, Templates, and Auto-Selection.
<code>colsFirst</code>	boolean	—	No	<code>false</code>	When <code>true</code> , well selection iteration proceeds column-first (top-to-bottom within a column, then right to the next column); when <code>false</code> , row-first.
<code>rowsFirst</code>	boolean	—	No	<code>false</code>	When <code>true</code> , well selection iteration proceeds row-first (left-to-right within a row, then down to the next row). If both <code>colsFirst</code> and <code>rowsFirst</code> are <code>false</code> , defaults to column-first.
<code>startAtMark</code>	boolean	—	No	<code>false</code>	Span-8 Only. When <code>true</code> , well iteration starts at the mark position on the labware (requires the deck to have marked this labware). When <code>false</code> , starts at <code>startAtSelection</code> position.
<code>startAtSelection</code>	boolean	—	No	<code>true</code>	Span-8 Only. When <code>true</code> , iteration starts at the first selected well; when <code>false</code> , at the mark or position 1.
<code>setMark</code>	boolean	—	No	<code>true</code>	Span-8 Only. When <code>true</code> , the mark on the labware is updated after the transfer completes to the last well accessed.
<code>wellsX</code>	integer	—	No	Auto-detected	Number of columns in the labware (e.g., <code>12</code> for a 96-well plate). Auto-detected from labware class if not provided; used for pattern indexing.
<code>firstWell</code>	integer	—	No	<code>1</code>	1-based index of the first well to access if using well-number-based selection (not pattern-based).
<code>firstWellUsesExpression</code>	boolean	—	No	<code>false</code>	When <code>true</code> , <code>firstWell</code> is interpreted as an expression (e.g., <code>"=row"</code>).
<code>useExpression</code>	boolean	—	No	<code>false</code>	When <code>true</code> , well selection is determined by <code>sectionExpression</code> instead of <code>pattern</code> .
<code>sectionExpression</code>	string	—	Conditional	<code>""</code>	Expression that evaluates to a well index or section when <code>useExpression</code> is <code>true</code> .
<code>dataSetPattern</code>	boolean	—	No	<code>false</code>	Span-8 Only. When <code>true</code> , well selection is derived from a data set using <code>dataSetCondition</code> and <code>dataSetConditionType</code> .

Key	Type	Units	Required	Default	Description
localPattern	boolean	—	No	true	Span-8 Only. When <code>true</code> , <code>pattern</code> is internal to the step; when <code>false</code> , <code>referencedPattern</code> is used.
referencedPattern	string	—	Conditional	""	Span-8 Only. Name of a globally-defined pattern (from Well Patterns). Required when <code>localPattern</code> is <code>false</code> .
dataSetCondition	string	—	No	""	Span-8 Only. Condition string for data set filtering (e.g., <code>>0</code>). Used when <code>dataSetPattern</code> is <code>true</code> .
dataSetConditionType	string	—	No	""	Span-8 Only. Type of condition: e.g., <code>"Greater"</code> , <code>"Less"</code> , <code>"Equal"</code> . Used with <code>dataSetCondition</code> .
spacing	integer	—	No	Auto-detected	Well spacing between selected probes in a Multichannel or Fixed-8 context (e.g., <code>1</code> for 96-well, <code>2</code> for 384-well). Auto-detected from labware minimum interval if not set.
mixCount	integer	—	No	0	Unused in Transfer steps. Do not author this key.
ghosted	boolean	—	No	false	When <code>true</code> , this item is ignored during transfer. Used internally for deferred items. Do not author this key; if a method is opened in the editor, these items will be removed.

Enumerated / Constrained Values

stop

Value	Meaning
"Sources"	Stops when all source wells are exhausted; destination assignment continues until sources run out.
"Destinations"	Stops when all destination wells are filled; source assignment continues until destinations are full.
"Either"	Stops when either sources or destinations are exhausted (whichever comes first).

type

Value	Meaning
"Transfer"	One source → multiple destinations (this step).
"Combine"	Multiple sources → one destination (different step). Must always be <code>"Transfer"</code> for this step type.

podType

Value	Meaning
"Fixed8"	Fixed-8 pod — all 8 probes (or a subset of 1–8 via <code>numberOfTips</code>), using disposable tips.
"Span8"	Span-8 pod with 8 independent probe channels.
"MC"	Multichannel pod (96- or 384-channel head). This is the string the editor emits; <code>"Multichannel"</code> behaves identically.

operation (per item)

Value	Meaning
"Aspirate"	Item is a source; liquid is aspirated from this location. For Transfer steps, only the first item should be <code>"Aspirate"</code> .
"Dispense"	Item is a destination; liquid is dispensed to this location.

liquidType (per item)

Value	Meaning
"Well Contents"	Auto-detect liquid type from the labware's liquid data set at runtime. Valid for source (Aspirate) items only; invalid for Dispense.
"Tip Contents"	For Dispense items: use the actual liquid detected in the tips. For Aspirate: invalid.
Any named liquid (e.g., "Water", "Serum")	Specific liquid type for technique selection. Overrides auto-detection.

heightFrom (per item)

Value	Meaning
0 or "Liquid"	Measure height from the liquid surface in the well.
1 or "Bottom"	Measure height from the bottom of the well.
2 or "Top"	Measure height from the top of the well.

wash

Value	Meaning
true	Perform active wash (Span-8 or Multichannel) using specified solvent and technique.
false	No active wash.

span8Wash

Value	Meaning
true	Perform passive wash (Span-8 only) using configured wash volume and waste volume.
false	No passive wash.

useExpression (step-level) vs. useExpression (item-level)

Step-level (probe-selection subvariant; meaningful only in SELECTION mode — useFixedTips and useDisposableTips both false):

- true : Use mandrelExpression to determine probe selection at runtime.
- false : Use useProbes array for static probe selection.

Item-level:

- true : Use sectionExpression to determine well selection at runtime.
- false : Use the static boolean mask — selectionInfo on Fixed-8/Multichannel, pattern on Span-8 (see Rule 12).

Cross-Field Validation Rules

1. **At least one source and one destination required:** the step validates that at least one item is marked as source (source: true) and at least one is marked as destination (source: false). If either is missing, enqueue fails with a "no source/destination defined" error.
2. **Transfer type constraint:** For a Transfer step, only the first item (index 0) is marked as the source; all subsequent items are destinations. Multi-source pipetting is done with the **Combine** step (type: "Combine"), not Transfer — do not place multiple source items in a Transfer step's items array.
3. **Tip selection is mutually exclusive:** useFixedTips and useDisposableTips are opposite tip-handling modes on Span-8 — do **not** set both true . On a Span-8 disposable-tip step, leave both false and pick channels via useProbes (or mandrelExpression when

`useExpression` is `true`); disposable tips are the Span-8 default. Setting both is non-fatal — the engine reads them tolerantly and the first mode checked wins — but it is never correct to author. When both are `false`, the step falls through to per-probe selection via `useProbes` / `mandrelExpression`.

4. **Probe array constraints:** If `useFixedTips`, `useDisposableTips`, and `useExpression` are **all false** (SELECTION mode sourced from `useProbes`), then `useProbes` must be present with **at least one true element**. If all probes are `false`, enqueue fails with a "no usable probes" error. In FIXED or DISPOSABLE mode, or when `useExpression` is `true`, `useProbes` is ignored. (`useMandrelSelection` is inert UI-state and does not participate in this check.)
5. **Expression-based probe selection:** If `useExpression` is `true`, then `mandrelExpression` **must be non-empty** and must evaluate to a valid probe array at runtime.
6. **Tip location required for fresh tips:** If `useCurrentTips` is `false` and `useFixedTips` is `false`, then `tipLocation` **must be non-empty** and resolvable to a tip box on the deck. If not resolved, enqueue fails with a tips-not-found error.
7. **Number of tips constraint:** `numberOfTips` must be a string evaluating to an integer between 1 and 8. If invalid, tip selection fails.
8. **All selected tips must be same type and syringe size (Span-8):** At runtime, the step validates that all selected probes with tips have:
 - Same tip type (e.g., all Fixed or all Disposable)
 - Same syringe size (e.g., all 200 µL)
 If mismatched, enqueue fails with a same-tip-type / same-syringe-size error.
9. **Wash solvent required if wash is enabled:** If `wash` is `true`, then `solvent` **must be non-empty** and `washVolume` **must be non-empty**. The named `solvent` must resolve to a wash station on the deck; if none is found, tip cleaning throws a "no wash station" error, surfaced during the transfer as a tip-cleaning failure at the source position.
10. **Active wash vs. passive wash (Span-8):** `wash` (active) and `span8Wash` (passive) **should not both be true** at the same time.
11. **Wash technique specification:** If `wash` is `true` and `autoSelectActiveWashTechnique` is `false`, either `activeWashTechnique` (string name) or `customActiveWashTechnique` (object) **must be non-empty or defined**. If neither is present and `autoSelectActiveWashTechnique` is `false`, the step may fail or skip washing.
12. **Well selection in each item:** every item selects wells through exactly one mode, and **which key is the operative selector depends on the pod family:**

Mode (item keys)	Operative selector
Default — <code>localPattern: true</code> , <code>dataSetPattern: false</code> , <code>useExpression: false</code> , ON Fixed-8 / Multichannel	<code>selectionInfo</code>
Default — <code>localPattern: true</code> , <code>dataSetPattern: false</code> , <code>useExpression: false</code> , ON Span-8	<code>pattern</code>
<code>localPattern: false</code>	<code>referencedPattern</code> (name of a globally-defined pattern)
<code>dataSetPattern: true</code>	the named data set + <code>dataSetCondition</code> / <code>dataSetConditionType</code>
<code>useExpression: true</code>	<code>sectionExpression</code>

13. **Volume specification:** For each item, **at least one way to specify volume must be present:**
 - o `volume` key (string, µL) — preferred for most items, or
 - o `amount` and `amounts` — if per-probe volumes are needed (Span-8 specific) An **omitted** `volume` key defaults to `0` (which may cause technique validation errors). An **empty string** `""` is **not** the same as omitted — it fails at enqueue with `Invalid volume specified for <source|destination> N`. Every item that carries volume must give a non-empty numeric value or an evaluable expression; to leave it unset, omit the key rather than authoring `""`.
14. **Volume per source vs. per destination:** For a Transfer step, always set the volume on the destination.
15. **Repeats and repeatsByVolume constraints:**
 - o If `repeatsByVolume` is `false`: `repeats` must evaluate to a number ≥ 1 indicating a number of dispenses per aspirate.

- If `repeatsByVolume` is `true`: `repeats` must evaluate to a number indicating a volume to aspirate for multidispensing.
16. **Stop condition and well assignment:** The choice of `stop` ("Sources", "Destinations", "Either") determines how the algorithm pairs sources with destinations:
- "Sources": All source wells are processed; destination wells accessed as needed (looping through destinations when they are exhausted).
 - "Destinations": All destination wells are processed; source wells accessed as needed (looping through sources when they are exhausted).
 - "Either": Algorithm stops when first limit is hit. An invalid value raises a "stop condition" error.
17. **Split volume constraints:** If `splitVolume` is `true`:
- **Splitting engages only when each aspirate matches a single dispense.** It is not valid when `repeats` is greater than 1 or `repeatsByVolume` is `true`.
 - Each split cycle inherits its technique's overhead (trailing air gap / blowout).
 - The split calculator sums those per-cycle aspirates against the source well's tracked volume, so split sources must be trackable — author `volumeType`: "Known" with realistic `evalAmounts` in Instrument Setup (see Instrument Setup). An "Unknown" source under `splitVolume`: `true` cannot be validated and fails enqueue.
 - If `splitVolumeCleaning` is also `true` and either `changeTipsBetweenSources` or `changeTipsBetweenDests` is `true`, tips are cleaned (changed or washed) between split cycles.
18. **Leave tips vs. discard:** If `leaveTipsOn` is `true`, tips are left on the pod; if `false`, they are discarded to the location specified by `unloadLocation` (Fixed-8 only) or as defined in Instrument Setup (Multichannel, Span-8).
19. **Item count constraint:** `items` must contain at least 2 elements (one source, one destination).
20. **Item `labwareClass` must match (or be compatible with) the labware at `position`:** At enqueue the resolved labware class is checked against each item's `labwareClass`. A mismatch that is not a compatible substitute raises a class-mismatch error; an entirely unknown expected class raises a "labware class not found" error.
21. **`useCurrentTips` requires tips already on the pod:** If `useCurrentTips` is `true` but no tips are loaded on the pod when the step runs, enqueue fails with a "no tips loaded" error.
22. **Non-LLS tips are incompatible with LLS-enabled techniques (Span-8):** The stock Span-8 pipetting techniques enable liquid level sensing on the initial descent, but non-LLS tip classes (e.g., T190F, T50F — filtered tips with `LLS`: `false` in the catalog) cannot liquid-level-sense. Combining them with the default auto-selected Span-8 technique hard-fails enqueue with a wrong-tip error. To use non-LLS tips, either pick an LLS-capable tip class or set `overrideHeight`: `true` on the item with a fixed-reference `heightFrom` (1 / "Bottom" or 2 / "Top") so the technique's initial LLS branch is bypassed.
23. **Volume tracking (cross-reference):** The engine tracks a running per-well volume for the source and destination wells and rejects both over-dispense (fills exceeding the well/labware capacity) and under-aspirate (drawing more than the tracked source volume). Author source starting volumes and reservoir dead volumes in Instrument Setup — see the "Volume tracking" note there.
24. **`numberOfTips` for Fixed-8 must match the pattern of selected wells:** For a Fixed-8 pod, the number of selected wells in a given column must be divisible evenly by `numberOfTips`. For example, to pipet to four rows in a destination, `numberOfTips` can be 1, 2, or 4, but cannot be 3 or anything higher than 4.

Tip Budget

Tip consumption scales with pod family and `changeTips*` settings:

- **Fixed-8:** one tip-load per aspirate cycle draws `numberOfTips` tips (1-8). A whole-plate 96-well transfer with `numberOfTips`: 8 and `changeTipsBetweenSources` / `changeTipsBetweenDests` at their defaults uses 96 tips (12 columns x 8).
- **Span-8:** one tip-load draws up to 8 tips (one per active probe from `useProbes` / `mandrelExpression`). A whole-plate 96-well transfer with all 8 probes active uses 96 tips.
- **Multichannel:** 96 or 384 tips per load, depending on the head.

Place enough tip boxes of the right class on the deck to cover the worst-case tip count for the method — or use `changeTipsBetween*:false` where cross-contamination is acceptable to reduce consumption.

Multichannel: what a Transfer item can and cannot address

On a Multichannel pod a Transfer item's well selection resolves to **whole sections or quadrants** of the labware, not to an arbitrary set of wells. The item-level `selectionInfo` is a list of integer section/quadrant indices (see the `selectionInfo` row above and Multichannel Aspirate §Enumerated) — there is no boolean 96-well mask on this pod family.

That is enough for whole-plate and whole-quadrant work, and not enough for a large class of routine science:

- partial-plate layouts, where only some columns carry sample;
- control wells — an NTC, a no-RT control, reference wells — that must **not** receive what everything else receives;
- replicate patterning that reuses one tip load across a defined set of wells.

Authoring these as a plain Multichannel Transfer does not fail at enqueue. It silently widens the operation to the whole section, which dispenses into the very wells the controls depended on staying clean. A method can therefore be structurally valid and scientifically void.

For that work use the **Multichannel Select Tips** container family instead (Multichannel Select Tips and the child Aspirate/Dispense/Mix/Load/Unload steps, docs 27–35), which selects individual rows and columns of tips and addresses wells accordingly.

Before authoring a Multichannel Transfer, check whether every well in the target section should really receive the same thing. If the answer is no, this is not the step.

Mixing in place

Mixing at a well is a **technique** property on every pod family, not an item-level Transfer key. An item-level `mixCount` on a Transfer item is inert (see the `mixCount` row in the item table) and does nothing at run time regardless of pod.

Multichannel additionally offers a standalone `Multichannel Mix` step and Fixed-8 offers `Fixed-8 Mix` when a mix is wanted without an accompanying transfer. Span-8 has no standalone Mix step — use a technique with a built-in mix.

The technique's mix volume goes through the same volume tracker as any aspirate, and which side the mix runs on decides what it is budgeted against: an Aspirate-side mix is checked before the transfer draws from the source, a Dispense-side mix after the delivery lands. See Well Volume Tracking.

Dispensing to waste

A Transfer dispenses to a labware position — there is no separate "liquid trash" destination for it, and the `Trash` characteristic (`<Any Trash>`, `WhenDone`, `DiscardTipsLocation`) routes **labware** and **tips**, not liquid. Liquid still in the tips is commonly discarded at a **wash station** instead: the Span-8 and Multichannel Wash Tips steps empty tip contents there (`dispenseTipContentsOnly`, or the passive `toWaste` volume), usually followed by a rinse that clears it — see Span-8 Wash Tips. Whether a particular liquid is appropriate to send through the wash station depends on the liquid and your instrument's plumbing and is outside the scope of these docs.

To discard bulk liquid **with a Transfer**, author a Transfer whose destination item names either a wash station or a labware position you have designated as waste — typically an empty reservoir placed on a free deck position for that purpose in Instrument Setup. From the step's point of view this is an ordinary destination: the item carries a normal `position`, `labwareClass`, `volume`, `pattern` / `selectionInfo`, and `prototype`; nothing about the destination is special-cased in the engine. If a wash station is selected as a destination, then the `labwareClass` must be the appropriate permanent labware (e.g. `"WashStation"` or `"WashStationSpan8"`).

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Simple Fixed-8 Transfer (single source to single destination, current tips)

```
{
  "stepType": "Transfer",
  "parameters": {
    "activeWashTechnique": "",
    "autoSelectActiveWashTechnique": false,
    "changeTipsBetweenDestis": false,
    "changeTipsBetweenSources": false,
    "dynamic?": true,
    "items": [
      {
        "autoSelectPrototype": false,
        "colsFirst": true,
        "customHeight": false,
        "dataSetPattern": false,
        "height": -2.0,
        "heightFrom": 0,
        "labwareClass": "BCFlat96",
        "liquidType": "Well Contents",
        "localPattern": true,
        "overrideHeight": false,
        "pattern": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [true, false, false, false, false, false, false, false, false, false, false, false, true, false, false, false, f
        ],
        "position": "p9",
        "prototype": "F8 Medium",
        "referencedPattern": "",
        "rowsFirst": false,
        "sectionExpression": "",
        "selectionInfo": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [true, false, false, false, false, false, false, false, false, false, false, false, true, false, false, false, f
        ],
        "setMark": true,
        "startAtMark": false,
        "startAtSelection": true,
        "useExpression": false,
        "volume": "10",
        "wellsX": 12
      },
      {
        "autoSelectPrototype": false,
        "colsFirst": true,
        "customHeight": false,
        "dataSetPattern": false,
        "height": 0.0,
        "heightFrom": 0,
        "labwareClass": "BCFlat96",
        "liquidType": "Tip Contents",
        "localPattern": true,
        "overrideHeight": false,
        "pattern": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [false, true, false, false, false, false, false, false, false, false, false, false, false, true, false, false, f
        ],
        "position": "p9",
        "prototype": "F8 Medium",
        "referencedPattern": "",
        "rowsFirst": false,
        "sectionExpression": "",
        "selectionInfo": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [false, true, false, false, false, false, false, false, false, false, false, false, false, true, false, false, f
        ],
        "setMark": true,
        "startAtMark": false,
        "startAtSelection": true,

```

```

    "useExpression": false,
    "volume": "10",
    "wellsX": 12
  }
],
"leaveTipsOn": false,
"mandrelExpression": "",
"numberOfTips": "8",
"pod": "Pod1",
"podType": "Fixed8",
"repeats": "1",
"repeatsByVolume": false,
"replicates": "1",
"showTipHandlingDetails": true,
"showTransferDetails": false,
"solvent": "Water",
"span8Wash": false,
"span8WashVolume": "2",
"span8WasteVolume": "1",
"splitVolume": false,
"splitVolumeCleaning": false,
"stop": "Destinations",
"tipLocation": "BC230",
"useCurrentTips": true,
"useDisposableTips": false,
"useExpression": false,
"useFixedTips": false,
"useJIT": true,
"useMandrelSelection": true,
"useProbes": {
  "_biomekType": "comArray",
  "arraySubtype": "boolean",
  "values": [true, true, true, true, true, true, true, true]
},
"wash": false,
"washCycles": "3",
"washVolume": "110%",
"wizard": false
}
}

```

Common Mistakes

- **Per-item pattern / selectionInfo shorter than the labware's well count** — the `values` `comArray` must be full-length, not just the intended wells. Full-length means one entry per **addressable position** on that labware, which is not always 96: `wells X × wells Y` for a titer plate or tube rack (96 booleans for a 96-well plate), but the **section count** for a reservoir — `[true]` for a one-section `BCFullReservoir`, four entries for a `ModularReservoir`. Short arrays fail enqueue with a pattern-shape error.
- **Treating `podType` as pod-family selection** — pod family is resolved from the named `pod`. Setting `podType: "Span8"` on a Fixed-8 pod does not redirect the step to a Span-8 pod — but it is still wrong: keep `podType / span8` in sync with the resolved pod; see `$Parameters` row `podType`.
- **`autoSelectPrototype: true` on a Fixed-8 item**. Fixed-8 items must set `autoSelectPrototype: false` with a named `prototype`.
- **Forgetting to update `labwareClass` /tip-box after a deck-class change** — when the labware class on a position changes in Instrument Setup, every downstream `labwareClass / tipLocation / what` field must follow, or the step fails at enqueue with a class-mismatch error.
- **Wash enabled without a resolvable `solvent`** — the named solvent must resolve to a wash station on the deck; missing/misnamed stations surface only mid-run as a tip-cleaning error at the source position, not at authoring time.
- **`tipLocation` set to a tip class when the deck box carries a distinct instance name** — a class-only query only matches boxes whose instance name is empty or equal to the class name. If the deck's tip box has been renamed (its `properties.name` differs from the class), the tips-not-found error at Rule 6 fires — author the box's instance name or its deck position instead.
- **Omitting `changeTipsBetweenDests` on a Fixed-8 or Multichannel Transfer** — the default for those pod families is `false` (reuse tips across destinations), so omission yields the cross-contamination-risky behavior even though the same key defaults to `true` on Span-8. Author `changeTipsBetweenDests: true` whenever cross-contamination is not the intent; see the `changeTipsBetweenDests` row.

Combine

Property	Value
stepType	"Combine"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 — see note below

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `defaultCaption` , `dynamic?` , ...) are documented there, not here.

Implementation note. Combine shares implementation with **Transfer**. Because the two steps share every key, **this doc mirrors Transfer**; only the Combine-specific deltas are spelled out in full. Read the Transfer doc for the deep item-level detail and the shared `_biomekType: "comArray"` shapes.

Behavior Summary

The Combine step is the mirror image of Transfer: it moves liquid from **multiple source locations into a single destination** (many → one), whereas Transfer is one → many. At enqueue time it verifies that all source wells and destination wells are defined with proper selections, resolves both source and destination labware positions and classes, and builds one or more transfer groups optimized for efficient pipetting. Tip handling, washing, repeats/replicates, and large-volume splitting are identical to Transfer.

The primary difference from Transfer is that the list of items includes multiple sources and a single destination, and transfer volumes are defined on each source, rather than on the destination.

Hardware note. Combine is not available on a Fixed-8 pod, unlike Transfer. The shared engine supports Span-8 and multichannel pods for Combine; name the pod you want and set the `span8` / `podType` discriminators to match it.

Parameters Reference Table

Note. Combine is implemented as a subclass of the Transfer step and shares Transfer's entire enqueue implementation, so its key set, defaults, and required-ness are identical to Transfer — only the compatible hardware, caption, and `Type` value differ.

Every step-level and item-level key is **identical to Transfer**. The table below is the Transfer key set with Combine-specific semantics called out; see Transfer for the full per-key prose, enum tables, and comArray shapes. Keys marked **(Combine delta)** differ in meaning or presence from Transfer.

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>type</code>	string	—	Yes	"Combine"	(Combine delta) Must be set to "Combine" .
<code>pod</code>	string	—	Yes	""	Name of the pipetting pod (e.g., "Pod1"). Must resolve at runtime to a valid pod ((Combine delta) Span-8 or Multichannel only.).

Key	Type	Units	Required	Default	Description
podType	string	—	No	derived from pod	Pod-family label ("Span8" / "MC" — see \$podType). Keep it in sync with the resolved pod. Which pod family actually runs is resolved from the named pod , so podType does not select the family; but it must still be correct, because it selects the pod-family wording of the printed method (mandrel-vs-section labels, Span-8 lines, tip count, Combine/Transfer header) — a stale value prints a method that contradicts the pod.
span8	boolean	—	No	(absent)	Span-8 pod-family discriminator. Keep it in sync with the resolved pod. Set to true for a Span-8 pod and false for any other pod.
items	array (object[])	—	Yes	[]	Array of source and destination item dictionaries. Each item describes a labware location, well selection pattern, volume, and pipetting parameters. Position is what assigns the role: (Combine delta) all items except the last item are sources; the last item is the destination . The array must contain at least one of each (so at least two entries). The item-level source key is written positionally at enqueue and any authored value is overwritten (see the source row in Item-Level Keys). See Item-Level Keys subsection below.
stop	string	—	No	"Sources"	Stopping condition: "Sources" (exhaust all sources), "Destinations" (fill all destinations), or "Either" (stop when either is exhausted). Controls iteration over wells in Transfer assignment. The fallback default differs by pod family. Author it explicitly.
repeats	string	—	No	"1"	This key is used for configuring multidispense behavior. The default is to allow a single dispense for each aspirate, represented by a value of 1 for this key. If repeatsByVolume is true , this is the maximum volume in microliters that can be aspirated at once for repeated dispensing (i.e. aspirate once, dispense multiple times). If repeatsByVolume is false , this is the maximum number of dispenses allowed for a single aspirate (minimum 1). Multidispensing is not a common usage and requires specialized pipetting techniques. This should typically be authored and set to "1" .
repeatsByVolume	boolean	—	No	false	When true , repeats specifies the maximum volume to aspirate for multidispensing; when false , repeats specifies the maximum number of dispenses allowed per aspirate.
replicates	string	—	No	"1"	Number replicates to create for each source well. Expression-capable. Minimum 1. This key impacts the mapping from source wells to destination wells, ensuring that each source well is transferred to the given number of destination wells before moving on to the next source well.
useJIT	boolean	—	No	true	When true , aspirate and dispense operations are bundled into a single Just-In-Time block to ensure no other actions occur in between the aspirate and dispense.
useFixedTips	boolean	—	No	false	One of the Span-8 tip-mode keys. When true , only fixed-tip probes are used and useProbes is ignored (tip

Key	Type	Units	Required	Default	Description
					physicality is inferred from the selected probes). Do not also set <code>useDisposableTips</code> .
<code>useDisposableTips</code>	boolean	—	No	<code>false</code>	One of the Span-8 tip-mode keys. When <code>true</code> , all non-fixed (disposable-tip) probes are used and <code>useProbes</code> is ignored. Do not also set <code>useFixedTips</code> .
<code>useMandrelSelection</code>	boolean	—	No	<code>true</code>	Inert UI-state (engine does not read it; it persists the editor's selection mode). Execution branches on <code>mandrelExpression</code> / <code>useProbes</code> (<code>UseExpression</code>), <code>useFixedTips</code> , and <code>useDisposableTips</code> .
<code>useProbes</code>	comArray (boolean[8])	—	Conditional	All <code>true</code>	Array of eight booleans indicating which probes (1–8) are active on Span-8 pods. Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]}</code> . Required (with at least one <code>true</code> element) when <code>useFixedTips</code> , <code>useDisposableTips</code> , and <code>useExpression</code> are all <code>false</code> — i.e., SELECTION mode sourced from <code>useProbes</code> . In FIXED (<code>useFixedTips: true</code>) or DISPOSABLE (<code>useDisposableTips: true</code>) mode, or when <code>useExpression</code> is <code>true</code> , <code>useProbes</code> is ignored. Multichannel ignores this key — MC head geometry (96 or 384 channels) is not per-probe selectable at the step level.
<code>useExpression</code>	boolean	—	No	<code>false</code>	When <code>true</code> , probe selection is determined by <code>mandrelExpression</code> instead of <code>useProbes</code> .
<code>mandrelExpression</code>	string	—	Conditional	<code>""</code>	Expression that evaluates to the probe selection at runtime (e.g., <code>"=[1,2,3,4,5,6,7,8]"</code> for Span-8). Required when <code>useExpression</code> is <code>true</code> .
<code>numberOfTips</code>	string	—	No	<code>"8"</code>	(Combine delta) Inert on Combine. This key is not used on Combine as it only applies to Fixed-8 pods, which are not supported for Combine. Do not author it.
<code>useCurrentTips</code>	boolean	—	No	<code>false</code>	When <code>true</code> , the step reuses tips already loaded on the pod from a previous step instead of loading new tips.
<code>changeTipsBetweenSources</code>	boolean	—	No	<code>true</code> on Multichannel, <code>false</code> on Span-8	For a Multichannel pod, when <code>true</code> , tips are changed (or washed) between aspirations from different source locations. For a Span-8 pod, whether or not tips are changed/washed is the result of a logical OR of <code>changeTipsBetweenSources</code> and <code>changeTipsBetweenDestinations</code> . When authoring a Span-8 Combine step, leave this set to <code>false</code> and utilize <code>changeTipsBetweenDests</code> . (Combine delta) — for many→one, sources are iterated, so this is the primary tip-change control.
<code>changeTipsBetweenDests</code>	boolean	—	No	<code>false</code> on Multichannel, <code>true</code> on Span-8	When <code>true</code> , tips are changed (or washed) between after dispensing. While this key may suggest it applies when destinations are different, it actually should be interpreted as "change (or wash) tips after dispensing".
<code>leaveTipsOn</code>	boolean	—	No	<code>false</code>	When <code>true</code> , tips are left on the pod after the step completes; when <code>false</code> , tips are unloaded.
<code>splitVolume</code>	boolean	—	No	<code>false</code>	When <code>true</code> , a transfer volume larger than the tip capacity is split into several aspirate-dispense cycles.

Key	Type	Units	Required	Default	Description
<code>splitVolumeCleaning</code>	boolean	—	No	<code>false</code>	When <code>true</code> and <code>splitVolume</code> is <code>true</code> , tips are cleaned between each split cycle (if <code>changeTipsBetweenSources</code> or <code>changeTipsBetweenDestis</code> is <code>true</code>).
<code>tipLocation</code>	string	—	Conditional	<code>""</code>	Tip source used when loading fresh tips — a tip-box labware name , a deck position (e.g. <code>"P1"</code>), or a tip labware class (e.g. <code>"BC230"</code>). Required when loading new disposable tips (i.e., when <code>useCurrentTips</code> is <code>false</code> and <code>useFixedTips</code> is <code>false</code>). Class-name resolution only matches a box whose instance name is empty or equal to the class name — when the tip box on deck carries a distinct <code>properties.name</code> (renamed instance), author the box's instance name or its deck position.
<code>unloadLocation</code>	string	—	No	<code>"<where they came from>"</code>	(Combine delta) Inert on Combine. This key is not used on Combine as it only applies to Fixed-8 pods, which are not supported for Combine. Do not author it.
<code>wash</code>	boolean	—	No	<code>false</code>	For Multichannel and Span-8 pods: when <code>true</code> , an active wash (using specified solvent and wash technique) is performed between tip handling events. Mutually exclusive with <code>span8Wash</code> .
<code>span8Wash</code>	boolean	—	No	<code>false</code>	For Span-8 pods: when <code>true</code> , a passive wash is performed (uses <code>span8WashVolume</code> and <code>span8WasteVolume</code>). Mutually exclusive with <code>wash</code> .
<code>washVolume</code>	string	μL or %	Conditional	<code>""</code>	Volume of solvent to use in active wash, or percentage of previously used tip capacity (e.g., <code>"110%"</code>). Expression-capable. Required when <code>wash</code> is <code>true</code> .
<code>span8WashVolume</code>	string	mL	No	<code>"2"</code>	Volume of passive wash solvent for Span-8 pods (e.g., <code>"2"</code>), in mL . Expression-capable. Default 2 mL.
<code>span8WasteVolume</code>	string	mL	No	<code>"1"</code>	Volume of waste liquid discarded after passive wash for Span-8 pods (e.g., <code>"1"</code>), in mL . Expression-capable. Default 1 mL.
<code>washCycles</code>	string	—	Conditional	<code>""</code>	Number of wash cycles to perform (e.g., <code>"3"</code>). Expression-capable. Required when <code>wash</code> is <code>true</code> .
<code>solvent</code>	string	—	Conditional	<code>"Water"</code>	Name of the washing solvent (e.g., <code>"Water"</code> , <code>"Ethanol"</code>). Required when <code>wash</code> is <code>true</code> . When configuring an active wash, the <code>solvent</code> must match the liquid type configured for an active wash station ALP on the deck.
<code>autoSelectActiveWashTechnique</code>	boolean	—	No	<code>false</code>	When <code>true</code> , an active wash technique is auto-selected based on liquid type and pod; when <code>false</code> , use <code>activeWashTechnique</code> (if a string) or <code>customActiveWashTechnique</code> (if an object). Tolerant read at enqueue; omission does not throw.
<code>activeWashTechnique</code>	string	—	No	<code>""</code>	Name of a predefined wash technique (e.g., <code>"Standard Wash"</code>). Ignored if <code>autoSelectActiveWashTechnique</code> is <code>true</code> or <code>customActiveWashTechnique</code> is bound.
<code>customActiveWashTechnique</code>	object	—	No	Not present	Custom wash technique definition (inline technique object). Takes precedence over <code>activeWashTechnique</code> .
<code>showTipHandlingDetails</code>	boolean	—	No	<code>false</code>	UI flag: when <code>true</code> , the tip handling summary is displayed in the method editor.

Key	Type	Units	Required	Default	Description
<code>showTransferDetails</code>	boolean	—	No	<code>false</code>	UI flag: when <code>true</code> , the transfer mapping summary is displayed in the method editor.
<code>wizard</code>	boolean	—	No	<code>false</code>	Internal UI flag. Not read at runtime.
<code>groupTransfer</code>	boolean	—	No	<code>false</code>	Obsolete and unused key. Do not author.

The step-level `tip/split/wash/UI` keys above (`useFixedTips`, `useDisposableTips`, `useMandrelSelection`, `useExpression`, `splitVolume`, `splitVolumeCleaning`, `activeWashTechnique`, `showTipHandlingDetails`, `showTransferDetails`, `wizard`) are persisted by the editor but stay *Optional* — omission does not throw at enqueue. However, they should be authored so that a method imported into the Biomek editor displays the appropriate values in the UI.

Universal base keys (`caption`, `defaultCaption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Item-Level Keys

Item-level keys are **identical to Transfer** — see Transfer for the full table (`source`, `position`, `labwareClass`, `volume`, `pattern`, `selectionInfo`, `operation`, `liquidType`, `prototype`, `autoSelectPrototype`, `customPrototype`, `overrideHeight`, `height`, `heightFrom`, `customHeight`, `colsFirst`, `rowsFirst`, `startAtMark`, `startAtSelection`, `setMark`, `wellsX`, `firstWell`, `firstWellUsesExpression`, `useExpression`, `sectionExpression`, `dataSetPattern`, `localPattern`, `referencedPattern`, `dataSetCondition`, `dataSetConditionType`, `spacing`, `mixCount`, `ghosted`). Two items carry **Combine-specific semantics**:

Key	Type	Units	Required	Default	Description
<code>source</code>	boolean	—	No (auto-set)	<code>true</code> or <code>false</code> (auto-marked)	Set automatically at enqueue based on item position — (Combine delta) for a Combine step, the last item is the destination and every previous item is a source. Any value you author here is OVERWRITTEN, so do not set it manually.
<code>volume</code>	string	μL	Conditional	<code>""</code>	Volume to aspirate (if source) or dispense per well (if destination). Expression-capable. (Combine delta) For Combine, set the volume on each source.

Enumerated / Constrained Values

stop

Value	Meaning
<code>"Sources"</code>	Stops when all source wells are exhausted; destination assignment continues until sources run out.
<code>"Destinations"</code>	Stops when all destination wells are filled; source assignment continues until destinations are full.
<code>"Either"</code>	Stops when either sources or destinations are exhausted (whichever comes first).

type

Value	Meaning
<code>"Transfer"</code>	One source → multiple destinations (different step). Must always be <code>"Combine"</code> for this step type.
<code>"Combine"</code>	Multiple sources → one destination (this step).

podType

Value	Meaning
<code>"Fixed8"</code>	Fixed-8 pod — all 8 probes (or a subset of 1–8 via <code>numberOfTips</code>), using disposable tips.

Value	Meaning
"Span8"	Span-8 pod with 8 independent probe channels.
"MC"	Multichannel pod (96- or 384-channel head). This is the string the editor emits; "Multichannel" behaves identically.

operation (per item)

Value	Meaning
"Aspirate"	Item is a source; liquid is aspirated from this location.
"Dispense"	Item is a destination; liquid is dispensed to this location. For Combine steps, only the last item should be "Dispense".

liquidType (per item)

Value	Meaning
"Well Contents"	Auto-detect liquid type from the labware's liquid data set at runtime. Valid for source (Aspirate) items only; invalid for Dispense.
"Tip Contents"	For Dispense items: use the actual liquid detected in the tips. For Aspirate: invalid.
Any named liquid (e.g., "Water", "Serum")	Specific liquid type for technique selection. Overrides auto-detection.

heightFrom (per item)

Value	Meaning
0 or "Liquid"	Measure height from the liquid surface in the well.
1 or "Bottom"	Measure height from the bottom of the well.
2 or "Top"	Measure height from the top of the well.

wash

Value	Meaning
true	Perform active wash (Span-8 or Multichannel) using specified solvent and technique.
false	No active wash.

span8Wash

Value	Meaning
true	Perform passive wash (Span-8 only) using configured wash volume and waste volume.
false	No passive wash.

useExpression (step-level) vs. useExpression (item-level)

Step-level (probe-selection subvariant; meaningful only in SELECTION mode — useFixedTips and useDisposableTips both false):

- true : Use mandrelExpression to determine probe selection at runtime.
- false : Use useProbes array for static probe selection.

Item-level:

- true : Use sectionExpression to determine well selection at runtime.
- false : Use the static boolean mask — selectionInfo on Multichannel, pattern on Span-8 (see Rule 12).

Cross-Field Validation Rules

Validation rules are largely the same as the Transfer step. The numbering here matches the numbering in the Transfer step, with modifications noted with **Combine delta**.

- At least one source and one destination required:** the step validates that at least one item is marked as source (`source: true`) and at least one is marked as destination (`source: false`). If either is missing, enqueue fails with a "no source/destination defined" error.
- Transfer type constraint: Combine delta** For a Combine step, only the last item is marked as the destination; all previous items are sources. Multi-destination pipetting is done with the **Transfer** step, not Combine — do not place multiple destination items in a Combine step's `items` array.
- Tip selection is mutually exclusive:** `useFixedTips` and `useDisposableTips` are opposite tip-handling modes on Span-8 — do **not** set both `true` . On a Span-8 disposable-tip step, leave both `false` and pick channels via `useProbes` (or `mandrelExpression` when `useExpression` is `true`); disposable tips are the Span-8 default. Setting both is non-fatal — the engine reads them tolerantly and the first mode checked wins — but it is never correct to author. When both are `false` , the step falls through to per-probe selection via `useProbes` / `mandrelExpression` .
- Probe array constraints:** If `useFixedTips` , `useDisposableTips` , and `useExpression` are **all false** (SELECTION mode sourced from `useProbes`), then `useProbes` must be present with **at least one true element**. If all probes are `false` , enqueue fails with a "no usable probes" error. In FIXED or DISPOSABLE mode, or when `useExpression` is `true` , `useProbes` is ignored. (`useMandrelSelection` is inert UI-state and does not participate in this check.)
- Expression-based probe selection:** If `useExpression` is `true` , then `mandrelExpression` **must be non-empty** and must evaluate to a valid probe array at runtime.
- Tip location required for fresh tips:** If `useCurrentTips` is `false` and `useFixedTips` is `false` , then `tipLocation` **must be non-empty** and resolvable to a tip box on the deck. If not resolved, enqueue fails with a tips-not-found error.
- Combine delta:** Item 7 in the Transfer step's validation rules does not apply, as Fixed-8 pods are not valid for Combine.
- All selected tips must be same type and syringe size (Span-8):** At runtime, the step validates that all selected probes with tips have: - Same tip type (e.g., all Fixed or all Disposable) - Same syringe size (e.g., all 200 µL) If mismatched, enqueue fails with a same-tip-type / same-syringe-size error.
- Wash solvent required if wash is enabled:** If `wash` is `true` , then `solvent` **must be non-empty** and `washVolume` **must be non-empty**. The named `solvent` must resolve to a wash station on the deck; if none is found, tip cleaning throws a "no wash station" error, surfaced during the transfer as a tip-cleaning failure at the source position.
- Active wash vs. passive wash (Span-8):** `wash` (active) and `span8Wash` (passive) **should not both be true** at the same time.
- Wash technique specification:** If `wash` is `true` and `autoSelectActiveWashTechnique` is `false` , either `activeWashTechnique` (string name) or `customActiveWashTechnique` (object) **must be non-empty or defined**. If neither is present and `autoSelectActiveWashTechnique` is `false` , the step may fail or skip washing.
- Well selection in each item:** every item selects wells through exactly one mode, and **which key is the operative selector depends on the pod family:**

Mode (item keys)	Operative selector
Default — <code>localPattern: true</code> , <code>dataSetPattern: false</code> , <code>useExpression: false</code> , on Multichannel	<code>selectionInfo</code>
Default — <code>localPattern: true</code> , <code>dataSetPattern: false</code> , <code>useExpression: false</code> , on Span-8	<code>pattern</code>
<code>localPattern: false</code>	<code>referencedPattern</code> (name of a globally-defined pattern)

Mode (item keys)	Operative selector
<code>dataSetPattern: true</code>	the named data set + <code>dataSetCondition / dataSetConditionType</code>
<code>useExpression: true</code>	<code>sectionExpression</code>

13. **Volume specification:** For each item, **at least one way to specify volume must be present:**

- `volume` key (string, μL) — preferred for most items, or
- `amount` and `amounts` — if per-probe volumes are needed (Span-8 specific) An **omitted** `volume` key defaults to `0` (which may cause technique validation errors). An **empty string** `""` is **not** the same as omitted — it fails at enqueue with `Invalid volume specified for <source|destination> N`. Every item that carries volume must give a non-empty numeric value or an evaluable expression; to leave it unset, omit the key rather than authoring `""`.

14. **Volume per source vs. per destination:** For a Combine step, always set the volume on the source.

15. **Repeats and repeatsByVolume constraints:**

- If `repeatsByVolume` is `false`: `repeats` must evaluate to a number ≥ 1 indicating a number of dispenses per aspirate.
- If `repeatsByVolume` is `true`: `repeats` must evaluate to a number indicating a volume to aspirate for multidispensing.

16. **Stop condition and well assignment:** The choice of `stop` ("Sources", "Destinations", "Either") determines how the algorithm pairs sources with destinations:

- "Sources" : All source wells are processed; destination wells accessed as needed (looping through destinations when they are exhausted).
- "Destinations" : All destination wells are processed; source wells accessed as needed (looping through sources when they are exhausted).
- "Either" : Algorithm stops when first limit is hit. An invalid value raises a "stop condition" error.

17. **Split volume constraints:** If `splitVolume` is `true`:

- **Splitting engages only when each aspirate matches a single dispense.** It is not valid when `repeats` is greater than 1 or `repeatsByVolume` is `true`.
- Each split cycle inherits its technique's overhead (trailing air gap / blowout).
- The split calculator sums those per-cycle aspirates against the source well's tracked volume, so split sources must be trackable — author `volumeType: "Known"` with realistic `evalAmounts` in Instrument Setup (see Instrument Setup). An `"Unknown"` source under `splitVolume: true` cannot be validated and fails enqueue.
- If `splitVolumeCleaning` is also `true` and either `changeTipsBetweenSources` or `changeTipsBetweenDests` is `true`, tips are cleaned (changed or washed) between split cycles.

18. **Leave tips vs. discard:** If `leaveTipsOn` is `true`, tips are left on the pod; if `false`, they are discarded to the location defined in Instrument Setup.

19. **Item count constraint:** `items` must contain at least 2 elements (one source, one destination).

20. **Item `labwareClass` must match (or be compatible with) the labware at `position`:** At enqueue the resolved labware class is checked against each item's `labwareClass`. A mismatch that is not a compatible substitute raises a class-mismatch error; an entirely unknown expected class raises a "labware class not found" error.

21. **`useCurrentTips` requires tips already on the pod:** If `useCurrentTips` is `true` but no tips are loaded on the pod when the step runs, enqueue fails with a "no tips loaded" error.

22. **Non-LLS tips are incompatible with LLS-enabled techniques (Span-8):** The stock Span-8 pipetting techniques enable liquid level sensing on the initial descent, but non-LLS tip classes (e.g., `T190F`, `T50F` — filtered tips with `LLS: false` in the catalog) cannot liquid-level-sense. Combining them with the default auto-selected Span-8 technique hard-fails enqueue with a wrong-tip error. To use non-LLS tips, either pick an LLS-capable tip class or set `overrideHeight: true` on the item with a fixed-reference `heightFrom` (`1 / "Bottom"` or `2 / "Top"`) so the technique's initial LLS branch is bypassed.

23. **Volume tracking (cross-reference):** The engine tracks a running per-well volume for the source and destination wells and rejects both over-dispense (fills exceeding the well/labware capacity) and under-aspirate (drawing more than the tracked source volume). Author source starting volumes and reservoir dead volumes in Instrument Setup — see the "Volume tracking" note there.
24. **Combine delta:** Item 24 in the Transfer step's validation rules does not apply, as Fixed-8 pods are not valid for Combine.

Structural Context

Leaf step — no `subSteps` .

Canonical Example

Two sources combined into one destination (multichannel-style pod; `span8: false`). Well-selection `comArrays` truncated for readability — supply a full boolean array per item as in Transfer.

```
{
  "stepType": "Combine",
  "parameters": {
    "type": "Combine",
    "pod": "Pod1",
    "span8": false,
    "dynamic?": true,
    "items": [
      {
        "source": true,
        "operation": "Aspirate",
        "position": "P5",
        "labwareClass": "BCFlat96",
        "liquidType": "Well Contents",
        "volume": "50",
        "prototype": "MC",
        "autoSelectPrototype": false,
        "overrideHeight": false,
        "height": 0.0,
        "heightFrom": 0,
        "customHeight": false,
        "localPattern": true,
        "startAtSelection": true,
        "startAtMark": false,
        "setMark": true,
        "colsFirst": true,
        "rowsFirst": false,
        "useExpression": false,
        "sectionExpression": "",
        "referencedPattern": "",
        "dataSetPattern": false,
        "wellsX": 12,
        "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] },
        "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] }
      },
      {
        "source": true,
        "operation": "Aspirate",
        "position": "P6",
        "labwareClass": "BCFlat96",
        "liquidType": "Well Contents",
        "volume": "50",
        "prototype": "MC",
        "autoSelectPrototype": false,
        "overrideHeight": false,
        "height": 0.0,
        "heightFrom": 0,
        "customHeight": false,
        "localPattern": true,
        "startAtSelection": true,
        "startAtMark": false,
        "setMark": true,
        "colsFirst": true,
        "rowsFirst": false,
        "useExpression": false,
        "sectionExpression": "",
        "referencedPattern": "",
        "dataSetPattern": false,

```



```

    "wellsX": 12,
    "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] },
    "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] }
  },
  {
    "source": false,
    "operation": "Dispense",
    "position": "p9",
    "labwareClass": "BCFlat96",
    "liquidType": "Tip Contents",
    "volume": "100",
    "prototype": "MC",
    "autoSelectPrototype": false,
    "overrideHeight": false,
    "height": 0.0,
    "heightFrom": 0,
    "customHeight": false,
    "localPattern": true,
    "startAtSelection": true,
    "startAtMark": false,
    "setMark": true,
    "colsFirst": true,
    "rowsFirst": false,
    "useExpression": false,
    "sectionExpression": "",
    "referencedPattern": "",
    "dataSetPattern": false,
    "wellsX": 12,
    "pattern": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] },
    "selectionInfo": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, false, false, false] }
  }
],
"stop": "Sources",
"repeats": "1",
"repeatsByVolume": false,
"replicates": "1",
"useJIT": true,
"useFixedTips": false,
"useDisposableTips": false,
"useMandrelSelection": true,
"useProbes": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, true, true, true, true, true, true, true] },
"useExpression": false,
"mandrelExpression": "",
"useCurrentTips": false,
"changeTipsBetweenSources": false,
"changeTipsBetweenDest": true,
"leaveTipsOn": false,
"splitVolume": false,
"splitVolumeCleaning": false,
"tipLocation": "BC230",
"wash": false,
"span8Wash": false,
"washVolume": "",
"washCycles": "",
"solvent": "Water",
"span8WashVolume": "2",
"span8WasteVolume": "1",
"autoSelectActiveWashTechnique": false,
"activeWashTechnique": "",
"showTipHandlingDetails": false,
"showTransferDetails": false,
"wizard": false
}
}

```

Common Mistakes

- **Setting type to anything but "Combine"** : a Transfer-style value will silently reshape the semantics.
- **items array with fewer than 2 entries**: Combine marks items 0..N-2 as sources and the last as destination (Rule 1), so at least 2 are required. A truly single-entry `items` array fails enqueue with a "no source/destination defined" error. A 2-entry `items` (one source + one destination) is *legal* Combine and behaves like a Transfer — consider using Transfer instead for clarity.
- **Omitting useProbes / span8 on Span-8 hardware**: Include both when targeting a Span-8 pod.
- **Source labware-class mismatch**: When the deck position holds a class different from the source item's `items[i].labwareClass`, enqueue fails with a class-mismatch error.

Span-8 Transfer From File

Property	Value
stepType	"Span-8 Transfer From File"
Category	Leaf
Terminator	N/A
Compatible Hardware	Span-8 pods only (i5, i7)

Behavior Summary

Span-8 Transfer From File is a **Span-8-only** step that reads a comma-delimited text file (`.csv` / `.txt`) and builds one well-to-well aspirate/dispense per data row.

The step uses the file contents and configured source/destination items panels to generate a *pick list*. The pick list is five parallel lists of source position, source well, destination position, destination well and volume.

The pick list is pipetted using the Span-8 Transfer engine.

What the file supplies vs. what the step supplies. The file must always contain the **source well** column; it *may* also carry source position, destination position, destination well and volume. Each override/read boolean decides whether a piece of data comes from the file or from the step's own `items` panels:

- `overrideSource` / `overrideDestination` — **true** means "**do NOT read from file; use the fixed position configured on the source/destination item**"; **false** means "read the position from the file column." (This is inverted from the intuitive reading — see the Enumerated Values note and Validation Rule 3.)
- `readDestWells` — **true** reads the destination well from a file column; **false** computes the next unused destination well from the destination item's `pattern` /marks.
- `readVolume` — **true** reads the volume from a file column; **false** uses the destination item's `volume` .
- `skipZero` — drops rows whose volume evaluates to 0.

Use it when a downstream process (a hit-picking / cherry-picking output, a normalization table, a primer-plating list) already knows exactly which source well goes to which destination well and how much — i.e. an arbitrary, data-driven set of individual transfers that would be impractical to express as patterned Transfer/Combine steps.

The file-row → transfer mapping

- Each **line** of the file is either:
- a comment (first character `'` , an apostrophe — ignored)
 - the header row (skipped when `fileHeader` is **true**)
 - a data row that becomes **one aspirate + dispense**.
 - Fields are split on commas by the file parser.

Column indices (`sourceCol` , `sourceWellCol` , `destCol` , `destWellCol` , `volumeCol`) are **1-based** and are read directly from the step's parameters.

Which columns must exist depends on the booleans:

Datum	Read from file when	Column key	Column required (>0) when
Source position	<code>overrideSource == false</code>	<code>sourceCol</code>	<code>overrideSource == false</code>
Source well	<i>always</i>	<code>sourceWellCol</code>	<i>always</i>
Destination position	<code>overrideDestination == false</code>	<code>destCol</code>	<code>overrideDestination == false</code>
Destination well	<code>readDestWells == true</code>	<code>destWellCol</code>	<code>readDestWells == true</code>
Volume	<code>readVolume == true</code>	<code>volumeCol</code>	<code>readVolume == true</code>

Wells may be given as **numeric well numbers** (`23` , `32`) or **alphanumeric addresses** (`A5` , `B3`); the same file may mix both. They are converted against the resolved labware geometry.

Volume is parsed as a floating-point number; a negative or unparseable volume raises a "bad volume at line N" error.

Positions may be **deck positions** (`P4` , `P7`) or **labware names** (`Source1` , `Dest`); every field is first run through the expression engine, so cells may contain expressions.

Worked example (5 columns, header row present):

```
Source,SourceWell,Dest,DestWell,Volume
Primers,11,Dest,1,80
Primers,5,Dest,1,35
...
```

The matching configuration is: `fileHeader: true` , `overrideSource: false` , `overrideDestination: false` , `readDestWells: true` , `readVolume: true` , `sourceCol: 1` , `sourceWellCol: 2` , `destCol: 3` , `destWellCol: 4` , `volumeCol: 5` . Row 2 aspirates 80 µL from labware `Primers` well 11 into `Dest` well 1; Row 3 aspirates 35 µL from labware `Primers` well 5 into the same `Dest` well 1 (two separate transfers into the same destination well). Other typical shapes include a 5-column layout with numeric wells only (`SamplePlate,HitWell,Dest,DestWell,Vol`), and mixed layouts where source wells are alphanumeric (`A5` , `B3` , ...).

When the destination well is *not* read from file (`readDestWells: false`): the step ignores `destWellCol` and instead walks the destination item's selection `pattern` , handing out the next unused well each row; when the current destination panel's wells are exhausted it advances to the next destination item, and raises a "not enough destination wells" error if all destinations run out.

Parameters Reference Table

Serialized (first-letter-lowercase) casing is shown.

Universal base keys (`caption` , `defaultCaption` , `disabled` , ...) — see Base Keys & Serialization Conventions.

Envelope / `comArray` shape — see Method JSON Structure.

Pipetting keys (`prototype` / `autoSelectPrototype` / `overrideHeight` / `height` / `heightFrom`) — see Pipetting Techniques, Templates, and Auto-Selection.

Unlike the plain `Transfer` step and other span-8 pipetting steps (such as 15-Span-8-Aspirate / 16-Span-8-Dispense) some keys are not serialized for this step.

Not serialized by this step: `type` / `Type` , `podType` , `stop` , `replicates` , `numberOfTips` , `unloadLocation` , `groupTransfer` , `dynamic?` . Do **not** author these keys for this step type.

Step-Level Keys — Identity & Pod

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	""	Name of the Span-8 pod. Must resolve to a Span-8 pod; otherwise enqueue fails (Rule 1). This argument is used as a literal pod name. This step cannot use an expression here.
span8	boolean	—	Yes	true	Always true . Marks the step as Span-8. Real, functional ..

Step-Level Keys — File Source (the distinguishing feature)

Key	Type	Units	Required	Default	Description
destCol	integer	—	Yes	0	1-based file column for destination position. Used/validated only when <code>overrideDestination == false</code> .
destWellCol	integer	—	Yes	0	1-based file column for destination well. Used/validated only when <code>readDestWells == true</code> .
filename	string (expression-capable)	—	Yes	""	Full path to the <code>.csv</code> / <code>.txt</code> data file. Evaluated with the expression engine, so <code>=</code> -expressions and variables are allowed. An empty filename fails validation. In a Validated Run the file is auto-registered.
fileHeader	boolean	—	Yes	false	<code>true</code> $\Rightarrow$ the first data line is a header and is skipped.
fileInfoCollapsed	boolean	—	No	false	Inert (UI-state) . Whether the File Properties panel is collapsed in the editor. No runtime effect.
overrideDestination	boolean	—	Yes	false	<code>true</code> = use the fixed destination item position for every row; <code>false</code> = read destination position from <code>destCol</code> . Same inversion as <code>overrideSource</code> .
overrideSource	boolean	—	Yes	false	true = do NOT read the source position from the file; use the fixed position on the source item (<code>items[0].position</code>) . false = read the source position from <code>sourceCol</code> . (Inverted from intuition)
readDestWells	boolean	—	Yes	false	<code>true</code> = read destination well from <code>destWellCol</code> ; <code>false</code> = compute the next unused destination well from the destination item's <code>pattern</code> /marks.
readVolume	boolean	—	Yes	false	<code>true</code> = read the transfer volume from <code>volumeCol</code> ; <code>false</code> = use the destination item's <code>volume</code> .
skipZero	boolean	—	Yes	false	<code>true</code> = rows whose volume evaluates to 0 are dropped.
sourceCol	integer	—	Yes	0	1-based file column for source position. Used/validated only when <code>overrideSource == false</code> .
sourceWellCol	integer	—	Yes	0	1-based file column for source well. Always required (>0).
volumeCol	integer	—	Yes	0	1-based file column for volume. Used/validated only when <code>readVolume == true</code> .

Step-Level Keys — Probe / Tip Selection

Semantics identical to the `Transfer` and Span-8 aspirate/dispense steps.

See Probe & Mandrel Selection.

Key	Type	Units	Required	Default	Description
mandrelExpression	string	—	Conditional	""	Expression evaluating to the probe selection; required when <code>useExpression == true</code> .
useDisposableTips	boolean	—	Yes	false	One of the Span-8 tip-mode keys. When <code>true</code> , all non-fixed (disposable-tip) probes are used and <code>useProbes</code> is ignored. Do not also set <code>useFixedTips</code> .

Key	Type	Units	Required	Default	Description
<code>useExpression</code>	boolean	—	Yes	<code>false</code>	<code>true</code> ⇒ probe selection comes from <code>mandrelExpression</code> instead of <code>useProbes</code> .
<code>useFixedTips</code>	boolean	—	Yes	<code>false</code>	One of the Span-8 tip-mode keys. When <code>true</code> , only fixed-tip probes are used and <code>useProbes</code> is ignored. Do not also set <code>useDisposableTips</code> .
<code>useMandrelSelection</code>	boolean	—	Yes	<code>true</code>	Inert (UI-state). The engine does not read it; it persists the editor's selection mode.
<code>useProbes</code>	comArray (boolean[8])	—	Yes	All <code>true</code>	Which of the 8 Span-8 probes are active. <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...8...]}</code> . At least one <code>true</code> (Rule 11).

Step-Level Keys — Tip Handling

Key	Type	Units	Required	Default	Description
<code>useCurrentTips</code>	boolean	—	Yes	<code>false</code>	<code>true</code> = reuse tips already on the pod; <code>false</code> = load fresh tips from <code>tipLocation</code> .
<code>tipLocation</code>	string	—	Conditional	<code>""</code>	Tip labware class / deck position to load when <code>useCurrentTips == false</code> .
<code>leaveTipsOn</code>	boolean	—	Yes	<code>false</code>	<code>true</code> = keep tips on the pod after the step; <code>false</code> = unload when finished.
<code>changeTipsBetweenSources</code>	boolean	—	No	<code>false</code>	Change/wash tips when the source position changes between transfers.
<code>changeTipsBetweenDests</code>	boolean	—	No	<code>true</code>	Change/wash tips when the destination changes.
<code>splitVolume</code>	boolean	—	Yes	<code>false</code>	Split large transfers into evenly divided aspirations, chunked to the smaller of the tip's usable capacity (net of blowout and air gap) and the syringe's usable per-stroke capacity (net of system air gap, blowout and trailing air gap, then adjusted for calibration) — not the raw syringe maximum. The single-dispense-per-draw restriction is enforced by the editor only: a method authored directly in JSON can set <code>splitVolume</code> to <code>true</code> alongside a repeat count above 1 and will not be rejected when the method runs.
<code>splitVolumeCleaning</code>	boolean	—	Yes	<code>false</code>	Clean tips between partial (split) transfers. Only meaningful when <code>splitVolume</code> and a between-transfer change/wash are on.
<code>useJIT</code>	boolean	—	Yes	<code>true</code>	When <code>true</code> , aspirate and dispense operations are bundled into a single Just-In-Time block to ensure no other actions occur in between the aspirate and dispense.
<code>repeats</code>	string (expression-capable)	count or μL	Yes	<code>"1"</code>	Configuration for multidispense behavior. If <code>repeatsByVolume == false</code> : number of dispenses per draw. If <code>true</code> : max μL to aspirate per draw for repeat-dispensing. Multidispensing is not a common usage and requires specialized pipetting techniques. This should typically be authored and set to <code>"1"</code> .
<code>repeatsByVolume</code>	boolean	—	Yes	<code>false</code>	Selects the meaning of <code>repeats</code> (see above).

Step-Level Keys — Wash

Key	Type	Units	Required	Default	Description
<code>span8Wash</code>	boolean	—	Yes	<code>false</code>	Passive (flush) Span-8 wash between transfers. Mutually exclusive with <code>wash</code> in the editor.
<code>span8WashVolume</code>	string (expression-capable)	mL	Yes	<code>"2"</code>	Passive-wash volume (mL).

Key	Type	Units	Required	Default	Description
<code>span8WasteVolume</code>	string (expression-capable)	mL	Yes	<code>"1"</code>	Volume dispensed to waste before/after the passive wash (mL).
<code>wash</code>	boolean	—	Yes	<code>false</code>	Active wash (aspirate/dispense at a wash station) between transfers.
<code>solvent</code>	string	—	Yes	<code>"Water"</code>	Active-wash solvent name.
<code>washVolume</code>	string (expression-capable)	μL or %	Yes	<code>"110%"</code>	Active-wash volume; a trailing % means percent of the previously used capacity of the tip.
<code>washCycles</code>	string (expression-capable)	count	Yes	<code>"3"</code>	Number of active-wash cycles.
<code>autoSelectActiveWashTechnique</code>	boolean	—	Yes	<code>false</code>	Auto-select the active-wash technique.
<code>activeWashTechnique</code>	string	—	Yes	<code>""</code>	Named active-wash technique; ignored when <code>autoSelectActiveWashTechnique == true</code> or a custom technique is bound.
<code>customActiveWashTechnique</code>	object (<code>_biomekType:"technique"</code>)	—	No	<i>(absent)</i>	Inline custom wash technique. Present only when a custom technique is defined. Takes precedence over <code>activeWashTechnique</code> .

Step-Level Keys — UI State / Inactive

Key	Type	Units	Required	Default	Description
<code>showTipHandlingDetails</code>	boolean	—	No	<code>true</code>	Inert (UI-state). Tip-handling panel expanded (stored as the negation of "collapsed").
<code>showTransferDetails</code>	boolean	—	No	<code>true</code>	Inert (UI-state). Transfer-details panel expanded.
<code>wizard</code>	boolean	—	No	<code>false</code>	Inactive — not read at runtime for authored methods — wizard-created flag; hides the pod panel in the editor only.
<code>items</code>	array (object[])	—	Yes	<code>[]</code>	The two pipetting panels: <code>items[0]</code> = source, <code>items[1]</code> = destination. See Item-Level Keys. Additional destination items are allowed only when neither <code>overrideDestination</code> nor <code>readDestWells</code> is set (extra dest panels for pattern-based multi-destination filling).

Item-Level Keys

`items` is the serialized form of the source/destination pipetting panels. `items[0]` is the source, `items[1]` the (primary) destination, Additional destination items are allowed only when neither `overrideDestination` nor `readDestWells` is set (extra dest panels for pattern-based multi-destination filling).

These keys mirror the `Transfer` item keys (see Transfer) and drive the per-row aspirate/dispense build at enqueue.

Key	Type	Units	Required	Default	Description
<code>source</code>	boolean	—	Yes	<code>true</code> (item0)/ <code>false</code> (item1)	Marks source vs destination panel.
<code>position</code>	string (expression-capable)	—	Yes	<code>""</code>	Fixed deck position / labware name. Always required non-empty on both <code>items[0]</code> and <code>items[1]</code> (Rule 2) — enqueue validates it regardless of <code>overrideSource</code> / <code>overrideDestination</code> . Its <i>value</i> is only used for pipetting when the corresponding <code>override*</code> is <code>true</code> (or as the destination when <code>readDestWells == false</code>); when the file supplies the position it is otherwise informational, but must still be

Key	Type	Units	Required	Default	Description
					non-empty. An empty position fails validation.
labwareClass	string	—	Yes	""	Labware class at this position. Empty triggers the same "step not configured" validation (Rule 2). Also used to size the generated well pattern.
volume	string (expression-capable)	μL	Conditional	""	Per-transfer volume for the destination item; used when readVolume == false. Ignored when volume is read from the file.
liquidType	string	—	Yes	"Water"	Liquid type for technique selection (source: e.g. "Well Contents" ; dest: e.g. "Tip Contents" or a named liquid).
prototype	string	—	Required	""	Named pipetting technique; required when autoSelectPrototype == false. Key must be present on every item, but may be empty.
autoSelectPrototype	boolean	—	No	false	Auto-select the technique.
customPrototype	object (_biomekType: "technique")	—	No	(absent)	Inline custom technique; overrides prototype. Copied through only when bound.
overrideHeight	boolean	—	Yes	false	Override the technique's tip height with height / heightFrom.
height	number/expression string	mm	Conditional	0.0	Tip-height offset when overrideHeight == true.
heightFrom	integer or string	—	Conditional	0	Height reference: 0 / "Liquid", 1 / "Bottom", 2 / "Top".
customHeight	boolean	—	No	false	Encoding discriminator for height: false = numeric offset; true = text/custom height, which also implies overrideHeight. Use false with numeric heights. See Pipetting Techniques, Templates, and Auto-Selection.
pattern	comArray (boolean[wells])	—	Conditional	(local)	Well-selection mask for the destination item when readDestWells == false (drives the next-unused-well walk). Required (non-empty selection) in that mode or a "pattern not specified" error fires (Rule 9).
selectionInfo	comArray (boolean[wells]) or int[]	—	Conditional	mirrors pattern	Section/quadrant indices or a duplicate of pattern. Keep it in sync with pattern — on the destination item it must be present and correct whenever readDestWells == false and item-level useExpression == false; an absent or malformed value fails the step.
sectionExpression	string	—	Conditional	""	Section expression when useExpression == true (item-level).
useExpression	boolean	—	No	false	Item-level: use sectionExpression instead of pattern.

Key	Type	Units	Required	Default	Description
<code>localPattern</code>	boolean	—	No	<code>true</code>	<code>pattern</code> is stored inline. When <code>false</code> , <code>referencedPattern</code> names a global pattern.
<code>dataSetPattern</code>	boolean	—	No	<code>false</code>	Derive the selection from a data set (<code>dataSetCondition</code> / <code>dataSetConditionType</code>).
<code>referencedPattern</code>	string	—	Conditional	<code>""</code>	Named global pattern when <code>localPattern == false</code> ; a missing/unknown name fails validation.
<code>dataSetConditionType</code>	string	—	No	<code>""</code>	Data-set condition operator (used with <code>dataSetPattern</code>).
<code>dataSetCondition</code>	string	—	No	<code>""</code>	Data-set condition value.
<code>wellsX</code>	integer	—	No	auto	Column count of the labware; used for pattern indexing / print layout.
<code>colsFirst</code>	boolean	—	No	<code>true</code>	Column-major fill order (left-to-right, then down) for pattern-based destination assignment. Complement of <code>rowsFirst</code> ; the editor's <code>MarkInfoGrid</code> always serializes both.
<code>rowsFirst</code>	boolean	—	No	<code>false</code>	Row-major fill order (top-to-bottom, then right) for pattern-based destination assignment. The Transfer From File step reads <i>this</i> key (not <code>colsFirst</code>) to select row-major walking when <code>readDestWells == false</code> . Complement of <code>colsFirst</code> ; author both together (<code>rowsFirst: true, colsFirst: false</code> for row-major; <code>rowsFirst: false, colsFirst: true</code> for column-major).
<code>startAtSelection</code>	boolean	—	No	<code>true</code>	Start from the beginning of the selection (vs. from the mark).
<code>startAtMark</code>	boolean	—	No	<code>false</code>	Start from the labware mark.
<code>setMark</code>	boolean	—	No	<code>false</code>	When <code>true</code> , the mark on the labware is updated after the transfer completes to the last well accessed.

Enumerated / Constrained Values

`overrideSource` / `overrideDestination` (note the inversion)

Value	Meaning
<code>false</code>	Read the position from the file (<code>sourceCol</code> / <code>destCol</code>). This is the common case; the editor checkbox "File specifies ... position in column" is checked .
<code>true</code>	Do not read from file — use the single fixed position on the source/destination <code>item</code> . Editor checkbox unchecked .

`readDestWells` / `readVolume`

Value	Meaning
<code>true</code>	Read destination well / volume from the file column (<code>destWellCol</code> / <code>volumeCol</code>).

Value	Meaning
false	Compute destination well from the destination item's pattern/marks / use the item's <code>volume</code> .

repeatsByVolume

Value	repeats means
false	Number of dispenses per aspirate (draw).
true	Maximum μ L to aspirate per draw (repeat-dispensing).

Both boolean values are valid.

heightFrom (per item)

Value	Meaning
0 / "Liquid"	From liquid surface.
1 / "Bottom"	From well bottom.
2 / "Top"	From well top.

Cross-Field Validation Rules

Each rule describes the enqueue-time behavior it enforces.

- Span-8 pod required.** `pod` must resolve and be a Span-8 pod. A missing pod or a non-Span-8 pod each fail with a distinct error.
- Both panels must be configured.** `items` must have ≥ 2 entries and both the source (`items[0]`) and destination (`items[1]`) must have non-empty `position` and `labwareClass` ; otherwise the step is treated as not configured.
- File-column validity depends on the booleans:** each required column that is missing or ≤ 0 fails with a distinct "invalid column" error.
- Filename required.** Empty `filename` (after expression evaluation) fails validation; a path that cannot be opened fails at file-open time.
- Row parsing.** A data row with fewer fields than a referenced column fails to parse.
- Position / labware resolution** (per row): an empty source or destination position, an unknown position, or missing labware / missing section info each fail with a distinct message.
- Well validity.** A source or destination well outside `1..WellsX*WellsY` (numeric or after alphanumeric translation) fails validation.
- Volume validity.** When `readVolume == true` , a non-numeric or negative cell fails at that row.
- Pattern required for computed destinations.** When `readDestWells == false` , the destination item must provide a selection: an empty selection or an unknown named pattern each fail. When `readDestWells == true` , `pattern` is **still required**. It may be set to all `true` but it must exist.
- Destination capacity.** With computed destinations, running out of unused wells across all destination items fails.
- Probe selection:** `useProbes` must have ≥ 1 `true` , or when `useExpression == true` , `mandrelExpression` must be non-empty. All selected probes must share tip type and syringe size at runtime.
- Active vs. passive wash are mutually exclusive** in the editor (`wash` clears `span8Wash` and vice-versa); do not author both `true` .
- Transfer step validations.** Because the step hands the pipetting off to the ordinary Span-8 Transfer, that engine surfaces additional enqueue errors, including: a blank liquid type; an unknown named liquid; an unknown technique; `useCurrentTips: true` with no tips on the pod; a bad wash / waste / repeat count value; and an active wash whose solvent has no wash station.
- Do not use passive wash with disposable tips.** Passive wash may only be used with fixed tips. Only use active wash when disposable tips are loaded.

Structural Context

Leaf step; no `subSteps` . Omit `subSteps` or provide `[]` .

Canonical Example

Reads a 5-column file (source position, source well, dest position, dest well, volume) with a header row onto Span-8 Pod1 , auto-selecting techniques, loading fresh BC230 tips, changing tips between transfers.

The pod name is per-instrument configuration — read it from the target instrument rather than copying the example's value.

```
{
  "stepType": "Span-8 Transfer From File",
  "parameters": {
    "activeWashTechnique": "",
    "autoSelectActiveWashTechnique": false,
    "changeTipsBetweenDestS": false,
    "changeTipsBetweenSources": true,
    "defaultCaption": "Transfer From File",
    "destCol": 3,
    "destWellCol": 4,
    "fileHeader": true,
    "fileInfoCollapsed": false,
    "filename": "C:\\Biomek\\Data\\transferfromfile.csv",
    "items": [
      {
        "autoSelectPrototype": true,
        "colsFirst": true,
        "customHeight": false,
        "dataSetPattern": false,
        "height": 0.0,
        "heightFrom": 0,
        "labwareClass": "BCFlat96",
        "liquidType": "Well Contents",
        "localPattern": true,
        "overrideHeight": false,
        "position": "P5",
        "pattern": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true]
        },
        "prototype": "",
        "referencedPattern": "",
        "rowsFirst": false,
        "sectionExpression": "",
        "setMark": false,
        "source": true,
        "startAtMark": false,
        "startAtSelection": true,
        "useExpression": false,
        "volume": "",
        "wellsX": 12
      },
      {
        "autoSelectPrototype": true,
        "colsFirst": true,
        "customHeight": false,
        "dataSetPattern": false,
        "height": 0.0,
        "heightFrom": 0,
        "labwareClass": "BCFlat96",
        "liquidType": "Tip Contents",
        "localPattern": true,
        "overrideHeight": false,
        "pattern": {
          "_biomekType": "comArray",
          "arraySubtype": "boolean",
          "values": [true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true, true]
        },
        "position": "P9",
        "prototype": "",
        "referencedPattern": "",
        "rowsFirst": false,
        "sectionExpression": "",
        "setMark": false,
        "source": false,
        "startAtMark": false,

```

```

        "startAtSelection": true,
        "useExpression": false,
        "volume": "",
        "wellsX": 12
    }
],
"leaveTipsOn": false,
"mandrelExpression": "",
"overrideDestination": false,
"overrideSource": false,
"pod": "Pod1",
"readDestWells": true,
"readVolume": true,
"repeats": "1",
"repeatsByVolume": false,
"showTipHandlingDetails": true,
"showTransferDetails": true,
"skipZero": false,
"solvent": "Water",
"sourceCol": 1,
"sourceWellCol": 2,
"span8": true,
"span8Wash": false,
"span8WashVolume": "2",
"span8WasteVolume": "1",
"splitVolume": false,
"splitVolumeCleaning": false,
"tipLocation": "BC230",
"useCurrentTips": false,
"useDisposableTips": true,
"useExpression": false,
"useFixedTips": false,
"useJIT": true,
"useMandrelSelection": true,
"useProbes": {
    "_biomekType": "comArray",
    "arraySubtype": "boolean",
    "values": [true, true, true, true, true, true, true, true]
},
"volumeCol": 5,
"wash": false,
"washCycles": "3",
"washVolume": "110%",
"wizard": false
}
}

```

Common Mistakes

- **Missing filename or unreadable path:** the step reads `filename` at enqueue and opens the CSV file up-front; a bad path fails before any pipetting starts.
- **CSV column indices off-by-one:** `sourceCol`, `sourceWellCol`, `destCol`, `destWellCol`, `volumeCol` are **1-based**. Zero-based indices resolve to the wrong column silently — the transfer runs but reads incorrect data.
- **Header row mis-set:** setting `fileHeader: false` when the file has a header treats the header text as data; setting it `true` when the file has no header skips a real row. Verify against the actual file.
- **Mixing overrideSource: true with a non-zero sourceCol:** when `overrideSource` is `true` the file's source-position column is *not* read, so `sourceCol` is ignored. Leave `sourceCol: 0` (or reset it) when overriding, to avoid confusion for later readers. The same applies to `overrideDestination` / `destCol`, `readDestWells` / `destWellCol`, and `readVolume` / `volumeCol`.
- **Assuming labware auto-derives:** each item carries its own `labwareClass` (`items[0].labwareClass` for the source panel, `items[1].labwareClass` for the destination) and must match the deck positions the file's rows reference. On mismatch the runtime throws a class-mismatch error.

Span-8 Serial Dilution

Property	Value
stepType	"Span-8 Serial Dilution"
Category	Leaf (expands into many enqueued child steps at runtime)
Terminator	N/A
Compatible Hardware	i5, i7 (Span-8 pod required)

Behavior Summary

The Span-8 Serial Dilution step performs a serial dilution across a contiguous run of wells on a plate (or tube rack) using a Span-8 pod.

When enqueued, the step:

. validates the step configuration, including:

- the pod is a Span-8 pod
- the dilution plate (or tube rack) resolves properly
- the well selection pattern is contiguous along the chosen `direction`
- the probe selection is valid

. When the step is configured to pre-fill the plate with diluent from a reservoir (`prepareDiluentInPlate`),

- it aspirates from the diluent reservoir and dispenses to the diluent labware
- the diluent volume is computed to meet the ratio `1:dilutionRatio` given the dilution `volume`.
- the diluent step uses `diluentAspirateProperties` / `diluentDispenseProperties` to control the pipetting behavior.
- Tips are changed or washed per the distinct diluent tip handling settings.

. It then walks the wells transferring `volume` `µL` of `liquidtype`

between adjacent wells along `direction` * Tips are changed or washed per the passive/active wash settings * Aspirate/dispense technique and height for the serial transfers come from the `aspirateProperties` / `dispenseProperties` sub-dictionaries;

. When `transferToWaste` is true, discard the excess from the last wells to waste.

Parameters Reference Table

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `defaultCaption` , `disabled` , ...) live there and are **not** repeated here.

Note: unlike the atomic Span-8 steps this step does **not** carry `dynamic?` at the top level.

Key-naming decoder (the cryptic `a` -prefix families):

- `a...` = tip cleaning settings for the *main* serial transfer
- `aDil...` = tip cleaning settings for the *diluent* transfer.

Step-Level Keys — Position, labware, pod, volume

Key	Type	Units	Required	Default	Description
pod	string (expression)	—	Yes	""	Span-8 pod name. Must resolve to a Span-8 pod.
where	string (expression)	—	Yes	""	Deck position of the dilution plate; validated at enqueue.
position	string	—	Yes	""	Deck position of the dilution plate. Must be the same value as <code>where</code> .
what	string	—	Yes	""	Labware class of the dilution plate; validated at enqueue.
labwareClass	string	—	Yes	""	Labware class of the dilution plate. Must be the same value as <code>what</code> .
volume	string (expression)	µL	Yes	""	Transfer volume per serial step.
liquidtype	string	—	Yes	"Water"	Liquid type for the serial transfers. Serialized as <code>liquidtype</code> (all lowercase) — differs from Fixed-8 steps which use <code>liquidType</code> ; the runtime dictionary is case-insensitive.
direction	string (expression)	—	Yes	"Left to Right"	Dilution direction. Only "Left to Right" or "Top to Bottom" are legal.
dilutionRatio	string (expression)	ratio	Yes	"2"	Dilution ratio 1:N; must be ≥ 1.0 . Diluent volume = $\text{volume} \times (\text{dilutionRatio} - 1)$.
wellsX	integer	—	No	(from labware)	Number of columns of the dilution plate; the editor writes it from the labware class (-1 if unknown).

Step-Level Keys — Tip handling

Key	Type	Units	Required	Default	Description
tips	string	—	Yes	""	Selected tip type / labware class.
tipLocation	string	—	Conditional	""	Tip source used when loading fresh tips — a tip labware class (what the editor writes, e.g. "BC230"), a tip-box labware name, or a deck position.
loadTips	boolean	—	Yes	true	When <code>true</code> , the step auto-loads/refreshes tips as needed; when <code>false</code> , it requires tips already on the pod.
useFixedTips	boolean	—	No	false	Use the pod's fixed tips. Mutually exclusive with <code>useDisposableTips</code> /mandrel selection.
useDisposableTips	boolean	—	No	false	Use disposable tips on the pod.
useMandrelSelection	boolean	—	No	false	Inert UI-state: persists the editor's probe-selection mode (fixed / disposable / mandrel); not read by the enqueue engine.
useCurrentTips	boolean	—	No	true (editor writes true)	UI-written constant (<code>true</code>); the runtime tip lifecycle is decided per substep, not from this key. Treat as inert-UI-state .
leaveTipsOn	boolean	—	No	true (editor writes true)	UI-written constant (<code>true</code>). Inert at this level; the enqueued substeps set their own <code>LeaveTipsOn</code> .

Step-Level Keys — Probe & well selection

Key	Type	Units	Required	Default	Description
<code>useProbes</code>	comArray (boolean[8])	—	No	<i>(all true)</i>	Per-probe selection. <code>{"_biomekType":"comArray","arraySubtype":"boolean","values":[...]}</code> . Reads with a tolerant fallback to the all-probes-selected default, so omission does not throw (consistent with the Span-8 Load/Unload/Wash docs).
<code>mandrelExpression</code>	string	—	No	""	Expression-based probe selection.
<code>useMandrelExpression</code>	boolean	—	No	false	Authoritative flag: use <code>mandrelExpression</code> for probes.
<code>useSectionExpression</code>	boolean	—	No	false	Authoritative flag: use <code>sectionExpression</code> for well selection.
<code>useExpression</code>	boolean	—	No	false	Scratch/overloaded key — both the mandrel selector and the labware view persist to this same key (<code>UseExpression</code>), so its exported value is not authoritative. Prefer the two authoritative flags above; preserve <code>useExpression</code> verbatim on round-trip.
<code>sectionExpression</code>	string	—	No	""	Well-selection expression. Alternative to <code>selectionInfo</code> / <code>pattern</code> when <code>useSectionExpression: true</code> .
<code>selectionInfo</code>	comArray (boolean)	—	Yes	<i>(empty)</i>	Boolean well-selection mask over the plate wells. One of two mutually-exclusive selection modes — either <code>selectionInfo</code> + <code>pattern</code> , or <code>useSectionExpression: true</code> + <code>sectionExpression</code> .
<code>pattern</code>	comArray (boolean)	—	Yes	<i>(derived)</i>	Boolean well pattern (parallel to <code>selectionInfo</code>). Always required — even when using <code>useSectionExpression</code> , the engine reads <code>pattern</code> via a throwing accessor before checking the expression flag, so include a full-length placeholder (e.g., all-false array of the correct length) when using expression-based selection.

Well numbering. Wells are 1-based and **row-major**: on a 96-well plate `A1` = 1, `A12` = 12, `B1` = 13, ..., `H12` = 96. See [Well Numbering](#).

Serial Dilution selection keys vs. simple Span-8 steps. On Serial Dilution, author `useMandrelExpression` (probe selection) and `useSectionExpression` (well selection) — not `useExpression` , which is a non-authoritative scratch key overwritten at enqueue.

Step-Level Keys — Dilution behavior

Key	Type	Units	Required	Default	Description
<code>transferToWaste</code>	boolean	—	No	false	Aspirate and discard the excess volume from the last well of each dilution series.
<code>prepareDiluentInPlate</code>	boolean	—	No	false	Pre-fill the plate wells with diluent from a reservoir before the serial transfers (adds diluent at <code>1:dilutionRatio</code>).
<code>prepareFirstWell</code>	boolean	—	No	false	When pre-filling, also add diluent to the first wells of the series.

Step-Level Keys — Main transfer, passive wash (fixed tips) / tip change

Key	Type	Units	Required	Default	Description
<code>washBetweenTransfers</code>	boolean	—	Yes	false	Passive-wash (fixed tips) or change tips (disposable) between serial transfers.
<code>washVolume</code>	string (expression)	mL	Conditional	""	Passive-wash rinse volume (fixed tips).
<code>wasteVolume</code>	string (expression)	mL	Conditional	""	Passive-wash waste volume (fixed tips).

Key	Type	Units	Required	Default	Description
aTipCleanMode	string	—	Conditional	"After Dispensing"	Passive clean/change timing: "After Dispensing" Or "After Every Sample" . (The a here is the wash-mode prefix, not "active".)
preWashVolume	string (expression)	mL	No	"2"	Pre-wash rinse volume applied once at the start if fixed tips are already dirty. Read by enqueue but not written by the editor — safe to omit.
preWasteVolume	string (expression)	mL	No	"2"	Pre-wash waste volume for the same start-of-run wash. Not written by the editor.

Step-Level Keys — Main transfer, active wash

Key	Type	Units	Required	Default	Description
activeWashBetweenTransfers	boolean	—	Yes	false	Use an active wash between serial transfers.
aSolvent	string	—	Conditional	"Water"	Active-wash solvent liquid type.
aWashVolume	string (expression)	μL or %	Conditional	"0"	Active-wash volume per cycle.
aWashCycles	string (expression)	—	Conditional	"0"	Active-wash cycle count.
aTipActiveCleanMode	string	—	Conditional	"After Dispensing"	Active clean timing: "After Dispensing" Or "After Every Sample" .
activeWashTechnique	string	—	Conditional	""	Active-wash technique name.
autoSelectActiveWashTechnique	boolean	—	No	false	Auto-select the active-wash technique. Persisted by the editor but read tolerantly (defaults to false if omitted); not a runtime requirement.
customActiveWashTechnique	object (technique)	—	No	(absent)	Inline custom active-wash technique. Present only when a custom technique is defined.

Step-Level Keys — Diluent transfer, passive wash / active wash

Key	Type	Units	Required	Default	Description
washBetweenDiluentTransfers	boolean	—	No	false	Passive-wash (fixed tips) or change tips between diluent dispenses. Read tolerantly (defaults to false if omitted).
diluentWashVolume	string (expression)	mL	Conditional	"2"	Diluent passive-wash rinse volume.
diluentWasteVolume	string (expression)	mL	Conditional	"2"	Diluent passive-wash waste volume.
activeWashBetweenDiluentTransfers	boolean	—	No	false	Use an active wash between diluent transfers. Read tolerantly (defaults to false if omitted).
aDilSolvent	string	—	Conditional	"Water"	Diluent active-wash solvent.
aDilWashVolume	string (expression)	μL or %	Conditional	"0"	Diluent active-wash volume per cycle.
aDilWashCycles	string (expression)	—	Conditional	"0"	Diluent active-wash cycle count.
aDilWashTechnique	string	—	Conditional	""	Diluent active-wash technique name (root name "ADilWashTechnique").

Key	Type	Units	Required	Default	Description
<code>autoSelectADilWashTechnique</code>	boolean	—	No	<code>false</code>	Auto-select the diluent active-wash technique. Read tolerantly (defaults to <code>false</code> if omitted).
<code>customADilWashTechnique</code>	object (technique)	—	No	<i>(absent)</i>	Inline custom diluent active-wash technique. Present only when defined.

Step-Level Keys — Technique sub-dictionaries (nested dictionary objects)

Key	Type	Required	Description
<code>aspirateProperties</code>	object	Yes	Technique/height for the serial-transfer aspirate . See sub-dict shape below.
<code>dispenseProperties</code>	object	Yes	Technique/height for the serial-transfer dispense .
<code>diluentAspirateProperties</code>	object	Conditional	Source config + technique/height for the diluent aspirate (from a reservoir). Required when <code>prepareDiluentInPlate</code> is <code>true</code> .
<code>diluentDispenseProperties</code>	object	Conditional	Technique/height for the diluent dispense into the plate.

Inert / UI-state keys

The keys below are written by the editor but never read by the enqueue path — they are UI mirrors or panel state. Safe to omit; preserve verbatim on round-trip if present.

Key	Written by editor	Classification	Notes
<code>amount</code>	always	inert UI-mirror	Mirror of <code>volume</code> . The engine reads <code>volume</code> , never <code>amount</code> .
<code>firstLoad</code>	always	inert UI-state	Tracks whether the "Load Tips" checkbox has been applied once; UI bookkeeping only.
<code>displayDiluentProps</code>	always	inert UI-state	Collapse state of the diluent panel. Cosmetic.
<code>isUserSelected</code>	conditional	inert UI-state	Tracks whether the tip type was manually chosen. Cosmetic.
<code>defaultCaption</code>	always	base	See Base Keys & Serialization Conventions.

Nested sub-dictionary shapes

`aspirateProperties` / `dispenseProperties`

Plain dictionary (no `_biomekType`). Keys written by the editor:

Sub-key	Type	Meaning
<code>prototype</code>	string	Technique name. Required whenever the parent sub-dictionary is present — the key must appear (the value may be <code>""</code> when <code>autoSelectPrototype</code> is <code>true</code>), or the step fails.
<code>autoSelectPrototype</code>	boolean	Auto-select technique. Required whenever the parent sub-dictionary is present — omitting the key fails the step, regardless of its value.
<code>customPrototype</code>	object	Custom technique (only when defined)
<code>operation</code>	string	"Aspirate" / "Dispense"
<code>height</code>	number/expression	Tip height offset. Required whenever the parent sub-dictionary is present — the key must appear even when <code>overrideHeight</code> is <code>false</code> .
<code>heightFrom</code>	integer	<code>0</code> =Liquid, <code>1</code> =Bottom, <code>2</code> =Top. Required whenever the parent sub-dictionary is present , on the same terms as <code>height</code> .

Sub-key	Type	Meaning
<code>overrideHeight</code>	boolean	Override technique height. Required whenever the parent sub-dictionary is present — omitting the key fails the step, regardless of its value.
<code>customHeight</code>	boolean	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/custom height, which also implies <code>overrideHeight</code> . See Pipetting Techniques, Templates, and Auto-Selection. Required whenever the parent sub-dictionary is present , on the same terms as <code>height</code> and <code>heightFrom</code> .
<code>firstWell</code>	integer	1-based first well
<code>firstWellExpression</code>	string	First-well expression
<code>useWellExpression</code>	boolean	When true use <code>firstWellExpression</code> , when false use the well index <code>firstWell</code> .

See Pipetting Techniques, Templates, and Auto-Selection for `prototype` / `height` semantics.

`diluentAspirateProperties`

Everything in `aspirateProperties`, **plus** the diluent source config and a well selection:

Sub-key	Type	Meaning
<code>diluentLocation</code>	string	Deck position of the diluent reservoir
<code>diluentLiquidType</code>	string	Diluent liquid type
<code>diluentLabwareClass</code>	string	Reservoir labware class (must be a reservoir)
<code>selectionInfo</code> / <code>pattern</code>	comArray (boolean)	Reservoir section selection
<code>wellsX</code>	integer	Reservoir columns
<code>sectionExpression</code> / <code>useExpression</code>	string / boolean	Expression-based section selection

`diluentDispenseProperties`

Same shape as `aspirateProperties` / `dispenseProperties` (technique + height + well-selection keys).

Enumerated / Constrained Values

`direction`

Value	Meaning
<code>"Left to Right"</code>	Dilute along rows, left→right (probes may address multiple rows at once).
<code>"Top to Bottom"</code>	Dilute along columns, top→bottom (single-probe pipetting).

Both are source constants and the only legal values (Rule 4).

`aTipCleanMode` / `aTipActiveCleanMode`

Value	Meaning
<code>"After Dispensing"</code>	Clean/change tips after every dispense.
<code>"After Every Sample"</code>	Clean/change tips only between samples (dilution series).

Both are source constants.

Cross-Field Validation Rules

Each rule describes the enqueue-time behavior it enforces.

1. **Span-8 pod required** — the pod must be a Span-8 pod; an unresolvable pod name is a distinct error.
2. **Position present** — an unresolved `where` / `position` fails validation.
3. **Configured** — missing labware/pattern fails validation; the editor summary shows a "not yet configured" placeholder when `aspirateProperties` is absent.
4. **Direction** — anything other than the two legal values is rejected.
5. **Labware kind** — the dilution plate must be a titerplate or tube rack; any other labware class is rejected.
6. **Contiguous selection** — the selected wells must be contiguous along `direction`; a gap fails validation.
7. **Probes selected** — the resolved probe mask must not be empty.
8. **Tips loaded when not loading** — with `loadTips: false` there must be tips on the pod.
9. **Reachability** — the selected probes must be able to reach every row/column; unreachable rows/columns are reported.
10. **Diluent (only when `prepareDiluentInPlate` is `true`)** — the diluent side has its own configuration checks: labware type, labware kind (reservoir), section selection, position, and ratio.

Structural Context

This step is a **leaf** — no authored `subSteps`.

Canonical Example

Minimal Left-to-Right dilution on a 96-well plate, disposable tips, tip change between transfers, no diluent pre-fill. The `pattern` and `selectionInfo` comArrays below are full-length (96 values, one per well of a 12-column x 8-row plate) — columns 1-4 are selected. Array length must equal the plate's well count; a truncated pattern trips the "not configured" check.

The Span-8 pod is `Pod1` on a standalone Span-8 instrument (i5-Span, i7-Span) and `Pod2` (the second/rightmost pod) on an i7 Hybrid. The example uses `Pod1` because it is valid on a standalone Span-8 — use whichever pod your instrument defines.

```
{
  "stepType": "Span-8 Serial Dilution",
  "parameters": {
    "aDilSolvent": "Water",
    "aDilWashCycles": "0",
    "aDilWashTechnique": "",
    "aDilWashVolume": "0",
    "aSolvent": "Water",
    "aTipActiveCleanMode": "After Dispensing",
    "aTipCleanMode": "After Dispensing",
    "aWashCycles": "0",
    "aWashVolume": "0",
    "activeWashBetweenDiluentTransfers": false,
    "activeWashBetweenTransfers": false,
    "activeWashTechnique": "",
    "amount": "20",
    "aspirateProperties": {
      "autoSelectPrototype": true, "customHeight": false, "firstWell": 1,
      "firstWellExpression": "", "height": 0.0, "heightFrom": 0,
      "operation": "Aspirate", "overrideHeight": false, "prototype": "",
      "useWellExpression": false
    },
    "autoSelectADilWashTechnique": false,
    "autoSelectActiveWashTechnique": false,
    "diluentAspirateProperties": {
      "autoSelectPrototype": true, "customHeight": false,
      "diluentLabwareClass": "", "diluentLiquidType": "", "diluentLocation": "",
      "firstWell": 1, "firstWellExpression": "", "height": 0.0, "heightFrom": 0,
      "operation": "Aspirate", "overrideHeight": false, "prototype": "",
      "sectionExpression": "", "useExpression": false, "useWellExpression": false, "wellsX": 1
    },
    "diluentDispenseProperties": {
      "autoSelectPrototype": true, "customHeight": false, "firstWell": 1,
      "firstWellExpression": "", "height": 0.0, "heightFrom": 2,
      "operation": "Dispense", "overrideHeight": false, "prototype": "",
      "useWellExpression": false
    },
    "diluentWashVolume": "2",
    "diluentWasteVolume": "2",
    "dilutionRatio": "2",
    "direction": "Left to Right",
    "dispenseProperties": {
```

```

    "autoSelectPrototype": true, "customHeight": false, "firstWell": 1,
    "firstWellExpression": "", "height": 0.0, "heightFrom": 2,
    "operation": "Dispense", "overrideHeight": false, "prototype": "",
    "useWellExpression": false
  },
  "displayDiluentProps": false,
  "firstLoad": true,
  "labwareClass": "BCFlat96",
  "leaveTipsOn": true,
  "liquidType": "Water",
  "loadTips": true,
  "mandrelExpression": "",
  "pattern": {
    "_biomekType": "comArray", "arraySubtype": "boolean",
    "values": [
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false
    ]
  },
  "pod": "Pod1",
  "position": "P13",
  "prepareDiluentInPlate": false,
  "prepareFirstWell": false,
  "selectionInfo": {
    "_biomekType": "comArray", "arraySubtype": "boolean",
    "values": [
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false,
      true, true, true, true, false, false, false, false, false, false, false, false
    ]
  },
  "sectionExpression": "",
  "tipLocation": "BC230",
  "tips": "BC230",
  "transferToWaste": false,
  "useCurrentTips": true,
  "useDisposableTips": true,
  "useExpression": false,
  "useFixedTips": false,
  "useMandrelExpression": false,
  "useMandrelSelection": false,
  "useProbes": {
    "_biomekType": "comArray", "arraySubtype": "boolean",
    "values": [true, true, true, true, true, true, true, true, true]
  },
  "useSectionExpression": false,
  "volume": "20",
  "washBetweenDiluentTransfers": false,
  "washBetweenTransfers": true,
  "washVolume": "2",
  "wasteVolume": "2",
  "wellsX": 12,
  "what": "BCFlat96",
  "where": "P13"
}

```

Common Mistakes

- **Non-contiguous well selection along direction** : the selected wells must form a contiguous run along the chosen direction ("Left to Right" or "Top to Bottom"). A gap trips the contiguity check (Rule 6). Gaps in the perpendicular axis are allowed.
- **Using a non-titerplate/tuberack**: the dilution plate must be a titerplate or tube rack. Any other labware class (reservoir, tip box, etc.) fails Rule 5.
- **Forgetting to mirror where / position and what / labwareClass** : both mirror-pairs must carry the same value. Omitting either half fails validation.

- **Confusing** `aTipCleanMode` **with** `aTipActiveCleanMode` : `aTipCleanMode` is the **passive** clean/change timing (fixed tips or disposable tip-change) and pairs with `washBetweenTransfers` . `aTipActiveCleanMode` is the **active** clean timing and pairs with `activeWashBetweenTransfers` . They are not interchangeable.
- **Omitting diluent-side properties when** `prepareDiluentInPlate: true` : turning on diluent pre-fill requires `diluentAspirateProperties` (with source config — `diluentLocation` , `diluentLabwareClass` , `diluentLiquidType` , and a reservoir well selection) **and** `diluentDispenseProperties` . Missing/empty either → the diluent validation strings in Rule 10 fire.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — technique/height keys inside the four `*Properties` sub-dicts.
- **Method JSON Structure** — `comArray` encoding for `useProbes` / `selectionInfo` / `pattern` .
- **Base Keys & Serialization Conventions** — base keys, casing, `disabled` .

[†] Table of contents

Span-8 Aspirate

Property	Value
stepType	"Span-8 Aspirate"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7

Behavior Summary

The Span-8 Aspirate step removes liquid from source labware using one or more probes on a Span-8 pod. It provides direct control over individual probe aspiration volumes and well access.

At enqueue time the step validates pod type (must be a Span-8 pod), verifies selected probes have tips of the same type and syringe size, resolves the target labware position and first well, and optionally refreshes tips. If tips were last used for a dispense, a wash step is automatically inserted before aspiration to discard residual liquid.

The step shares its implementation with Span-8 Dispense (the `aspirate` boolean key distinguishes the two).

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>aspirate</code>	boolean	—	No	<code>true</code>	Must be <code>true</code> for this step type. Distinguishes Span-8 Aspirate from Span-8 Dispense (which sets this to <code>false</code>). Omission defaults to <code>true</code> (correct for aspirate), but author explicitly for clarity and to distinguish from Span-8 Dispense.
<code>pod</code>	string	—	Yes	<code>""</code>	Name of the Span-8 pod to use (e.g., <code>"Pod1"</code> on a standalone Span-8 — i5-Span / i7-Span; <code>"Pod2"</code> on an i7 Hybrid). Must resolve to a Span-8 pod; an unresolved name raises an "unknown pod" error, and a non-Span-8 pod raises a "wrong pod type" error.
<code>podType</code>	string	—	No	<code>"Span8"</code>	Pod-family label — always <code>"Span8"</code> for this step type. If you write it, keep it correct: an out-of-sync value misrepresents the step wherever the label is shown. It does not select the pod family — that comes from the named <code>pod</code> — and omitting it does not fail the step, so it is not author-required. Every editor-produced step carries <code>podType: "Span8"</code> .
<code>where</code>	string	—	Yes	<code>""</code>	Deck position or labware name where the source labware is located. Supports expressions (prefix with <code>=</code>).
<code>what</code>	string	—	Yes	<code>""</code>	Labware class name of the source labware (e.g., <code>"BCF1at96"</code>). Must be a pipettable labware class. This field uses the short labware class name; the

Key	Type	Units	Required	Default	Description
					LabwareClasses\\ path prefix is used only in Instrument Setup's <code>deckItems</code> .
<code>useProbes</code>	<code>comArray (boolean[8])</code>	—	No	All <code>true</code>	Array of eight booleans indicating which probes (1–8) are active. Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]}</code> . Omission does not throw (consistent with the Span-8 Load/Unload/Wash docs).
<code>amount</code>	<code>string</code>	μL	Conditional	<code>""</code>	Volume to aspirate, uniform across all selected probes. Supports expressions. Required unless <code>amounts</code> is specified.
<code>amounts</code>	<code>comArray (string[8])</code>	μL	Conditional	Not present	Array of eight volume strings, one per probe, for individual probe volumes. Overrides <code>amount</code> when present. Supports expressions. Serialized as a <code>comArray</code> envelope — <code>{"_biomekType": "comArray", "arraySubtype": "string", "values": [...]}</code> — a <code>SafeArray</code> of strings.
<code>spacing</code>	<code>string</code>	wells	No	<code>""</code>	Number of wells between probes. Determines how far apart selected probes address wells in the labware. Supports expressions. When empty, the step substitutes the labware's minimum interval (labware-dependent: 1 for 96-well, 2 for 384-well).
<code>firstWell</code>	<code>integer</code>	—	Yes	<code>-1</code> (unconfigured)	1-based numeric index of the first well to access (the well assigned to the lowest-numbered active probe). <code>-1</code> is only the truly-unconfigured sentinel (no labware picked yet; <code>enqueue</code> raises a "no well selected" error). Once a labware is configured the editor resolves it to <code>1</code> ; a configured method carries a valid 1-based index (or uses the well expression).
<code>firstWellExpression</code>	<code>string</code>	—	No	<code>""</code>	Expression or well coordinate string used to calculate the first well when <code>useWellExpression</code> is <code>true</code> . Supports alphanumeric well references (e.g., <code>"A1"</code>) and expressions (e.g., <code>"=col"</code>).
<code>useWellExpression</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , the step uses <code>firstWellExpression</code> instead of <code>firstWell</code> to determine the starting well. Omission defaults to <code>false</code> .
<code>liquidtype</code>	<code>string</code>	—	Yes	<code>"Well Contents"</code>	The liquid type for technique selection. May be a named liquid, <code>"Well Contents"</code> (auto-detect from labware), or <code>"Tip Contents"</code> (invalid for aspirate). Serialized all-lowercase (<code>liquidtype</code>) in method exports; case-insensitive import means either casing works, but author the lowercase form to match real exports.
<code>refreshTips</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>true</code> , discards current tips and loads fresh tips from the specified tip labware class before aspirating.
<code>tipLabwareClass</code>	<code>string</code>	—	Conditional	<code>""</code>	Labware class name of tips to load when <code>refreshTips</code> is <code>true</code> (e.g., <code>"BC230"</code>).
<code>autoSelectPrototype</code>	<code>boolean</code>	—	Yes	—	When <code>true</code> , the technique is auto-selected based on liquid type, labware, tip type, and volume.

Key	Type	Units	Required	Default	Description
<code>prototype</code>	string	—	Conditional	<code>""</code>	Name of the pipetting technique to use. Required when <code>autoSelectPrototype</code> is <code>false</code> .
<code>customPrototype</code>	object (<code>_biomekType:"technique"</code>)	—	No	<i>(absent)</i>	Inline custom pipetting technique object. When present, takes precedence over <code>prototype</code> . Omission is safe.
<code>overrideHeight</code>	boolean	—	No	<code>false</code>	When <code>true</code> , the step overrides the technique's default tip height with the values in <code>height</code> and <code>heightFrom</code> . Omission defaults to <code>false</code> (no height override).
<code>height</code>	number or expression-capable string	mm	Conditional	<code>0.0</code>	Tip height offset from the reference point when <code>overrideHeight</code> is <code>true</code> . May be negative (e.g., <code>-2.0</code> means 2 mm below the reference). Expression-capable.
<code>heightFrom</code>	integer or string	—	Conditional	<code>0</code>	Reference point for height measurement: <code>0</code> or <code>"Liquid"</code> = liquid surface, <code>1</code> or <code>"Bottom"</code> = well bottom, <code>2</code> or <code>"Top"</code> = well top. String aliases are case-insensitive. Expression-capable (prefix with <code>=</code>).
<code>customHeight</code>	boolean	—	No	<code>false</code>	Encoding discriminator for <code>height</code> / <code>heightFrom</code> . When <code>false</code> , those keys carry numeric/enumerated values; when <code>true</code> , they carry expression text. The editor writes it on every save. Set it to match the form of <code>height</code> you author.
<code>operation</code>	string	—	No	<code>"Aspirate"</code>	Operation identifier saved by the technique selector. Always <code>"Aspirate"</code> for this step type.
<code>useExpression</code>	boolean	—	No	<code>false</code>	When <code>true</code> , the probe selection is determined by <code>mandrelExpression</code> instead of <code>useProbes</code> .
<code>mandrelExpression</code>	string	—	Conditional	<code>""</code>	Expression that evaluates to the probe selection at runtime. Used when <code>useExpression</code> is <code>true</code> .
<code>emptyTips</code>	boolean	—	No	<code>false</code>	Not applicable to aspirate (used by dispense). Should be <code>false</code> .
<code>discardExcess</code>	boolean	—	No	<code>false</code>	Not applicable to aspirate (used by dispense). Should be <code>false</code> .

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*. Key matching is case-insensitive (§1), so the `stepUI` / `stepui` casing seen in `exports` / `imports` either way; author the first-letter-lowercase form.

Well numbering. Wells are 1-based and **row-major**: on a 96-well plate `A1` = 1, `A12` = 12, `B1` = 13, ..., `H12` = 96. See *Well Numbering*.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
<code>aspirate</code>	<code>true</code>	Must always be <code>true</code> for Span-8 Aspirate
<code>podType</code>	<code>"Span8"</code>	Always <code>"Span8"</code> for this step type

Parameter	Allowed Values	Notes
heightFrom	0, 1, 2 (Or "Liquid", "Bottom", "Top")	0 / "Liquid" = from liquid surface, 1 / "Bottom" = from well bottom, 2 / "Top" = from well top. String aliases are case-insensitive.
operation	"Aspirate"	Always "Aspirate" for this step type
liquidtype	Any named liquid, "Well Contents"	"Tip Contents" is invalid for aspirate steps
useProbes values	true, false	Exactly 8 elements; at least one must be true
spacing	Positive integer (as string), expression, or ""	If empty, uses labware minimum interval. If provided, minimum depends on labware geometry (e.g., minimum 2 for 384-well plates). Supports expressions.

Cross-Field Validation Rules

1. aspirate **must** be true. If false, the step behaves as Span-8 Dispense.
2. The named pod **must** resolve to a Span-8 pod. An unresolved name raises an "unknown pod" error; a pod that is not Span-8 raises a "wrong pod type" error. podType is written as "Span8" by the step itself and is not the source of pod-family resolution.
3. If refreshTips is true, then tipLabwareClass **must** be a non-empty string naming a valid tip labware class.
4. If autoSelectPrototype is false, then prototype **must** be a non-empty string naming a valid pipetting technique.
5. If overrideHeight is true, then height and heightFrom **must** be provided with valid values.
6. If useWellExpression is true, then firstWellExpression **must** be a non-empty string. If false, firstWell must be a valid 1-based well index.
7. If useExpression is true, then mandrelExpression **must** be a non-empty expression string. If false, useProbes must be present with at least one true element.
8. amounts and amount are mutually exclusive: if amounts is present, amount is ignored. One or the other must provide a valid volume.
9. All selected probes (those with true in useProbes) must have the **same tip type** and the **same syringe size** at runtime.
10. liquidtype **must not** be "Tip Contents" for an aspirate step — using it raises a "cannot be used" error. "Well Contents" is valid and means the liquid type is auto-detected from the labware data.
11. emptyTips should be false for aspirate steps; setting it true has no meaningful effect on aspiration.
12. discardExcess should be false for aspirate steps; it is only meaningful for dispense.
13. spacing must be compatible with the labware geometry — e.g., 384-well plates require a minimum spacing of 2. An empty string is valid (substitutes the labware's minimum interval).

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

The Span-8 pod is Pod1 on a standalone Span-8 instrument (i5-Span, i7-Span) and Pod2 (the second/rightmost pod) on an i7 Hybrid. The examples below use Pod1 because it is valid on any Span-8-equipped instrument — use whichever pod your instrument defines.

Example 1: Basic Span-8 Aspirate from a 96-well plate (all 8 probes, auto-select technique)

```
{  
  "stepType": "Span-8 Aspirate",  
}
```

```

"parameters": {
  "amount": "10",
  "aspirate": true,
  "autoSelectPrototype": true,
  "customHeight": false,
  "discardExcess": false,
  "emptyTips": false,
  "firstWell": 1,
  "firstWellExpression": "",
  "height": 0.0,
  "heightFrom": 0,
  "liquidtype": "Well Contents",
  "mandrelExpression": "",
  "operation": "Aspirate",
  "overrideHeight": false,
  "pod": "Pod1",
  "prototype": "",
  "refreshTips": false,
  "spacing": "1",
  "tipLabwareClass": "",
  "useExpression": false,
  "useProbes": {
    "_biomekType": "comArray",
    "arraySubtype": "boolean",
    "values": [true, true, true, true, true, true, true, true]
  },
  "useWellExpression": false,
  "what": "BCFlat96",
  "where": "P13"
}
}

```

Example 2: Aspirate with specific technique, height override, tip refresh, and expression-based first well

```

{
  "stepType": "Span-8 Aspirate",
  "parameters": {
    "amount": "=aspirateVolume",
    "aspirate": true,
    "autoSelectPrototype": false,
    "customHeight": false,
    "discardExcess": false,
    "emptyTips": false,
    "firstWell": 1,
    "firstWellExpression": "=col",
    "height": -2.0,
    "heightFrom": 0,
    "liquidtype": "Water",
    "mandrelExpression": "",
    "operation": "Aspirate",
    "overrideHeight": true,
    "pod": "Pod1",
    "prototype": "S8 1000 High",
    "refreshTips": true,
    "spacing": "1",
    "tipLabwareClass": "BC230",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "useWellExpression": true,
    "what": "BCFlat96",
    "where": "P13"
  }
}

```

Example 3: Aspirate with individual probe volumes and subset of probes

```

{
  "stepType": "Span-8 Aspirate",
  "parameters": {
    "amounts": {
      "_biomekType": "comArray",
      "arraySubtype": "string",
      "values": ["10", "10", "0", "0", "0", "0", "0", "0"]
    },
    "aspirate": true,
    "autoSelectPrototype": false,

```

```

"customHeight": false,
"discardExcess": false,
"emptyTips": false,
"firstWell": 5,
"firstWellExpression": "",
"height": 1.0,
"heightFrom": 1,
"liquidtype": "Serum",
"mandrelExpression": "",
"operation": "Aspirate",
"overrideHeight": true,
"pod": "Pod1",
"prototype": "S8 1000 Low",
"refreshTips": false,
"spacing": "2",
"tipLabwareClass": "",
"useExpression": false,
"useProbes": {
  "_biomekType": "comArray",
  "arraySubtype": "boolean",
  "values": [true, true, false, false, false, false, false, false]
},
"useWellExpression": false,
"what": "BCFlat96",
"where": "P5"
}

```

Common Mistakes

- **"Tip Contents" as liquidtype**: raises a "cannot be used" error. Use **"Well Contents"** or a named liquid.
- **Mixing probe tip types or syringe sizes across the selected probes**: the runtime rejects the mismatch. Applies at the pod, not the step — a probe with a fixed tip and a probe with a disposable tip cannot be co-selected.
- **spacing too small for the labware**: 384-well plates need `spacing >= 2`. `""` (empty string) is fine — the step substitutes the labware's minimum interval.
- **Confusing useExpression (probes) with useWellExpression (well)**: independent flags. Similarly, the serialized key for the well-by-expression flag is `useWellExpression`, not `firstWellByExpression`.
- **Providing both amounts and amount**: `amounts` takes precedence; `amount` is silently ignored. Pick one.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — Technique auto-selection algorithm, height override mechanics, `C__` pipetting variable reference, and template architecture. Covers how `autoSelectPrototype`, `prototype`, `overrideHeight`, `height`, and `heightFrom` interact with the technique system.
- **Mixing in place** — mixing at a well is a technique property. Configure it on the Span-8 technique's **Mix** tab in the Technique Editor (mix volume, cycles, aspirate/dispense heights and speeds, follow-liquid-level, touch-tips, leading-air-gap-and-blowout). An item-level `mixCount` is inert on every pod family and Span-8 Aspirate does not read it. See [Transfer](#) → [Mixing in place](#).

Span-8 Dispense

Property	Value
stepType	"Span-8 Dispense"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7

Behavior Summary

The Span-8 Dispense step delivers liquid from the tips of a Span-8 pod into destination labware. It shares its implementation with Span-8 Aspirate, distinguished by `aspirate` being `false`. At enqueue time the step validates the pod type (must implement `IPodS8`), verifies that all selected probes have tips of the same type and syringe size, resolves the target labware position and first well, and builds one or more group collections based on the selected technique. When `emptyTips` is `true`, the step ignores the `amount` / `amounts` values and instead dispenses the entire current contents of each tip (the volume is determined dynamically from the tip contents at runtime). If `discardExcess` is `true`, after the dispense completes a wash step is enqueued that discards any liquid remaining in the tips to waste. The step sets the pod's valve state to output and records "Dispense" as the last operation on the pod. For liquid type resolution, "Tip Contents" is the standard convention for dispense (it resolves the actual liquid type from the last non-air content in the tips), while "Well Contents" is invalid for dispense steps.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>aspirate</code>	boolean	—	No	<code>false</code>	Must be <code>false</code> for this step type. Distinguishes Span-8 Dispense from Span-8 Aspirate. Omission defaults to <code>true</code> (aspirate behavior) , so this key must be explicitly set to <code>false</code> for correct Span-8 Dispense operation.
<code>pod</code>	string	—	Yes	<code>""</code>	Name of the Span-8 pod to use (e.g., "Pod1" on a standalone Span-8 — i5-Span / i7-Span; "Pod2" on an i7 Hybrid). Must resolve to a Span-8 pod.
<code>podType</code>	string	—	No	<code>"Span8"</code>	Pod-family label — always <code>"Span8"</code> for this step type. If you write it, keep it correct: an out-of-sync value misrepresents the step wherever the label is shown. It does not select the pod family — that comes from the named <code>pod</code> — and omitting it does not fail the step, so it is not author-required. Every editor-produced step carries <code>podType: "Span8"</code> .
<code>where</code>	string	—	Yes	<code>""</code>	Deck position or labware name of the destination. Supports expressions (prefix with <code>=</code>).
<code>what</code>	string	—	Yes	<code>""</code>	Labware class name of the destination labware (e.g., "BCF1at96"). Must be a pipettable labware class.
<code>useProbes</code>	comArray (boolean[8])	—	No	All <code>true</code>	Array of eight booleans indicating which probes (1–8) are active. Serialized as <code>{"_biomekType":</code>

Key	Type	Units	Required	Default	Description
					"comArray", "arraySubtype": "boolean", "values": [...]}. Omission does not throw (consistent with the Span-8 Load/Unload/Wash docs).
amount	string	μL	Conditional	""	Volume to dispense, uniform across all selected probes. Supports expressions (prefix with =). Required unless amounts is specified or emptyTips is true.
amounts	comArray (string[8])	μL	Conditional	Not present	Array of eight volume strings, one per probe, for individual probe volumes. Overrides amount when present. Supports expressions. Serialized as a comArray envelope — {"_biomekType": "comArray", "arraySubtype": "string", "values": [...]} — a SafeArray of strings, the same wrapper useProbes uses (there with arraySubtype: "boolean").
spacing	string	wells	No	""	Number of wells between probes. Determines how far apart selected probes address wells in the labware. Supports expressions. When empty, the step substitutes the labware's minimum interval (labware-dependent: 1 for 96-well, 2 for 384-well).
firstWell	integer	—	Yes	-1 (unconfigured)	1-based numeric index of the first well to access (the well assigned to the lowest-numbered active probe). -1 is only the truly-unconfigured sentinel (no labware picked yet; enqueue raises a "no well selected" error). Once a labware is configured the editor resolves it to 1; a configured method carries a valid 1-based index (or uses the well expression).
firstWellExpression	string	—	No	""	Expression or well coordinate string used to calculate the first well when useWellExpression is true. Supports alphanumeric well references (e.g., "A1") and expressions (e.g., "=col").
useWellExpression	boolean	—	No	false	When true, the step uses firstWellExpression instead of firstWell to determine the starting well. Omission defaults to false.
liquidtype	string	—	Yes	"Tip Contents"	The liquid type for technique selection. May be a named liquid or "Tip Contents" (auto-detect from tip contents). "Well Contents" is invalid for dispense steps. Serialized all-lowercase (liquidtype) in method exports; case-insensitive import means either casing works, but author the lowercase form to match real exports.
emptyTips	boolean	—	No	false	When true, the step dispenses the entire current contents of each tip, ignoring amount / amounts.
discardExcess	boolean	—	No	false	When true, after the dispense any remaining liquid in the tips is discarded to waste via a wash step.
autoSelectPrototype	boolean	—	Yes	—	When true, the technique is auto-selected based on liquid type, labware, tip type, and volume.
prototype	string	—	Conditional	""	Name of the pipetting technique (transfer prototype) to use. Required when

Key	Type	Units	Required	Default	Description
					<code>autoSelectPrototype</code> is <code>false</code> .
<code>customPrototype</code>	object (<code>_biomekType:"technique"</code>)	—	No	(<i>absent</i>)	Inline custom pipetting technique object. When present, takes precedence over <code>prototype</code> . Omission is safe.
<code>overrideHeight</code>	boolean	—	Yes	<code>false</code>	When <code>true</code> , the step overrides the technique's default tip height with the values in <code>height</code> and <code>heightFrom</code> .
<code>height</code>	number or expression-capable string	mm	Conditional	<code>0.0</code>	Tip height offset from the reference point when <code>overrideHeight</code> is <code>true</code> . May be negative. Expression-capable (prefix with <code>=</code>).
<code>heightFrom</code>	integer or string	—	Conditional	<code>0</code>	Reference point for height measurement: <code>0</code> or <code>"Liquid"</code> = liquid surface, <code>1</code> or <code>"Bottom"</code> = well bottom, <code>2</code> or <code>"Top"</code> = well top. String aliases are case-insensitive. Expression-capable (prefix with <code>=</code>).
<code>customHeight</code>	boolean	—	No	<code>false</code>	Encoding discriminator for <code>height</code> / <code>heightFrom</code> . When <code>false</code> , those keys carry numeric/enumerated values; when <code>true</code> , they carry expression text. The editor writes it on every save. Set it to match the form of <code>height</code> you author.
<code>operation</code>	string	—	No	<code>"Dispense"</code>	Operation identifier saved by the technique selector. Always <code>"Dispense"</code> for this step type.
<code>useExpression</code>	boolean	—	No	<code>false</code>	When <code>true</code> , the probe selection is determined by <code>mandrelExpression</code> instead of <code>useProbes</code> .
<code>mandrelExpression</code>	string	—	Conditional	<code>""</code>	Expression that evaluates to the probe selection at runtime. Used when <code>useExpression</code> is <code>true</code> .

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Well numbering. Wells are 1-based and **row-major**: on a 96-well plate `A1` = 1, `A12` = 12, `B1` = 13, ..., `H12` = 96. See *Well Numbering*.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
<code>aspirate</code>	<code>false</code>	Must always be <code>false</code> for Span-8 Dispense
<code>podType</code>	<code>"Span8"</code>	Always <code>"Span8"</code> for this step type
<code>heightFrom</code>	<code>0</code> , <code>1</code> , <code>2</code> (Or <code>"Liquid"</code> , <code>"Bottom"</code> , <code>"Top"</code>)	<code>0</code> / <code>"Liquid"</code> = from liquid surface, <code>1</code> / <code>"Bottom"</code> = from well bottom, <code>2</code> / <code>"Top"</code> = from well top. String aliases are case-insensitive.
<code>operation</code>	<code>"Dispense"</code>	Always <code>"Dispense"</code> for this step type
<code>liquidtype</code>	Any named liquid, <code>"Tip Contents"</code>	<code>"Well Contents"</code> is invalid for dispense steps
<code>useProbes</code> values	<code>true</code> , <code>false</code>	Exactly 8 elements; at least one must be <code>true</code>

Parameter	Allowed Values	Notes
spacing	Positive integer (as string), expression, or ""	If empty, uses labware minimum interval. If provided, minimum depends on labware geometry (e.g., minimum 2 for 384-well plates). Supports expressions.

Cross-Field Validation Rules

1. `aspirate` **must** be `false`. If `true`, the step behaves as Span-8 Aspirate.
2. `podType` is **not author-required** (see §Parameters row) and is safely omitted from authored JSON. The runtime pod-family check is on the *resolved* `pod` (must be a Span-8 pod), not on this key — the step writes `"Span8"` itself at construction, so exports may carry it but authoring is free to leave it out.
3. If `autoSelectPrototype` is `false`, then `prototype` **must** be a non-empty string naming a valid pipetting technique.
4. If `overrideHeight` is `true`, then `height` and `heightFrom` **must** be provided with valid values.
5. If `useWellExpression` is `true`, then `firstWellExpression` **must** be a non-empty string. If `false`, `firstWell` must be a valid 1-based well index.
6. If `useExpression` is `true`, then `mandrelExpression` **must** be a non-empty expression string. If `false`, `useProbes` must be present with at least one `true` element.
7. `amounts` and `amount` are mutually exclusive: if `amounts` is present, `amount` is ignored. One or the other must provide a valid volume — unless `emptyTips` is `true`, in which case both are ignored.
8. All selected probes (those with `true` in `useProbes`) must have the **same tip type** and the **same syringe size** at runtime.
9. `liquidtype` **must not** be `"Well Contents"` for a dispense step; using it raises a "cannot be used" error. Use `"Tip Contents"` or a named liquid type.
10. When `emptyTips` is `true`, `amount` may be empty (`""`) or `"0"`; `amounts` may be absent. The runtime determines volumes dynamically from tip contents.
11. `discardExcess` is only meaningful for dispense steps. When `true`, a wash step is enqueued after the dispense that discards remaining tip contents to waste.
12. `spacing` must be compatible with the labware geometry — e.g., 384-well plates require a minimum spacing of 2. An empty string is valid (substitutes the labware's minimum interval).
13. When `emptyTips` is `true` and `autoSelectPrototype` is `true`, each probe may get a different technique based on its individual tip volume.
14. The dispense volume (`amount` or a per-probe `amounts` entry) **must not** exceed the tip contents: dispensing more than a tip holds raises a "dispense-too-much" error.
15. The dispensed volume **must not** overflow the destination well: exceeding well capacity raises a well-overflow error. Both checks apply per-probe for `amounts`.
16. When `emptyTips` is `true`, the step ignores `amount` / `amounts` **and** `discardExcess` is moot: the editor disables Discard Excess, Volume, Individual Volumes, and Edit... whenever Empty Tips is checked. A method that sets both `emptyTips: true` and `discardExcess: true` is contradictory — `emptyTips` wins and the tips are already emptied, so the discard is redundant.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

The examples use `"pod": "Pod1"`. The Span-8 pod is `Pod1` on a standalone Span-8 instrument (i5-Span, i7-Span) and `Pod2` (the second/rightmost pod) on an i7 Hybrid. `Pod1` is valid on a standalone Span-8. Use whichever pod your instrument defines; an unresolved name raises an "unknown pod" error.

Example 1: Basic Span-8 Dispense to a 96-well plate (all 8 probes, specified technique)

```

{
  "stepType": "Span-8 Dispense",
  "parameters": {
    "amount": "5",
    "aspirate": false,
    "autoSelectPrototype": false,
    "customHeight": false,
    "discardExcess": false,
    "emptyTips": false,
    "firstWell": 9,
    "firstWellExpression": "=col",
    "height": 0.0,
    "heightFrom": 2,
    "liquidtype": "Tip Contents",
    "mandrelExpression": "",
    "operation": "Dispense",
    "overrideHeight": false,
    "pod": "Pod1",
    "prototype": "S8 MultiDispense",
    "spacing": "1",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "useWellExpression": false,
    "what": "BCFlat96",
    "where": "P13"
  }
}

```

Example 2: Empty Tips dispense (dispense all tip contents)

```

{
  "stepType": "Span-8 Dispense",
  "parameters": {
    "amount": "",
    "aspirate": false,
    "autoSelectPrototype": false,
    "customHeight": false,
    "discardExcess": false,
    "dynamic?": true,
    "emptyTips": true,
    "firstWell": 7,
    "firstWellExpression": "",
    "height": 0.0,
    "heightFrom": 2,
    "liquidtype": "Tip Contents",
    "mandrelExpression": "",
    "operation": "Dispense",
    "overrideHeight": false,
    "pod": "Pod1",
    "prototype": "S8 MultiDispense",
    "spacing": "1",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "useWellExpression": false,
    "what": "BCFlat96",
    "where": "P13"
  }
}

```

Example 3: Dispense with height override, expression-based well, and subset of probes

```

{
  "stepType": "Span-8 Dispense",
  "parameters": {
    "amount": "10",
    "aspirate": false,
    "autoSelectPrototype": true,
    "customHeight": false,
    "discardExcess": false,
    "emptyTips": false,
    "firstWell": 1,

```



```
    "firstWellExpression": "=col",
    "height": -2.0,
    "heightFrom": 0,
    "liquidtype": "Tip Contents",
    "mandrelExpression": "",
    "operation": "Dispense",
    "overrideHeight": true,
    "pod": "Pod1",
    "prototype": "",
    "spacing": "1",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, false, false, false, false]
    },
    "useWellExpression": true,
    "what": "BCFlat96",
    "where": "P5"
  }
}
```

Common Mistakes

- **"Well Contents" as liquidtype** : raises a "cannot be used" error. Use `"Tip Contents"` or a named liquid.
- **Mixing probe tip types or syringe sizes across the selected probes**: the runtime rejects the mismatch — a probe with a fixed tip and a probe with a disposable tip cannot be co-selected.
- **spacing too small for the labware**: 384-well plates need `spacing >= 2` . `""` (empty string) is fine — the step substitutes the labware's minimum interval.
- **Confusing useExpression (probes) with useWellExpression (well)**: independent flags. The serialized well-by-expression key is `useWellExpression` , not `firstWellByExpression` .
- **Authoring refreshTips / tipLabwareClass** : aspirate-only. Not written by the span-8 dispense step and should be omitted.

Differences from Span-8 Aspirate

Aspect	Span-8 Aspirate	Span-8 Dispense
aspirate key	true	false
operation key	"Aspirate"	"Dispense"
liquidtype default	"Well Contents"	"Tip Contents"
liquidtype restrictions	"Tip Contents" is invalid	"Well Contents" is invalid
emptyTips	Not applicable (should be false)	Dispense all tip contents when true
discardExcess	Not applicable (should be false)	Discard remaining tip contents after dispense
refreshTips / tipLabwareClass	Available (load fresh tips before aspiration)	Not available (not set at creation)
Post-operation behavior	Records "Aspirate" as last operation	Records "Dispense" as last operation; if next operation is aspirate, a wash is auto-inserted

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — Technique auto-selection algorithm, height override mechanics, `C__` pipetting variable reference, and template architecture. Covers how `autoSelectPrototype` , `prototype` , `overrideHeight` , `height` , and `heightFrom` interact with the technique system.

- **Mixing in place** — mixing at a well is a technique property. Configure it on the Span-8 technique's **Mix** tab in the Technique Editor (mix volume, cycles, aspirate/dispense heights and speeds, follow-liquid-level, touch-tips, leading-air-gap-and-blowout). An item-level `mixCount` is inert on every pod family and Span-8 Dispense does not read it. See Transfer → Mixing in place.

Span-8 Load Tips

Property	Value
stepType	"Span-8 Load Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 (Span-8 pod required)

Behavior Summary

The Span-8 Load Tips step loads disposable tips onto the Span-8 pod's probes. It first discards any currently loaded tips on the selected probes, then finds and loads new tips of the specified type from available tip boxes on the deck. The probe selection determines which of the 8 probes participate — either via a boolean array or a mandrel expression.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	""	Name of the Span-8 pod (e.g., "Pod1"). Must resolve to a Span-8 pod.
tips	string	—	Yes	""	Tip labware class name, labware instance name, or deck position to load from (e.g., "BC80", "BC230_LLS", "TipBox6", "P3"). The step first checks whether the value names a deck position (loading from it directly); otherwise it searches all reachable tip boxes whose class or instance name matches. Supports expressions (prefix with =).
useProbes	comArray (boolean[8])	—	No	(all true)	8-element boolean array (indices 0–7), one per Span-8 mandrel. true = probe participates. Serialized as {"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]} .
mandrelExpression	string	—	No	""	Expression-based probe selection using 1-indexed probe numbers (e.g., "1,3,4" selects probes 1, 3, and 4). Also accepts Biomek expressions (e.g., "=myProbeVar"). Used when useExpression is true . Probe numbers below 1 raise a "must be greater than zero" error.
useExpression	boolean	—	No	false	If true, use mandrelExpression instead of useProbes for probe selection.

The probe-selection keys (useProbes , mandrelExpression , useExpression) stay Optional — omission does not throw; the selector falls back to the standard all-probes-selected default.

Universal base keys (caption , dynamic? , stepUI , ...) apply to this step but are omitted here — see Base Keys & Serialization Conventions.

Enumerated / Constrained Values

No enumerated constraints beyond those listed in the table.

Cross-Field Validation Rules

- pod **must** resolve to a Span-8 pod.
- When useExpression is true , mandrelExpression is used for probe selection; otherwise useProbes is used. The selected probe subset is honored — only those probes are loaded.

3. tips **must** name a tip type retrievable on the deck (by class name, instance name, or deck position). **The deck-position form only works if that position currently holds a tip box.** A position that exists but was left empty in Instrument Setup falls through to the class-and-instance search, which then fails reporting your position name as though it were a tip type (Unable to find and retrieve tips of type TL4 for Pod2.). When the name in that message is a deck position rather than a labware class, the position is empty; place a tip box there. If the value matches a stocked deck position, the step loads from that position directly; otherwise it searches reachable tip boxes and, if none yields tips, raises a "unable to find tips" error with per-box skip reasons appended. Partially used boxes are accepted (unlike the Multichannel path). Automatic tip-box retrieval will relocate and de-lid a sealed box from storage when no reachable open box is available.
4. Loading onto probes that carry non-disposable (fixed) tips raises a "cannot load on fixed tips" error — see the probe-selection note above the second example. Whether a probe is fixed or disposable is **instrument state, not a method choice**: the pod reads it from its own per-probe tip configuration, and no key in this step (or any other) can override it. An instrument whose Span-8 probes are all configured with fixed tips will reject this step on every probe selection. Check the target instrument's `podSettings`. `<pod>.settings.tip1...tip8` before authoring — see Fixed vs. disposable probes (in *Biomek Instrument & Project Reference*).

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

The examples use `"pod": "Pod1"`. The Span-8 pod is `Pod1` on a standalone Span-8 instrument (i5-Span, i7-Span) and `Pod2` (the second/rightmost pod) on an i7 Hybrid. **Do not treat "Pod2" as canonical** — read the pod name from the target instrument's configuration.

Load tips on all probes:

```
{
  "stepType": "Span-8 Load Tips",
  "parameters": {
    "pod": "Pod1",
    "tips": "BC230_LLS",
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "useExpression": false
  }
}
```

`useProbes` is defaulted to "all true" internally but the enqueue path for Span-8 pods that mix fixed and disposable probes routes based on this array; omitting it can raise a "cannot load on fixed tips" error when the default routing chooses a fixed-tip probe. Include the explicit boolean array for a disposable-tip method.

Load tips on specific probes:

```
{
  "stepType": "Span-8 Load Tips",
  "parameters": {
    "pod": "Pod1",
    "tips": "BC80",
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, false, false, false, false]
    },
    "useExpression": false
  }
}
```

Common Mistakes

- **Omitting `useProbes` on a pod that mixes fixed and disposable probes** — the "all-true" internal default can route through a fixed probe and raise a "cannot load on fixed tips" error. Always author the explicit array on such pods.
- **Authoring a disposable-tip method against a fixed-tip instrument** — narrowing `useProbes`, changing `tips`, or adding an unload first will not help: every probe is fixed, and the method has no way to ask for disposable ones. The instrument's probe configuration must be changed (Hardware Setup), or the method must run on an instrument already configured for disposable tips. See Fixed vs. disposable probes (in *Biomek Instrument & Project Reference*).
- **Setting `useExpression: true` but leaving `mandrelExpression` empty** — the `useProbes` array is then ignored and there is no probe selection to consume; author both keys as a pair.
- **Naming the wrong pod for the instrument family** — `Pod1` on standalone Span-8 (i5-Span/i7-Span), `Pod2` on i7 Hybrid. A Multichannel pod name resolves but fails the Span-8 pod check.
- **Assuming a used tip box will be re-picked** — boxes whose tips have all been consumed are skipped with a "has been used" skip reason. The step searches for unused or partially used boxes on the deck, and can automatically retrieve and de-lid a sealed box from storage if no open box is reachable.
- **Searching a named box by its class** — if Instrument Setup gave the tip box a name, `tips` must be that name; its class alone is skipped as a named-instance mismatch, and the error names the box's own name back to you. Author the instance name (or the box's deck position), not the class.
- **Naming a deck position that has no box** — the position-name form of `tips` only works when that position actually holds a tip box. A real-but-empty position falls through to the class search and fails with your position name reported as though it were a tip type.

Span-8 Unload Tips

Property	Value
stepType	"Span-8 Unload Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 (Span-8 pod required)

Behavior Summary

The Span-8 Unload Tips step discards disposable tips from the Span-8 pod's selected probes. Fixed-tip probes in the selection are silently skipped. Typically executed at the end of a pipetting sequence or when switching tip types. Unload tips before the method's Finish step.

Where the tips go

This step takes **no unload-location parameter**. The destination is resolved by the pod from the deck and the tip box the loaded tips came from. Resolution order:

1. If the pod does not remember a source tip box, the location is `<Any Trash>`.
2. Otherwise the pod reads the `DiscardTips` and `DiscardTipsLocation` routing keys off the **tip-box labware object** configured in the Instrument Setup step — not off this step. Span-8 defaults `DiscardTips` to `true` and `DiscardTipsLocation` to `<Any Trash>` when the box does not set them.
3. A **named** position is used only when `DiscardTips` is `true` **and** `DiscardTipsLocation` is a position name. That position must exist on the deck, or the step raises a "deck position does not exist" error: `<pod> cannot discard tips. Deck position "<position>" does not exist.` The pod moves directly to the position, using the position's Span-8 tip-disposal offsets, so it must be a real tip-disposal position.
4. In every other case the pod searches the deck for the **closest position it can reach that carries the "Can Discard Tips" characteristic** — a tip-trash / waste ALP. If no such position is both present and reachable by this pod, the step raises the "tip discard location" error:

`<pod>` is unable to find a tip discard location. This differs from the named-position error above; the wording distinguishes a missing name from a missing/unreachable trash.

The Span-8 pod has no return-to-box path for disposable tips. `DiscardTips: false` and `DiscardTipsLocation: "<TipBox>"` (which sets `DiscardTips` to `false`) still force the trash search, producing the same "tip discard location" error on a deck with no reachable trash.

Deck prerequisite

A method that loads and unloads Span-8 disposable tips requires a tip-trash / waste ALP with the "Can Discard Tips" characteristic within the Span-8 pod's reach. **"Can Discard Tips" is a deck-position characteristic, not a step parameter.** It is set in the instrument's deck configuration (Deck Editor or factory scripts); this step can only reference positions that already exist. On a deck with no reachable tip trash, no step parameters or tip-box routing keys can resolve the problem — the remedy is in Instrument Setup or the deck layout.

Reachability is pod-specific. The auto-search filters candidates through this pod's own reach check, and on a two-pod instrument the other pod's bridge narrows that reach — so a trash present on the deck can still be unreachable here. Move it toward this pod's side or reassign which pod unloads; see *Reachability and Access (in Biomek Instrument & Project Reference)* §"Two pods on one bridge".

Parameters Reference Table

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	""	Name of the Span-8 pod (e.g., "Pod1"). Must resolve to a Span-8 pod.
useProbes	comArray (boolean[8])	—	No	(all true)	8-element boolean array (indices 0–7), one per Span-8 mandrel. true = probe participates. Serialized as { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [...] } .
mandrelExpression	string	—	No	""	Expression-based probe selection. Used when useExpression is true .
useExpression	boolean	—	No	false	If true , use mandrelExpression instead of useProbes for probe selection.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Enumerated / Constrained Values

No enumerated constraints.

Cross-Field Validation Rules

1. `pod` **must** name a pod that exists on the instrument and resolve to a Span-8 pod. An unresolvable name raises an "unknown pod" error; a wrong pod type raises a validation error.
2. A tip discard location **must** be reachable **by this pod** (pod-specific, narrowed by the other pod's bridge on two-pod instruments). See [Where the tips go](#) and [Deck prerequisite](#).
3. When `useExpression` is `true` , `mandrelExpression` is used for probe selection; otherwise `useProbes` is used.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

The examples use `"pod": "Pod1"` . The Span-8 pod is `Pod1` on a standalone Span-8 instrument (i5-Span, i7-Span) and `Pod2` (the second/rightmost pod) on an i7 Hybrid. **Do not treat "Pod2" as canonical** — read the pod name from the target instrument's configuration.

Unload tips from all probes:

```
{
  "stepType": "Span-8 Unload Tips",
  "parameters": {
    "pod": "Pod1"
  }
}
```

Unload tips from specific probes:

```
{
  "stepType": "Span-8 Unload Tips",
  "parameters": {
    "pod": "Pod1",
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, false, false, false, false]
    },
    "useExpression": false
  }
}
```

```
}  
}
```

Common Mistakes

- **Naming the wrong pod for the instrument family** — `Pod1` on standalone Span-8, `Pod2` on i7 Hybrid; a Multichannel pod name resolves but fails the Span-8 pod check at Enqueue.
- **Setting `useExpression: true` but leaving `mandrelExpression` empty** — the `useProbes` array is then ignored and the expression evaluates to an empty probe set (silent no-op, no error). Author both keys as a pair.
- **No reachable tip-discard location for *this* pod** — raises the "tip discard location" error. No step parameter can fix this; see Deck prerequisite.
- **Trying to steer the discard from this step** — this step has no location key. The only routing controls are `DiscardTips` / `DiscardTipsLocation` on the tip-box labware in Instrument Setup, and on Span-8 they select *which* trash, never a return to the box.
- **Expecting `DiscardTipsLocation: "<TipBox>"` to avoid the trash on Span-8** — it forces the trash search instead. `<TipBox>` returns tips to their box on the Multichannel pod only.
- **Assuming this step throws when probes carry no disposable tips** — the step silently skips probes that have no tips or that carry fixed tips; when no probe qualifies, no pod motion occurs and no error is raised.

Span-8 Wash Tips

Property	Value
stepType	"Span-8 Wash Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 (Span-8 pod required)

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `defaultCaption` , `dynamic?` , `disabled` , ...) are documented once in that file and are **not** repeated here.

Behavior Summary

The Span-8 Wash Tips step rinses tips or the disposable tip mandrels on a Span-8 pod at a wash station. It runs in one of two modes.

In **passive** mode (`activeWash: false` , the default) the step moves the selected probes to a passive Span-8 wash station (deck characteristic `"VWash Station"`), dispenses `toWaste` mL to the waste channel, then dispenses `amount` mL of system liquid while over the wells to rinse; both volumes are entered in **mL and multiplied by 1000 to µL** internally. In **active** mode (`activeWash: true`) the step instead enqueues a Span-8 mix at an active wash station (characteristic `"Wash Station"`), performing `activeWashCycles` cycles of `activeVolume` of the `activeLiquidType` solvent using the selected (or auto-selected) technique.

At enqueue the step resolves the pod (which must be a Span-8 pod), reads the probe selection, eliminates probes whose tips are clean when `dirtyTipsOnly` is set (or that have no content when `dispenseTipContentsOnly` is set), verifies all remaining tips are the same type, and finds the wash station — using `where` if given, otherwise the nearest reachable station of the correct type. After washing it removes tip contents, marks the tips clean (unless `dispenseTipContentsOnly` is set, which only empties them), and records `"Wash"` as the pod's last operation. The step is a **leaf**: it expands into lower-level pod motions (and, for active wash, a Span-8 mix) at runtime, but is authored with no `subSteps` .

Dispensing waste through the wash station. Emptying tip contents into the wash station's waste channel (the `toWaste` volume, or `dispenseTipContentsOnly`) is a common way to discard liquid during a run. It is generally paired with a rinse that dilutes and clears what was sent down — `amount` mL of system fluid over the wells on a passive wash, or an active-wash cycle. Whether a **particular** liquid is appropriate to send through the wash station depends on the liquid and your instrument's plumbing, and is outside the scope of these docs. If you are not sure a liquid is safe to dispense through the wash station, check with Beckman Coulter before relying on it.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>pod</code>	string (expression)	—	Yes	<code>""</code>	Name of the Span-8 pod (e.g. <code>"Pod1"</code>). Must resolve to a Span-8 pod.
<code>where</code>	string (expression)	—	No	<code>""</code> (auto-find)	Deck position of the wash station. Blank $\Rightarrow$ the step finds the nearest reachable station — a <code>"VWash Station"</code> for passive wash or a <code>"Wash Station"</code> (matching <code>activeLiquidType</code>) for active wash.
<code>activeWash</code>	boolean	—	No	<code>false</code>	<code>false</code> = passive wash (dispense to waste + rinse in wells); <code>true</code> = active wash (mix <code>activeVolume</code> $\times$ <code>activeWashCycles</code> at an active

Key	Type	Units	Required	Default	Description
					wash station). Selects which volume/technique keys apply.
amount	string (expression)	mL	Conditional	"1"	Passive rinse volume dispensed while over the wells (converted $\times 1000$ to μL). Used only when <code>activeWash</code> is <code>false</code> . A trailing <code>%</code> is allowed when tips are loaded; entering the volume as a percent specifies the volume of wash fluid as a percent of the maximum volume of fluid contained in the tips in previous steps. For example, if the maximum volume of fluid transferred is 50 μL , and the <code>%</code> is set for 110%, the Wash step washes the tips with 55 μL of solution. However, if no fluid has been transferred and the volume is entered as a percent, no action is taken and tips are not washed.
toWaste	string (expression)	mL	No	"2"	Volume dispensed to the waste channel before the in-well rinse (converted $\times 1000$ to μL). The StepUI writes "2"; if omitted, enqueue falls back to 1 mL.
delayTime	string (expression)	ms	No	"300"	Delay after each dispense cycle. Must be a non-negative number. The StepUI writes "300"; if omitted, enqueue falls back to 0 ms.
dirtyTipsOnly	boolean	—	No	true	When <code>true</code> , only probes whose tips are marked <i>dirty</i> are washed; clean tips are skipped. Defaults to <code>true</code> when omitted.
dispenseTipContentsOnly	boolean	—	No	false	Passive only. When <code>true</code> , the step only empties the remaining tip contents to waste (no fresh rinse); probes with empty tips are skipped.
activeVolume	string (expression)	μL or %	Conditional	""	Active wash volume per cycle. Used only when <code>activeWash</code> is <code>true</code> . A trailing <code>%</code> is allowed when tips are loaded; entering the volume as a percent specifies the volume of wash fluid as a percent of the maximum volume of fluid contained in the tips in previous steps. For example, if the maximum volume of fluid transferred is 50 μL , and the <code>%</code> is set for 110%, the Wash step washes the tips with 55 μL of solution. However, if no fluid has been transferred and the volume is entered as a percent, no action is taken and tips are not washed.
activeWashCycles	string (expression)	—	Conditional	""	Active wash cycle count — passed to the enqueued mix as <code>C__MixCount</code> . Used only when <code>activeWash</code> is <code>true</code> .
activeLiquidType	string	—	Conditional	""	Active wash solvent liquid type — it becomes the <code>liquidType</code> of the mix the step enqueues at the wash station. Required whenever <code>activeWash</code> is <code>true</code> : the key must be present (it is read with no fallback), so omitting it fails the step even when <code>where</code> names the station explicitly. Additionally, when <code>where</code> is blank it selects the wash station by solvent, and <i>that</i> lookup alone tolerates omission by falling back to "Water". Ignored when <code>activeWash</code> is <code>false</code> .
operation	string	—	No	"Mix"	Technique-selector operation tag. Always "Mix" for this step.
autoSelectPrototype	boolean	—	Conditional	false	Active-wash technique auto-selection. Only relevant when <code>activeWash</code> is <code>true</code> . Omission defaults to <code>false</code> (named-technique mode), which then requires a non-empty <code>prototype</code> ; set it explicitly to <code>true</code> to opt into auto-selection. See Pipetting Techniques, Templates, and Auto-Selection.
prototype	string	—	Conditional	""	Active-wash technique name (used when <code>autoSelectPrototype</code> is <code>false</code>).
customPrototype	object (technique)	—	No	(absent)	Inline custom active-wash technique object. Present only when a custom technique was defined in the editor — absent from normal exports.
useProbes	comArray (boolean[8])	—	No	(all fixed probes)	Per-probe selection (indices 0–7). Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "boolean", "values": [...]}</code> . When

Key	Type	Units	Required	Default	Description
					omitted, enqueue falls back to the pod's fixed-probe array.
mandrelExpression	string	—	No	""	Expression-based probe selection. Used when useExpression is true .
useExpression	boolean	—	No	false	When true , probe selection comes from mandrelExpression instead of useProbes .
openPosition	boolean	—	No	true	When true , the deck position is opened before and closed after washing (passive path). Read by enqueue but not written by the editor ; safe to omit.

Inert / inactive keys

Key	Classification	Notes
useSpeedPump	Inactive — not read at runtime	Not referenced by the i-Series Span-8 step; left over from previous Biomek FX/NX generation. This key has no effect on this step. Do not rely on it.
caption	base	See Base Keys & Serialization Conventions. Node-level disabled toggles whether the step runs.

There is no items array and no sub-object array — the Item-Level Keys section does not apply.

Enumerated / Constrained Values

activeWash

Value	Meaning
false	Passive wash: dispense toWaste mL to waste, then rinse amount mL in the wells (fixed tips, empty disposable tips, or disposable tip mandrel dispense only).
true	Active wash: mix activeWashCycles cycles of activeVolume of activeLiquidType at an active wash station.

operation

Value	Meaning
"Mix"	Only value. The wash is executed as a mix operation for technique selection.

amount / activeVolume percent form

Form	Meaning
plain number (e.g. "2")	Absolute volume (mL for passive amount / toWaste ; µL for active activeVolume).
trailing % (e.g. "110%")	Percent of the tip's MaxVolumeUsed . Requires tips to be present (else validation fails).

Cross-Field Validation Rules

Each rule describes the enqueue-time behavior it enforces.

1. **Pod type** — pod must be a Span-8 pod, else the step raises a wrong-pod-type error.
2. **Wash volume parses to a number** — a non-numeric amount / activeVolume raises a "bad wash volume" error.
3. **Wash volume non-negative** — a negative value raises a "must be positive" error; an overflow raises a "too large" error.
4. **Percent volume needs tips** — a % volume with no tips loaded raises a "cannot be specified as percent" error.
5. **Tip present on selected probe** (mixed tip state) — a selected probe with no tip raises a per-probe error.
6. **Uniform tip type** — all selected probes with tips must share a type, else a mixed-type error fires.
7. **Disposable tips + passive wash** — disposable tips cannot be passively washed unless dispenseTipContentsOnly is set.

8. **Wash station resolvable** — if `where` is blank and none is found, or a named `where` is absent, or a named station is the wrong kind, each fails with a distinct error.
9. **Dispense-tip-contents needs tips** — `dispenseTipContentsOnly` with no tips raises a "no tips loaded" error.
10. **Delay time** — non-numeric, negative, and overflow values each fire a distinct error.
11. **Probe selection non-empty** — if no probes remain selected after the dirty/empty filters the step simply does nothing; it is **not** an error.

Structural Context

This step is a **leaf** — it does not support authored `subSteps`. (At runtime it enqueues pod motions and, for active wash, a Span-8 mix; those are engine-generated, not part of the authored method tree.)

Canonical Examples

The examples use `"pod": "Pod1"`. The Span-8 pod is `Pod1` on a standalone Span-8 instrument (i5-Span, i7-Span) and `Pod2` (the second/rightmost pod) on an i7 Hybrid. `Pod1` is valid on a standalone Span-8. Use whichever pod your instrument defines.

Example 1: Passive wash, all probes, auto-find station

```
{
  "stepType": "Span-8 Wash Tips",
  "parameters": {
    "activeLiquidType": "Water",
    "activeVolume": "",
    "activeWash": false,
    "activeWashCycles": "",
    "amount": "1",
    "autoSelectPrototype": true,
    "delayTime": "300",
    "dirtyTipsOnly": true,
    "dispenseTipContentsOnly": false,
    "mandrelExpression": "",
    "operation": "Mix",
    "pod": "Pod1",
    "prototype": "",
    "toWaste": "2",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true, true, true, true, true, true, true, true]
    },
    "where": ""
  }
}
```

Example 2: Active wash, 3 cycles of 150 µL Ethanol, named station, subset of probes

```
{
  "stepType": "Span-8 Wash Tips",
  "parameters": {
    "activeLiquidType": "Ethanol",
    "activeVolume": "150",
    "activeWash": true,
    "activeWashCycles": "3",
    "amount": "1",
    "autoSelectPrototype": false,
    "delayTime": "300",
    "dirtyTipsOnly": true,
    "dispenseTipContentsOnly": false,
    "mandrelExpression": "",
    "operation": "Mix",
    "pod": "Pod1",
    "prototype": "S8 Active Wash",
    "toWaste": "2",
    "useExpression": false,
    "useProbes": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",

```

```

    "values": [true, true, true, true, false, false, false, false]
  },
  "where": "AW1"
}
}

```

Common Mistakes

- **Treating `toWaste` / `amount` as booleans** — both are volumes in **mL** (strings/expressions), not "send to waste?" flags. The unit split: passive `amount` / `toWaste` are mL ($\times 1000$ internally), but active `activeVolume` is μL . `delayTime`, sitting alongside them, is neither — it is milliseconds. Three units in one parameter block, so read the Units column rather than pattern-matching the neighboring keys; a value copied from `amount` into `activeVolume` is a $1000\times$ volume error.
- **Mixing passive and active volume keys** — `activeWash: false` ignores `activeVolume` / `activeWashCycles` / `activeLiquidType`; `activeWash: true` ignores `amount` / `toWaste`. Leave the unused side as empty strings (as the StepUI does) rather than co-authoring both.
- **Passive-washing disposable tips without `dispenseTipContentsOnly`** — passive station rejects disposables outright; the tip-contents flag is the only escape hatch.
- **Providing `useProbes` as a plain JSON array** — must be the `{"_biomekType": "comArray", ...}` wrapper object.
- **Pointing active wash at a non-wash-station position** — an active wash needs an **active** wash station ALP (a station plumbed for the solvent) on the deck, and `where` names it (leave it empty, as above, to let the engine locate it). The stock i5/i7 Span-8 sample decks include only a *passive* station (`W1`) and no active one, so a real active wash requires a site-specific active-wash position your instrument actually has — not a plate slot.
- **Authoring `useSpeedPump`** — inactive; ignored. Do not add it.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — how `autoSelectPrototype` / `prototype` / `customPrototype` resolve the active-wash technique.
- **Base Keys & Serialization Conventions** — base keys, casing, `disabled`.

Multichannel Aspirate

Property	Value
stepType	"Multichannel Aspirate"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `dynamic?` , `disabled` , ...) are **not** re-documented here.

Behavior Summary

The Multichannel Aspirate step removes liquid from source labware using a fixed multichannel head (96-channel or 384-channel). At enqueue time the step validates that the specified pod is a multichannel pod (only multichannel pods are accepted; a Span-8 or Fixed-8 pod will produce an error. It resolves the deck position, verifies that labware is present and on top of any stack at that position, checks that the labware class is compatible with the actual labware at the position, and confirms the labware class has a "Can Pipette" characteristic. If `refreshTips` is `true` , existing tips are discarded and fresh tips are loaded from the specified tip labware class. It then uses the given pipetting technique to aspirate the given volume from the selected wells.

After aspirating with a Multichannel Aspirate step, use a Multichannel Dispense step to dispense the liquid.

Note that Multichannel Aspirate, Multichannel Dispense, Multichannel Mix, and Multichannel Wash Tips share much of their implementation. As a result, some parameters are listed here that are only used by those other steps. See each step's documentation for details.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>operation</code>	string	—	Yes	"Aspirate"	Step action identity. Must be "Aspirate" .
<code>pod</code>	string	—	Yes	""	Name of the multichannel pod to use (e.g., "Pod1"). Must resolve to a multichannel pod. The MC pod is "Pod1" on a standalone Multichannel instrument (i5-MC, i7-MC) and on an i7 Hybrid; it is "Pod2" only when the MC is the right-hand pod of a dual-pod i7. Never author "MC" / "MPod" . See Introduction to Biomek Method JSON §"Instrument Variants" / "Pod Types".
<code>location</code>	string	—	Yes	""	Deck position or labware name where the source labware is located. Supports expressions (prefix with <code>=</code>). Accepts a deck-position name (e.g. P5) or a labware instance name (assigned via the labware's <code>properties.name</code> in Instrument Setup) — matched case-insensitively. Example 1's "SourcePlate" refers to a placed labware instance named "SourcePlate". See Instrument Setup and Labware and Position Resolution.
<code>labwareClass</code>	string	—	Yes	""	Expected labware class at the source position (e.g., "BCF1at96"). Must be compatible with the actual labware. This field uses the

Key	Type	Units	Required	Default	Description
					short labware class name; the <code>LabwareClasses\</code> path prefix is used only in Instrument Setup's <code>deckItems</code> .
<code>liquidtype</code>	string	—	Yes	""	Source liquid classification. Commonly "Well Contents" (auto-detect from labware) or a named liquid type. Must be non-empty.
<code>volume</code>	number or string	μL	Yes	0.0	Amount to aspirate. Positive numeric value or expression string (e.g., "20", "=AspVol"). An aspirate cannot drive the plunger past the installed head's capacity — the pod's D-axis maximum, in μL (e.g. 300 for the 300 μL MC-96 head). The limit applies to the plunger position the move <i>ends</i> at — the pod's current D position plus this move's calibrated volume — so whatever the tips already hold counts against it: a 200 μL aspirate on top of 150 μL already in the tips overruns a 300 μL head even though neither volume alone does. Violations fail at enqueue, before any motion, with <code>Move of <pod> to D of <n> μL violates D axis maximum of <max> μL</code> . Read the ceiling from <code>podSettings.max.d</code> in the instrument-settings export — see Pod Settings Format (in <i>Biomek Instrument & Project Reference</i>) §"What <code>max.d</code> means, and when it is not the pipetting ceiling". Watch computed volumes (base + conditioning/excess terms) that quietly push one aspirate over the ceiling. Separately, an aspirate also cannot exceed the loaded tip's own capacity (e.g. 230 μL for a BC230 tip) — that is a different limit and fails with <code>Too much aspirated into tip(s); max volume is <n> μL and a total of <m> μL has been aspirated</code> .
<code>selectionInfo</code>	comArray (<code>arraySubtype: "integer"</code>)	—	Conditional	"" (empty)	Array of 1-based section/quadrant indices to aspirate from. On these low-level Multichannel steps <code>selectionInfo</code> is a <code>comArray</code> of <code>arraySubtype: "integer"</code> carrying section/quadrant indices (e.g. [1]) — different from the item-level <code>selectionInfo</code> on Transfer/Combine, which is a well selection whose subtype varies by pod (boolean well-mask on Fixed-8, integer on Span-8/MC). Must contain exactly one element — the step requires exactly one target group per invocation (see Cross-Field rule 11); to process multiple sections, use a Loop with <code>sectionExpression</code> . Required when <code>useExpression</code> is <code>false</code> . Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "integer", "values": [...]}</code> .
<code>useExpression</code>	boolean	—	No	false	When <code>true</code> , selection is determined by <code>sectionExpression</code> instead of <code>selectionInfo</code> .
<code>sectionExpression</code>	string	—	Conditional	""	Expression used for section/column/quadrant selection at runtime. Required when <code>useExpression</code> is <code>true</code> .
<code>refreshTips</code>	boolean	—	Yes	false	Whether to discard current tips and load fresh tips before aspirating.
<code>tipLabwareClass</code>	string	—	Conditional	""	The tips to use, specified either as a labware class, a labware instance name, or a position name. Expression capable. Required when <code>refreshTips</code> is <code>true</code> .
<code>autoSelectPrototype</code>	boolean	—	Yes	false	Whether the software auto-selects a technique based on pod type, tip type, labware, liquid type, and volume.
<code>prototype</code>	string	—	Conditional	""	Explicit technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is not present.
<code>customPrototype</code>	object	—	No	Not present	Inline technique object (<code>_biomekType: "technique"</code> payload). When present, overrides both <code>prototype</code> and <code>autoSelectPrototype</code> .

Key	Type	Units	Required	Default	Description
overrideHeight	boolean	—	Yes	false	Whether to override the technique's default aspirate height.
height	number or string	mm	Conditional	0.0	Height offset from the reference point. Required when <code>overrideHeight</code> is <code>true</code> . Supports expressions.
heightFrom	integer or string	—	Conditional	0	Reference point for height: <code>0</code> = liquid surface, <code>1</code> = well bottom, <code>2</code> = well top. Also accepts string aliases <code>"Liquid"</code> , <code>"Bottom"</code> , <code>"Top"</code> .

A liquid-relative height needs a locatable liquid surface. With `heightFrom = 0 / "Liquid"` (or a technique that follows the liquid), the target labware's surface must be computable at run time: its `volumeType` — set in Instrument Setup / Guided Setup — must be `"Nominal"` or `"Known"` **with** an `evalAmounts` fill. Multichannel (96-/384-channel) and Fixed-8 pods do **not** support liquid-level sensing (LLS is Span-8 only), so on those pods an `"Unknown"` well cannot be rescued at run time — the move throws. See [Pipetting Techniques and Templates §"Liquid types" / §"Liquid-relative heights need a locatable liquid surface"](#).

Editor-persisted keys (not read by the Aspirate engine)

These are written on every export by the editor but are **not** read. Include them to match editor output; classify as **inert UI-state**.

Key	Type	Units	Default	Description / classification
customHeight	boolean	—	false	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/custom height, which also implies <code>overrideHeight</code> . Use <code>false</code> with numeric heights. See Pipetting Techniques, Templates, and Auto-Selection .
pattern	comArray (<code>arraySubtype</code> : "boolean")	—	(well-count mask)	Inert UI-state. Boolean well-selection mask maintained by the labware view to restore the visual selection. Engine uses <code>selectionInfo</code> , not <code>pattern</code> .
aspirateTime	expression-capable string	s	"1"	Inactive and obsolete. This key is not used.
mixCount	expression-capable string	—	"1"	Inert UI-state on Aspirate. This key is only used for the Multichannel Mix step. Not read by the Aspirate engine.
washSettleTime	expression-capable string	s	"0"	Inactive and obsolete. This key is not used.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Aspirate"	Must always be "Aspirate" for this step type.
heightFrom	0, 1, 2 (or "Liquid", "Bottom", "Top")	Integer form preferred in JSON. String aliases are resolved at runtime.
liquidtype	Any named liquid, "Well Contents"	"Tip Contents" is invalid for aspirate operations.
selectionInfo values	Positive integers (1-based)	The array iterates the <i>sections</i> the head touches, so the valid values match how many sections the labware exposes to the head — not the plate's column count. For a 96-well plate + 96-channel head there is exactly one section, so only <code>[1]</code> is valid; authoring <code>[1,2,3]</code> fails because the extra indices do not match sections the head exposes on that labware. For a 384-well plate + 96-channel head or 1536-well plate + 384-channel head each labware exposes four interleaved quadrants and the valid values are quadrant indices <code>1 –</code>

Parameter	Allowed Values	Notes
		4 . However, the Multichannel Aspirate step requires that there be only a single section used, so this array must have exactly one value.

Cross-Field Validation Rules

1. `operation` **must** equal `"Aspirate"` . The step defaults to this value.
2. `pod` **must** resolve to a Multichannel pod. A Span-8 pod or Fixed-8 pod will produce a runtime error.
3. `location` **must** resolve to a valid, reachable deck position with labware present and on top of any stack.
4. `labwareClass` **must** be compatible with the actual labware class at the resolved position.
5. The labware class **must** have a `"Can Pipette"` characteristic, or the step raises a runtime error
6. If `refreshTips` is `true` , then `tipLabwareClass` **must** be a non-empty string naming a valid tip labware class, tip box instance name, or position name.
7. If `autoSelectPrototype` is `false` , then `prototype` **must** be a non-empty string naming a valid technique. If the technique cannot be found, the step raises a technique-not-found error.
8. If `autoSelectPrototype` is `true` , at least one matching technique must exist for the given context (pod type, head type, tip class, labware, liquid type, volume), or the step raises a technique-selection error.
9. If `overrideHeight` is `true` , both `height` and `heightFrom` **must** be provided with valid values.
10. `heightFrom` should be `0` , `1` , or `2` (or `"Liquid"` , `"Bottom"` , `"Top"`). Unrecognized string values raise an invalid-height-from error. Any `heightFrom` value outside `{0, 1, 2}` raises the same height-from error at run time when `overrideHeight` is `true` .
11. `selectionInfo` (or the result of `sectionExpression`) **must** resolve to exactly one valid target group (either 1 for a whole plate where the plate wells match the head geometry, or a quadrant if the plate has more wells than the head). Zero targets or multiple targets raise a selection error.
12. If `useExpression` is `true` , then `sectionExpression` **must** be a non-empty, evaluable expression string. If `false` , `selectionInfo` must be a non-empty array of valid section indices.
13. `liquidtype` **must not** be `"Tip Contents"` for an aspirate step. Use `"Well Contents"` or a named liquid type.
14. When `liquidtype` is `"Well Contents"` , all wells in the target group must contain the same liquid type at runtime, or the step raises a liquid-mismatch error.
15. `labwareClass` must be reachable/pipettable by the pod. An incompatible labware+pod combination raises a labware-access error.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Basic Multichannel Aspirate from all wells of a 96-well plate (auto-select technique)

Includes the editor-persisted keys so it matches a real export.

```
{
  "stepType": "Multichannel Aspirate",
  "parameters": {
    "operation": "Aspirate",
    "pod": "Pod1",
    "location": "SourcePlate",
    "labwareClass": "BCFlat96",
    "liquidtype": "Well Contents",
    "volume": 20,
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": [1]
    },
  },
  "useExpression": false,
```

```

    "sectionExpression": "",
    "refreshTips": false,
    "tipLabwareClass": "",
    "autoSelectPrototype": true,
    "prototype": "",
    "overrideHeight": false,
    "height": 0.0,
    "heightFrom": 0,
    "customHeight": false,
    "pattern": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": []
    },
    "aspirateTime": "1",
    "mixCount": "1",
    "washSettleTime": "0"
  }
}

```

The `pattern` array is shown empty for brevity; in a real export it is a full-length boolean mask matching the labware's well count. The engine ignores `pattern` and uses `selectionInfo`.

Example 2: Aspirate from quadrant 2 of a 384-well plate with tip refresh, explicit technique, and height override

```

{
  "stepType": "Multichannel Aspirate",
  "parameters": {
    "operation": "Aspirate",
    "pod": "Pod1",
    "location": "P4",
    "labwareClass": "CostarFlat384Square",
    "liquidtype": "Serum",
    "volume": "=AspVol",
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": [2]
    },
    "useExpression": false,
    "sectionExpression": "",
    "refreshTips": true,
    "tipLabwareClass": "BC230",
    "autoSelectPrototype": false,
    "prototype": "MC",
    "overrideHeight": true,
    "height": -2.0,
    "heightFrom": 0
  }
}

```

Example 3: Aspirate from a 384-well plate using expression-based quadrant selection

```

{
  "stepType": "Multichannel Aspirate",
  "parameters": {
    "operation": "Aspirate",
    "pod": "Pod1",
    "location": "P7",
    "labwareClass": "CostarFlat384Square",
    "liquidtype": "Water",
    "volume": 5,
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": []
    },
    "useExpression": true,
    "sectionExpression": "=CurrentQuadrant",
    "refreshTips": false,
    "tipLabwareClass": "",
    "autoSelectPrototype": true,
    "prototype": "",
    "overrideHeight": true,
    "height": 1.5,
    "heightFrom": 1
  }
}

```

```
}  
}
```

Common Mistakes

- **Confusing column indices with well indices:** For a 96-well plate with a 96-channel head, `selectionInfo` values are section indices (1-only on 96-well with 96-format head or 384-well with 384-format head; quadrants 1–4 on 384-well with 96-format head or 1536-well with 384-format head), not individual well numbers. For a 384-well plate with a 96-channel head, they are **quadrant indices** (1–4).
- **Selection that resolves to more than one target group:** The selection resolver must produce exactly one target — non-contiguous selections or incompatible section indices raise a selection error.
- **Wrong sign convention for `height`:** From Liquid or Top the offset is negative (below reference); from Bottom it is positive.
- **Confusing `useExpression` with Span-8 well-expression keys:** Here it toggles `sectionExpression` vs `selectionInfo`. There is no `useWellExpression` / `firstWellExpression` on this step.
- **Assuming `autoSelectPrototype` defaults to `true`:** It defaults to `false`. Set it explicitly or `prototype` is required.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — Technique auto-selection algorithm, height override mechanics, `c__` pipetting variable reference, and template architecture. Covers how `autoSelectPrototype`, `prototype`, `overrideHeight`, `height`, and `heightFrom` interact with the technique system.

† Table of contents

Multichannel Dispense

Property	Value
stepType	"Multichannel Dispense"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `dynamic?` , `disabled` , ...) are **not** re-documented here.

Behavior Summary

The Multichannel Dispense step dispenses liquid into destination labware using a fixed multichannel head (96-channel or 384-channel). At enqueue time the step validates that the specified pod is a multichannel pod (only multichannel pods are accepted; a Span-8 or Fixed-8 pod will produce an error). It resolves the deck position, verifies that labware is present and on top of any stack at that position, checks that the labware class is compatible with the actual labware at that position, confirms the labware class has a "Can Pipette" characteristic, and verifies that tips are loaded. It then uses the given pipetting technique to dispense the given volume, or the volume in tips if `emptyTips` is `true` , into the selected wells.

Use the Multichannel Dispense step to dispense liquid that was aspirated with a Multichannel Aspirate step.

Note that Multichannel Aspirate, Multichannel Dispense, Multichannel Mix, and Multichannel Wash Tips share much of their implementation. As a result, some parameters are listed here that are only used by those other steps. See each step's documentation for details.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>operation</code>	string	—	Yes	"Dispense"	Step action identity. Must be "Dispense" .
<code>pod</code>	string	—	Yes	""	Name of the multichannel pod to use (e.g., "Pod1"). Must resolve to a multichannel pod. The MC pod is "Pod1" on a standalone Multichannel instrument (i5-MC, i7-MC) and on an i7 Hybrid; it is "Pod2" only when the MC is the right-hand pod of a dual-pod i7. Never author "MC" / "MPod" . See Introduction to Biomek Method JSON §"Instrument Variants" / "Pod Types".
<code>location</code>	string	—	Yes	""	Deck position or labware name where the destination labware is located. Supports expressions (prefix with <code>=</code>). Accepts a deck-position name (e.g. <code>P5</code>) or a labware instance name (assigned via the labware's <code>properties.name</code> in Instrument Setup) — matched case-insensitively. Example 1's "DestPlate" refers to a placed labware instance named "DestPlate". See Instrument Setup and Labware and Position Resolution.
<code>labwareClass</code>	string	—	Yes	""	Expected labware class at the destination position (e.g., "BCF1at96"). Must be compatible with the actual labware. This

Key	Type	Units	Required	Default	Description
					field uses the short labware class name; the <code>LabwareClasses\</code> path prefix is used only in Instrument Setup's <code>deckItems</code> .
<code>liquidtype</code>	string	—	Yes	""	Destination liquid classification. Commonly <code>"Tip Contents"</code> (auto-detect from tips) or a named liquid type. Must be non-empty.
<code>volume</code>	number or string	μL	Yes (key must be present)	"-1"	Amount to dispense. Positive numeric value or expression string. The key must be present , even when <code>emptyTips</code> is <code>true</code> , in which case the authored value is otherwise ignored for the dispense amount. Default <code>"-1"</code> is a sentinel; authored values must be positive. A dispense is bounded on the D-axis minimum, not the maximum: it drives the plunger down, away from the head's capacity, so the head rating in <code>podSettings</code> . <code><pod>.settings.max.d</code> is not the limit here — that ceiling applies to aspirates. Driving the plunger below the pod's D-axis minimum fails at enqueue with <code>Move of <pod> to D of <n> μL violates D axis minimum of <min> μL.</code> , though in practice rule 16 (dispensing more than the tips hold) catches the same authoring error first. See Pod Settings Format (in <i>Biomek Instrument & Project Reference</i>) §"What <code>max.d</code> means, and when it is not the pipetting ceiling".
<code>emptyTips</code>	boolean	—	Yes (key must be present)	false	When <code>true</code> , the step ignores <code>volume</code> and dispenses the entire tip contents (trailing air gap + liquid + blow-out) at the initial dispense height, overriding technique "Follow liquid level" settings. The key must be present ; always author it. If technique height and following liquid are important, set <code>emptyTips</code> to <code>false</code> and ensure that <code>volume</code> is set to the actual volume in the tips.
<code>selectionInfo</code>	comArray (<code>arraySubtype</code> : "integer", long[])	—	Conditional	"" (empty)	Array of 1-based section or quadrant indices to dispense into. On these low-level Multichannel steps <code>selectionInfo</code> is a <code>comArray</code> of <code>arraySubtype</code> : "integer" carrying section/quadrant indices (e.g. <code>[1]</code>) — different from the item-level <code>selectionInfo</code> on Transfer/Combine, which is a well selection whose subtype varies by pod (boolean well-mask on Fixed-8, integer on Span-8/MC). Required when <code>useExpression</code> is <code>false</code> . Serialized as <code>{"_biomekType": "comArray", "arraySubtype": "integer", "values": [...]}</code> . Must contain exactly one element — the step requires exactly one target group per invocation; to process multiple sections, use a Loop with <code>sectionExpression</code> or use multiple Multichannel Dispense steps.
<code>useExpression</code>	boolean	—	No	false	When <code>true</code> , selection is determined by <code>sectionExpression</code> instead of <code>selectionInfo</code> .
<code>sectionExpression</code>	string	—	Conditional	""	Expression used for section/column/quadrant selection at runtime. Required when <code>useExpression</code> is <code>true</code> .
<code>refreshTips</code>	boolean	—	Yes	false	Inactive. This key must be authored, but it is ignored by the dispense step. Always author as <code>false</code> .
<code>tipLabwareClass</code>	string	—	No	""	Inactive. This key is used only in aspirate, not in dispense steps. Do not author it.
<code>autoSelectPrototype</code>	boolean	—	Yes	false	Whether the software auto-selects a technique based on pod type, head type, tip class, labware, liquid type, and volume.
<code>prototype</code>	string	—	Conditional	""	Explicit technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is not present.

Key	Type	Units	Required	Default	Description
customPrototype	object (_biomekType: "technique")	—	No	Not present	Inline technique object; when present and autoSelectPrototype is false , used instead of the named prototype . When autoSelectPrototype is true , customPrototype is ignored.
overrideHeight	boolean	—	Yes	false	Whether to override the technique's default dispense height.
height	number or string	mm	Conditional	0.0	Height offset from the reference point. Required when overrideHeight is true . Supports expressions.
heightFrom	integer or string	—	Conditional	0	Reference point for height: 0 = liquid surface, 1 = well bottom, 2 = well top. Also accepts string aliases "Liquid" , "Bottom" , "Top" .

A liquid-relative height needs a locatable liquid surface. With heightFrom = 0 / "Liquid" (or a technique that follows the liquid), the target labware's surface must be computable at run time: its volumeType — set in Instrument Setup / Guided Setup — must be "Nominal" or "Known" **with** an evalAmounts fill. Multichannel (96-/384-channel) and Fixed-8 pods do **not** support liquid-level sensing (LLS is Span-8 only), so on those pods an "Unknown" well cannot be rescued at run time — the move throws. See [Pipetting Techniques and Templates §"Liquid types" / §"Liquid-relative heights need a locatable liquid surface"](#).

Editor-persisted keys (not read by the Dispense engine)

These are written on every export by the editor but are **not** read at enqueue. Include them to match editor output; classify as **inert UI-state**.

Key	Type	Units	Default	Description / classification
customHeight	boolean	—	false	Encoding discriminator for height : false = numeric offset; true = text/custom height, which also implies overrideHeight . Use false with numeric heights. See Pipetting Techniques, Templates, and Auto-Selection.
pattern	comArray (arraySubtype: "boolean")	—	(well-count mask)	Inert UI-state. Boolean well-selection mask maintained by the labware view to restore the visual selection. Engine uses selectionInfo , not pattern .
aspirateTime	expression-capable string	s	"1"	Inactive and obsolete. This key is not used.
mixCount	expression-capable string	—	"1"	Inert UI-state on Dispense. This key is only used for the Multichannel Mix step. Not read by the Dispense engine.
washSettleTime	expression-capable string	s	"0"	Inactive and obsolete. This key is not used.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Dispense"	Must always be "Dispense" for this step type.
heightFrom	0 , 1 , 2 (or "Liquid" , "Bottom" , "Top")	Integer form preferred in JSON. String aliases are resolved at runtime.
liquidtype	Any named liquid, "Tip Contents"	"Tip Contents" is the editor default for dispense. "Well Contents" is invalid for dispense operations.

Parameter	Allowed Values	Notes
<code>selectionInfo</code> values	Positive integers (1-based)	The array iterates the <i>sections</i> the head touches, so the valid values match how many sections the labware exposes to the head — not the plate's column count. For a 96-well plate + 96-channel head there is exactly one section, so only <code>[1]</code> is valid; authoring <code>[1,2,3]</code> fails because the extra indices do not match sections the head exposes on that labware. For a 384-well plate + 96-channel head or 1536-well plate + 384-channel head each labware exposes four interleaved quadrants and the valid values are quadrant indices <code>1 – 4</code> . However, the Multichannel Dispense step requires that there be only a single section used, so this array must have exactly one value.

Cross-Field Validation Rules

- `operation` **must** equal `"Dispense"`. The step defaults to this value.
- `pod` **must** resolve to a Multichannel pod. A Span-8 or Fixed-8 pod will produce a runtime error.
- `location` **must** resolve to a valid, reachable deck position with labware present and on top of any stack.
- `labwareClass` **must** be compatible with the actual labware class at the resolved position.
- The labware class **must** have a `"Can Pipette"` characteristic, or the step raises a runtime error.
- Tips **must** be loaded on the pod at runtime. If no tips are present, the step raises a no-tips error.
- If `emptyTips` is `false`, `volume` **must** be a positive value. When `emptyTips` is `true`, `volume` is ignored and may be any value.
- If `autoSelectPrototype` is `false`, then `prototype` **must** be a non-empty string naming a valid technique. If the technique cannot be found, the step raises a technique-not-found error.
- If `autoSelectPrototype` is `true`, at least one matching technique must exist for the given context (pod type, head type, tip class, labware, liquid type, volume), or the step raises a technique-selection error.
- If `overrideHeight` is `true`, both `height` and `heightFrom` **must** be provided with valid values.
- `heightFrom` should be `0`, `1`, or `2` (or `"Liquid"`, `"Bottom"`, `"Top"`). Unrecognized string values raise an invalid-height-from error. Any `heightFrom` value outside `{0, 1, 2}` raises the same height-from error at run time when `overrideHeight` is `true`.
- `selectionInfo` (or the result of `sectionExpression`) **must** resolve to exactly one valid target group (either 1 for a whole plate where the plate wells match the head geometry, or a quadrant if the plate has more wells than the head). Zero targets or multiple targets raise a selection error.
- If `useExpression` is `true`, then `sectionExpression` **must** be a non-empty, evaluable expression string. If `false`, `selectionInfo` must be a non-empty array of valid section indices.
- `liquidtype` **must not** be `"Well Contents"` for a dispense step. Use `"Tip Contents"` or a named liquid type.
- When `liquidtype` is `"Tip Contents"` and the tips contain no non-air contents, the liquid type falls back to the labware's liquid dataset for the first well in the target group.
- `volume` **must not** exceed the current tip contents: dispensing more than the tips hold raises a dispense-too-much error.
- The dispensed volume **must not** overflow the destination well: exceeding well capacity raises a well-overflow error.
- `labwareClass` **must** be reachable/pipettable by the pod, else the step raises a labware-access error.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Basic Multichannel Dispense to section 1 of a 96-well plate (auto-select technique)

Includes the editor-persisted keys so it matches a real export.

```
{
  "stepType": "Multichannel Dispense",
  "parameters": {
    "operation": "Dispense",
```

```

"pod": "Pod1",
"location": "DestPlate",
"labwareClass": "BCFlat96",
"liquidtype": "Tip Contents",
"volume": 20,
"emptyTips": false,
"selectionInfo": {
  "_biomekType": "comArray",
  "arraySubtype": "integer",
  "values": [1]
},
"useExpression": false,
"sectionExpression": "",
"refreshTips": false,
"tipLabwareClass": "",
"autoSelectPrototype": true,
"prototype": "",
"overrideHeight": false,
"height": 0.0,
"heightFrom": 0,
"customHeight": false,
"pattern": {
  "_biomekType": "comArray",
  "arraySubtype": "boolean",
  "values": []
},
"aspirateTime": "1",
"mixCount": "1",
"washSettleTime": "0"
}
}

```

The `pattern` array is shown empty for brevity; in a real export it is a full-length boolean mask matching the labware's well count. The engine ignores `pattern` and uses `selectionInfo`.

Example 2: Dispense with Empty Tips, height override, and explicit technique

```

{
  "stepType": "Multichannel Dispense",
  "parameters": {
    "operation": "Dispense",
    "pod": "Pod1",
    "location": "P6",
    "labwareClass": "BCDeep96Square",
    "liquidtype": "Tip Contents",
    "volume": 0,
    "emptyTips": true,
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": [1]
    },
    "useExpression": false,
    "sectionExpression": "",
    "refreshTips": false,
    "tipLabwareClass": "",
    "autoSelectPrototype": false,
    "prototype": "MC",
    "overrideHeight": true,
    "height": -1.0,
    "heightFrom": 2
  }
}

```

Example 3: Expression-based selection dispensing to a quadrant of a 384-well plate defined in a variable

```

{
  "stepType": "Multichannel Dispense",
  "parameters": {
    "operation": "Dispense",
    "pod": "Pod1",
    "location": "P7",
    "labwareClass": "CostarFlat384Square",
    "liquidtype": "Water",
    "volume": "=DispenseVol",
    "emptyTips": false,
    "selectionInfo": {

```



```

    "_biomekType": "comArray",
    "arraySubtype": "integer",
    "values": []
  },
  "useExpression": true,
  "sectionExpression": "=CurrentQuadrant",
  "refreshTips": false,
  "tipLabwareClass": "",
  "autoSelectPrototype": true,
  "prototype": "",
  "overrideHeight": false,
  "height": 0.0,
  "heightFrom": 0
}
}

```

Common Mistakes

- **Confusing column indices with well indices:** For a 96-well plate with a 96-channel head, `selectionInfo` values are section indices (1-only on 96-well with 96-format head or 384-well with 384-format head; quadrants 1–4 on 384-well with 96-format head or 1536-well with 384-format head), not individual well numbers. For a 384-well plate with a 96-channel head, they are **quadrant indices** (1–4).
- **Selection that resolves to more than one target group:** The selection resolver must produce exactly one target — non-contiguous selections or incompatible section indices raise a selection error.
- **Wrong sign convention for `height`:** From Liquid or Top the offset is negative (below reference); from Bottom it is positive.
- **Empty Tips height behavior is not "follow liquid":** With `emptyTips: true` the technique's Follow-Liquid-Level setting is overridden — the entire tip contents (trailing air gap + liquid + blow-out) are expelled at the *initial* dispense height. The tip does not track the surface downward.
- **Expecting volume to matter when `emptyTips` is `true`:** The runtime sums all tip content volumes (including air gaps) and uses that for technique selection; the authored `volume` is bypassed entirely. Set it to `0` for clarity.
- **Confusing `useExpression` with Span-8 well-expression keys:** Here it toggles `sectionExpression` vs `selectionInfo`. There is no `useWellExpression` / `firstWellExpression` on this step.
- **Assuming `autoSelectPrototype` defaults to `true`:** It defaults to `false`. Set it explicitly or `prototype` is required.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — Technique auto-selection algorithm, height override mechanics, `c__` pipetting variable reference, and template architecture. Covers how `autoSelectPrototype`, `prototype`, `overrideHeight`, `height`, and `heightFrom` interact with the technique system.

Multichannel Load Tips

Property	Value
stepType	"Multichannel Load Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Behavior Summary

The Multichannel Load Tips step loads disposable tips onto all mandrels of the Multichannel (96/384-channel) pod head. It locates an available tip box matching the requested tips and loads tips from it.

Note that while tips must be loaded from a TipLoad1x1 ALP, the tip box does not have to be present on that ALP. The step will move the tip box to an empty TipLoad1x1 in order to load it.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
pod	string	—	Yes	(must be provided)	Name of the multichannel pod to use (e.g., "Pod1"). Must resolve to a multichannel pod. The MC pod is "Pod1" on a standalone Multichannel instrument (i5-MC, i7-MC) and on an i7 Hybrid; it is "Pod2" only when the MC is the right-hand pod of a dual-pod i7. Never author "MC" / "MPod" . See Introduction to Biomek Method JSON §"Instrument Variants" / "Pod Types".
tips	string	—	Yes	(must be provided)	Tip labware class name, labware instance name, or deck position to load from (e.g., "BC230" , "BC190F" , "TipBox6" , "P3"). If a labware class or instance name is provided that matches more than one tip box, the step will select an appropriate tip box from the matching tip boxes.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Enumerated / Constrained Values

No enumerated constraints. The `tips` value must match a labware class name, a labware instance name, or a deck position name defined in the project's labware library / deck.

Cross-Field Validation Rules

- `pod` **must** resolve to a Multichannel pod.
- `tips` **must** name a tip type retrievable on the deck (by class name, instance name, or deck position). **The deck-position form only works if that position currently holds a tip box.** Naming a position that exists on the deck but was left empty in Instrument Setup is not an error in itself — the value simply falls through to the class-and-instance search, which then fails reporting your position

name as though it were a tip type (Unable to find and retrieve tips of type TL1 for Pod1.). If you get that message and the name in it is a deck position rather than a labware class, the position is empty; place a tip box there. If the value matches a stocked deck position, the step loads from that position directly; otherwise it searches all reachable tip boxes and, if none yields tips, the step fails and reports the reason it skipped each candidate box (used, partially used, named-instance mismatch when searched by class, unreachable, or no tip-load locations found).

3. The tip density **must** match the head: loading a 384-tip box onto a 96-channel head (or vice-versa) fails.
4. The tip box must be fully populated — all tips must be present. A partially-used box is skipped as partially used; an empty (fully consumed) box is skipped as used. Automatic tip-box retrieval will relocate a sealed/lidded box from storage and de-lid it (moving the lid to trash) before loading when no reachable open box is available.
5. The tip box must have remaining uses, as defined in the Instrument Setup step. See Instrument Setup, § `evalAmounts` for tip boxes.

Budgeting boxes: typically, one Load Tips consumes one box. If the tip box is created with default settings in the Instrument Setup step, the tips may be loaded. This means that **every** Multichannel Load Tips step needs its own fully-populated box. The number of tip changes a Multichannel method can perform is therefore capped by how many boxes you can place, and how many `tl*` tip-load positions are available.

To reuse tips from the same box, configure the number of usages in the Instrument Setup step (Instrument Setup, § `evalAmounts` for tip boxes).

On a hybrid instrument, keep each pod's boxes separate. Where a Multichannel and a Span-8 pod share one deck, the two pods have opposite partial-box rules: Multichannel skips a partially-used box (rule 4), while Span-8 accepts one (Span-8 Load Tips rule 3). So a Span-8 load that draws a few tips from a shared box leaves that box partial and thereby **permanently removes it from the Multichannel pod's pool** — the box still looks full to a reader of the method, and no error is raised at the Span-8 step that consumed it. Declare dedicated boxes per pod, and point each pod's steps only at its own.

The failure lands well away from its cause: it appears at the *later* Multichannel step as `Unable to find and retrieve tips of type <class> for <pod>`, reported against that step's source position rather than against the Span-8 load. Enqueue-time scheduling can also raise it before any tips have physically moved. If a hybrid method fails this way, audit which boxes the Span-8 steps touch before re-examining the Multichannel step.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Load multichannel tips:

```
{
  "stepType": "Multichannel Load Tips",
  "parameters": {
    "pod": "Pod1",
    "tips": "BC230"
  }
}
```

Common Mistakes

- **Tip density mismatch with head:** A 384-tip box on a 96-channel head (or vice-versa) fails at load time.
- **Invalid tip name:** `tips` must match a labware class name, labware instance name, or deck position defined in the project — check Instrument Setup for available types. Silently misspelling produces the generic "unable to find and retrieve tips" error.

- **Assuming a partial/used box will do:** on a Multichannel pod, a partially-used box is always skipped and a fully-consumed box is reported as used; automatic retrieval will de-lid a fresh box only if one is reachable.
 - **Searching a named box by class:** if a tip box on deck has been given a custom instance name, referencing it by its class alone is skipped as a named-instance mismatch — use the instance name instead.
-

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — tip class selection interacts with technique auto-selection on subsequent Multichannel Aspirate / Dispense / Mix steps.

[†] Table of contents

Multichannel Unload Tips

Property	Value
stepType	"Multichannel Unload Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Behavior Summary

The Multichannel Unload Tips step removes disposable tips from the Multichannel (96/384-channel) pod head. If tips are currently loaded on the pod, the step unloads them; if no tips are loaded, the step does nothing.

Where the tips go

This step takes **no unload-location parameter** — `pod` is its whole parameter set. The destination is resolved by the pod from the deck and the tip box the loaded tips came from. To change where tips are unloaded to, set the `DiscardTips` and `DiscardTipsLocation` on the source tip box in the Instrument Setup step (see Instrument Setup).

- **No tips loaded** — the step does nothing.
- **Source tip box configured to discard to trash** (`DiscardTips` `true` with `DiscardTipsLocation` `<Any Trash>` , or no remembered source box) — the pod picks the **closest reachable position carrying the "Can Discard Tips" characteristic**. `Can Discard Tips` is a deck-position characteristic set in the instrument's deck configuration, not a step parameter. Reachability is **pod-specific**: on a two-pod instrument the other pod's bridge narrows each pod's usable X range. If no qualifying position is reachable by this pod, the step fails with:

`<pod> cannot find a location to discard tips.` The pod name at the start identifies which pod's discard path failed. The search and the deck-side remedy are in Reachability and Access (in *Biomek Instrument & Project Reference*) §"Symptom to likely cause". - **Source tip box configured to return tips to the tip box** (`DiscardTips` `false` , e.g. `DiscardTipsLocation`: `"<TipBox>"`) — the pod looks for the original box's current position, then for an **empty box of the same class** at the source position (not claimed by the other pod). If neither is found it falls back to the closest reachable trash. No reachable trash at that point raises a "cannot find the tip box" error. A trashless deck is still a risk on the return-to-box path. - **Tips loaded via a Select-tips step** — if the selected tips cannot be placed back into the source box, the pod falls back to the closest reachable trash on the same terms. Note that tips loaded via Select Tips steps do NOT decrement the tip usage count. It is recommended not to share tip boxes between Select Tips steps and regular Multichannel tip load/unload steps.

A named destination must be a position the pod can legitimately unload at: if it holds no labware it must carry the "Can Discard Tips" characteristic, otherwise the unload is rejected.

The `DiscardTips` / `DiscardTipsLocation` routing keys live on the **tip-box labware object** in Instrument Setup, not on this step.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	<code>string</code>	—	Yes	<i>(must be provided)</i>	Name of the multichannel pod (e.g., <code>"Pod1"</code>). Must resolve to a Multichannel pod.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

No enumerated constraints.

Cross-Field Validation Rules

1. `pod` **must** resolve to a Multichannel pod.
2. A discard location **must** be reachable **by this pod** (pod-specific, narrowed by the other pod's bridge on two-pod instruments). See Where the tips go and Span-8 Unload Tips "Deck prerequisite". If no motion path exists, the failure is wrapped as an unload-tips path error; a missing position yields a "deck position not found" error.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Unload multichannel tips:

```
{
  "stepType": "Multichannel Unload Tips",
  "parameters": {
    "pod": "Pod1"
  }
}
```

Common Mistakes

- **No reachable discard location for *this* pod**: ensure a `Can Discard Tips` position is reachable by the pod named on this step, not merely present on the deck. No step parameter can substitute; see Where the tips go.
- **Assuming `DiscardTipsLocation: "<TipBox>"` removes the need for a trash**: the return-to-box path still falls back to the closest reachable trash whenever the original box is unavailable, so it can fail on a trashless deck with a "cannot find the tip box" error instead.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — tip lifecycle context; pairs with Multichannel Load Tips.

Multichannel Mix

Property	Value
stepType	"Multichannel Mix"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `dynamic?` , `disabled` , ...) are **not** re-documented here.

Behavior Summary

The Multichannel Mix step aspirates and re-dispenses liquid in place, `mixCount` times, to mix well contents using a fixed multichannel head (96- or 384-channel). It shares behavior with Multichannel Aspirate and Dispense, differing only in that it defaults `operation` to "Mix" and adds a `mixCount` property. At enqueue time the step validates that the pod is a multichannel pod (a Span-8 or Fixed-8 pod produces an error), resolves the deck position, verifies labware is present and on top of any stack, checks labware-class compatibility and the "Can Pipette" characteristic, and confirms tips are loaded. If `refreshTips` is `true` , current tips are discarded and fresh tips loaded from `tipLabwareClass` . It then uses the given pipetting technique to mix the given volume the given number of times in the selected wells. The number of mix cycles and the per-cycle mix volume are driven by `mixCount` and `volume` through the technique's mix action.

Note that Multichannel Aspirate, Multichannel Dispense, Multichannel Mix, and Multichannel Wash Tips share much of their implementation. As a result, some parameters are listed here that are only used by those other steps. See each step's documentation for details.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>operation</code>	string	—	Yes	"Mix"	Step action identity. Must be "Mix" .
<code>pod</code>	string	—	Yes	""	Name of the multichannel pod to use (e.g., "Pod1"). Must resolve to a multichannel pod. The MC pod is "Pod1" on a standalone Multichannel instrument (i5-MC, i7-MC) and on an i7 Hybrid; it is "Pod2" only when the MC is the right-hand pod of a dual-pod i7. Never author "MC" / "MPod" . See Introduction to Biomek Method JSON §"Instrument Variants" / "Pod Types".
<code>location</code>	string	—	Yes	""	Deck position or labware name where the source labware is located. Supports expressions (prefix with <code>=</code>). Accepts a deck-position name (e.g. <code>P5</code>) or a labware instance name (assigned via the labware's <code>properties.name</code> in Instrument Setup) — matched case-insensitively. See Instrument Setup and Labware and Position Resolution.
<code>labwareClass</code>	string	—	Yes	""	Expected labware class at the position (e.g., "BCF1a96"). Must be compatible with the actual labware. This field uses the short labware

Key	Type	Units	Required	Default	Description
					class name; the <code>LabwareClasses\</code> path prefix is used only in Instrument Setup's <code>deckItems</code> .
<code>liquidtype</code>	string	—	Yes	""	Liquid classification for technique selection. Commonly "Well Contents" (auto-detect from wells) or a named liquid type.
<code>volume</code>	number or string	μL	Yes	0.0	Volume aspirated/dispensed per mix cycle. Positive numeric value or expression.
<code>mixCount</code>	expression-capable string	—	Yes	"1"	Number of mix cycles. Positive integer value or expression.
<code>selectionInfo</code>	comArray (arraySubtype: "integer")	—	Conditional	"" (empty)	Array of 1-based section/quadrant indices to mix in. On these low-level Multichannel steps <code>selectionInfo</code> is a <code>comArray</code> of <code>arraySubtype: "integer"</code> carrying section/quadrant indices (e.g. [1]) — different from the item-level <code>selectionInfo</code> on Transfer/Combine, which is a well selection whose subtype varies by pod (boolean well-mask on Fixed-8, integer on Span-8/MC). Must contain exactly one element — the step requires exactly one target group per invocation (see Cross-Field rule 10); to process multiple sections, use a Loop with <code>sectionExpression</code> or multiple Multichannel Mix steps. Required when <code>useExpression</code> is <code>false</code> . Serialized as <code>{ "_biomekType": "comArray", "arraySubtype": "integer", "values": [...] }</code> .
<code>useExpression</code>	boolean	—	No	false	When <code>true</code> , selection comes from <code>sectionExpression</code> instead of <code>selectionInfo</code> .
<code>sectionExpression</code>	string	—	Conditional	""	Expression used for section selection at runtime. Required when <code>useExpression</code> is <code>true</code> .
<code>refreshTips</code>	boolean	—	Yes	false	Whether to discard current tips and load fresh tips before mixing.
<code>tipLabwareClass</code>	string	—	Conditional	""	The tips to use, specified either as a labware class, a labware instance name, or a position name. Expression capable. Required when <code>refreshTips</code> is <code>true</code> .
<code>autoSelectPrototype</code>	boolean	—	Yes	false	Whether the software auto-selects a technique based on pod type, tip type, labware, liquid type, and volume.
<code>prototype</code>	string	—	Conditional	""	Explicit technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is absent.
<code>customPrototype</code>	object (_biomekType: "technique")	—	No	Not present	Inline technique object (<code>_biomekType: "technique"</code> payload). When present, overrides both <code>prototype</code> and <code>autoSelectPrototype</code> .
<code>overrideHeight</code>	boolean	—	Yes	false	Inactive on Mix. Mix steps do not allow overriding technique height. However, this key must be present. Author it as <code>false</code> .
<code>height</code>	number or string	mm	No	0.0	Inactive on Mix. Do not author this key.
<code>heightFrom</code>	integer or string	—	No	0 (long)	Inactive on Mix. Do not author this key.

A liquid-relative height needs a locatable liquid surface. With a technique that follows the liquid, the target labware's surface must be computable at run time: its `volumeType` — set in Instrument Setup / Guided Setup — must be "Nominal" or "Known" with an `evalAmounts` fill. Multichannel (96-/384-channel) and Fixed-8 pods do **not** support liquid-level sensing, so on those pods an "Unknown" well cannot be rescued at run time — the move throws. See [Pipetting Techniques and Templates §"Liquid types"](#) / [§"Liquid-relative heights need a locatable liquid surface"](#).

Editor-persisted keys (not read by the Mix engine)

These are written on every export by the editor but are **not** read at enqueue. Include them to match editor output; classify as **inert UI-state**.

Key	Type	Units	Default	Description / classification
customHeight	boolean	—	false	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/custom height, which also implies <code>overrideHeight</code> . For Mix the height panel is disabled, so this is effectively always <code>false</code> . See Pipetting Techniques, Templates, and Auto-Selection.
pattern	comArray (<code>arraySubtype</code> : "boolean")	—	(well- count mask)	Inert UI-state . Boolean well-selection mask maintained by the labware view to restore the visual selection; the engine uses <code>selectionInfo</code> , not <code>pattern</code> . In real exports this is a full-length boolean array matching the labware's well count.
aspirateTime	expression-capable string	s	"1"	Inactive and obsolete . This key is not used.
washSettleTime	expression-capable string	s	"0"	Inactive and obsolete . This key is not used.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Mix"	Must always be "Mix" for this step type.
heightFrom	0, 1, 2 (or "Liquid", "Bottom", "Top")	Inactive on Mix.
liquidtype	Any named liquid, "Well Contents"	"Well Contents" is the editor default for Mix. "Tip Contents" is invalid for Mix (only valid for Dispense), raising a runtime error.
selectionInfo values	Positive integers (1- based)	The array iterates the <i>sections</i> the head touches, so the valid values match how many sections the labware exposes to the head — not the plate's column count. For a 96-well plate + 96-channel head there is exactly one section, so only <code>[1]</code> is valid; authoring <code>[1,2,3]</code> fails because the extra indices do not match sections the head exposes on that labware. For a 384-well plate + 96-channel head or 1536-well plate + 384-channel head each labware exposes four interleaved quadrants and the valid values are quadrant indices <code>1–4</code> . However, the Multichannel Mix step requires that there be only a single section used, so this array must have exactly one value.

Cross-Field Validation Rules

- `operation` **must** equal "Mix". The step defaults to this value.
- `pod` **must** resolve to a Multichannel pod. A Span-8 or Fixed-8 pod will produce a runtime error.
- `location` **must** resolve to a valid, reachable deck position with labware present and on top of any stack.
- `labwareClass` **must** be compatible with the actual labware class at the resolved position.
- The labware class **must** have a "Can Pipette" characteristic, or the step raises a runtime error.
- Tips **must** be loaded on the pod at runtime. If no tips are present, the step raises a no-tips error.
- If `refreshTips` is `true`, `tipLabwareClass` **must** name a valid tip labware class.
- If `autoSelectPrototype` is `false`, then `prototype` **must** be a non-empty string naming a valid technique. If the technique cannot be found, the step raises a technique-not-found error.
- If `autoSelectPrototype` is `true`, at least one matching technique must exist for the given context (pod type, head type, tip class, labware, liquid type, volume), or the step raises a technique-selection error.

10. `selectionInfo` (or the result of `sectionExpression`) **must** resolve to exactly one valid target group (either 1 for a whole plate where the plate wells match the head geometry, or a quadrant if the plate has more wells than the head). Zero targets or multiple targets raise a selection error.
11. If `useExpression` is `true`, then `sectionExpression` **must** be a non-empty, evaluable expression string. If `false`, `selectionInfo` must be a non-empty array of valid section indices.
12. `liquidtype` **must not** be "Tip Contents" for a Mix step. When "Well Contents" is selected, all wells in the target group must share one liquid type at runtime, or a liquid-mismatch error is raised
13. `volume` (the per-cycle mix volume) runs through the same technique aspirate/dispense checks as the other MC steps: a volume greater than tip capacity raises a dispense-too-much error, and a volume that would overflow the well raises a well-overflow error. Additionally, `volume` must not exceed the well's tracked contents. See Well Volume Tracking.
14. `mixCount` must resolve to a non-negative integer at runtime. A negative value raises a negative-mix-count error. A value of `0` is technically accepted and produces zero mix cycles.
15. Tips must be empty before a mix begins, or the step raises an error. Air gaps do not trip the check; anything classified as liquid does. Reusing tips that still hold liquid — a Multichannel Aspirate followed by a standalone Mix — is the common cause.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Basic Multichannel Mix (auto-select technique), 3 cycles of 80 uL across all wells (section 1)

Includes the editor-persisted keys so it matches a real export.

```
{
  "stepType": "Multichannel Mix",
  "parameters": {
    "operation": "Mix",
    "pod": "Pod1",
    "location": "P4",
    "labwareClass": "BCFlat96",
    "liquidtype": "Well Contents",
    "volume": 80,
    "mixCount": "3",
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": [1]
    },
    "useExpression": false,
    "sectionExpression": "",
    "refreshTips": false,
    "tipLabwareClass": "",
    "autoSelectPrototype": true,
    "prototype": "",
    "overrideHeight": false,
    "height": 0.0,
    "heightFrom": 0,
    "customHeight": false,
    "pattern": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": []
    },
    "aspirateTime": "1",
    "washSettleTime": "0"
  }
}
```

The `pattern` array is shown empty for brevity; in a real export it is a full-length boolean mask matching the labware's well count. The engine ignores `pattern` and uses `selectionInfo`.

Example 2: Explicit technique with tip refresh, expression-based selection

Engine-relevant keys only (the software supplies UI-state keys on next save).

```
{
  "stepType": "Multichannel Mix",
  "parameters": {
    "operation": "Mix",
    "pod": "Pod1",
    "location": "P7",
    "labwareClass": "BCDeep96Square",
    "liquidtype": "Water",
    "volume": "=MixVol",
    "mixCount": "=MixCycles",
    "selectionInfo": {
      "_biomekType": "comArray",
      "arraySubtype": "integer",
      "values": []
    },
    "useExpression": true,
    "sectionExpression": "=CurrentQuadrant",
    "refreshTips": true,
    "tipLabwareClass": "BC230",
    "autoSelectPrototype": false,
    "prototype": "MC",
    "overrideHeight": false
  }
}
```

Common Mistakes

- **Expecting `overrideHeight` / `height` to move the mix heights**: the Mix editor disables height editing, and mix aspirate/dispense heights come from the technique's Mix parameters, not the step-level. Leave `overrideHeight: false` — but the key must still be present, or enqueue throws.
- **Reading meaning into `aspirateTime` / `washSettleTime` / `pattern`**: these are inert UI-state, not engine inputs. `customHeight` is not in that group — it is the `height` encoding discriminator (see the parameter table) — but because Mix takes its heights from the technique, `false` is the only value worth authoring here.
- **`mixCount` of empty or negative**: `mixCount` must resolve to a non-negative integer at runtime. A negative value throws a negative-mix-count error; a value of `0` is technically accepted but results in no mixing (silent no-op).
- **Providing `selectionInfo` as a plain JSON array**: must be a `comArray` object (`arraySubtype: "integer"`).
- **Assuming `autoSelectPrototype` defaults to `true`**: default is `false`. Set it explicitly.

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — technique auto-selection, height override mechanics, the `C__Mix*` context variables, and `customHeight` vs `overrideHeight`.
- **Method JSON Structure** — envelope, `_biomekType` / `comArray` encoding, structural rules.

Multichannel Wash Tips

Property	Value
stepType	"Multichannel Wash Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5 or i7 with a Multichannel pod

Base keys + casing: see Base Keys & Serialization Conventions. Universal base keys (`caption` , `dynamic?` , `disabled` , ...) are **not** re-documented here.

Behavior Summary

The Multichannel Wash Tips step washes the tips currently on a multichannel head by mixing them in a wash-station reservoir. It validates the pod, tips, and wash station, then performs a mix step in the wash station.

Tips must be empty before washing. Dispense the contents first using a Multichannel Dispense step, then wash.

Note that Multichannel Aspirate, Multichannel Dispense, Multichannel Mix, and Multichannel Wash Tips share much of their implementation. As a result, some parameters are listed here that are only used by those other steps. See each step's documentation for details.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>pod</code>	string	—	Yes	<code>""</code>	Name of the multichannel pod to use (e.g., <code>"Pod1"</code>). Must resolve to a multichannel pod. The MC pod is <code>"Pod1"</code> on a standalone Multichannel instrument (i5-MC, i7-MC) and on an i7 Hybrid; it is <code>"Pod2"</code> only when the MC is the right-hand pod of a dual-pod i7. Never author <code>"MC"</code> / <code>"MPod"</code> . See Introduction to Biomek Method JSON §"Instrument Variants" / "Pod Types".
<code>location</code>	string	—	No	<code>""</code>	Deck position of the wash station. If empty, the step auto-selects the first reachable wash station whose solvent matches <code>liquidtype</code> . If set, it must be a wash station stocked with that solvent. Supports <code>=</code> expression strings (e.g., <code>"=xTips_WashPosition"</code>).
<code>liquidtype</code>	string	—	Yes (key must be present)	<code>""</code>	Solvent/liquid type used for the wash. The key must be present , and there must be a wash station on the deck that has that liquid type assigned. Casing note: the exported key is all-lowercase <code>liquidtype</code> (not <code>liquidType</code>) for this step — author it that way to match the editor's output. Expression-capable.
<code>volume</code>	string (number, percent, or expression)	μL or %	Yes	<code>"0"</code>	Wash volume. A plain number string (e.g., <code>"170"</code>) is μL; a trailing- <code>%</code> string (e.g., <code>"110%"</code>) is a percentage of the tips' max-volume-used, clamped to tip capacity; a leading- <code>=</code> string (e.g., <code>"=TipWashVol1"</code>) is evaluated as an expression. Empty string raises a volume-not-specified error; a computed value <code>≤ 0</code> makes the step a no-op.

Key	Type	Units	Required	Default	Description
					Canonical export form is always a string; a plain JSON number imports correctly but does not match the editor's output.
mixCount	string (integer or expression)	—	Yes	"1"	Number of wash (mix) repetitions within the wash solution. Canonical export form is a string (e.g., "2", "4", or an expression like "=TipWashCycles"). Must evaluate to a positive integer.
autoSelectPrototype	boolean	—	Yes	false	Whether the software auto-selects the wash technique. It is possible that auto-select may result in choosing a technique not suitable for washing. When possible, set autoSelectPrototype to false and explicitly set prototype to a wash technique.
prototype	string	—	Yes (key must be present; may be "")	""	Explicit technique name. The key must be present , even when autoSelectPrototype is true . Provide a real technique name unless auto-selecting or supplying customPrototype , in which case an empty string is acceptable but the key must still appear. When selecting a technique, choose one intended for washing. Typically wash techniques have "Wash" in their names.
customPrototype	object (_biomekType: "technique")	—	No	Not present	Inline technique object; when present overrides prototype / autoSelectPrototype .
operation	string	—	Yes (persisted by UI)	"Mix"	Always "Mix" .

Keys deliberately NOT present. The shared pipetting editor keys (aspirateTime , customHeight , pattern , washSettleTime , selectionInfo , useExpression , sectionExpression , refreshTips , tipLabwareClass , overrideHeight , height , heightFrom , labwareClass) do **not** appear on this step; do not author them here.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Mix"	Fixed. The wash is implemented as a mix.
volume	positive number string, "<n>%" , or "=<expr>"	% form = percentage of tips' max-volume-used, clamped to tip capacity. = form is evaluated as a method expression.
liquidtype	a solvent liquid-type name	Must match a wash station's stocked solvent.

Cross-Field Validation Rules

- pod **must** resolve to a Multichannel pod. A Span-8 or Fixed-8 pod will produce an error.
- Tips **must** be loaded, or a no-tips error is raised.
- Tips **must** be empty (no liquid content), or a tips-not-empty error is raised.
- Auto-locate wash station** (empty location): if no position has a "Wash Station" characteristic, a no-wash-stations error is raised; if wash stations exist but liquidtype is empty, a liquid-type-not-specified error is raised; if none stock the requested solvent, a no-station-with-solvent error is raised.
- Explicit location** : if the position is not a wash station, a location-is-not-wash-station error is raised; if it is a station but liquidtype is empty, a liquid-type-not-specified error is raised; if the station's solvent doesn't match, a wash-station-selection-invalid error is raised.

6. volume **must** be a non-empty string, or a volume-not-specified error is raised. A computed volume ≤ 0 returns success without washing (no-op).
7. If autoSelectPrototype is false, then prototype **must** be a non-empty string naming a valid technique. If the technique cannot be found, the step raises a technique-not-found error.
8. If autoSelectPrototype is true, at least one matching technique must exist for the given context (pod type, head type, tip class, labware, liquid type, volume), or the step raises a technique-selection error.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Example 1: Editor-typical explicit-station wash (percent volume, named technique)

Uses the editor's default shape: all string-valued volume/mixCount, autoSelectPrototype: false, an explicit prototype.

```
{
  "stepType": "Multichannel Wash Tips",
  "parameters": {
    "pod": "Pod1",
    "location": "WS1",
    "liquidtype": "Water",
    "volume": "110%",
    "mixCount": "2",
    "autoSelectPrototype": false,
    "prototype": "MC Active Wash",
    "operation": "Mix"
  }
}
```

Example 2: Auto-located station, auto-select technique

```
{
  "stepType": "Multichannel Wash Tips",
  "parameters": {
    "pod": "Pod1",
    "location": "",
    "liquidtype": "Water",
    "volume": "200",
    "mixCount": "3",
    "autoSelectPrototype": true,
    "prototype": "",
    "operation": "Mix"
  }
}
```

Example 3: Expression-driven volume, cycles, and location

Pattern where the volume, cycle count, and wash position are method expressions resolved at run time.

```
{
  "stepType": "Multichannel Wash Tips",
  "parameters": {
    "pod": "Pod1",
    "location": "=xTips_WashPosition",
    "liquidtype": "Water",
    "volume": "=TipWashVol",
    "mixCount": "=TipWashCycles",
    "autoSelectPrototype": false,
    "prototype": "MC Active Wash",
    "operation": "Mix"
  }
}
```

Common Mistakes

- Washing non-empty tips: the step requires empty tips. Dispense before washing.
- **Expecting pipetting keys here:** `selectionInfo`, `refreshTips`, `height / heightFrom`, `labwareClass`, `pattern`, `aspirateTime`, `washSettleTime` are **not** Wash keys. Authoring them here has no effect.
- A "`<n>%`" **volume that rounds to ≤ 0** : silently no-ops (returns success without washing) when tips have used very little liquid. An empty-string `volume`, by contrast, errors.
- Explicit `location` with mismatched solvent: raises a wash-station-solvent-mismatch error — the station must both be a wash station *and* stock the requested `liquidtype` (case-insensitive).

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — technique auto-selection (applies to the internal Mix operation).
- **Method JSON Structure** — `envelope`, `_biomekType / comArray` encoding, structural rules.

[†] Table of contents

Multichannel Select Tips

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips"
Category	Free-form container
Terminator	"Multichannel Select Tips End"
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips container groups a sequence of select-tips operations (aspirate, dispense, mix, load, unload) that work with a specific multichannel pod using partial-tip patterns. At runtime, the container validates that the pod starts without tips, stores the pod and rearrange position globally for child steps to reference, enqueues all child steps in order, then cleans up. The pod **must have no tips loaded** by the time the "Multichannel Select Tips End" terminator executes (display caption "End Using Select Tips", but the emitted stepType is "Multichannel Select Tips End") — a Select Tips group that loads tips must also unload them before the terminator.

The rearrange position is an optional tip-box position used as temporary storage when loading complex tip patterns that require multiple load/unload cycles.

Use Multichannel Select Tips when you need to load **less than a full head** of tips on a multichannel pod. If you are using a full head of tips, do **not** use Multichannel Select Tips; use Multichannel Load Tips and Multichannel Unload Tips instead.

Declaring the rearrange position's empty tip box

When a child Load actually needs the rearrange position, that position must hold a tip box which is of the **same tip type** as the box being loaded from and is **empty** — no tips in any well. A box that still holds tips fails with `The tip box on position "<position>" must be empty.`

A tip box's tip inventory is stored in the same per-well amounts array that `evalAmounts` seeds for liquid labware: a well holding a tip has a positive amount, and an empty well is `0`. So an empty tip box is declared in Instrument Setup exactly like a plate seeded to zero — set `volumeType` to "Known" and give `evalAmounts` a full-length array of `0.0`:

```
{
  "_biomekType": "labware",
  "class": "LabwareClasses\\BC230",
  "volumeType": "Known",
  "evalAmounts": {
    "_biomekType": "comArray", "arraySubtype": "variant",
    "values": [0.0, 0.0, /* ...96 total, one per well... */ 0.0]
  }
}
```

Place that object on a position carrying the multichannel tip-load characteristic (`tl*`), the same as any other MC tip box, and name it in the container's `rearrangePos`. Note the array must be full-length: a shortened array does **not** zero the remaining wells — see `$ evalAmounts / evalLiquids` in Instrument Setup.

Two related points that are easy to get backwards:

- **A declared tip box occupies the deck whether or not a step addresses it.** Obstruction follows declaration, not use, so an unused box on a neighboring position can still block the head. See Reachability and Access (in *Biomek Instrument & Project Reference*).
- **Clearing `rearrangePos` is not a way to avoid the rearrange machinery.** The value is only consulted when a load pattern genuinely needs staging; emptying it does not change which pattern you asked for, it just removes the position the load was going to use.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	string	—	Yes	""	Multichannel pod name (e.g., "Pod1").
<code>rearrangePos</code>	string	—	Conditional	""	Optional rearrange position for multi-step tip loading.

`rearrangePos` becomes **conditionally required** only when a child that needs a temporary rearrange position runs — the child load step then raises a `rearrange-position-not-specified` error.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

None. `pod` and `rearrangePos` are free-form strings validated at runtime.

Cross-Field Validation Rules

1. Pod **must not** have tips loaded when this step begins, or a start-with-tips error is raised.
2. Pod **must have no tips loaded** when "Multichannel Select Tips End" executes, or an exit-with-tips error is raised. Include a Select Tips Unload / Advanced Unload child before the terminator.
3. A pipetting child step (Aspirate, Dispense, Mix, Serial Dilution) that runs before tips are loaded raises a pod-no-tips error; Unload / Advanced Unload raise a similar unload-no-tips error for the same condition.
4. Any Select Tips child step placed outside a "Multichannel Select Tips" container raises a container-required error.
5. If `rearrangePos` is specified, it must be a valid deck position that can load tips.

Structural Context

This step is a **free-form container**. Child steps are placed in `subSteps` , terminated by "Multichannel Select Tips End" .

Valid child step types: "Multichannel Select Tips Aspirate" , "Multichannel Select Tips Dispense" , "Multichannel Select Tips Mix" , "Multichannel Select Tips Load" , "Multichannel Select Tips Unload" , "Multichannel Select Tips Advanced Load" , "Multichannel Select Tips Advanced Unload" , "Multichannel Select Tips Serial Dilution" (all from the Select Tips palette).

```
{
  "stepType": "Multichannel Select Tips",
  "parameters": { ... },
  "subSteps": [
    { "stepType": "Multichannel Select Tips Load", ... },
    { "stepType": "Multichannel Select Tips Aspirate", ... },
    { "stepType": "Multichannel Select Tips Dispense", ... },
    { "stepType": "Multichannel Select Tips Unload", ... },
    { "stepType": "Multichannel Select Tips End", "parameters": { } }
  ]
}
```

```
}  
}
```

Canonical Examples

Basic select tips workflow:

```
{  
  "stepType": "Multichannel Select Tips",  
  "parameters": {  
    "pod": "Pod1",  
    "rearrangePos": ""  
  },  
  "subSteps": [  
    {  
      "stepType": "Multichannel Select Tips Load",  
      "parameters": {  
        "tipType": "BC190F",  
        "tipsLocation": "TL1",  
        "pattern": "Columns",  
        "columns": "1"  
      }  
    },  
    {  
      "stepType": "Multichannel Select Tips Aspirate",  
      "parameters": {  
        "location": "P3",  
        "labwareClass": "BCFlat96",  
        "volume": "100",  
        "columnOffset": "1",  
        "rowOffset": "1",  
        "liquidtype": "Well Contents",  
        "autoSelectPrototype": true  
      }  
    },  
    {  
      "stepType": "Multichannel Select Tips Dispense",  
      "parameters": {  
        "location": "P4",  
        "labwareClass": "BCFlat96",  
        "volume": "100",  
        "columnOffset": "1",  
        "rowOffset": "1",  
        "liquidtype": "Tip Contents",  
        "autoSelectPrototype": true  
      }  
    },  
    {  
      "stepType": "Multichannel Select Tips Unload",  
      "parameters": {  
        "unloadPosition": "TL1"  
      }  
    },  
    { "stepType": "Multichannel Select Tips End", "parameters": { } }  
  ]  
}
```

Common Mistakes

- **Tips on the pod at entry or exit:** Select Tips brackets a *tips-off* → *tips-off* region. Pre-existing tips at start raise a start-with-tips error (see rule 1); tips loaded inside and not unloaded before the terminator raise an exit-with-tips error (see rule 2).
- **Omitting `rearrangePos` when a child Load needs it:** the container accepts an empty value, but Load raises a rearrange-position-not-specified error at runtime when the pattern requires a temporary tip-box position.
- **Using Select-Tips child steps outside this container:** any of the Select-Tips-family leaves raises a container-required error at method root (see rule 4).

Related Guides

- **Pipetting Techniques, Templates, and Auto-Selection** — technique auto-selection applies to Select-Tips-family child pipetting steps.

† Table of contents

Multichannel Select Tips End

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips End"
Category	Terminator (auto-generated)
Terminator	N/A (this IS the terminator)
Compatible Hardware	Any / not restricted

Behavior Summary

The "Multichannel Select Tips End" step (display caption "End Using Select Tips") is the mandatory terminator for a "Multichannel Select Tips" container. It performs a single validation: the multichannel pod **must have no tips loaded** at this point. If tips are still on the pod, it throws an error. This ensures every Select Tips block unloads its tips before exiting — a Select Tips group that loaded tips must unload them (via a Select Tips Unload / Advanced Unload child) before the terminator.

This step cannot be copied, moved, or edited by the user via the user interface. When authoring a method via JSON, include it explicitly as the last element of the container's `subSteps` array.

Parameters Reference Table

None. This step has no user-configurable parameters.

Cross-Field Validation Rules

1. The multichannel pod **must have no tips loaded** when this step executes. If tips remain, it raises an exit-with-tips error. Unload tips inside the container before this terminator.

Structural Context

This step is the **terminator** for "Multichannel Select Tips" . It must always be the last element in `subSteps` .

```
{ "stepType": "Multichannel Select Tips End", "parameters": {} }
```

Common Mistakes

- **Placing anything after the terminator inside `subSteps`** : the terminator must always be the final element in the container's `subSteps` array.
- **Manually editing this step**: it is auto-generated and read-only; do not add parameters or move it out of the container.

- **Reaching the terminator with tips still on the pod:** include a Select Tips Unload (or Advanced Unload) child before this step. See rule 1.
-

Related Guides

- **Multichannel Select Tips** — the container this step terminates; explains the tips-off → tips-off bracket and rearrange-position semantics.

¹ Table of contents

Multichannel Select Tips Serial Dilution

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Serial Dilution"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips Serial Dilution step performs a complete serial dilution protocol using the multichannel pod's currently loaded tips. It orchestrates multiple aspirate/dispense operations internally: optionally transferring diluent to the plate, optionally transferring source compound from a master plate, then performing sequential dilution transfers across rows or columns. It can optionally wash tips between operations and discard excess volume at the end.

The Select Tips Serial Dilution step can perform dilution across rows or across columns, depending which arrangement of tips is loaded on the pod. It requires equally spaced columns or rows of tips to be loaded.

Parameters Reference Table

*Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.*

Decision Flags

Key	Type	Units	Required	Default	Description
<code>addDiluent?</code>	<code>boolean</code>	—	No	<code>false</code>	Include diluent transfer to plate before dilution.
<code>getSourceCompound?</code>	<code>boolean</code>	—	No	<code>false</code>	Transfer source from master plate to first well set.
<code>discardExcess?</code>	<code>boolean</code>	—	No	<code>false</code>	Discard excess volume from last well set.
<code>diluteFirstWells?</code>	<code>boolean</code>	—	No	<code>false</code>	When <code>addDiluent?</code> is <code>true</code> , also add diluent to the first row/column (otherwise the first row/column is left undiluted as the concentrated reference). Has no effect when <code>addDiluent?</code> is <code>false</code> .
<code>washAfterDiluent?</code>	<code>boolean</code>	—	No	<code>false</code>	Wash tips after diluent transfer.
<code>washAfterSource?</code>	<code>boolean</code>	—	No	<code>false</code>	Wash tips after master/source transfer.
<code>washWhenDone?</code>	<code>boolean</code>	—	No	<code>false</code>	Wash tips when serial dilution is complete.

UI Display Flags

All four are **editor-persistence only; not read at runtime**.

Key	Type	Units	Required	Default	Description
<code>showTips?</code>	<code>boolean</code>	—	No	<code>false</code>	Show tips panel in UI. Editor-only.

Key	Type	Units	Required	Default	Description
<code>showPlate?</code>	<code>boolean</code>	—	No	<code>true</code>	Show dilution plate panel in UI. Editor-only.
<code>showDiluent?</code>	<code>boolean</code>	—	No	<code>true</code>	Show diluent panel in UI. Editor-only.
<code>showMaster?</code>	<code>boolean</code>	—	No	<code>false</code>	Show master plate panel in UI. Editor-only.

Wash Configuration (used when any `washAfter*?` / `washWhenDone?` flag is `true`)

Key	Type	Units	Required	Default	Description
<code>activeWashTechnique</code>	<code>string</code>	—	Conditional	<code>""</code>	Name of the active-wash technique to apply between operations. Required when <code>autoSelectActiveWashTechnique</code> is <code>false</code> and any wash decision flag (<code>washAfterDiluent?</code> , <code>washAfterSource?</code> , or <code>washWhenDone?</code>) is <code>true</code> .
<code>autoSelectActiveWashTechnique</code>	<code>boolean</code>	—	No	<code>false</code>	If <code>true</code> , auto-select the wash technique instead of using the named <code>activeWashTechnique</code> .
<code>washCycles</code>	<code>expression-capable string</code>	count	Conditional	<code>""</code>	Number of wash cycles per wash operation. Required when any wash decision flag is <code>true</code> .
<code>washLiquid</code>	<code>string</code>	—	Conditional	<code>""</code>	Wash-solvent name (e.g. "Water"). Required when any wash decision flag is <code>true</code> .
<code>washVolume</code>	<code>expression-capable string</code>	μ L or %	Conditional	<code>""</code>	Wash volume per cycle (e.g. "110%" or an absolute μ L string). Required when any wash decision flag is <code>true</code> .
<code>customActiveWashTechnique</code>	object or <code>null</code>	—	No	<code>null</code>	Inline wash technique object (embedded <code>_biomekType: "technique"</code>). Populated by the technique editor; for JSON authoring, prefer <code>activeWashTechnique</code> by name or <code>autoSelectActiveWashTechnique: true</code> . Preserve on round-trip if present.

Plate Configurations (nested dictionaries)

Key	Type	Units	Required	Default	Description
<code>basePlate</code>	<code>object</code>	—	Yes	—	Dilution plate configuration (see Plate Dictionary below).
<code>diluentPlate</code>	<code>object</code>	—	Conditional	—	Diluent source configuration. Required when <code>addDiluent?</code> is <code>true</code> .
<code>masterPlate</code>	<code>object</code>	—	Conditional	—	Master/source configuration. Required when <code>getSourceCompound?</code> is <code>true</code> .

Plate Dictionary Structure

The three plate dictionaries have **different** key sets — they are structured differently and do not share a common shape. All three share the following common keys:

Key	Type	Required	Default	Description
<code>location</code>	<code>string</code>	Yes	—	Deck position name or labware instance name. Expression-capable.
<code>volume</code>	<code>string</code>	Conditional	—	Transfer volume (expression-capable). Required on <code>basePlate</code> and <code>masterPlate</code> ; on <code>diluentPlate</code> this key is UI-persisted but not read at enqueue — the diluent transfer volume is computed from <code>basePlate.volume</code> and <code>dilutionRatio</code> .
<code>labwareClass</code>	<code>string</code>	Yes	—	Labware class name.
<code>liquidtype</code>	<code>string</code>	Conditional	—	Liquid classification for technique selection.

Key	Type	Required	Default	Description
<code>usingPlate?</code>	<code>boolean</code>	No	match labware type	Selects how the diluent/master plate is rendered in the printed/summary method — titer-plate (well column/row) vs. reservoir (section). Keep it in sync with the plate's actual labware type. It does not gate whether the plate participates (that is <code>addDiluent?</code> / <code>getSourceCompound?</code>), and it is not used for the base plate.
<code>autoSelectPrototype</code>	<code>boolean</code>	No	<code>false</code>	Per-plate technique auto-select (see Pipetting Techniques).
<code>prototype</code>	<code>string</code>	Conditional	<code>""</code>	Named pipetting technique for this plate. Required when <code>autoSelectPrototype</code> is <code>false</code> and no inline <code>customPrototype</code> is set.
<code>customPrototype</code>	<code>object</code>	Conditional	(unset)	Inline pipetting-technique object. Alternative to <code>prototype</code> / <code>autoSelectPrototype</code> ; populated by the editor.
<code>operation</code>	<code>string</code>	No	(set by editor)	Editor-persistence only; written by the technique selector for its own filtering. Not read at runtime. Preserve on round-trip if present.
<code>overrideHeight</code>	<code>boolean</code>	No	<code>false</code>	Per-plate tip-height override — <code>true</code> $\Rightarrow$ use <code>height</code> / <code>heightFrom</code> instead of the technique default.
<code>height</code>	<code>number</code>	Conditional	—	Tip-height offset (mm); used when <code>overrideHeight</code> is <code>true</code> .
<code>heightFrom</code>	integer or string	Conditional	—	Height reference: <code>0</code> =Liquid, <code>1</code> =Bottom, <code>2</code> =Top. Also accepts string aliases "Liquid", "Bottom", "Top", matching the Multichannel Aspirate/Dispense/Mix and Select Tips Aspirate steps. Used when <code>overrideHeight</code> is <code>true</code> .
<code>customHeight</code>	<code>boolean</code>	No	<code>false</code>	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/custom height, which also implies <code>overrideHeight</code> . Use <code>false</code> with numeric heights and carry the override on <code>overrideHeight</code> / <code>height</code> / <code>heightFrom</code> . Preserve on round-trip if present. See Pipetting Techniques, Templates, and Auto-Selection.

_Each plate dictionary **must** have a pipetting technique configured — at least one of `autoSelectPrototype`: `true` , a named `prototype` , or an inline `customPrototype` must be present. Otherwise enqueue raises a no-technique error.

*Each plate dictionary carries the **full per-plate pipetting + height sub-config** above, in addition to its geometry keys below. These behave exactly as the like-named keys on the Select Tips Multichannel Aspirate-Dispense steps.*

basePlate (dilution plate) adds *only* the following keys — it does not require `columnOffset` , `rowOffset` , or `reservoirSection` :

Key	Type	Description
<code>startWells</code>	<code>string</code>	Starting well index for dilution range to position the back left tip over
<code>endWells</code>	<code>string</code>	Ending well index for dilution range to position the back left tip over
<code>wellSet</code>	<code>string</code>	Offset rows/columns to dilute, when mandrel spacing does not match well spacing. For example, if performing a column dilution with a 96 head in a 384-well plate, a value of <code>1</code> hits rows 1, 3, etc., while <code>2</code> hits rows 2, 4, etc.
<code>discardExcessPosition</code>	<code>string</code>	Position for excess discard

diluentPlate (diluent source) adds *only* the following keys — it has **no** `startWells` , `endWells` , or `wellSet` :

Key	Type	Description
<code>columnOffset</code>	<code>string</code>	Starting column (1-based, expression-capable) to position the back left tip over
<code>rowOffset</code>	<code>string</code>	Starting row (1-based, expression-capable) to position the back left tip over
<code>reservoirSection</code>	<code>string</code>	Reservoir section (for reservoir labware) to position the back left tip over
<code>dilutionRatio</code>	expression-capable string (real ≥ 1.0)	Dilution ratio (e.g., "2.0" for 1:2 dilution). Enqueue rejects values < 1.0 .

masterPlate (master/source) adds *only* the following keys — it has **no** `startWells` , `endWells` , `wellSet` , or `dilutionRatio` :

Key	Type	Description
columnOffset	string	Starting column (1-based, expression-capable) to position the back left tip over
rowOffset	string	Starting row (1-based, expression-capable) to position the back left tip over
reservoirSection	string	Reservoir section (for reservoir labware) to position the back left tip over

Cross-Field Validation Rules

1. This step **must** be inside a "Multichannel Select Tips" container.
2. The loaded tip pattern **must** be equally spaced.
3. Tips must already be loaded on the pod, or a pod-no-tips error is raised.
4. When addDiluent? is true, diluentPlate must be provided with valid location and volume.
5. When getSourceCompound? is true, masterPlate must be provided.
6. When discardExcess? is true, basePlate.discardExcessPosition must be set.
7. diluentPlate.dilutionRatio **must** evaluate to a real ≥ 1.0 ; enqueue rejects values below 1.0.
8. Each active plate dictionary (basePlate always; diluentPlate when addDiluent? is true; masterPlate when getSourceCompound? is true) **must** have a pipetting technique configured — either autoSelectPrototype: true, a named prototype, or an inline customPrototype. Otherwise enqueue raises a no-technique error.

Structural Context

This step is a leaf — it goes inside a "Multichannel Select Tips" container but does not have its own subSteps.

Canonical Examples

Simple serial dilution (1:2 across columns):

```
{
  "stepType": "Multichannel Select Tips Serial Dilution",
  "parameters": {
    "addDiluent?": true,
    "getSourceCompound?": true,
    "discardExcess?": true,
    "washAfterSource?": true,
    "basePlate": {
      "location": "P4",
      "volume": "100",
      "labwareClass": "BCFlat96",
      "liquidtype": "Well Contents",
      "autoSelectPrototype": true,
      "startWells": "1",
      "endWells": "12",
      "wellSet": "1",
      "discardExcessPosition": "AW1"
    },
    "diluentPlate": {
      "location": "P3",
      "volume": "100",
      "labwareClass": "Reservoir",
      "liquidtype": "Well Contents",
      "autoSelectPrototype": true,
      "reservoirSection": "1",
      "dilutionRatio": "2.0"
    },
    "masterPlate": {
      "location": "P5",
      "volume": "100",
      "labwareClass": "BCFlat96",
      "liquidtype": "Well Contents",
      "autoSelectPrototype": true,
      "columnOffset": "1",
      "rowOffset": "1"
    }
  }
}
```

Common Mistakes

- **Uneven tip pattern loaded:** unequally-spaced rows/columns fail — the pattern loaded by the preceding Select Tips Load must be a regular row or column pitch (see rule 2).
- **Enabling a decision flag without its plate/config:** `addDiluent?` needs `diluentPlate`; `getSourceCompound?` needs `masterPlate`; `discardExcess?` needs `basePlate.discardExcessPosition`; wash flags need the wash-configuration keys.
- **Assuming plate dictionaries share a shape:** `basePlate` has `startWells` / `endWells` / `wellSet`; `diluentPlate` and `masterPlate` instead have `columnOffset` / `rowOffset` / `reservoirSection` (and only `diluentPlate` has `dilutionRatio`) — copying keys across them is a common export drift.

Multichannel Select Tips Aspirate

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Aspirate"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips Aspirate step picks up liquid from a labware position using the multichannel pod's currently loaded tip pattern. It validates that tips are loaded, positions the pod over the target wells (calculated from row/column offset and current tip pattern), performs the aspirate using the selected technique, then returns to safe height. Only the tips that are physically loaded participate in the operation.

This step must be placed inside a `"Multichannel Select Tips"` container, after tips have been loaded.

To perform an aspirate operation using a **full** head of tips (96 for a 96 head, 384 for a 384 head), do not use Multichannel Select Tips steps. Use Multichannel Aspirate.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>location</code>	expression-capable string	—	Yes	<code>""</code>	Deck position name or labware instance name of source labware. Supports expressions (prefix with <code>=</code>).
<code>volume</code>	expression-capable string	µL	Yes	<code>"0"</code>	Volume to aspirate. Must be non-negative and less than or equal to the volume in the source.
<code>labwareClass</code>	string	—	Yes	<code>""</code>	Expected labware class at <code>location</code> . Empty or missing values raise a missing-labware-class error. A non-empty, valid class must be supplied.
<code>liquidtype</code>	expression-capable string	—	Yes	<code>""</code>	Source liquid classification. Commonly <code>"Well Contents"</code> (auto-detect from labware) or a named liquid type. Must be non-empty.
<code>columnOffset</code>	expression-capable string	—	Conditional	<code>"1"</code>	Starting column (1-based) to place the back left loaded tip over. Required when source labware is a microplate or tube rack.
<code>rowOffset</code>	expression-capable string	—	Conditional	<code>"1"</code>	Starting row (1-based) to place the back left loaded tip over. Required when source labware is a microplate or tube rack.
<code>reservoirSection</code>	expression-capable string	—	Conditional	<code>"1"</code>	Reservoir section number to place the back left loaded tip over. Required when source labware is a reservoir.

Key	Type	Units	Required	Default	Description
prototype	string	—	Conditional	""	Technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is not present.
autoSelectPrototype	boolean	—	No	false	If <code>true</code> , auto-select technique.
customPrototype	object	—	No	—	Inline technique object (<code>_biomekType</code> : "technique" payload). When present, overrides both <code>prototype</code> and <code>autoSelectPrototype</code> . Preserve on round-trip if present; prefer using named techniques to custom.
overrideHeight	boolean	—	No	false	If <code>true</code> , use custom height to aspirate.
height	string	mm	Conditional	—	Custom pipetting height. Required when <code>overrideHeight</code> is <code>true</code> .
heightFrom	integer or string	—	Conditional	—	Height reference — <code>0</code> / <code>1</code> / <code>2</code> or the aliases "Liquid" / "Bottom" / "Top". Runtime accepts either form. Required when <code>overrideHeight</code> is <code>true</code> .
doPositionOpenAndClose	boolean	—	No	true	Whether to open the position before accessing and close when done. Preserve on round-trip if present; otherwise do not author.
operation	string	—	No	"Aspirate"	Defaults to "Aspirate". Do not change; no other operations are valid.
customHeight	boolean	—	No	false	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/expression, which also implies <code>overrideHeight</code> . Use <code>false</code> with numeric heights and author <code>overrideHeight</code> / <code>height</code> / <code>heightFrom</code> directly. Preserve on round-trip if present. See Pipetting Techniques, Templates, and Auto-Selection.
usingPlate?	boolean	—	No	—	Selects how the step renders in the printed/summary method — titer-plate (well column/row) vs. reservoir (section). Keep it in sync with the actual labware type.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Aspirate"	Defaults to "Aspirate"; do not change.
heightFrom	0, 1, 2 (Or "Liquid", "Bottom", "Top")	Integer form preferred in JSON; string aliases resolve at runtime. Same encoding as Multichannel Aspirate / Dispense / Mix.
liquidtype	Any named liquid, "Well Contents"	"Tip Contents" is invalid for aspirate.

Cross-Field Validation Rules

- This step **must** be inside a "Multichannel Select Tips" container.
- Pod **must** have tips loaded, or a pod-no-tips error is raised.
- Tips must have been loaded via Select Tips Load, else a wrong-tip-load error is raised.
- `volume` must be non-negative, or a negative-volume error is raised.
- Position must be reachable and contain labware.

6. Every active tip must map to a valid well, else a tip-miss error is raised. (Runtime motion collisions are a separate, later hardware check, not this enqueue validation.)
7. When `overrideHeight` is `true`, `height` and `heightFrom` must be provided.
8. If `autoSelectPrototype` is `false`, then either `prototype` **must** name a valid technique or an inline `customPrototype` object **must** be bound. `customPrototype` takes precedence over both `autoSelectPrototype` and `prototype`.

Structural Context

This step is a leaf inside a "Multichannel Select Tips" container. It does not support subSteps.

Canonical Examples

```
{
  "stepType": "Multichannel Select Tips Aspirate",
  "parameters": {
    "location": "P3",
    "volume": "50",
    "labwareClass": "BCFlat96",
    "columnOffset": "1",
    "rowOffset": "3",
    "liquidtype": "Well Contents",
    "autoSelectPrototype": true,
    "usingPlate?": true
  }
}
```

Common Mistakes

- **Tip-well mismatch:** The loaded tip pattern combined with `columnOffset` / `rowOffset` must land every active tip in a well (see rule 6).
- **Empty `labwareClass`:** The default is `""` (empty), which fails the non-empty check — supply a real class name.
- **Authoring `customHeight` or `operation`:** `customHeight` only selects how `height` is encoded — leave it `false` unless an expression is required, and carry the override on `overrideHeight` / `height` / `heightFrom`. `operation` defaults to `"Aspirate"` — leave it as-is.

[†] Table of contents

Multichannel Select Tips Dispense

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Dispense"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips Dispense step delivers liquid to a labware position using the multichannel pod's currently loaded tip pattern. When `emptyTips` is `true`, it dispenses all liquid currently in the tips (volume calculated from tip contents) rather than using the `volume` parameter. After dispensing, it updates the destination labware's liquid type only if it is not already set. Only the tips that are physically loaded participate in the operation.

This step must be placed inside a `"Multichannel Select Tips"` container, after tips have been loaded and liquid has been aspirated.

To perform a dispense operation using a **full** head of tips (96 for a 96 head, 384 for a 384 head), do not use Multichannel Select Tips steps. Use Multichannel Dispense.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>location</code>	expression-capable string	—	Yes	<code>""</code>	Deck position name or labware instance name of destination labware. Supports expressions (prefix with <code>=</code>).
<code>volume</code>	expression-capable string	µL	Conditional	<code>"0"</code>	Volume to dispense. Required when <code>emptyTips</code> is <code>false</code> . Must be non-negative.
<code>emptyTips</code>	boolean	—	No	<code>false</code>	If <code>true</code> , dispense all liquid in tips (ignores <code>volume</code>).
<code>labwareClass</code>	string	—	Yes	<code>""</code>	Expected labware class at <code>location</code> . Empty or missing values raise a missing-labware-class error. A non-empty, valid class must be supplied.
<code>liquidtype</code>	expression-capable string	—	Yes	<code>""</code>	Destination liquid classification. Commonly <code>"Tip Contents"</code> (auto-detect from tips) or a named liquid type. Must be non-empty.
<code>columnOffset</code>	expression-capable string	—	Conditional	<code>"1"</code>	Starting column (1-based) to place the back left loaded tip over. Required when destination labware is a microplate or tube rack.
<code>rowOffset</code>	expression-capable string	—	Conditional	<code>"1"</code>	Starting row (1-based) to place the back left loaded tip over. Required when destination labware is a microplate or tube rack.

Key	Type	Units	Required	Default	Description
reservoirSection	expression-capable string	—	Conditional	"1"	Reservoir section number to place the back left loaded tip over. Required when destination labware is a reservoir.
prototype	string	—	Conditional	—	Technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is not present.
autoSelectPrototype	boolean	—	No	false	If <code>true</code> , auto-select technique.
customPrototype	object	—	No	—	Inline technique object (<code>_biomekType</code> : "technique" payload). When present, overrides both <code>prototype</code> and <code>autoSelectPrototype</code> . Preserve on round-trip if present; prefer using named techniques to custom.
overrideHeight	boolean	—	No	false	If <code>true</code> , use custom height to dispense.
height	number or expression-capable string	mm	Conditional	—	Custom pipetting height. Required when <code>overrideHeight</code> is <code>true</code> .
heightFrom	integer or string	—	Conditional	—	Height reference — <code>0</code> / <code>1</code> / <code>2</code> or the aliases "Liquid" / "Bottom" / "Top". Runtime accepts either form. Required when <code>overrideHeight</code> is <code>true</code> .
doPositionOpenAndClose	boolean	—	No	true	Whether to open the position before accessing and close when done. Preserve on round-trip if present; otherwise do not author.
operation	string	—	No	"Dispense"	Defaults to "Dispense". Do not change; no other operations are valid.
customHeight	boolean	—	No	false	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/expression, which also implies <code>overrideHeight</code> . Use <code>false</code> with numeric heights and author <code>overrideHeight</code> / <code>height</code> / <code>heightFrom</code> directly. Preserve on round-trip if present. See Pipetting Techniques, Templates, and Auto-Selection.
usingPlate?	boolean	—	No	—	Selects how the step renders in the printed/summary method — titer-plate (well column/row) vs. reservoir (section). Keep it in sync with the actual labware type.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

Parameter	Allowed Values	Notes
operation	"Dispense"	Defaults to "Dispense"; do not change.
heightFrom	0, 1, 2 (or "Liquid", "Bottom", "Top")	Integer form preferred in JSON; string aliases resolve at runtime. Same encoding as Multichannel Aspirate / Dispense / Mix.
liquidtype	Any named liquid, "Tip Contents"	"Well Contents" is invalid for dispense.

Cross-Field Validation Rules

1. This step **must** be inside a "Multichannel Select Tips" container.

2. Pod **must** have tips loaded, or a pod-no-tips error is raised.
3. Tips must have been loaded via Select Tips Load, else a wrong-tip-load error is raised.
4. When `emptyTips` is `false`, `volume` must be provided and non-negative — a negative volume raises a negative-volume error.
5. When `emptyTips` is `true`, volume is calculated from tip contents (non-air volumes summed).
6. Position must be reachable and contain labware.
7. Every active tip must map to a valid well, else a tip-miss error is raised. (Runtime motion collisions are a separate, later hardware check, not this enqueue validation).
8. When `overrideHeight` is `true`, `height` and `heightFrom` must be provided.
9. If `autoSelectPrototype` is `false`, then either `prototype` **must** name a valid technique or an inline `customPrototype` object **must** be bound. `customPrototype` takes precedence over both `autoSelectPrototype` and `prototype`.

Structural Context

This step is a leaf inside a "Multichannel Select Tips" container. It does not support subSteps.

Canonical Examples

Standard dispense:

```
{
  "stepType": "Multichannel Select Tips Dispense",
  "parameters": {
    "location": "P4",
    "volume": "50",
    "labwareClass": "BCFlat96",
    "columnOffset": "2",
    "rowOffset": "1",
    "liquidtype": "Tip Contents",
    "autoSelectPrototype": true,
    "usingPlate?": true
  }
}
```

Empty tips:

```
{
  "stepType": "Multichannel Select Tips Dispense",
  "parameters": {
    "location": "P4",
    "emptyTips": true,
    "labwareClass": "BCFlat96",
    "columnOffset": "1",
    "rowOffset": "1",
    "liquidtype": "Tip Contents",
    "autoSelectPrototype": true,
    "usingPlate?": true
  }
}
```

Common Mistakes

- **Tip-well mismatch:** The loaded tip pattern combined with `columnOffset` / `rowOffset` must land every active tip in a well (see rule 6).
- **Providing volume with `emptyTips: true`:** `volume` is ignored — actual amount comes from summed non-air tip contents.
- **Empty `labwareClass`:** The default is "" (empty), which fails the non-empty check — supply a real class name.
- **Destination `liquidtype` unexpectedly unchanged:** The step only writes `liquidtype` to destination wells that don't already have one; existing labels are preserved.
- **Authoring `customHeight` or `operation`:** `customHeight` only selects how `height` is encoded — leave it `false` unless an expression is required, and carry the override on `overrideHeight` / `height` / `heightFrom`. `operation` defaults to "Dispense" — leave it as-is.

Multichannel Select Tips Mix

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Mix"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Multichannel Select Tips Mix step performs repeated aspirate/dispense cycles at a single position to mix well contents. It uses the multichannel pod's currently loaded tip pattern. The mix count controls how many cycles are performed, and the volume controls how much liquid is moved per cycle.

This step must be placed inside a `"Multichannel Select Tips"` container, after tips have been loaded.

To perform a mix operation using a **full** head of tips (96 for a 96 head, 384 for a 384 head), do not use Multichannel Select Tips steps. Use Multichannel Mix.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>location</code>	<code>string</code>	—	Yes	<code>""</code>	Deck position name or labware instance name of source labware. Supports expressions (prefix with <code>=</code>).
<code>volume</code>	<code>expression-capable string</code>	μ L	Yes	<code>"0"</code>	Volume per mix cycle. Must be non-negative and less than or equal to the volume in the labware. Unlike other mix steps, percentages are not accepted; volume must be in μ L.
<code>mixCount</code>	<code>expression-capable string</code>	—	Yes	<code>"1"</code>	Number of mix cycles. Must be a positive integer.
<code>labwareClass</code>	<code>string</code>	—	Yes	<code>""</code>	Expected labware class at <code>location</code> . Empty or missing values raise a missing-labware-class error. A non-empty, valid class must be supplied.
<code>liquidtype</code>	<code>expression-capable string</code>	—	Yes	<code>""</code>	Source liquid classification. Commonly <code>"Well Contents"</code> (auto-detect from labware) or a named liquid type. Must be non-empty.
<code>columnOffset</code>	<code>expression-capable string</code>	—	Conditional	<code>"1"</code>	Starting column (1-based) to place the back left loaded tip over. Required when source labware is a microplate or tube rack.
<code>rowOffset</code>	<code>expression-capable string</code>	—	Conditional	<code>"1"</code>	Starting row (1-based) to place the back left loaded tip over. Required when source labware is a microplate or tube rack.
<code>reservoirSection</code>	<code>expression-capable</code>	—	Conditional	<code>"1"</code>	Reservoir section number to place the back left loaded tip over. Required when source labware is a reservoir.

Key	Type	Units	Required	Default	Description
	string				
prototype	string	—	Conditional	""	Technique name. Required when <code>autoSelectPrototype</code> is <code>false</code> and <code>customPrototype</code> is not present.
autoSelectPrototype	boolean	—	No	false	If <code>true</code> , auto-select technique.
customPrototype	object	—	No	—	Inline technique object (<code>_biomekType</code> : "technique" payload). When present, overrides both <code>prototype</code> and <code>autoSelectPrototype</code> . Preserve on round-trip if present; prefer using named techniques to custom.
overrideHeight	boolean	—	No	false	If <code>true</code> , use custom height to aspirate.
height	string	mm	Conditional	—	Custom pipetting height. Required when <code>overrideHeight</code> is <code>true</code> .
heightFrom	integer or string	—	Conditional	—	Height reference — <code>0 / 1 / 2</code> or the aliases "Liquid" / "Bottom" / "Top". Runtime accepts either form. Required when <code>overrideHeight</code> is <code>true</code> .
doPositionOpenAndClose	boolean	—	No	true	Whether to open the position before accessing and close when done. Preserve on round-trip if present; otherwise do not author.
operation	string	—	No	"Mix"	Defaults to "Mix"; do not change.
customHeight	boolean	—	No	false	Encoding discriminator for <code>height</code> : <code>false</code> = numeric offset; <code>true</code> = text/expression, which also implies <code>overrideHeight</code> . Use <code>false</code> with numeric heights and author <code>overrideHeight / height / heightFrom</code> directly. Preserve on round-trip if present. See Pipetting Techniques, Templates, and Auto-Selection.
usingPlate?	boolean	—	No	—	Selects how the step renders in the printed/summary method — titer-plate (well column/row) vs. reservoir (section). Keep it in sync with the actual labware type.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Cross-Field Validation Rules

- This step **must** be inside a "Multichannel Select Tips" container.
- Pod **must** have tips loaded, or a pod-no-tips error is raised.
- Tips must have been loaded via Select Tips Load, else a wrong-tip-load error is raised.
- `volume` must be non-negative, or a negative-volume error is raised.
- Position must be reachable and contain labware.
- Every active tip must map to a valid well, else a tip-miss error is raised. (Runtime motion collisions are a separate, later hardware check, not this enqueue validation.)
- `mixCount` must evaluate to a positive integer (*throws error if empty or ≤ 0*).
- The per-cycle `volume` must not exceed the well's tracked contents — the same bound that governs any other aspirate, raising Cannot pipette <v> uL; the well only has <x> uL in it. `mixCount` does not multiply this budget, and no conditioning or pre-wet overhead is added to a mix cycle. See Well Volume Tracking.
- Tips must be empty before the mix begins, or the step raises The mix operation cannot begin until tips are empty. Air gaps do not trip the check; anything classified as liquid does.
- When `overrideHeight` is `true`, `height` and `heightFrom` must be provided.
- If `autoSelectPrototype` is `false`, then either `prototype` **must** name a valid technique or an inline `customPrototype` object **must** be bound. `customPrototype` takes precedence over both `autoSelectPrototype` and `prototype`.

Structural Context

This step is a leaf inside a "Multichannel Select Tips" container. It does not support subSteps.

Canonical Examples

```
{
  "stepType": "Multichannel Select Tips Mix",
  "parameters": {
    "location": "P4",
    "volume": "80",
    "mixCount": "5",
    "labwareClass": "BCFlat96",
    "columnOffset": "1",
    "rowOffset": "1",
    "liquidtype": "Well Contents",
    "autoSelectPrototype": true,
    "usingPlate?": true
  }
}
```

Mix in a reservoir section using expressions.

```
{
  "stepType": "Multichannel Select Tips Mix",
  "parameters": {
    "location": "=reservoir",
    "volume": "=mixVolume",
    "mixCount": "=mixCount",
    "labwareClass": "ModularReservoir",
    "reservoirSection": "3",
    "liquidtype": "Well Contents",
    "autoSelectPrototype": true,
    "usingPlate?": false
  }
}
```

Common Mistakes

- **mixCount of 0 or empty:** Must evaluate to a positive integer, else the step throws at enqueue.
- **Redundant technique keys:** `customPrototype` always wins when present (checked before `autoSelectPrototype`). When `customPrototype` is absent and `autoSelectPrototype: true`, `prototype` is ignored (auto-selection is used instead).
- **Tip-well mismatch:** The loaded tip pattern combined with `columnOffset` / `rowOffset` must land every active tip in a well (see rule 6).
- **Empty labwareClass:** The default is "" (empty), which fails the non-empty check — supply a real class name.
- **Authoring customHeight or operation:** `customHeight` only selects how `height` is encoded — leave it `false` unless an expression is required, and carry the override on `overrideHeight` / `height` / `heightFrom`. `operation` defaults to "Mix" — leave it as-is.

Multichannel Select Tips Load

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Load"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips Load step picks up tips from a tip box using the multichannel pod, loading a specific pattern of tips (single tip, specific rows, or specific columns). It supports a primary tip location and an optional backup location. When loading rows or columns, if tips are not available contiguously, the step can use the container's rearrange position to perform multi-step loading (load partial, unload to rearrange, load more).

This step must be placed inside a "Multichannel Select Tips" container. The pod must not already have tips loaded.

When the requested "Rows" / "Columns" pattern cannot be satisfied by a single pick from the available tips (e.g. the requested rows/columns aren't the topmost/rightmost tips in the box), the parent "Multichannel Select Tips" container must have `rearrangePos` set to a deck position holding an empty tip box of the same type — the child Load rearranges through that position. Without it, enqueue throws a rearrange-position-not-specified error; with a position whose box still holds tips, it throws `The tip box on position "<position>" must be empty.` For the JSON that declares an empty tip box — which is not the same shape as omitting the box — see §Declaring the rearrange position's empty tip box in Multichannel Select Tips.

Budget the rearrange box in **addition** to the boxes you draw tips from: it occupies a tip-load position for the whole method and supplies no tips.

It is recommended to name tip boxes used by Multichannel Select Tips steps and reference them by name, to avoid sharing boxes between regular Multichannel Load Tips/Unload Tips steps and Select Tips steps. Select Tips steps do **not** decrement the tip reuse counter, so boxes used by select tips will appear usable to Multichannel Load Tips steps after the Select Tips group even though they may be dirty.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
tipType	string	—	Yes	""	Tip box labware-class name, bare and unprefixed (e.g. "BC190F" , "BC50F"). Must match a labware class installed on the instrument, and its <code>WellsX</code> / <code>WellsY</code> must match the pod's mandrel grid. Must not be empty. See Catalog: Labware Classes (in <i>Biomek Instrument & Project Reference</i>).
tipsLocation	string	—	Yes	""	Deck position containing the tip box (e.g. "P1") or labware instance name (as specified in Instrument Setup step). This is not a labware-class name — the class is <code>tipType</code> . Update it only when the tip box moves to a different position, not when its class changes; contrast Multichannel/Span-8 Load Tips, where <code>tips</code> names a tip <i>class</i> . See Labware & Position Resolution §4-5.

Key	Type	Units	Required	Default	Description
backupTipsLocation	string	—	No	""	Backup tip box position (used if primary is empty).
pattern	string	—	No	"Single"	Loading pattern. See enumeration below.
rows	expression-capable string	—	Conditional	"1"	Comma-separated row numbers (1-based). Used when pattern is "Rows" .
columns	expression-capable string	—	Conditional	"1"	Comma-separated column numbers (1-based). Used when pattern is "Columns" .

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

pattern Values

Value	Meaning
"Single"	Load exactly one tip
"Rows"	Load all tips in specified row(s)
"Columns"	Load all tips in specified column(s)

Note that the pattern indicates **where on the pod the tips will be loaded**, not the location in the box to load. For example, selecting the right-most column of a tip box will load those tips onto the right-most column of mandrels on the pod.

Cross-Field Validation Rules

- Pod **must not** have tips loaded, or a load-with-tips error is raised.
- `tipType` **must not** be empty, or a tips-none error is raised.
- Tip type must be compatible with the pod.
- Primary position must contain a tip box with the specified tip type.
- If `backupTipsLocation` is specified, it must also contain the correct tip type.
- For "Rows" pattern: rows must be valid integers; tip box must have complete rows available.
- For "Columns" pattern: columns must be valid integers; tip box must have complete columns available.
- Partial rows/columns are not allowed (*throws "incomplete row/column"*).

Structural Context

This step is a leaf inside a "Multichannel Select Tips" container.

Canonical Examples

Load a full column of tips:

```
{
  "stepType": "Multichannel Select Tips Load",
  "parameters": {
    "tipType": "BC190F",
    "tipsLocation": "P1",
    "pattern": "Columns",
    "columns": "1"
  }
}
```

```
}  
}
```

Load rows 1 and 2:

```
{  
  "stepType": "Multichannel Select Tips Load",  
  "parameters": {  
    "tipType": "BC50F",  
    "tipsLocation": "P1",  
    "backupTipsLocation": "P2",  
    "pattern": "Rows",  
    "rows": "1,2"  
  }  
}
```

Load a single tip:

```
{  
  "stepType": "Multichannel Select Tips Load",  
  "parameters": {  
    "tipType": "BC190F",  
    "tipsLocation": "P1",  
    "pattern": "Single"  
  }  
}
```

Common Mistakes

- **Partial row/column anywhere in the box:** The step scans the tip box (right-to-left for "Columns" , bottom-to-top for "Rows") looking for full columns/rows. If it encounters any column (or row) with some tips present and some missing — even one that wasn't specifically requested — enqueue throws. Fully empty columns/rows are fine and are skipped. The step will not silently pick up a partial set.
- **backupTipsLocation misunderstood:** It's only consulted when the primary box is empty for the requested tips; it is not a second-pattern load.
- **Missing rows / columns for the chosen pattern:** pattern: "Rows" requires rows ; pattern: "Columns" requires columns . Set the value to match your layout; the default "1" loads only the first row or column.

The Error cleaning tips for transfer from <position>: message

This message does **not** come from any Select Tips step. It is emitted by a **Transfer** step running on a Multichannel or Fixed-8 pod, which wraps its internal tip-provisioning routine in this prefix. <position> is the **source** deck position of the transfer that was about to run — it is not the tip box, and not the failing location. The real diagnosis is the message text **after the colon**, which is whatever the wrapped work raised.

It is not a reliable indicator of tip reuse. Despite the "cleaning" wording, the routine is the transfer's general tip-provisioning path, and it is invoked for the **first transfer of the step unconditionally** — before any between-source or between-destination consideration applies. Turning changeTipsBetweenDests off, turning changeTipsBetweenSources off, or turning useJIT off will not suppress it.

What the wrapped routine does, in order:

1. Reads the pod's tips — the whole-head Tips on a Multichannel pod, or the validated per-probe set on a Fixed-8. A pod that exposes neither raises No tips .
2. On the first (checking) call it can **exit doing nothing**: when the pod already carries tips whose class name, labware name or origin position matches the transfer's tip location — or when useCurrentTips is set — and those tips are unused (or clean, if active wash is on).
3. Otherwise it either **washes** the tips (active wash, and the tips are already the right type), or **enqueues a tip load**: a Multichannel Load Tips step on a Multichannel pod, or an Unload-then-Load pair on a Fixed-8. The tip location it passes is the transfer's own tipLocation .

So the whole failure surface of tip loading, tip unloading and washing can surface behind this one prefix. Bodies you are most likely to see:

Message after the colon	Meaning
Tips not specified.	The transfer has no tip location and is not reusing current tips.
Cannot use current tips; no tips loaded.	useCurrentTips is set but the pod is empty.
No tips	The pod exposes no tip state — usually the wrong pod type for the step.
Unable to find and retrieve tips of type <value> for <pod>.	The tip load failed; the appended per-box skip reasons say why. See Multichannel Load Tips.
The fixed-8 pod does not support washing tips.	Active wash was requested on a Fixed-8 pod.
a tip-discard or unload error	The Fixed-8 unload half failed — see Multichannel Unload Tips for how a discard location is resolved.

† Table of contents

Multichannel Select Tips Unload

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Unload"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Select Tips Unload step removes tips from the multichannel pod to a given location. If the destination is a tip trash, tips are simply discarded. If the destination is a tip box, the step finds an available row/column position (first-fit scan) and returns the tips to the box. The step determines the unload position automatically based on the current tip pattern.

This step must be placed inside a `"Multichannel Select Tips"` container.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>unloadPosition</code>	expression-capable string	—	Yes	""	Location to unload tips to (tip trash, position name, tip box class, or tip box instance name).

*Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.*

Cross-Field Validation Rules

- Pod **must** have tips loaded, or an unload-no-tips error is raised.
- `unloadPosition` must evaluate to a tip trash, position containing a tip box, or a tip box instance, or an invalid-unload-position error is raised.
- If unloading to a tip box, it must have space for the current tip pattern, or a no-space error is raised.
- When unloading to a tip box, the tips must have been loaded via a `"Multichannel Select Tips Load"` step, or a wrong-tip-load error is raised.

Structural Context

This step is a leaf inside a `"Multichannel Select Tips"` container.

Canonical Examples

Unload to tip trash:

```
{
  "stepType": "Multichannel Select Tips Unload",
  "parameters": {
```



```
    "unloadPosition": "TR1"  
  }  
}
```

Return tips to named box:

```
{  
  "stepType": "Multichannel Select Tips Unload",  
  "parameters": {  
    "unloadPosition": "MyTips"  
  }  
}
```

Common Mistakes

- **Tip box lacks space for the current pattern:** The step auto-picks the destination cells, but the box must actually have those cells free — otherwise enqueue throws.
- **Trying to unload to a plate/reservoir:** `unloadPosition` must resolve to a tip trash or a tip box; any other labware fails.

Related Steps

- **Multichannel Select Tips** — the parent container; explains the tips-off bracket and valid child steps.

¹ Table of contents

Multichannel Select Tips Advanced Load

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Advanced Load"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Advanced Load Tips step loads tips from a specific row and column position in a tip box, giving explicit control over which tips are picked up (unlike the standard Load Tips which auto-selects). This step handles precise tip placement when automatic pattern-based loading is insufficient. It also handles the case where a tip box has been preconfigured with a pattern of tips.

The `row` and `column` values allow specifying where mandrel A1 is positioned over the tip box. Whatever tips are underneath the head at that point are loaded onto the pod.

This step must be placed inside a `"Multichannel Select Tips"` container. The pod must not already have tips loaded.

Note that Multichannel Select Tips Advanced Load does not decrement the tip reuse counter when tips are later returned to the box.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
tipType	string	—	Yes	""	Tip box labware-class name, bare and unprefixed (e.g. "BC230", "BC190F", "BC1025F"). Same grammar as Multichannel Select Tips Load <code>tipType</code> ; see Catalog: Labware Classes (in <i>Biomek Instrument & Project Reference</i>).
tipsLocation	expression-capable string	—	Yes	""	Deck position containing the tip box, tip box type, or tip box instance name.
row	expression-capable string	—	Yes	"1"	Row number in tip box to position mandrel A1 over (1-based; negative or zero values are allowed to offset the pod mandrel grid for partial-tip loading).
column	expression-capable string	—	Yes	"1"	Column number in tip box to position mandrel A1 over (1-based; negative or zero values are allowed to offset the pod mandrel grid for partial-tip loading).

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Cross-Field Validation Rules

- Pod **must not** have tips loaded, or a load-with-tips error is raised.
- `tipType` **must not** be empty.
- Tip type must be compatible with the pod.
- Position must contain a tip box with the correct tip type, or if a named tip box or tip box class is specified, it must match a tip box on deck.

5. `row` must evaluate to a valid integer; an empty or non-integer value raises a row-not-specified or invalid-row error.
6. `column` must evaluate to a valid integer (*same rules as `row`*).
7. The `(row, column)` pair must resolve to at least one tip on the pod's mandrel grid; otherwise the step throws a loads-no-tips error.
8. This step **must** be inside a `"Multichannel Select Tips"` container; at method root it raises a container-required error.

Structural Context

This step is a leaf inside a `"Multichannel Select Tips"` container.

Canonical Examples

```
{
  "stepType": "Multichannel Select Tips Advanced Load",
  "parameters": {
    "tipType": "BC230",
    "tipsLocation": "TL1",
    "row": "1",
    "column": "3"
  }
}
```

Common Mistakes

- **Fictional or wrongly-shaped `tipType`**: Use a real, bare tip-box labware class name (e.g. `"BC230"`, `"BC190F"`, `"BC1025F"`, `"BC50F"`, `"BC90"`). Free-text descriptions like `"P250 Tips"` throw an unknown-labware-class error at runtime, and so does a `TipClasses\ / LabwareClasses\` path — the prefixed form belongs on an Instrument Setup labware object, not here.
- **Out-of-range `row / column` that load nothing**: `(row, column)` may fall outside the tip box (including negative values, which offset the mandrel grid for partial-tip loading), but the resolved intersection must land on at least one tip — otherwise the step throws a loads-no-tips error.
- **Using this where standard Load Tips would do**: Advanced Load is for precise, single-position picks; for automatic full head loading, use `"Multichannel Select Tips Load"` instead.

Multichannel Select Tips Advanced Unload

Reference manual: Chapter 20 documents these as *Select Tips* steps (the manual omits the *Multichannel* prefix); the JSON `stepType` keeps the `Multichannel Select Tips ...` name.

Property	Value
stepType	"Multichannel Select Tips Advanced Unload"
Category	Leaf (does not accept child steps)
Terminator	N/A
Compatible Hardware	Multichannel pod only

Behavior Summary

The Advanced Unload Tips step unloads tips to a specific row and column position in a tip box, giving explicit control over where tips are placed. The step verifies that tips were originally loaded via a Select Tips family load step (`Multichannel Select Tips Load` or `Multichannel Select Tips Advanced Load` — not via standard multichannel tip loading), then places them at the specified location.

This step must be placed inside a `"Multichannel Select Tips"` container.

Note that Multichannel Select Tips steps do not decrement the tip reuse counter, so after unloading tips, the tips are available to reuse.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>tipsLocation</code>	expression-capable string	—	Yes	<code>" "</code>	Deck position containing the tip box, tip box type, or tip box instance name.
<code>row</code>	expression-capable string	—	Yes	<code>"1"</code>	Row number in tip box to position mandrel A1 over (1-based; negative or zero values are allowed).
<code>column</code>	expression-capable string	—	Yes	<code>"1"</code>	Column number in tip box to position mandrel A1 over (1-based; negative or zero values are allowed).

*Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.*

Cross-Field Validation Rules

- Pod **must** have tips loaded, or an unload-no-tips error is raised.
- Tips **must** have been loaded via a Select Tips Load or Advanced Load step.
- Destination must contain a tip box with matching tip type.
- `row` must evaluate to a valid integer.
- `column` must evaluate to a valid integer.
- The destination must have space in the box for the tips to unload.
- The position of mandrel A1 must not leave any loaded tips outside of the box.

Structural Context

This step is a leaf inside a "Multichannel Select Tips" container.

Canonical Examples

```
{
  "stepType": "Multichannel Select Tips Advanced Unload",
  "parameters": {
    "tipsLocation": "TL1",
    "row": "1",
    "column": "5"
  }
}
```

Common Mistakes

- **Tips not loaded via a Select Tips Load or Advanced Load step:** Only tips whose source is tracked can be advanced-unloaded; tips picked up by a plain multichannel Load Tips will be rejected.
- **Destination tip box has the wrong type:** The box at `tipsLocation` must accept the currently-loaded tip type; there is no auto-conversion.
- **Destination tip box must have empty cells to unload all of the tips to:** The loaded tips must be entirely over the target box, and each cell a tip is over must be empty.

[†] Table of contents

Fixed-8 Aspirate

Property	Value
stepType	"Fixed-8 Aspirate"
Category	Leaf
Terminator	N/A
Compatible Hardware	i3 (Fixed-8 pod required)

Behavior Summary

The Fixed-8 Aspirate step picks up liquid from a labware position using the Fixed-8 pod (8 channels in a column that move together). Tips must already be loaded before this step executes. The step accepts a manually specified technique via `prototype` (a stored technique name) or via `customPrototype` (an inline technique object). **Fixed-8 never auto-selects a technique — this is a permanent design choice.** Setting `autoSelectPrototype: true` raises an enqueue error. Always set `autoSelectPrototype: false` and name a technique. The labware must be a TiterPlate, TubeRack, or Reservoir with `CanPipette = true`.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	expression-capable string	—	Yes	—	Name of the Fixed-8 pod (e.g., "Pod1").
<code>where</code>	expression-capable string	—	Yes	—	Deck position name or labware instance name where the source labware is located.
<code>amount</code>	expression-capable string	µL	Yes	—	Volume to aspirate. Must be non-negative and less than or equal to the volume in the well.
<code>firstWell</code>	integer	—	Conditional	—	1-based well index to position the backmost tip over when aspirating. Used when <code>useWellExpression</code> is <code>false</code> .
<code>useWellExpression</code>	boolean	—	No	<code>false</code>	If <code>true</code> , use <code>firstWellExpression</code> instead of <code>firstWell</code> .
<code>firstWellExpression</code>	expression-capable string	—	Conditional	—	Expression for first well. Used when <code>useWellExpression</code> is <code>true</code> .
<code>liquidType</code>	expression-capable string	—	Yes	—	Liquid type for technique selection. Must not be "Tip Contents" for aspirate. Accepts "Well Contents" or any named liquid type.
<code>prototype</code>	expression-capable string	—	Conditional	—	Technique name. Required when <code>customPrototype</code> is not present.
<code>customPrototype</code>	object	—	Conditional	Not present	Inline technique object (a <code>_biomekType: "technique"</code> payload). When present, overrides <code>prototype</code> .
<code>autoSelectPrototype</code>	boolean	—	No	<code>false</code>	Must be false — Fixed-8 Aspirate does not support auto-selection by design (permanent). Setting <code>true</code> raises an enqueue error.
<code>overrideHeight</code>	boolean	—	No	<code>false</code>	If <code>true</code> , use <code>custom height</code> and <code>heightFrom</code> values.

Key	Type	Units	Required	Default	Description
height	expression-capable string	mm	Conditional	—	Custom pipetting height. Used when <code>overrideHeight</code> is <code>true</code> .
heightFrom	expression-capable string or integer	—	Conditional	—	Reference point for height. Accepts case-insensitive strings <code>"Liquid"</code> / <code>"Bottom"</code> / <code>"Top"</code> or integer forms <code>0</code> / <code>1</code> / <code>2</code> (same meaning). Used when <code>overrideHeight</code> is <code>true</code> .
what	string	—	No	""	Expected labware class name for the labware compatibility check (e.g., <code>"BCF1at96"</code>). When non-empty and the actual labware class at <code>where</code> differs, the compatibility check raises an enqueue error. Empty string skips the check. This field uses the short labware class name (e.g. <code>"BCF1at96"</code>); the <code>LabwareClasses\</code> path prefix is used only in Instrument Setup's <code>deckItems</code> .
emptyTips	boolean	—	No	<code>false</code>	Not used for aspirate (dispense only). Should be <code>false</code> .
mixCount	expression-capable string	—	No	—	Ignored for Aspirate — Not applicable to this operation.
podType	string	—	No	<code>"Fixed8"</code>	Pod type identifier. Every editor-produced step carries <code>"Fixed8"</code> ; do not change.
operation	string	—	Yes	—	Must be present and equal <code>"Aspirate"</code> . Every editor-produced step carries it, but when authoring from scratch, if the value doesn't match the step's operation (empty string when the key is omitted), enqueue raises an error. Include it when authoring from scratch. See rule 8.
customHeight	boolean	—	No	<code>false</code>	Encoding discriminator for <code>height</code> / <code>heightFrom</code> . When <code>false</code> , those keys carry numeric/enumerated values; when <code>true</code> , they carry expression text. Every editor-produced step carries it. Set it to match the form of <code>height</code> you author.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Well numbering. Wells are 1-based and **row-major** — on a 96-well plate `A1` = 1, `A12` = 12, `B1` = 13, ..., `H12` = 96. The Fixed-8 pod places its lowest-numbered active probe at `firstWell` and strides **down the column by `+wellsX`**, so `firstWell` = 1 on a 96-well plate hits wells 1, 13, 25, 37, 49, 61, 73, 85 — all of column 1. To sweep plate columns, use `firstWellExpression` = `"=Col"`, **not** `"=1+(Col-1)*8"`: multiplying by the probe count assumes column-major numbering and overflows the plate. See **Probe & Mandrel Selection**.

Liquid-relative heights need a locatable liquid surface. A `heightFrom` of `"Liquid"` (or a follow-liquid technique) needs the run-time liquid level of the labware at `where`. Fixed-8 and Multichannel (96-/384-channel) pods do **not** have liquid-level sensing (LLS is Span-8 only), so an `"Unknown"` well cannot be sensed at run time on these pods — that labware's `volumeType` (seeded in Instrument Setup / Guided Setup) must be `"Nominal"` or `"Known"` **with an `evalAmounts` fill**, or the move throws at run time. See **Pipetting Techniques and Templates §"Liquid types"** / **§"Liquid-relative heights need a locatable liquid surface"**.

Enumerated / Constrained Values

liquidType Values

Value	Meaning
<code>"Well Contents"</code>	Auto-detect liquid from the labware's liquid data set at runtime

Value	Meaning
Any named liquid (e.g., "Water", "Serum")	Use the named liquid for technique selection

"Tip Contents" is NOT valid for aspirate — only for dispense.

heightFrom Values

Accepted both as integers and case-insensitive strings:

Integer	String alias	Meaning
0	"Liquid"	Height measured from the liquid surface
1	"Bottom"	Height measured from the well bottom
2	"Top"	Height measured from the well top

Cross-Field Validation Rules

- 1. amount **must** be non-negative — otherwise enqueue raises a negative-volume error.
- 2. When useWellExpression is true, firstWellExpression is used; otherwise firstWell is required.
- 3. autoSelectPrototype **must** be false — otherwise enqueue raises an auto-select-not-supported error.
- 4. Either customPrototype **must** be present or prototype **must** name a valid stored technique. If neither resolves to a technique, enqueue fails.
- 5. When overrideHeight is true, height and heightFrom must be provided.
- 6. liquidType **must not** be "Tip Contents" for aspirate.
- 7. The labware at where must be a TiterPlate, TubeRack, or Reservoir with CanPipette = true.
- 8. operation **must** be present and equal "Aspirate" — otherwise enqueue raises an operation-mismatch error (when the key is omitted, the value used is the empty string).
- 9. amount **must** keep the pod inside its pipetting range — a separate pod setting, 0–1050 µL in the factory configuration. The check runs at enqueue against the D position the move ends at, meaning the pod's current position plus the calibrated volume, so whatever the tips already hold counts against it. A breach raises The D axis position of <n> µL is outside the supported pipetting range of the pod. **No export carries that maximum**, and settings.max.d is not it: on a Fixed-8 the plunger and gripper share one motor, so max.d is the far end of the combined travel. Tip capacity is normally lower still and is what a method meets first, so this surfaces mainly with a computed volume or a large-capacity tip. See Pod Settings Format (in Biomek Instrument & Project Reference) §"What max.d means, and when it is not the pipetting ceiling".

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Basic aspirate from well A1 with a named technique:

```
{
  "stepType": "Fixed-8 Aspirate",
  "parameters": {
    "pod": "Pod1",
    "where": "P3",
    "operation": "Aspirate",
    "amount": "100",
    "firstWell": 1,
    "liquidType": "Well Contents",
    "autoSelectPrototype": false,
    "prototype": "F8 Medium"
  }
}
```


Aspirate with explicit technique and height override:

```
{
  "stepType": "Fixed-8 Aspirate",
  "parameters": {
    "pod": "Pod1",
    "where": "P5",
    "operation": "Aspirate",
    "amount": "50",
    "firstWell": 9,
    "liquidType": "Well Contents",
    "prototype": "F8 Medium",
    "autoSelectPrototype": false,
    "overrideHeight": true,
    "height": "2.0",
    "heightFrom": "Bottom"
  }
}
```

Common Mistakes

- **Setting** `autoSelectPrototype: true` — Fixed-8 Aspirate/Dispense/Mix reject auto-selection at enqueue. Always pair `autoSelectPrototype: false` with an explicit `prototype` (or `customPrototype`).
- **Omitting** `operation` — every editor-produced step carries it, but when authoring from scratch, omission is read as the empty string and enqueue raises an error. Include `"operation": "Aspirate"`.
- **Using** `"Tip Contents"` as `liquidType` for aspirate — only `"Well Contents"` or a named liquid is valid here.
- **Omitting** `prototype` with no `customPrototype` — with auto-select disabled, one of them must resolve to a technique or enqueue fails.
- **Reading** `settings.max.d` as the **Fixed-8 pipetting ceiling** — it is the far end of the plunger/gripper travel the two share, not a pipetting limit. The pipetting maximum is a separate pod setting that no export carries; see rule 9.

[†] Table of contents

Fixed-8 Dispense

Property	Value
stepType	"Fixed-8 Dispense"
Category	Leaf
Terminator	N/A
Compatible Hardware	i3 (Fixed-8 pod required)

Behavior Summary

The Fixed-8 Dispense step delivers liquid to a labware position using the Fixed-8 pod. Tips must already be loaded and contain liquid. When `emptyTips` is `true`, the step dispenses all liquid currently in the tips (volume is calculated from tip contents) rather than using the `amount` parameter. The technique is specified via `prototype` (a stored technique name) or `customPrototype` (an inline technique object). **Like all Fixed-8 pipetting steps, this step never auto-selects a technique — a permanent design choice.** Setting `autoSelectPrototype: true` raises an enqueue error. The labware must be a TiterPlate, TubeRack, or Reservoir with `CanPipette = true`.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	expression-capable string	—	Yes	—	Name of the Fixed-8 pod (e.g., <code>"Pod1"</code>).
<code>where</code>	expression-capable string	—	Yes	—	Deck position name or labware instance name where the destination labware is located.
<code>amount</code>	expression-capable string	µL	Conditional	—	Volume to dispense. Required when <code>emptyTips</code> is <code>false</code> . Must be non-negative.
<code>emptyTips</code>	boolean	—	No	<code>false</code>	If <code>true</code> , dispense all liquid in tips (ignores <code>amount</code>).
<code>firstWell</code>	integer	—	Conditional	—	1-based well index to place the backmost tip over when dispensing. Used when <code>useWellExpression</code> is <code>false</code> .
<code>useWellExpression</code>	boolean	—	No	<code>false</code>	If <code>true</code> , use <code>firstWellExpression</code> instead of <code>firstWell</code> .
<code>firstWellExpression</code>	expression-capable string	—	Conditional	—	Expression for first well. Used when <code>useWellExpression</code> is <code>true</code> .
<code>liquidType</code>	expression-capable string	—	Yes	—	Liquid type for technique selection. Must not be <code>"Well Contents"</code> for dispense. Accepts <code>"Tip Contents"</code> or any named liquid type.
<code>prototype</code>	expression-capable string	—	Conditional	—	Technique name. Required when <code>customPrototype</code> is not present.
<code>customPrototype</code>	object	—	Conditional	Not present	Inline technique object (<code>_biomekType: "technique"</code> payload). When present, overrides <code>prototype</code> .
<code>autoSelectPrototype</code>	boolean	—	No	<code>false</code>	Must be <code>false</code> — Fixed-8 Dispense does not support auto-selection by design (permanent). Setting <code>true</code> raises an enqueue error.

Key	Type	Units	Required	Default	Description
overrideHeight	boolean	—	No	false	If true, use custom height and heightFrom values.
height	expression-capable string	mm	Conditional	—	Custom pipetting height. Used when overrideHeight is true.
heightFrom	expression-capable string or integer	—	Conditional	—	Reference point for height. Accepts case-insensitive "Liquid" / "Bottom" / "Top" or integer 0 / 1 / 2. Used when overrideHeight is true.
what	string	—	No	""	Expected labware class name for the labware compatibility check (e.g., "BCFlat96"). When non-empty and the actual labware class at where differs, the compatibility check raises an enqueue error. Empty string skips the check. This field uses the short labware class name (e.g. "BCFlat96"); the LabwareClasses\\ path prefix is used only in Instrument Setup's deckItems.
mixCount	expression-capable string	—	No	—	Ignored for Dispense — Not applicable to this operation.
podType	string	—	No	"Fixed8"	Pod type identifier. Every editor-produced step carries "Fixed8"; do not change.
operation	string	—	Yes	—	Must be present and equal "Dispense" . Every editor-produced step carries it, but when authoring from scratch, if the value doesn't match the step's operation (empty string when the key is omitted), enqueue raises an error. Include it when authoring from scratch. See rule 7.
customHeight	boolean	—	No	false	Encoding discriminator for height / heightFrom. When false, those keys carry numeric/enumerated values; when true, they carry expression text. Every editor-produced step carries it. Set it to match the form of height you author.

Universal base keys (caption, dynamic?, stepUI, ...) apply to this step but are omitted here — see Base Keys & Serialization Conventions.

Well numbering. Wells are 1-based and **row-major** — on a 96-well plate A1 = 1, A12 = 12, B1 = 13, ..., H12 = 96. The Fixed-8 pod places its lowest-numbered active probe at firstWell and strides **down the column by +wellsX**, so firstWell = 1 on a 96-well plate hits wells 1, 13, 25, 37, 49, 61, 73, 85 — all of column 1. To sweep plate columns, use firstWellExpression = "=Col", **not** "=1+(Col-1)*8": multiplying by the probe count assumes column-major numbering and overflows the plate. See Probe & Mandrel Selection.

Liquid-relative heights need a locatable liquid surface. A heightFrom of "Liquid" (or a follow-liquid technique) needs the run-time liquid level of the labware at where. Fixed-8 and Multichannel (96-/384-channel) pods do **not** have liquid-level sensing (LLS is Span-8 only), so an "Unknown" well cannot be sensed at run time on these pods — that labware's volumeType (seeded in Instrument Setup / Guided Setup) must be "Nominal" or "Known" **with an evalAmounts fill**, or the move throws at run time. See Pipetting Techniques and Templates §"Liquid types" / §"Liquid-relative heights need a locatable liquid surface".

Enumerated / Constrained Values

liquidType Values

Value	Meaning
"Tip Contents"	Auto-detect liquid type from tip contents for technique selection

Value	Meaning
Any named liquid (e.g., "Water", "Serum")	Use the named liquid for technique selection

"Well Contents" is NOT valid for dispense — only for aspirate.

heightFrom Values

Accepted both as integers and case-insensitive strings:

Integer	String alias	Meaning
0	"Liquid"	Height measured from the liquid surface
1	"Bottom"	Height measured from the well bottom
2	"Top"	Height measured from the well top

Cross-Field Validation Rules

- 1. When `emptyTips` is `false`, `amount` **must** be provided and non-negative — otherwise enqueue raises a negative-volume error.
- 2. When `emptyTips` is `true`, `amount` is ignored — volume is calculated from tip contents.
- 3. When `useWellExpression` is `true`, `firstWellExpression` is used; otherwise `firstWell` is required.
- 4. `autoSelectPrototype` **must** be `false` — otherwise enqueue raises an auto-select-not-supported error. Provide `customPrototype` or a `prototype` naming a valid technique.
- 5. When `overrideHeight` is `true`, `height` and `heightFrom` must be provided.
- 6. `liquidType` **must not** be "Well Contents" for dispense.
- 7. `operation` **must** be present and equal "Dispense" — otherwise enqueue raises an operation-mismatch error (when the key is omitted, the value used is the empty string).
- 8. Every plunger move the step makes **must** keep the pod inside its pipetting range — a separate pod setting, 0–1050 µL in the factory configuration. The check runs at enqueue against the D position the move *ends* at, meaning the pod's current position plus the calibrated volume, so whatever the tips already hold counts against it. A breach raises `The D axis position of <n> µL is outside the supported pipetting range of the pod.` **No export carries that maximum**, and `settings.max.d` is not it: on a Fixed-8 the plunger and gripper share one motor, so `max.d` is the far end of the combined travel. Tip capacity is normally lower still and is what a method meets first, so this surfaces mainly with a computed volume or a large-capacity tip. See Pod Settings Format (in *Biomek Instrument & Project Reference*) §"What `max.d` means, and when it is not the pipetting ceiling".

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Basic dispense to well A1 with a named technique:

```
{
  "stepType": "Fixed-8 Dispense",
  "parameters": {
    "pod": "Pod1",
    "where": "P4",
    "operation": "Dispense",
    "amount": "100",
    "firstWell": 1,
    "liquidType": "Tip Contents",
    "autoSelectPrototype": false,
    "prototype": "F8 Medium"
  }
}
```

Empty tips (dispense all remaining liquid):

```
{
  "stepType": "Fixed-8 Dispense",
  "parameters": {
    "pod": "Pod1",
    "where": "P4",
    "operation": "Dispense",
    "firstWell": 1,
    "emptyTips": true,
    "liquidType": "Tip Contents",
    "autoSelectPrototype": false,
    "prototype": "F8 Medium"
  }
}
```

Common Mistakes

- **Setting** `autoSelectPrototype: true` — Fixed-8 steps reject auto-selection at enqueue. Pair `autoSelectPrototype: false` with an explicit `prototype` (or `customPrototype`).
- **Omitting** `operation` — every editor-produced step carries it, but when authoring from scratch, omission is read as the empty string and enqueue raises an error. Include `"operation": "Dispense"`.
- **Using** `"Well Contents"` as `liquidType` **for dispense** — only `"Tip Contents"` or a named liquid is valid here.
- **Silent** `amount` **override with** `emptyTips: true` — when `emptyTips` is `true`, `amount` is ignored (volume comes from tip contents); a stale value in place raises no error.
- **Reading** `settings.max.d` as the **Fixed-8 pipetting ceiling** — it is the far end of the plunger/gripper travel the two share, not a pipetting limit. The pipetting maximum is a separate pod setting that no export carries; see rule 8.

¹ Table of contents

Fixed-8 Mix

Property	Value
stepType	"Fixed-8 Mix"
Category	Leaf
Terminator	N/A
Compatible Hardware	i3 (Fixed-8 pod required)

Base keys + casing: see Base Keys & Serialization Conventions.

Behavior Summary

The Fixed-8 Mix step mixes liquid **in place** at a labware position using the Fixed-8 pod (8 channels in a column that move together). It runs the selected technique's **Mix** operation `mixCount` times, aspirating and dispensing `amount` μ L at each mix cycle. Tips must already be loaded before this step executes. Fixed-8 Mix shares its implementation with Fixed-8 Aspirate/Dispense. **Like all Fixed-8 pipetting steps, this step never auto-selects a technique.** Setting `autoSelectPrototype: true` raises an enqueue error. The labware must be a TiterPlate, TubeRack, or Reservoir with `CanPipette = true`.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>pod</code>	expression-capable string	—	Yes	—	Name of the Fixed-8 pod (e.g. <code>"Pod1"</code>).
<code>where</code>	expression-capable string	—	Yes	—	Deck position or labware instance name to mix at.
<code>amount</code>	expression-capable string	μ L	Yes	—	Mix volume per cycle. Must be non-negative and less than or equal to the volume in the well.
<code>mixCount</code>	expression-capable string	—	Yes	"1"	(Mix-only functional key) Number of mix cycles. Evaluated to an integer.
<code>liquidType</code>	expression-capable string	—	Yes	—	Liquid type for technique selection. For Mix: <code>"Well Contents"</code> or a named liquid. "Tip Contents" is NOT valid for Mix (see below).
<code>firstWell</code>	integer	—	Conditional	—	1-based well index where the first (backmost) loaded mandrel mixes. Used when <code>useWellExpression</code> is <code>false</code> .
<code>useWellExpression</code>	boolean	—	No	<code>false</code>	If <code>true</code> , use <code>firstWellExpression</code> instead of <code>firstWell</code> .
<code>firstWellExpression</code>	expression-capable string	—	Conditional	—	Well expression (integer or letter+number like <code>"B7"</code>). Used when <code>useWellExpression</code> is <code>true</code> .
<code>prototype</code>	expression-capable string	—	Conditional	—	Technique name. Required when <code>customPrototype</code> is not present.
<code>customPrototype</code>	object	—	Conditional	(absent)	Inline technique object (<code>_biomekType: "technique"</code>). When present, overrides <code>prototype</code> .
<code>autoSelectPrototype</code>	boolean	—	No	<code>false</code>	Must be false — Fixed-8 Mix does not support auto-selection. <code>true</code> raises an enqueue error.
<code>overrideHeight</code>	boolean	—	No	<code>false</code>	If <code>true</code> , use custom <code>height</code> / <code>heightFrom</code> ; otherwise inherit from the technique's Mix settings.

Key	Type	Units	Required	Default	Description
height	expression-capable string	mm	Conditional	—	Custom pipetting height (used when <code>overrideHeight</code> is <code>true</code>).
heightFrom	expression-capable string or integer	—	Conditional	—	Height reference: case-insensitive <code>"Liquid"</code> / <code>"Bottom"</code> / <code>"Top"</code> or integer <code>0</code> / <code>1</code> / <code>2</code> .
what	string	—	No	""	Expected labware class name for the compatibility check.
emptyTips	boolean	—	No	false	Inactive — not read at runtime for Mix. Preserve for round-trip; do not rely on it.
podType	string	—	No	"Fixed8"	Every editor-produced step carries <code>"Fixed8"</code> . Do not change.
operation	string	—	Yes	—	Must be present and equal <code>"Mix"</code>. Include it when authoring from scratch.
customHeight	boolean	—	No	false	Encoding discriminator for <code>height</code> / <code>heightFrom</code> . When <code>false</code> , those keys carry numeric/enumerated values; when <code>true</code> , they carry expression text. Every editor-produced step carries it. Set it to match the form of <code>height</code> you author.

Universal base keys (`caption` , `disabled` , ...) — see [Base Keys & Serialization Conventions](#). Technique / height keys — see [Pipetting Techniques, Templates, and Auto-Selection](#).

Well numbering. Wells are 1-based and **row-major** — on a 96-well plate `A1` = 1, `A12` = 12, `B1` = 13, ..., `H12` = 96. The Fixed-8 pod places its lowest-numbered active probe at `firstWell` and strides **down the column by `+WellsX`**, so `firstWell` = 1 on a 96-well plate mixes wells 1, 13, 25, 37, 49, 61, 73, 85 — all of column 1. To sweep plate columns, use `firstWellExpression` = `"=Col"`, **not** `"=1+(Col-1)*8"`: multiplying by the probe count assumes column-major numbering and overflows the plate. See [Probe & Mandrel Selection](#).

Liquid-relative heights need a locatable liquid surface. A `heightFrom` of `"Liquid"` (or a follow-liquid technique, including the technique's own Mix height) needs the run-time liquid level of the labware at `where`. Fixed-8 and Multichannel (96-/384-channel) pods do **not** have liquid-level sensing, so an `"Unknown"` well cannot be sensed at run time on these pods — that labware's `volumeType` (seeded in Instrument Setup / Guided Setup) must be `"Nominal"` or `"Known"` **with an `evalAmounts` fill**, or the mix throws at run time. See [Pipetting Techniques and Templates §"Liquid types"](#) / [§"Liquid-relative heights need a locatable liquid surface"](#).

Enumerated / Constrained Values

liquidType Values (Mix)

Value	Meaning
<code>"Well Contents"</code>	Resolve the liquid type from the well(s) being mixed.
Any named liquid (e.g., <code>"Water"</code> , <code>"Serum"</code>)	Use the named liquid for technique selection.

`"Tip Contents"` **is NOT valid for Mix.** Using `"Tip Contents"` raises a dispense-only-liquid-type error. Mix's `liquidType` constraint matches Aspirate, not Dispense.

heightFrom Values

Accepted as integers or case-insensitive strings:

Integer	String alias	Meaning
0	"Liquid"	Height measured from the liquid surface
1	"Bottom"	Height measured from the well bottom
2	"Top"	Height measured from the well top

Cross-Field Validation Rules

1. **Operation must be "Mix"**. Enqueue compares the dictionary `operation` to the step's operation; mismatch raises an operation-mismatch error.
2. `amount` **must** be non-negative — otherwise enqueue raises a negative-volume error.
3. `mixCount` **is required and evaluated to an integer**.
4. **First well** — when `useWellExpression` is `true`, `firstWellExpression` is used and must resolve to a valid well; otherwise `firstWell` must be in `1..MaxWellNumber`.
5. **Auto-select is rejected**. `autoSelectPrototype: true` throws an auto-select-not-supported error. Provide `customPrototype` or a `prototype` naming a valid stored technique.
6. **Height override** — when `overrideHeight` is `true`, `height` and `heightFrom` are read; an unrecognized `heightFrom` raises an invalid-height-reference error.
7. **liquidType must not be "Tip Contents"** for Mix; it must resolve to an existing liquid type, else a no-liquid-type or ambiguous-liquid-type error is raised.
8. **Labware must support pipetting**. The labware at `where` must be a TiterPlate, TubeRack, or Reservoir with `CanPipette = true`.
9. **Tips must be loaded**. With no tips on the pod, an error is raised; mixed tip types raise a separate error.
10. The per-cycle `amount` must not exceed the well's tracked contents where the step runs, not the labware class's capacity — a mix cycle is an ordinary aspirate and takes the same below-empty check, raising `Cannot pipette <v> uL; the well only has <x> uL in it`. `mixCount` does not multiply this budget, and no conditioning or pre-wet overhead is added to a mix cycle. See Well Volume Tracking.
11. Tips must be empty before the mix begins, or the step raises `The mix operation cannot begin until tips are empty`. Air gaps do not trip the check; anything classified as liquid does.

Structural Context

Leaf step — no `subSteps`. Tips are not managed by this step; place it between the appropriate Fixed-8 Load Tips / Unload Tips steps.

Canonical Example

Mix 50 μ L, 3 cycles, at well A1 using Well Contents as the liquid type with a named technique:

```
{
  "stepType": "Fixed-8 Mix",
  "parameters": {
    "pod": "Pod1",
    "where": "P3",
    "operation": "Mix",
    "amount": "50",
    "mixCount": "3",
    "firstWell": 1,
    "liquidType": "Well Contents",
    "autoSelectPrototype": false,
    "prototype": "F8 Medium",
    "overrideHeight": false
  }
}
```

Editor-faithful form (includes the inert/non-functional keys the editor writes on every save):

```
{
  "stepType": "Fixed-8 Mix",
  "parameters": {
    "podType": "Fixed8",
```



```

"operation": "Mix",
"dynamic?": true,
"pod": "Pod1",
"where": "P5",
"what": "BCFlat96",
"amount": "50",
"mixCount": "5",
"liquidType": "Well Contents",
"useWellExpression": false,
"firstWell": 9,
"firstWellExpression": "",
"autoSelectPrototype": false,
"prototype": "F8 Medium",
"overrideHeight": true,
"height": 2.0,
"heightFrom": 1,
"customHeight": false,
"emptyTips": false
}
}

```

Common Mistakes

- **Setting** `autoSelectPrototype: true` — rejected at enqueue for all Fixed-8 steps. Pair `autoSelectPrototype: false` with an explicit prototype (or `customPrototype`).
- **Using** `"Tip Contents"` as `liquidType` — Mix matches Aspirate here, not Dispense; enqueue raises an error. Use `"Well Contents"` or a named liquid.
- **Expecting** `emptyTips` to do anything for Mix — it is Dispense-only; the editor writes `false` on every Mix step and execution ignores it.
- **Omitting** `mixCount` when authoring from scratch — every editor-produced step carries it (default `1`), but a hand-built dictionary without the key raises a mix-count-not-specified error at enqueue.
- **Reading** `settings.max.d` as the Fixed-8 pipetting ceiling — it is the far end of the plunger/gripper travel the two share, not a pipetting limit. The pipetting maximum is a separate pod setting that no export carries; see rule 10.

Porting: There Is No Span-8 Mix Step

When converting a Fixed-8 method to Span-8, there is **no** `"Span-8 Mix"` step type.

Preferred path — the technique. Mixing in place on Span-8 is configured on the pipetting technique, not with a step key. Select a technique that has is configured to mix before aspirate or mix after dispense with `prototype` on the Span-8 step that does the pipetting. See **Mixing in place** in Transfer. An item-level `mixCount` on a `Transfer` item does **not** produce a mix — it is inert on every pod family.

Explicit-loop path. If you instead expand the mix into `Span-8 Aspirate` + `Span-8 Dispense` inside a `Loop` around the same well, note that neither Span-8 Aspirate nor Span-8 Dispense reads a `mixCount` key — the mix count of a Fixed-8 Mix step has no counterpart on those two steps and simply doesn't apply. When porting, therefore:

- drop `mixCount` and the `operation: "Mix"` value from the Fixed-8 step — the loop's iteration count replaces `mixCount` in this expansion, and `operation` on the Span-8 Aspirate/Dispense steps is `"Aspirate"` / `"Dispense"`, not `"Mix"`.
- set `liquidType: "Well Contents"` on the aspirate and `"Tip Contents"` on the dispense;
- **change every** `prototype` from `F8 ...` to an `s8 ...` technique (see the F8/S8/MC technique-family rule in `Pipetting-Techniques-and-Templates.md` — a Fixed-8 technique on a Span-8 step will render the sub-steps incorrectly in the editor even when the JSON is otherwise valid).

Multichannel has its own `Multichannel Mix` step (`24-Multichannel-Mix.md`); this porting note applies to Span-8 conversions only.

Fixed-8 Load Tips

Property	Value
stepType	"Fixed-8 Load Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i3 (Fixed-8 pod required)

Behavior Summary

The Fixed-8 Load Tips step picks up disposable tips from a tip box on the deck and attaches them to the Fixed-8 pod mandrels. It supports loading a specific number of tips 1–8, automatically choosing which tips to load, or loading from a specific mandrel position on a specific tip box.

When automatically choosing tips to load, and multiple tip boxes match the specified tips, the step chooses the box with the fewest remaining tips, then the furthest right, then the furthest back. Tip boxes must be 96-density (8 rows × 12 columns).

Unless loading from specific positions is specified, tips are always loaded beginning from mandrel 1. When loading less than 8 tips, the pod will offset to the front of the tip box in order to position mandrel 1 inside the box. Be aware that additional tip boxes may not be placed in front of the tip box being loaded from in this scenario, or a collision error will be raised.

It is possible using the specific positions mode to offset the pod behind the tip box, to load tips on e.g. mandrels 5-8 without tips on 1-4. In this case, additional tip boxes may not be placed behind the tip box being loaded from, or a collision error will be raised.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
tips	expression-capable string	—	Yes	—	Identifies the tip source: deck position name (e.g. "P1"), tip box instance name (e.g. "MyTips" , may apply to multiple boxes), or tip box class name (e.g. "BC230").
mode	string	—	Yes	"NumberOfTips"	Loading mode. See enumeration below.
numberOfTips	expression-capable string	—	Conditional	"8"	Number of tips to load (1–8). Used when <code>mode</code> is "NumberOfTips" .
mandrel	integer	—	Conditional	1	Mandrel number (1–8) that aligns with <code>mandrelPosition</code> . Used when <code>mode</code> is "SpecificTips" . The editor never authors a value other than 1 or 8 . Prefer using 1 or 8 . For example, a single tip can be loaded onto mandrel 1 by positioning mandrel 1 at H1 , or onto mandrel 8 by positioning mandrel 8 at A1 .
mandrelPosition	expression-capable string	—	Conditional	"A1"	Well position in tip box position <code>mandrel</code> over. Used when <code>mode</code> is "SpecificTips" .

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

No `pod` key: Fixed-8 machines have a single Fixed-8 pod which the step resolves implicitly. Unlike Fixed-8 Aspirate/Dispense/Mix, do not add a `pod` key to this step.

Enumerated / Constrained Values

mode Values

Value	Meaning
"NumberOfTips"	Load 1–8 tips (uses <code>numberOfTips</code> parameter)
"SpecificTips"	Load at explicit mandrel/position (uses <code>mandrel</code> and <code>mandrelPosition</code> parameters)

Cross-Field Validation Rules

- 1. `numberOfTips` **must** evaluate to an integer 1–8 — otherwise enqueue raises an invalid-tip-count error.
- 2. `SpecificTips` mode requires that `tips` resolves to exactly one position; if multiple positions match, enqueue raises an error.
- 3. The matched tip box must be 96-density (8 rows × 12 columns).
- 4. The tip box must have enough available tips for the requested count.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Load 8 tips (default mode):

```
{
  "stepType": "Fixed-8 Load Tips",
  "parameters": {
    "tips": "P1",
    "mode": "NumberOfTips",
    "numberOfTips": "8"
  }
}
```

Load a single tip:

```
{
  "stepType": "Fixed-8 Load Tips",
  "parameters": {
    "tips": "P1",
    "mode": "NumberOfTips",
    "numberOfTips": "1"
  }
}
```

Load at specific position in the tip box:

```
{
  "stepType": "Fixed-8 Load Tips",
  "parameters": {
    "tips": "P1",
    "mode": "SpecificTips",
    "mandrel": 1,
    "mandrelPosition": "A3"
  }
}
```

Common Mistakes

- **Tips already loaded on the pod:** this step requires empty mandrels; if any mandrel is currently tipped the step raises a tips-already-loaded error. Pair with an Unload Tips step first when re-loading mid-method.
- **SpecificTips mode with an ambiguous tips source** — the resolver requires exactly one matching position; a class name that matches multiple boxes raises an enqueue error.

- **Assuming any tip-box density works** — must be 96-density (8 × 12); other densities fail the geometry check.

[†] Table of contents

Fixed-8 Unload Tips

Property	Value
stepType	"Fixed-8 Unload Tips"
Category	Leaf
Terminator	N/A
Compatible Hardware	i3 (Fixed-8 pod required)

Behavior Summary

The Fixed-8 Unload Tips step removes tips from the Fixed-8 pod mandrels. It supports returning tips to their original box, discarding to trash, or placing at a specific location. If no tips are currently loaded, the step is a no-op (succeeds silently). The step supports two modes: AutomaticSelection (finds an available position) and SpecificLocations (unloads to an explicit mandrel/position in the tip box).

Parameters Reference Table

Key	Type	Units	Required	Default	Description
tipDestination	expression-capable string	—	No	"<where they came from>"	Where to put the tips. Accepts a deck position name, tip box instance name, tip box class name, or a special token (see below).
mode	string	—	No	"AutomaticSelection"	Unload mode. See enumeration below.
mandrel	integer	—	Conditional	1	Mandrel number (1–8) that aligns with mandrelPosition . Used when mode is "SpecificLocations" . The editor never authors a value other than 1 or 8 . Prefer using 1 or 8 .
mandrelPosition	expression-capable string	—	Conditional	"A1"	Target position in tip box. Used in SpecificLocations mode.
isTipsTrash	boolean	—	No	false	Inert UI state — not read at runtime. Preserve for round-trip fidelity, but do not rely on it to route unloads to trash — use tipDestination ("<Any Trash>" or a trash-position name) instead.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

No `pod` key: Fixed-8 machines have a single Fixed-8 pod which the step resolves implicitly. Unlike Fixed-8 Aspirate/Dispense/Mix, do not add a `pod` key to this step.

Enumerated / Constrained Values

tipDestination Special Tokens

Value	Meaning
"<where they came from>"	Return tips to the position they were loaded from
"<Any Trash>"	Discard tips to any available trash position. Requires a deck that defines one; not every deck does (see rule 5).
"<Use Original Box Settings>"	Check the source tip box's discard settings configured in the Instrument Setup step

Any other string value is resolved against deck position names, tip box instance names, and tip box class names (in that precedence order).

mode Values

Value	Meaning
"AutomaticSelection"	Auto-find suitable location in the tip box
"SpecificLocations"	Unload to explicit mandrel/position. Cannot be used with trash positions.

Cross-Field Validation Rules

- 1. If no tips are loaded on the pod, the step succeeds immediately (no-op).
- 2. `SpecificLocations` mode requires that `tipDestination` resolves to exactly one position; if multiple match, enqueue raises an error.
- 3. The destination must have empty spaces corresponding to the loaded tips, or be a trash position.
- 4. When `tipDestination` is "`<Use Original Box Settings>`", the step reads the source box's discard-tips settings (configured in Instrument Setup) to determine the actual destination.
- 5. When `tipDestination` is "`<Any Trash>`" (or resolves to trash via "`<Use Original Box Settings>`"), the step searches for any deck position carrying the "Can Discard Tips" characteristic.
- 6. When `mode` is `SpecificLocations` , `tips` cannot point to a trash position.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Return tips to their source box:

```
{
  "stepType": "Fixed-8 Unload Tips",
  "parameters": {
    "tipDestination": "<where they came from>"
  }
}
```

Discard tips to trash:

```
{
  "stepType": "Fixed-8 Unload Tips",
  "parameters": {
    "tipDestination": "<Any Trash>"
  }
}
```

Unload to a specific position in the box:

```
{
  "stepType": "Fixed-8 Unload Tips",
  "parameters": {
    "tipDestination": "P1",
    "mode": "SpecificLocations",
    "mandrel": 1,
    "mandrelPosition": "A5"
  }
}
```

Common Mistakes

- **Relying on `isTipsTrash` to enable trash behavior:** `isTipsTrash` is **inactive — not read at runtime**. To discard tips, set `tipDestination` to "`<Any Trash>`" or a trash position (e.g. "`TR1`"), and use `mode` "`AutomaticSelection`" .
- **Trying to "empty" a tip box to make it a trash:** a position is a trash/discard target only when it has the "Can Discard Tips" deck characteristic (e.g. "`<Any Trash>`" / `TR1`). A tip box placed through `deckItems` in Instrument Setup tracks empty slots and accepts returned tips at specific row/column, but is not a trash. Routing unloads to a tip box returns tips there; it does not empty it.
- **Silent no-op when no tips are loaded:** the step succeeds without error even when there is nothing to unload — useful for defensive cleanup, but hides ordering bugs where tips were expected on the pod.
- **Misspelling special tokens:** the tokens are case-insensitive but must match the exact bracketed text (e.g., "`<where they came from>`").

Create Data Set

Property	Value
stepType	"Create Data Set"
Category	Leaf
Terminator	N/A
Compatible Hardware	All (i5, i7, i3) — no hardware restriction

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / `_biomekType` / `comArray` / structural rules: see Method JSON Structure.

Behavior Summary

Create Data Set reads a comma- or tab-delimited **text file** (which must reside on the local controller) or a **SQL Server table** and attaches the resulting values as one or more **data sets** to a piece of destination labware, one value per well. At enqueue time the step resolves the source and destination labware on the deck, verifies neither is a Reservation or Lid, checks that the source has no more wells than the destination, and then loads every row of the source file/table into a temporary SQL Server table (`tempdb` , `127.0.0.1` , integrated security — for the file path) or opens the user-specified database. It then builds a `#Platemap` temp table keyed on plate barcode + well number, joins it to the source rows, and writes each requested column back into a newly created data set on the destination labware. Rows are matched to wells in one of two modes: **by plate barcode / well number** (`useWellNum: true`) or **by sample ID** — a join between a chosen file/table column and an existing source data set (`useWellNum: false`). Each destination data set can be flagged independently as *reportable* (included in reports) and *copy-on-transfer* (tracked during pipetting). Use this step to bring externally-generated per-well data (sample IDs, concentrations, flags, etc.) into a method so later steps (Transfer/Combine well-pattern-from-data-set, Data Set Processing/Reporting, View Data Sets) can consume it.

Requires SQL Server. Even the "read from file" path opens a local SQL Server connection (`Provider=SQLOLEDB;Data Source=127.0.0.1;Initial Catalog=tempdb;Integrated Security=SSPI`) and uses `BULK INSERT` to stage the file. The manual notes database tables are imported only from SQL Server 2014, and that source string columns must be `VARCHAR` , not `NCHAR` (see Biomek reference manual pages 23-8 and 23-10).

Parameters Reference Table

Step-Level Keys

All keys below are read at enqueue unless marked **inactive — not read at runtime**. Casing shown is the serialized (camelCase) form; see Base Keys & Serialization Conventions §1. Universal base keys (`caption` , `disabled` , `dynamic?` , ...) are **not** listed — see Base Keys & Serialization Conventions. The editor writes all 25 config keys plus the 3 `panelState*` keys on every save.

Key	Type	Units	Required	Default	Description
<code>sourceFile</code>	string (expression-capable)	—	Conditional	<code>""</code>	Full path to the delimited text file to read. Required when <code>useDatabase</code> is <code>false</code> . Must exist on the local controller. Evaluated as an expression.
<code>sourcePos</code>	string (expression-capable)	—	Yes	<code>""</code>	Deck position of the source labware whose barcode/well or sample-ID data set is matched against the file/table. Evaluated as an expression; must resolve to a bound deck position.
<code>sourceDepth</code>	string (expression-capable)	stack index	No	<code>"0"</code>	Stack depth of the source labware (0 = top piece). Read as a string and evaluated as an integer expression. Editor default is

Key	Type	Units	Required	Default	Description
	capable integer)				"0" ; at enqueue an absent key falls back to 0 .
sourceDataSetName	string (expression-capable)	—	Conditional	""	Name of the existing source data set that holds sample IDs. Used only when useWellNum is false (Match Sample ID mode). Evaluated as an expression.
joinOnFieldName	string (expression-capable)	—	Conditional	""	Column name in the file/table to join against sourceDataSetName when matching by sample ID (useWellNum: false).
tableWellColName	string (expression-capable)	—	Conditional	""	Column name in the file/table containing the well number/ID . Used when useWellNum is true . Also used to auto-detect alpha vs. numeric well IDs.
tablePlateIDColName	string (expression-capable)	—	No	""	Column name in the file/table containing the plate barcode . Optional even in well-number mode: if empty, all rows are assumed to apply to the plate (manual 23-14).
destPos	string (expression-capable)	—	Yes	""	Deck position of the destination labware on which the data sets are created. Evaluated as an expression; must resolve to a bound deck position.
destDepth	string (expression-capable integer)	stack index	No	"0"	Stack depth of the destination labware (0 = top). Same handling as sourceDepth .
destDataSetNames	string (comma-delimited list)	—	Yes	""	Comma-separated names of the data sets to create on the destination labware. Must yield ≥1 name. A name of Volume is skipped (the Volume data set is reserved/auto).
destDataSetSources	string (comma-delimited list)	—	Yes	""	Comma-separated file/table column names , positionally paired with destDataSetNames : the <i>n</i> -th column supplies the values for the <i>n</i> -th created data set.
destDataSetReportable	comArray (boolean[]) or boolean	—	No	false	Per-data-set "Include in Reports" flags, indexed to destDataSetNames order. When a boolean array, sets each new data set's Reportable . A scalar false means "not an array" → no per-set override applied.
destDataSetCopyOnTransfer	comArray (boolean[]) or boolean	—	No	false	Per-data-set "Track Data While Pipetting" flags, indexed to destDataSetNames order. When a boolean array, sets each data set's CopyOnTransfer . Scalar false = no override.
firstRowHeader	boolean	—	No	false	true = the first row read is a header row supplying column names; false = columns are auto-named Column1 , Column2 , ...
readStartNum	integer	1-based row	No	1 (Enqueue) / 0 (editor initial)	Row number at which to begin reading the file. Enqueue default is 1 ; the editor's spin control initializes to 0 . Combined with firstRowHeader to compute the first row.
useWellNum	boolean	—	No	false	Matching mode: true = match by plate barcode + well number (uses tablePlateIDColName / tableWellColName); false = match by sample ID (uses joinOnFieldName / sourceDataSetName).
commaDelimited	boolean	—	No	true	File field delimiter: true = comma-delimited, false = tab-delimited. Ignored when useDatabase is true . Default is true .

Key	Type	Units	Required	Default	Description
useDatabase	boolean	—	No	false	Source mode: true = read from a SQL Server table; false = read from sourceFile .
databaseAddress	string (expression-capable)	—	Conditional	""	SQL Server data source / server address. Used when connectionString is empty.
databaseName	string (expression-capable)	—	Conditional	""	SQL Server initial catalog (database name). Used when connectionString is empty.
tableName	string (expression-capable)	—	Conditional	""	Database table name to read (Match column data is VARCHAR). In file mode this key is overwritten internally to #tmpTb1 ; the authored value only matters for useDatabase: true .
userName	string	—	No	"sa"	SQL login user name. Default is the literal "sa" . Ignored when ntAuth is true or a connectionString is supplied.
password	string	—	No	""	SQL login password. Ignored when ntAuth is true or a connectionString is supplied. Stored in cleartext in the method.
ntAuth	boolean	—	No	false	true = connect with Windows Integrated Security (SSPI) instead of userName / password . (Serialized casing is ntAuth ; source constant is NTAUTH .)
connectionString	string	—	No	""	Raw ADO connection string. When non-empty it overrides databaseAddress / databaseName / userName / password / ntAuth . Authoring note: the editor strips a leading @ when persisting this value, so the stored/JSON value is the connection string without the @ prefix the editor uses as its "raw string" marker.
panelStateFile	boolean	—	No	false	Inactive — not read at runtime. Collapsed state of the <i>File Options</i> panel in the editor.
panelStateInput	boolean	—	No	true	Inactive — not read at runtime. Collapsed state of the <i>Input Options</i> panel (default collapsed).
panelStateOutput	boolean	—	No	true	Inactive — not read at runtime. Collapsed state of the <i>Output Options</i> panel (default collapsed).

alphaPosID **is not a dictionary key**. It is computed at enqueue from the data (alpha well IDs like A1 vs. numeric), never read from or written to parameters . Do not author it.

destDataSets **is never persisted** to the method JSON. Do not author it.

Enumerated / Constrained Values

Source mode (useDatabase)

Value	Meaning
false	Read from the delimited text file at sourceFile (staged via BULK INSERT into tempdb).
true	Read from the SQL Server table tableName ON databaseAddress / databaseName .

Matching mode (useWellNum)

Value	Meaning
true	Match by plate barcode + well number . Uses tablePlateIDColName (optional) and tableWellColName .

Value	Meaning
false	Match by sample ID : join file/table column <code>joinOnFieldName</code> to existing source data set <code>sourceDataSetName</code> .

Delimiter (`commaDelimited`)

Value	Meaning
true	Comma-delimited file (<code>FIELDTERMINATOR</code> = <code>'</code> , <code>'</code>). Default.
false	Tab-delimited file (<code>FIELDTERMINATOR</code> = <code>'\t'</code>).

Authentication (`ntAuth`)

Value	Meaning
true	Windows Integrated Security (<code>Integrated Security=SSPI</code>); <code>userName</code> / <code>password</code> ignored.
false	SQL login using <code>userName</code> (default <code>"sa"</code>) and <code>password</code> .

Cross-Field Validation Rules

1. **Source labware must exist.** `sourcePos` is expression-evaluated and must be a bound deck position, else the step fails at enqueue.
2. **Source labware type restriction.** The source may not be a Reservation or Lid.
3. **Destination labware must exist.** `destPos` must resolve to a bound deck position, else the step fails at enqueue.
4. **Destination labware type restriction.** The destination may not be a Reservation or Lid.
5. **Source no larger than destination.** Compared via the Volume data set sizes; the step fails if the source has more wells than the destination.
6. **At least one destination data set.** `destDataSetNames` must parse to ≥ 1 name. A name of `volume` is silently skipped (reserved).
7. **File must exist (file mode).** When `useDatabase` is `false`, the step fails if the file is missing.
8. **File must have columns (file mode).** After header parsing, the step fails if no columns are found.
9. **Well column must exist in file (file + well-number mode).** The step fails if `tableWellColName` is not among the header columns.
10. **Table must be non-empty (database mode).** The step fails if the referenced table has no rows. Skipped when `connectionString` is non-empty (the raw-connection-string path exits before this check).
11. **Well column must exist in table (database + well-number mode).** The step fails if `tableWellColName` is not a column of the table. Also skipped when `connectionString` is non-empty.
12. **Stack depth must resolve to labware.** The step fails if no labware exists at the referenced deck position, or at the requested stack depth.
13. **Source-mode required keys.** `useDatabase: false` requires `sourceFile` (and honors `commaDelimited` / `firstRowHeader` / `readStartNum`). `useDatabase: true` requires `tableName` **plus** either a non-empty `connectionString` **or** `databaseAddress` + `databaseName` with credentials (`ntAuth: true`, or `userName` / `password`).
14. **Matching-mode required keys.** `useWellNum: true` requires `tableWellColName` (and optionally `tablePlateIDColName`); `useWellNum: false` requires `joinOnFieldName` **and** `sourceDataSetName`.
15. **Positional pairing.** Keep `destDataSetNames`, `destDataSetSources`, and (when arrays) `destDataSetReportable` / `destDataSetCopyOnTransfer` at equal length and order — the n -th entries are consumed together.

Positional pairing (not enqueue-guarded). `destDataSetSources` (columns) and `destDataSetNames` (data-set names) are consumed by parallel index; a length mismatch is not caught by an explicit validation. If `destDataSetSources` has **fewer** entries than `destDataSetNames`, the step raises a list-index error at run time; if it has **more**, the extra column names are silently ignored. An order mismatch (equal lengths but wrong pairing) silently misassigns columns. `destDataSetReportable` / `destDataSetCopyOnTransfer`, when arrays, are also indexed positionally against `destDataSetNames`. Keep all four lists the same length and order.

Structural Context

Leaf step. Do not include `subSteps` (omit, or an empty array).

Canonical Example

File-based, match-by-sample-ID (creates one `SampleID` data set from a comma-delimited file):

```
{
  "stepType": "Create Data Set",
  "parameters": {
    "commaDelimited": true,
    "connectionString": "",
    "databaseAddress": "",
    "databaseName": "",
    "destDataSetCopyOnTransfer": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true]
    },
    "destDataSetNames": "SampleID",
    "destDataSetReportable": {
      "_biomekType": "comArray",
      "arraySubtype": "boolean",
      "values": [true]
    },
    "destDataSetSources": "Sample",
    "destDepth": "0",
    "destPos": "P5",
    "firstRowHeader": true,
    "joinOnFieldName": "Barcode",
    "ntAuth": false,
    "panelStateFile": false,
    "panelStateInput": true,
    "panelStateOutput": true,
    "password": "",
    "readStartNum": 1,
    "sourceDataSetName": "SampleID",
    "sourceDepth": "0",
    "sourceFile": "C:\\\\BiomekData\\\\samples.csv",
    "sourcePos": "P5",
    "tableName": "",
    "tablePlateIDColName": "",
    "tableWellColName": "",
    "useDatabase": false,
    "userName": "sa",
    "useWellNum": false
  }
}
```

Common Mistakes

- **Confusing this step's `sourcePos` / `destPos` with `labwareAt` (used by 45 View Data Sets)** or with 42 Data Set Management's single- `position` key. Create Data Set has TWO deck positions — the source labware (barcode/well data joins against the file/table) and the destination labware (the new data sets are created here). Using `labwareAt` on this step is a no-op; enqueue then fails because the unbound `sourcePos` evaluates to an empty string.
- **Deck positions bound but no labware placed there at run-time:** `sourcePos` and `destPos` are validated at enqueue for a *bound* deck position. If an upstream Instrument Setup step hasn't placed labware at the referenced positions, the run halts with a no-labware-at-position error. Add an Instrument Setup upstream (or use runtime labware placement) so both `sourcePos` and `destPos` are populated when this step runs.
- **`destDataSetNames` and `destDataSetSources` length mismatch:** they are positionally paired — the n -th name is filled from the n -th column-source. A **shorter** `destDataSetSources` raises a list-index error when the step runs; a **longer** one silently ignores the extras. Keep both lists the same length and order.
- **`useDatabase: false` with `commaDelimited: true` but a tab-delimited file:** silently reads incorrect data — set `commaDelimited: false` for tab files.

Data Set Management

Property	Value
stepType	"Data Set Management"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7, i3 (hardware-independent data step)

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / `_biomekType` / structure: see Method JSON Structure.

Behavior Summary

The Data Set Management step manipulates *existing* data sets attached to a single piece of labware — it neither creates nor computes data (that is the job of Create Data Set and Data Set Processing). At enqueue it resolves the labware at `position / depth`, retrieves that labware's `DataSets` collection, and performs one of four operations selected by `operation`: **Copy** a data set to a new name, **Rename** a data set, **Remove** a data set, or **Change** the two flags (`reportable`, `copyOnTransfer`) on a data set. `Copy` and `Rename` require both a `source` and a `destination` name; `Remove` and `Change` use only `source`. Use it to duplicate/rename the `Sample ID` data set before a second Instrument Setup overwrites it, to clear data sets that are no longer needed, or to toggle whether a data set appears in reports / is carried during pipetting. The step runs entirely in the enqueue (method-build) phase; it performs no runtime actions. (Manual: *Configuring the Data Set Management Step*, ch. 23.)

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>position</code>	expression-capable string	—	Yes	(none — checked)	Deck position of the labware holding the data sets (e.g., "P1"). Evaluated as an expression.
<code>depth</code>	expression-capable string / integer	—	No	0	Stack depth at <code>position</code> . 0 = top piece of labware, 1 = second from top, etc. Evaluated as an expression.
<code>operation</code>	string (enum)	—	Yes	(none — checked)	Which management action to perform: "Copy", "Rename", "Remove", or "Change". Matched case-insensitively (see Enumerated Values).
<code>source</code>	expression-capable string	—	Yes	(none — checked)	Name of the data set to act on (copy/rename/remove/change). Evaluated as an expression.
<code>destination</code>	expression-capable string	—	Conditional	""	New data-set name. Required for <code>Copy</code> and <code>Rename</code> ; ignored for <code>Remove</code> and <code>Change</code> . Evaluated as an expression.
<code>reportable</code>	boolean	—	Conditional	true	Change only: sets the data set's <code>reportable</code> flag (include in reports). Ignored for other operations.
<code>copyOnTransfer</code>	boolean	—	Conditional	true	Change only: sets the data set's <code>copyOnTransfer</code> flag (data tracked during pipetting). Not applied to the built-in <code>Volume</code> data set. Ignored for other operations.

The editor writes all seven keys on every save.

Universal base keys (`caption`, `dynamic?`, `disabled`, ...) — see Base Keys & Serialization Conventions. This step has no item array and no editor-only state keys of its own.

Enumerated / Constrained Values

operation

The enqueue matches operation strings case-insensitively; the editor writes **title-case**, which is the form real exports carry. Prefer title-case.

Value	Meaning	Keys used
"Copy"	Duplicate <code>source</code> → new data set <code>destination</code> .	<code>source</code> , <code>destination</code>
"Rename"	Rename <code>source</code> → <code>destination</code> .	<code>source</code> , <code>destination</code>
"Remove"	Delete the <code>source</code> data set.	<code>source</code>
"Change"	Set the <code>reportable</code> / <code>copyOnTransfer</code> flags on <code>source</code> .	<code>source</code> , <code>reportable</code> , <code>copyOnTransfer</code>

Author these in title case (`"Copy"`). Matching is case-insensitive, so `"COPY"` imports too, but title case is the form the editor writes — use it.

Cross-Field Validation Rules

- 1. `position` , `operation` , and `source` must each be bound.
- 2. For `operation` = `Copy` or `Rename` , `destination` must be bound.
- 3. `position` must evaluate to a real deck position.
- 4. Labware must exist at the requested depth.
- 5. `operation` must be one of the four recognized values.
- 6. `Copy` / `Rename` will fail downstream if `destination` collides with an existing data set — each data set must have a unique name.

Structural Context

Leaf step. Do not include `subSteps` (or include an empty array).

Canonical Example

Rename the `Sample ID` data set on labware at P1:

```
{
  "stepType": "Data Set Management",
  "parameters": {
    "position": "P1",
    "depth": "0",
    "operation": "Rename",
    "source": "Sample ID",
    "destination": "Sample ID Run 1",
    "reportable": true,
    "copyOnTransfer": true
  }
}
```

Change the flags on a computed data set (Change operation):

```
{
  "stepType": "Data Set Management",
  "parameters": {
    "position": "P1",
    "depth": "0",
    "operation": "Change",
    "source": "Concentration",
    "destination": "",
    "reportable": true,
    "copyOnTransfer": false
  }
}
```

}
}

1 Table of contents

Data Set Processing

Property	Value
stepType	"Data Set Processing"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7, i3 (hardware-independent data step)

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / `_biomekType` / structure: see Method JSON Structure.

Behavior Summary

The Data Set Processing step applies a **transformation expression** to an existing (source) data set to build a **new** (destination) data set on the same labware. At enqueue it resolves the labware at `position / depth`, adds a data set named `destination`, and evaluates `expression` once per element (well or tube). When `source` is non-empty, the step iterates the source data set: for each element whose value is **not empty**, it binds the source data set's name as an expression variable (equal to that element's value), evaluates `expression`, and writes the result to the matching element of the destination — empty source elements are skipped. When `source` is left empty, the step computes without a source, sizing the destination from the labware geometry (reservoir section count, else `WellsX × WellsY`) and evaluating `expression` for every element. The result is whatever the expression evaluates to — commonly numeric, Boolean (`True / False` in reports), string, or date/time. Finally it sets the destination's `reportable` and `copyOnTransfer` flags. Use it to derive concentrations, dilution factors, pass/fail masks, etc. from measured data. (Manual: *Configuring the Data Set Processing Step*, ch. 23.)

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>position</code>	expression-capable string	—	Yes	<i>(none — checked)</i>	Deck position of the labware where the new data set is created. Evaluated as an expression.
<code>depth</code>	expression-capable string / integer	—	No	<code>0</code>	Stack depth at <code>position</code> (<code>0</code> = top). Evaluated as an expression.
<code>expression</code>	expression string	—	Yes	<i>(none — checked; also must be non-empty)</i>	Transformation applied per element. Must be prefixed with <code>=</code> (VBScript) or <code>==</code> (JScript). May reference the source data set by its name as a variable (e.g., <code>"=Concentration * 1.05"</code>).
<code>source</code>	expression-capable string	—	Yes	<i>(the key must exist)</i>	Name of the existing data set to transform. The key itself must be present — omitting it fails at enqueue. Its value may be empty to compute a fresh data set from labware geometry (no source binding). When non-empty, only non-empty source elements produce destination values.
<code>destination</code>	expression-capable string	—	Yes	<i>(none — checked; must be non-empty)</i>	Name of the new data set to create. Evaluated as an expression.
<code>reportable</code>	boolean	—	No	<code>true</code>	Sets the new data set's <code>reportable</code> flag (include in reports).
<code>copyOnTransfer</code>	boolean	—	No	<code>true</code>	Sets the new data set's <code>copyOnTransfer</code> flag (data tracked during pipetting).

The editor writes all seven keys on every save. All are scalar; this step has no item array and no editor-only state keys of its own.

Universal base keys (`caption` , `dynamic?` , `disabled` , ...) — see Base Keys & Serialization Conventions.

Enumerated / Constrained Values

No enumerated (magic-string) keys. `expression` is free-form VBScript/JScript.

Trap: without the leading `= / ==` the expression is treated as a literal and will not compute. Common VBScript operators: `+` `-` `*` `/` `\` (integer div) `MOD` `>` `<` `=` `<>` `^`. See manual Table 23.2–23.5.

Cross-Field Validation Rules

1. `position` , `expression` , and `destination` must be bound. **Note:** `source` is not part of that upfront bound-key check, but the key must still be present because it is read via a strict dictionary lookup that fails when missing. Its **value** may be the empty string to invoke the no-source branch (see `source` parameter description).
2. `position` must resolve.
3. Labware must exist at depth.
4. `destination` must evaluate to a non-empty string.
5. `expression` must be non-empty.
6. Duplicate destination names are rejected during validation — each data set on a labware must have a unique name.

Structural Context

Leaf step. Do not include `subSteps` .

Canonical Example

Create `Concentration` from an existing `Absorbance` data set:

```
{
  "stepType": "Data Set Processing",
  "parameters": {
    "position": "P2",
    "depth": "0",
    "source": "Absorbance",
    "destination": "Concentration",
    "expression": "=Absorbance * 1.05",
    "reportable": true,
    "copyOnTransfer": true
  }
}
```

Compute a constant data set with no source (fills every well):

```
{
  "stepType": "Data Set Processing",
  "parameters": {
    "position": "P2",
    "depth": "0",
    "source": "",
    "destination": "InitialVolume",
    "expression": "=100",
    "reportable": true,
    "copyOnTransfer": false
  }
}
```

Data Set Reporting

Property	Value
stepType	"Data Set Reporting"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7, i3 (hardware-independent data step)

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / `_biomekType` / structure: see Method JSON Structure.

Behavior Summary

The Data Set Reporting step generates a report of every reportable data set on every piece of labware known to the world — it sweeps labware on the deck, in stacker/carousel stacks, on SILAS devices, and in Cytomat storage, skips tip boxes and any labware class tagged `Do Not Report`, and hands them to the world's `ReportGenerator`. It then writes the report in the format chosen by `type`: a single **Text File**, **Per-Plate HTML Files** or **Per-Plate Text Files** (one file per plate in a directory), an **Excel Spreadsheet** (`.xls`; see caveat), or a **SQL Server** table. For SQL, the step opens the connection built from the UI fields (or a literal `connectionString`), appends a date-time stamp to `tableName`, and writes the table. The `Volume` data set and any data set with reporting enabled (see Data Set Management / Processing `reportable` flag) appear in the output. Insert it after transfer/pipetting operations to snapshot well volumes and computed data sets. Triggered at enqueue. (Manual: *Configuring the Data Set Reporting Step*, ch. 23.)

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>type</code>	string (enum)	—	Yes	(none — empty ⇒ "Invalid Report Type")	Report style. One of "SQL Server", "Text File", "Excel Spreadsheet", "Per-Plate Html Files", "Per-Plate Text Files". Evaluated as an expression.
<code>path</code>	expression-capable string	—	Yes	(none — required for all styles)	Output target. For file styles: full file path (Text/Excel) or directory (Per-Plate HTML/Text). For SQL Server: the Data Source / server (mirrors <code>server</code>). Evaluated as an expression.
<code>tableName</code>	expression-capable string	—	Conditional (SQL)	""	SQL table-name prefix. A <code>mm_dd_yyyy_hh_nn_ss</code> timestamp is appended directly at runtime (no leading <code>_</code>) so each run writes a fresh table. Must contain only letters, digits, and <code>_</code> .
<code>connectionString</code>	expression-capable string	—	Conditional (SQL)	""	ADO connection string for the SQL target. A single leading @ is silently stripped before connecting (see Traps). Normally auto-built by the editor from <code>server / catalog / userName / password / authentic</code> .
<code>catalog</code>	string	—	Conditional (SQL)	""	Database name → passed as <code>Initial Catalog / DB</code> on the SQL connection. Not expression-evaluated (read verbatim).
<code>userName</code>	string	—	No (editor state)	""	SQL login user. Read at enqueue but not used there — the editor folds it into <code>connectionString</code> on save. Persist it, but <code>connectionString</code> is what actually authenticates.
<code>password</code>	string	—	No (editor state)	""	SQL login password. Same status as <code>userName</code> : editor state, folded into <code>connectionString</code> ; not consumed by enqueue directly.

Key	Type	Units	Required	Default	Description
<code>server</code>	string	—	No (editor state)	""	SQL server name. Never read by the step — editor-only; duplicated into <code>path</code> (Data Source) and <code>connectionString</code> . Inactive — not read at runtime.
<code>authent</code>	boolean	—	No (editor state)	<code>false</code>	"Use NT Authentication" (Windows integrated auth). Never read by the step — the editor uses it to build <code>connectionString</code> (Integrated Security=SSPI when <code>true</code> , else <code>User ID / Password</code>). Inactive — not read at runtime.

The editor writes all nine keys on every save. File-style reports (Text/Per-Plate/Excel) ignore every SQL-only key; author them anyway to match what the editor produces.

Universal base keys (`caption`, `dynamic?`, `disabled`, ...) — see Base Keys & Serialization Conventions. No item array.

Enumerated / Constrained Values

type (report style)

Matched case-insensitively against the values listed below.

Value	Meaning
"SQL Server"	One database table (all plates), timestamp appended to the name. Uses <code>connectionString / tableName / catalog</code> .
"Text File"	Single delimited text file listing all plates. <code>path</code> = file path.
"Excel Spreadsheet"	Single <code>.xls</code> file. <code>path</code> = file path; filename must end <code>.xls</code> . Caveat: validation accepts it (directory / filename / <code>.xls</code> extension are checked) but the enqueue path has no Excel writer, so the step silently succeeds without producing any output. Do not author this style; it is also absent from the current manual's Report Style list.
"Per-Plate HTML Files"	Directory of one HTML file per plate. <code>path</code> = directory. (Enqueue constant spells it <code>Per-Plate HTML Files</code> ; the editor combo/manual show <code>Per-Plate HTML Files</code> — matching is case-insensitive so both match.)
"Per-Plate Text Files"	Directory of one text file per plate. <code>path</code> = directory.

Cross-Field Validation Rules

1. Unrecognized `type` (code 1).
2. `Text File`: directory of `path` must exist (code 2); filename must be present and not a directory (code 5).
3. `Excel Spreadsheet`: directory must exist (code 3); filename present with `.xls` extension (code 4).
4. `Per-Plate HTML Files`: `path` must be an existing directory (code 6).
5. `Per-Plate Text Files`: `path` must be an existing directory (code 7).
6. `SQL Server`: the connection must open (code 9); `tableName` must be non-empty (code 10); every `tableName` character must be a letter, digit, or `_` (code 8).
7. Any other configuration fault fails the step.

Traps (author carefully)

- `connectionString` **leading @ is silently stripped**. The `@` is the editor's marker for "Report-Location field holds a literal connection string". The runtime removes it before connecting. A literal connection string that genuinely starts with `@` will lose that character.
- **Windows-auth key is `authent`, not `ntAuth`**. The Create Data Set step uses `NTAuth`; this step persists `Authent`. `ntAuth` here is ignored.
- `connectionString` **wins over the individual SQL fields**. When authored directly, the step only additionally reads `catalog` (→ `Initial Catalog`) and `tableName`.

Structural Context

Leaf step. Do not include `subSteps`.

Canonical Examples

Text-file report (all nine keys present; SQL fields ignored for this style):

```
{
  "stepType": "Data Set Reporting",
  "parameters": {
    "type": "Text File",
    "path": "C:\\BiomekReports\\run1.csv",
    "tableName": "",
    "connectionString": "",
    "catalog": "",
    "userName": "",
    "password": "",
    "server": "",
    "authent": false
  }
}
```

SQL Server report with Windows authentication:

```
{
  "stepType": "Data Set Reporting",
  "parameters": {
    "type": "SQL Server",
    "path": "SQLHOST\\BIOMEK",
    "tableName": "RunResults",
    "connectionString": "Provider=SQLOLEDB.1;Integrated Security=SSPI;Initial Catalog=BiomekDB;Data Source=SQLHOST\\BIOMEK",
    "catalog": "BiomekDB",
    "userName": "",
    "password": "",
    "server": "SQLHOST\\BIOMEK",
    "authent": true
  }
}
```

Common Mistakes

- **Giving a file path where the style expects a directory (or vice-versa).** Per-Plate HTML Files and Per-Plate Text Files write one file per plate into a **directory** — `path` must be an existing directory. Text File and Excel Spreadsheet need a full **file path** (Text/Excel's parent directory must exist and the path must evaluate to a filename). Passing a file to a Per-Plate style, or a directory to Text/Excel, hits the corresponding invalid-path/filename error.
- **Forgetting the .xls extension for Excel.** Excel Spreadsheet requires the filename to end in `.xls`, else the step fails at enqueue.
- **Using NTAAuth / ntAuth casing instead of authent.** Windows-auth is represented as `authent` here (Create Data Set represents it as `NTAuth`). Anything else is silently ignored and the editor won't build Integrated-Security into `connectionString`.
- **Assuming userName / password / server / authent override connectionString.** They don't. At runtime, `connectionString` wins; the individual fields are editor-only state that the editor folds into `connectionString` on save. To change credentials at runtime, edit `connectionString`.
- **Leading @ in connectionString disappears.** The `@` is the editor's "raw string" marker and is silently stripped before connecting. A literal connection string that genuinely starts with `@` will lose that character.
- **Case-mismatch between source-form and editor/manual form for Per-Plate HTML.** The enqueue constant spells it `Per-Plate HTML Files`; the editor combo and manual show `Per-Plate HTML Files`. Matching is case-insensitive so either works; real exports and manual examples may show different casing.
- **Authoring only some of authent / server / userName / password without a connectionString.** Those fields are only wired through the editor; without a corresponding `connectionString` the runtime has nothing to connect with. Either author a full `connectionString`, or copy from an export where the editor built one.

View Data Sets

Property	Value
stepType	"View Data Sets"
Category	Leaf
Terminator	N/A
Compatible Hardware	All (i5, i7, i3) — no hardware restriction

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / `_biomekType` / `comArray` / structural rules: see Method JSON Structure.

Behavior Summary

View Data Sets is a **design-time inspection step**: it lets the method author view the data sets, labware properties, and global variables associated with a chosen piece of labware, and copy the values to the clipboard (tab-delimited or CSV) for pasting into Notepad/Excel. Per the manual it "aids beginning users in understanding data sets and how they are used [and] provides an easy means of checking data set values at any point in the Biomek method" (manual 23-36 to 23-38). Its enqueue and runtime behavior is a **no-op** — inserting the step performs no action and does no runtime validation; all five parameters exist purely to persist the editor's inspection state. It is safe to place anywhere between `Start` and `Finish`.

Parameters Reference Table

Step-Level Keys

All five keys are written and read **only** by the editor. They are therefore **inactive — not read at runtime**, but they *are* part of the persisted step configuration and every editor-produced step carries them, so document/preserve them. Casing shown is the serialized form. Universal base keys (`caption`, `disabled`, ...) are not listed — see Base Keys & Serialization Conventions.

Key	Type	Units	Required	Default	Description
<code>labwareAt</code>	string	—	No	<code>""</code>	Deck position of the labware whose data sets are displayed. Editor-only; no runtime effect.
<code>depth</code>	string (integer)	stack index	No	<code>"0"</code>	0-based stack depth from the top piece of labware at <code>labwareAt</code>. <code>0</code> = the top plate; <code>1</code> = the plate directly under it; <code>2</code> = the plate under that; etc. Stored as a string. Editor-only.
<code>viewAll</code>	boolean	—	No	<code>false</code> (see note)	<code>true</code> = show all data sets on the labware; <code>false</code> = show only the single data set named by <code>singleDatasetName</code> . Editor-only.
<code>singleDatasetName</code>	string	—	No	<code>""</code>	Name of the single data set to display when <code>viewAll</code> is <code>false</code> ; ignored when <code>viewAll</code> is <code>true</code> . Editor-only.
<code>copyCsvFormat</code>	boolean	—	No	<code>true</code>	Clipboard copy format toggle: <code>true</code> = CSV, <code>false</code> = tab-delimited. Editor-only.

`viewAll` **default note.** When the key is omitted, the fallback is `true`, but editor-produced steps carry an explicit `false` paired with a `singleDatasetName`. Prefer that shape to match the editor's output.

Enumerated / Constrained Values

viewAll

Value	Meaning
true	Display all data sets attached to the selected labware ("View All Datasets").
false	Display only the one data set named by <code>singleDatasetName</code> ("View One Dataset").

copyCsvFormat

Value	Meaning
true	Copy to clipboard as CSV (<code>.csv</code>).
false	Copy to clipboard as tab-delimited (for Excel).

Cross-Field Validation Rules

None at enqueue. The step performs no runtime validation and cannot fail a run on its own account.

The editor does surface two **design-time** messages while populating the inspection panel (shown in the editor only, not raised at enqueue/run):

- A "specified dataset not found" message shown when `viewAll` is `false` and the named data set is absent. **Also fires when `depth` exceeds the stack size** — going past the bottom of the stack means no labware is resolved, so the same string is displayed.
- A "no datasets found" message shown when `viewAll` is `true` and no data sets are found.

Structural Context

Leaf step. Do not include `subSteps` (omit, or an empty array).

Canonical Example

Typical editor-produced shape (`viewAll: false` paired with a `singleDatasetName` such as `"Volume"`):

```
{
  "stepType": "View Data Sets",
  "parameters": {
    "copyCsvFormat": true,
    "depth": "0",
    "labwareAt": "P5",
    "singleDatasetName": "Volume",
    "viewAll": false
  }
}
```

"View all data sets" variant (matches the omitted-key fallback):

```
{
  "stepType": "View Data Sets",
  "parameters": {
    "copyCsvFormat": true,
    "depth": "0",
    "labwareAt": "P5",
    "singleDatasetName": "",
    "viewAll": true
  }
}
```

Fly-By Read

Property	Value
stepType	"Fly-By Read"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 (Fly-By Bar Code Reader is an i5/i7 device; not i3)

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Fly-By Read step uses a gripper-equipped pod to pick up a stack of labware from a `source` deck position, "fly" each piece past a stationary Fly-By Bar Code Reader (FBBCR) device to read its bar code into the labware's `Properties.Barcode`, and then set the stack down at a `target` deck position. Common uses are entering bar codes for later decision-making (e.g. a downstream `If` step) and confirmatory reads that the correct labware was selected. Reads are accumulated for later use by the Fly-By Log step.

A Fly-By Bar Code Reader must be installed on the deck to use this step.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>pod</code>	string (expression)	—	Yes	<code>""</code>	Name of the gripper-equipped pod used to move the stack (e.g. <code>"Pod1"</code>). Expression-evaluated at enqueue.
<code>source</code>	string (expression)	—	Yes	<code>""</code>	Deck position from which to pick up the labware stack. Expression-evaluated.
<code>target</code>	string (expression)	—	Yes	<code>""</code>	Deck position at which to place the stack after reading. Expression-evaluated.
<code>device</code>	string (expression)	—	Yes	<code>""</code>	Name of the Fly-By Bar Code Reader device. Expression-evaluated. Must be a Fly-By Bar Code Reader configured on the current deck.
<code>gripSide</code>	string (enum)	—	Yes	<code>""</code>	Gripper orientation used to grip the stack: <code>"A1 near"</code> or <code>"A1 away from"</code> . Matched case-insensitively. See Enumerated Values.
<code>simulatedBarcode</code>	string (expression)	—	Yes	<code>"Simulating"</code> (editor pre-fill; enqueue fallback is <code>"Simulate"</code> when the key is absent)	Bar code value returned during simulation / method validation. Expression-evaluated.
<code>returnLid</code>	boolean	—	Yes	<code>true</code>	If <code>true</code> , the lid removed from a lidded stack is replaced onto the plate after reading. Genuine JSON boolean.
<code>override</code>	string <code>"True" / "False"</code>	—	Yes	<code>"False"</code> (consumed as <code>False</code> at enqueue)	Whether to override the device's default error handling. This is a <code>"True" / "False"</code> string, not a

Key	Type	Units	Required	Default	Description
					JSON boolean — write <code>"True"</code> / <code>"False"</code> , not <code>true</code> / <code>false</code> .
<code>errorHandling</code>	string (enum)	—	Yes	<code>"Ignore"</code>	Error-resolution mode used only when <code>override</code> is <code>"True"</code> : <code>"Ignore"</code> , <code>"Barcode"</code> , or <code>"Prompt"</code> .
<code>retries</code>	string (integer, expression)	count	Yes	<code>"1"</code>	Number of retries when a read error or mismatch occurs. Persisted as a string; evaluated as an integer at enqueue. Used only when <code>override</code> is <code>"True"</code> ; otherwise the device's own <code>Retries</code> (default <code>1</code>) is used.
<code>overrideBarcode</code>	string (expression)	—	Yes	<code>""</code>	Bar code value substituted when <code>errorHandling</code> is <code>"Barcode"</code> . Expression-evaluated.
<code>leaveBottomLabware</code>	boolean	—	Yes	<code>false</code>	If <code>true</code> , the stack is moved but the bottom piece of labware is left at the <code>source</code> position (moves <code>StackDepth - 1</code> pieces). Always emitted by the editor. Mutually exclusive with <code>depth</code> (see Stack Options).
<code>depth</code>	string (integer)	count	Conditional	<i>(absent)</i>	Number of topmost pieces of labware to move from the stack. Present only when the "Move depth" option is chosen. When absent (and <code>leaveBottomLabware</code> is <code>false</code>), the entire stack is moved. Mutually exclusive with <code>leaveBottomLabware</code> .

Universal base keys (`caption`, `node disabled`, etc.) — see Base Keys & Serialization Conventions. `dynamic?` is always written by this step's editor (so the canonical examples below include it for round-trip fidelity). Include `dynamic?` with value `true`.

Enumerated / Constrained Values

gripSide

Value	Meaning
<code>"A1 near"</code>	Grip with well A1 nearest the pod.
<code>"A1 away from"</code>	Grip with well A1 away from the pod.

The exact source constant values are `'A1 near'` and `'A1 away from'`. Matching is case-insensitive, but these are the values the editor writes, so author them verbatim. The editor lists `"A1 away from"` first, so it is the preferred/default choice when both are geometrically valid.

errorHandling (effective only when `override` is `"True"`)

Value	Meaning
<code>"Ignore"</code>	Ignore the error and keep the (possibly failed) bar code; continue.
<code>"Barcode"</code>	Substitute the <code>overrideBarcode</code> value for the read.
<code>"Prompt"</code>	Prompt the user (Retry / Ignore / Abort / enter a bar code); also enables duplicate-bar-code detection between consecutive plates.

The default (used when the string is unrecognized) is `"Ignore"`. `"Barcode"` is a valid option — author it (with `overrideBarcode`) when substitution is intended.

Stack Options (how `leaveBottomLabware` + `depth` select what moves)

Configuration	Pieces moved
<code>leaveBottomLabware: true</code>	<code>StackDepth - 1</code> (bottom piece stays at <code>source</code>).
<code>depth absent, leaveBottomLabware: false</code>	Entire stack (<code>StackDepth</code>).
<code>depth: "N"</code>	Topmost <code>N</code> pieces.

The three options are mutually exclusive.

Cross-Field Validation Rules

1. `device` must be set.
2. `device` must resolve in `PipettorDevices` .
3. `device` must be associated with a deck position — see Deck Layouts Format (in *Biomek Instrument & Project Reference*) §"The `device` block" for how the association is set up outside the method.
4. `gripSide` must be one of the two enum strings.
5. `retries` must be numeric when `override` is `"True"` .
6. `source` must be non-empty.
7. `pod` must resolve.
8. `source` must be a position on the current deck.
9. `target` must be a position on the current deck.
10. `source` must hold labware.
11. `pod` must have a gripper.
12. `device` must be on the current deck.
13. Stack must contain at least one non-lid plate to read.
14. A lidded stack requires a swap position for the removed lid.

Structural Context

This step is a **leaf** and does not support authored `subSteps` .

Canonical Example

Reads the whole stack at P2, overriding error handling to prompt after 3 retries, and places the stack at P3:

```
{
  "stepType": "Fly-By Read",
  "parameters": {
    "pod": "Pod1",
    "source": "P2",
    "target": "P3",
    "device": "FBBCR1",
    "gripSide": "A1 away from",
    "simulatedBarcode": "PLATE001",
    "returnLid": true,
    "leaveBottomLabware": false,
    "override": "True",
    "errorHandling": "Prompt",
    "retries": "3",
    "overrideBarcode": "",
    "dynamic?": true
  }
}
```

Minimal read using the device's default error handling and moving only the top 2 pieces:

```
{
  "stepType": "Fly-By Read",
  "parameters": {
    "pod": "Pod1",
    "source": "P2",
    "target": "P2",
    "device": "FBBCR1",
```

```
"gripSide": "A1 near",
"simulatedBarcode": "TEST",
"returnLid": false,
"leaveBottomLabware": false,
"depth": "2",
"override": "False",
"errorHandling": "Ignore",
"retries": "1",
"overrideBarcode": "",
"dynamic?": true
}
}
```

Common Mistakes

- **Writing `override` as a JSON boolean.** It is a `"True" / "False"` **string** — use `"True" / "False"`, never `true / false`. (`returnLid`, by contrast, is a real JSON boolean.)
- **Writing `retries` or `depth` as JSON numbers.** Both are persisted as strings (`"3"`, `"2"`).
- Setting `errorHandling` / `retries` / `overrideBarcode` while leaving `override: "False"` — those three keys are consumed only when `override` is `"True"`; otherwise the device's own defaults win.

† Table of contents

Fly-By Log

Property	Value
stepType	"Fly-By Log"
Category	Leaf
Terminator	N/A
Compatible Hardware	i5, i7 (companion to the Fly-By Bar Code Reader device; not i3)

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Fly-By Log step writes the accumulated Fly-By Bar Code Reader read results to a comma-separated text log file with a header row: **Time, Plate Name, Initial Barcode, Final Barcode, Recovery Action**. Each logged read records the time, plate name, initial bar code, final bar code, and recovery action — the same fields the Fly-By Read step accumulates onto the device's read list.

Place this step **after** one or more Fly-By Read steps that populated the device's read list.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
filename	string (expression)	—	Yes	""	Full path to the output log file (typically .txt ; the .txt convention comes from the editor's file-picker default and is not enforced at runtime). Must be non-blank; expression-evaluated at enqueue.

Universal base keys (caption , node disabled , dynamic? , ...) — see Base Keys & Serialization Conventions.

This step persists only the key filename . Other FBBCR-related keys (e.g., a log-name key or device-level keys) are not read by this step — do not author them.

Enumerated / Constrained Values

None — filename is a free-form path string (expression-capable).

Cross-Field Validation Rules

- filename must not be blank/empty.

Structural Context

This step is a **leaf** and does not support authored subSteps .

Canonical Example

```
{
  "stepType": "Fly-By Log",
```

```
"parameters": {  
  "filename": "C:\\Users\\Public\\Documents\\BarcodeReads.txt"  
}
```

Common Mistakes

- **Logging with no prior reads** — placing this step before any Fly-By Read produces an empty log; the device's `Reads` list is what gets emitted.
- **Authoring the wrong key name** — this step reads only the literal key `'filename'`; other logger-related keys do nothing here.

[†] Table of contents

Break

Property	Value
stepType	"Break"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Behavior Summary

The Break step terminates execution of one or more enclosing loops. When executed, it exits the specified number of enclosing loop iterations. A level of 1 breaks out of the innermost loop; a level of 2 breaks out of two nested loops, etc. The Break step must be placed inside a Loop (or nested Loops) to have any effect.

`levels` counts enclosing Loop steps only. Only the Loop step decrements the break counter. Non-Loop containers (`If` , `Group` , `Just In Time` , `Let` , ...) do not touch the break counter, so they are **not** counted when tallying `levels` .

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>levels</code>	expression-capable string	—	Yes	"1"	Number of enclosing Loop steps to break out of (see §Behavior Summary — <code>levels</code> counts Loops only, not intermediate containers such as <code>If/Group/Just In Time</code>). Evaluated as an integer. A value of <code>"1"</code> exits the innermost Loop; <code>"2"</code> exits two nested Loops.

Enumerated / Constrained Values

No enumerated constraints. `levels` accepts any expression that evaluates to a positive integer.

Cross-Field Validation Rules

- 1. `levels` **must** be bound in the dictionary; otherwise Enqueue fails.
- 2. `levels` **must** evaluate to a valid positive integer.
- 3. `levels` must not exceed the actual nesting depth of enclosing loops, and the Break must be inside at least one loop.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Break out of the innermost loop (default):

```
{
  "stepType": "Break",
  "parameters": {
    "levels": "1"
  }
}
```

Break out of two nested loops:

```
{
  "stepType": "Break",
  "parameters": {
    "levels": "2"
  }
}
```

Break using an expression:

```
{
  "stepType": "Break",
  "parameters": {
    "levels": "=BreakDepth"
  }
}
```

Common Mistakes

- **Omitting `levels`** : the step's built-in default does *not* apply during JSON import. Always emit the key.
- **`levels` exceeding actual loop nesting depth**: count the *enclosing* Loops, not the containers between them.
- **Counting non-Loop containers as levels**: `If` , `Group` , `Just In Time` , `Let` , etc. do **not** consume a break level. A Break inside `Loop`
→ `If` → `Loop` → `If` → (here) needs `levels: 2` to escape both Loops, not 4.

¹ Table of contents

Comment

Property	Value
stepType	"Comment"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Behavior Summary

The Comment step is a no-op at runtime — it does not generate any instrument commands. It carries `description` and `comment` text shown in the method editor and printed in method reports. Use it to annotate method logic or leave notes for other method authors.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>description</code>	<code>string</code>	—	No	<code>""</code>	Short description shown in the step tree. Every editor-produced step carries this key. Not expression-evaluated. Always author this key.
<code>comment</code>	<code>string</code>	—	No	<code>""</code>	Longer comment text. Editor-written; not read at runtime. Not expression-evaluated.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Inactive Keys — not read at runtime (may appear in existing methods)

If any additional keys appear on a `Comment` step in an exported method, they do not impact the behavior of the Comment step. Preserve them verbatim for round-tripping.

Enumerated / Constrained Values

None.

Cross-Field Validation Rules

None. Both parameters are optional free-text strings.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Simple comment:

```
{
  "stepType": "Comment",
  "parameters": {
    "description": "Transfer reagent to plate",
    "comment": "Using 200µL tips for this transfer to avoid cross-contamination."
  }
}
```

```
}  
}
```

Common Mistakes

- **Expecting runtime behavior:** Comment does nothing at runtime. Do not use it as a substitute for actual logic.
- **Using expressions in description/comment:** Expressions in these fields will not be evaluated; text is stored as-is.

[†] Table of contents

Define Pattern

Property	Value
stepType	"Define Pattern"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Behavior Summary

The Define Pattern step creates a named well-selection pattern that can be referenced by other steps (e.g., pipetting steps to limit operations to specific wells). The pattern is a boolean array where each element corresponds to a well in the labware. The pattern can be defined manually (a fixed array of booleans), read from a CSV file at runtime, or prompted from the user interactively. The pattern is stored globally and persists for the duration of the method run.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
name	expression-capable string	—	Yes	—	Name of the pattern being defined. Must not be empty.
type	expression-capable string	—	Yes	—	Labware class name (defines well geometry). Must match an installed labware class exactly (examples: BCF1at96 , Arrowhead_AbGene_96_PCR , Labcyte_384PPL).
readFromFile	boolean	—	No	false	If true , read pattern from a CSV file.
promptForPattern	boolean	—	No	false	If true , show a UI prompt at runtime.
file	expression-capable string	—	Conditional	—	Path to CSV file. Required when readFromFile is true .
headerRow	boolean	—	Conditional	false	If true , first row of CSV is treated as a header. Used with readFromFile .
matchColumn	integer	—	Conditional	0	0-based column index in CSV for matching. Used with readFromFile .
wellColumn	integer	—	Conditional	0	0-based column index in CSV for well identifiers. Used with readFromFile .
useAllLines	boolean	—	Conditional	false	If true , all rows in CSV contribute to pattern. If false , matching lines only are used. Used with readFromFile .
matchBarcode	boolean	—	Conditional	true	If true , the property and position defined in matchProperty and matchPosition are used to determine which lines in the CSV contribute to the pattern. If false , the expression defined in matchExpression is used to determine which lines contribute to the pattern. Used with readFromFile when useAllLines is false .
matchPosition	expression-capable string	—	Conditional	—	Deck position of the labware whose property will be read for matching. Used with readFromFile when matchBarcode is true and useAllLines is false .

Key	Type	Units	Required	Default	Description
<code>matchProperty</code>	expression-capable string	—	Conditional	"Barcode"	Name of the property to read from the labware at <code>matchPosition</code> . Used with <code>readFromFile</code> when <code>matchBarcode</code> is <code>true</code> and <code>useAllLines</code> is <code>false</code> .
<code>matchExpression</code>	expression-capable string	—	Conditional	—	Value (or <code>==</code> -prefixed case-insensitive regex) compared against each CSV row's match column to determine which rows in the CSV contribute to the pattern. Used with <code>readFromFile</code> when <code>matchBarcode</code> is <code>false</code> .
<code>pattern</code>	comArray of booleans (<code><comArray:boolean:N></code>)	—	Conditional	—	Manual well-selection array. Required when <code>readFromFile</code> is <code>false</code> and <code>promptForPattern</code> is <code>false</code> . Serializes as a typed comArray (<code><comArray:boolean:96></code> or <code><comArray:boolean:384></code>), not a plain JSON boolean array; see Method JSON Structure for comArray shape.
<code>sectionExpression</code>	expression-capable string	—	No	""	Expression-based pattern (advanced); Used only when <code>useExpression</code> is <code>true</code> and <code>readFromFile</code> is <code>false</code> .
<code>useExpression</code>	boolean	—	No	false	If <code>true</code> , evaluates <code>sectionExpression</code> instead of the manual <code>pattern</code> , when <code>readFromFile</code> is <code>false</code> .
<code>selectionInfo</code>	comArray of booleans (<code><comArray:boolean:N></code>)	—	No	—	Parallel copy of the well-selection array, written alongside <code>pattern</code> . Inactive at enqueue — not read at runtime (the enqueue read is <code>pattern</code>), but the editor always emits it. Always author this key alongside <code>pattern</code> for a manual pattern (<code>readFromFile=false</code> and <code>useExpression=false</code>).
<code>wellsX</code>	integer	—	No	1	Wells-per-row of the referenced labware class. Always emitted by the editor. Always author this key for a manual pattern (<code>readFromFile=false</code> and <code>useExpression=false</code>).
<code>wellsAreNumbers</code>	boolean	—	No	—	Obsolete and inactive — not read at runtime. Preserve if present in an existing method; do not author from scratch.

Universal base keys (`caption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

On always-present optional keys. The editor emits `readFromFile`, `promptForPattern`, `useExpression`, `sectionExpression`, `file`, `headerRow`, `matchColumn`, `wellColumn`, `useAllLines`, `matchBarcode`, `matchProperty`, `matchExpression`, `matchPosition`, `selectionInfo`, `wellsX`, and `pattern` on every editor-produced step, but enqueue reads them all tolerantly with defaults — except `pattern`, which is required in the manual branch (see its row).

Enumerated / Constrained Values

None.

Cross-Field Validation Rules

1. `name` **must not** be empty.
2. `type` **must** name a labware class defined in the project.
3. When `readFromFile` is `true`, `file` must name an existing CSV file.
4. When `readFromFile` is `true`, the CSV must have valid `matchColumn` and `wellColumn` indices.
5. Well identifiers in CSV must be valid for the labware type (numeric or alphanumeric A1 format).
6. When `readFromFile` is `false` and `promptForPattern` is `false`, `pattern` must be provided.

7. **CSV matching mode** (only when `readFromFile` is `true` and `useAllLines` is `false`) — `matchBarcode`, `matchPosition`, `matchProperty`, and `matchExpression` together pick which CSV rows apply: - `matchBarcode = true` (default): the step finds the labware at `matchPosition`, reads its `matchProperty` (default "Barcode"), and uses that value as the string each CSV row's match column is compared against. `matchPosition` is effectively required in this branch (an empty `matchPosition` will fail to resolve a labware); `matchExpression` is **not** used. - `matchBarcode = false`: `matchExpression` is used directly as the comparison target. If `matchExpression` starts with `==`, the remainder is treated as a case-insensitive regex; otherwise it is used as a literal string (after expression evaluation). `matchPosition` and `matchProperty` are **not** used in this branch. - When `useAllLines` is `true`, no matching is performed and all rows contribute regardless of `matchBarcode` / `matchPosition` / `matchProperty` / `matchExpression`.
8. `promptForPattern` is **not** a third pattern source — it is orthogonal to `readFromFile`. When `promptForPattern` is `true`, the runtime prompt is a *file* picker if `readFromFile` is `true` and a well-selection grid otherwise, and the prompted value supersedes the authored `file` / `pattern` for that run.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Manual pattern (wells 1–8 selected in a 96-well plate). Always author `wellsX` and `selectionInfo` alongside `pattern`.

[illegible]

Pattern from CSV file:

```
{
  "stepType": "Define Pattern",
  "parameters": {
    "name": "SamplePattern",
    "type": "BCFlat96",
    "readFromFile": true,
    "file": "C:\\Methods\\patterns\\sample_list.csv",
    "headerRow": true,
    "matchColumn": 0,
    "wellColumn": 1,
    "matchBarcode": true,
    "matchPosition": "P1",
    "matchProperty": "Barcode"
  }
}
```

Common Mistakes

- **Wrong pattern array length:** Must match the well count of the labware class (e.g., 96 booleans for a 96-well plate). Not statically enforced; fails at enqueue.
- **Serializing pattern as a plain JSON boolean array:** it must be a typed comArray (`<comArray:boolean:96>` / `:384>`). A raw `[true, false, ...]` will not round-trip. See the Parameters table entry and Method JSON Structure.
- **Using promptForPattern in automated contexts:** runtime prompts require a GUI and are unsuitable for unattended runs.

- **Authoring** `wellsAreNumbers` : inactive — not read at runtime; preserve if present on an existing method, do not add it to new steps.

[↑ Table of contents](#)

Group

Property	Value
stepType	"Group"
Category	Free-form container
Terminator	"End" (caption: "End Group")
Compatible Hardware	All

Behavior Summary

The Group step is a free-form container. At runtime it enqueues its child steps in order and does nothing else — no looping, no branching, no scope. The step tree renders it with an optional description; child steps are indented within it until the "End Group" terminator.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
description	string	—	No	""	Description text for the group. Not expression-evaluated. Editor-written; not read at runtime. Every editor-produced step carries this key, so exports always include it — emit "" when the group has no description, and always author this key .
comment	string	—	No	—	Optional comment text for the group. Not expression-evaluated. Editor-written; not read at runtime. Preserve when round-tripping.

Universal base keys (`caption` , `dynamic?` , `stepUI` , ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

If any additional keys appear on a `Group` step in an exported method, they do not impact the behavior of the Group step. Preserve them verbatim for round-tripping.

Enumerated / Constrained Values

None.

Cross-Field Validation Rules

None. The group is a simple container with no validation of its own.

Structural Context

This step is a **free-form container**. It uses `subSteps` and is terminated by an auto-generated "End" step (whose caption is "End Group"). **The terminator's stepType is "End" , not "End Group"** — "End Group" is only the display caption; there is no "End Group" step type registered, so serializing "stepType": "End Group" fails import.

```
{
  "stepType": "Group",
  "parameters": { ... },
  "subSteps": [
    { ... child steps ... },
    { "stepType": "End", "parameters": { "caption": "End Group" } }
  ]
}
```

```
]
}
```

The terminator must always be the last element in `subSteps` .

Canonical Examples

Group with child steps:

```
{
  "stepType": "Group",
  "parameters": {
    "description": "Reagent addition"
  },
  "subSteps": [
    {
      "stepType": "Fixed-8 Load Tips",
      "parameters": { "tips": "P1", "mode": "NumberOfTips", "numberOfTips": "8" }
    },
    {
      "stepType": "Fixed-8 Aspirate",
      "parameters": { "pod": "Pod1", "operation": "Aspirate", "where": "P3", "amount": "100", "firstWell": 1, "liquidType": "Well Cont"
    },
    {
      "stepType": "Fixed-8 Dispense",
      "parameters": { "pod": "Pod1", "operation": "Dispense", "where": "P4", "amount": "100", "firstWell": 1, "liquidType": "Tip Cont"
    },
    {
      "stepType": "Fixed-8 Unload Tips",
      "parameters": { "tipDestination": "<Any Trash>" }
    },
    { "stepType": "End", "parameters": { "caption": "End Group" } }
  ]
}
```

Empty group (description only):

```
{
  "stepType": "Group",
  "parameters": {
    "description": "Placeholder for future steps"
  },
  "subSteps": [
    { "stepType": "End", "parameters": { "caption": "End Group" } }
  ]
}
```

Common Mistakes

- **Serializing the terminator as** `"stepType": "End Group"` : `"End Group"` is only the display caption; the registered stepType is `"End"` . Emitting `"End Group"` fails import.
- **Expecting control flow**: Group does not loop or branch; it just runs children sequentially. Use `If` / `Loop` / `Worklist` instead.

† Table of contents

If

Property	Value
stepType	"If"
Category	Container (exact children)
Terminator	N/A (children are exact-match Named Containers, each of which ends with "End")
Compatible Hardware	All

Behavior Summary

At runtime the If step evaluates a VBScript/JScript expression and then executes exactly one of two branches: the Then branch (first Named Container child) if the condition is true, or the Else branch (second Named Container child) if the condition is false. The condition string is auto-prefixed with `=` if it does not already start with one, then evaluated as a script expression. An empty condition causes a runtime error. The If step is placed between operational steps inside any free-form container.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
condition	expression-capable string	—	Yes	""	A VBScript or JScript expression to evaluate. If it does not start with <code>=</code> , the runtime auto-prefixes one. Begin an expression with <code>==</code> for JScript evaluation. The result is coerced to boolean: truthy executes the Then branch, falsy executes the Else branch.

Universal base keys (`caption` , `dynamic?` , `stepUI` , `disabled` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions*.

Enumerated / Constrained Values

No enumerated value constraints — `condition` accepts any valid script expression.

Cross-Field Validation Rules

- `condition` **must** be a non-empty string.
- `condition` **must** be a valid VBScript/JScript expression that can be coerced to boolean.
- Expressions can reference variables defined by enclosing Let, Loop, Set Global, or Scripted Let steps. The `=` prefix marks the *whole condition* as an expression (not each reference individually) — e.g. `=MyVar > 10` treats the entire string as an expression that dereferences `MyVar` .

Structural Context

The If step is an **exact-children container** with exactly 2 "Named Container" children:

- Index 0:** Then branch — "Named Container" with caption "Then" .
- Index 1:** Else branch — "Named Container" with caption "Else" .

Each Named Container is itself a free-form container that must end with "End" . You may place any steps inside each branch before its "End" terminator.

Constraints:

- `subSteps` must contain exactly 2 elements.
- Both elements must be of type `"Named Container"`.
- Each Named Container's own `subSteps` must end with `"End"`.
- A branch with no operational steps is valid — it can contain just `"End"`.

Canonical Examples

Minimal If with empty branches:

```
{
  "stepType": "If",
  "parameters": {
    "condition": "=SampleCount > 0"
  },
  "subSteps": [
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Then" },
      "subSteps": [
        { "stepType": "End", "parameters": {} }
      ]
    },
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Else" },
      "subSteps": [
        { "stepType": "End", "parameters": {} }
      ]
    }
  ]
}
```

If with operational steps in both branches:

```
{
  "stepType": "If",
  "parameters": {
    "condition": "=Volume > 50"
  },
  "subSteps": [
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Then" },
      "subSteps": [
        { "stepType": "Transfer", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    },
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Else" },
      "subSteps": [
        { "stepType": "Pause", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    }
  ]
}
```

If using a variable from an enclosing Loop:

```
{
  "stepType": "If",
  "parameters": {
    "condition": "=IterationCount > 0"
  },
  "subSteps": [
    {
      "stepType": "Named Container",
      "parameters": { "caption": "Then" },
      "subSteps": [
        { "stepType": "Transfer", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    },
  ],
}
```



```

{
  "stepType": "Named Container",
  "parameters": { "caption": "Else" },
  "subSteps": [
    { "stepType": "End", "parameters": {} }
  ]
}

```

Common Mistakes

- **Placing operational steps directly inside If** : they must live inside the Then/Else Named Containers, not as direct children of `If` .
- **Substituting Group (or another container) for a branch**: both children must be `"Named Container"` — no other type is accepted by the exact-children structural rule.
- **Stylistic: prefer the = prefix on condition** : runtime auto-prefixes when it's missing, so `"Volume > 50"` and `"=Volume > 50"` both work. Not a correctness issue — the leading `=` marks the string as an expression.
- **Swapping Then and Else**: index 0 is Then, index 1 is Else — order is positional, not by any child key.

[†] Table of contents

Just In Time

Property	Value
stepType	"Just In Time"
Category	Free-form container
Terminator	"End" (caption: "End Just In Time")
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Just In Time step is a free-form container that wraps its child steps in a **queue-engine JIT block**, so their actions are scheduled "just in time" — as late as the schedule allows, instead of as early as their dependencies and resources permit. Children are still enqueued in order; only the timing of their actions changes. Unlike `Group`, which only organizes steps, Just In Time changes *when* the enclosed actions run; substituting `Group` changes the run schedule. Use it when the enclosed actions should happen as close as possible to the moment they are needed — for example, wrapping an aspirate/dispense pair so reagent is not drawn until the dispense is ready. Biomek always writes `slack` as `0` (see the parameter note).

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>slack</code>	number	seconds	Yes	<code>0</code>	Scheduling slop for the JIT block, consumed at enqueue. Biomek always writes <code>0</code> — the editor exposes no field for it — so author <code>0</code> . Required at enqueue: the key must be present, or Run fails on the missing-key lookup (see Base Keys & Serialization Conventions §0).

Universal base keys (`caption`, `isPreconfigured`, `dynamic?`, `disabled`, ...) — see Base Keys & Serialization Conventions; all are optional editor/serializer bookkeeping.

Enumerated / Constrained Values

`slack` must be a non-negative number of seconds. The queue engine rejects NaN, negative values, and values larger than 1e+8 seconds. Author the JSON number `0`.

Cross-Field Validation Rules

None. The step itself performs no enqueue-time validation of its parameters beyond the `slack` range check the queue engine applies (see Enumerated / Constrained Values). Structural validity is governed only by the container rules below.

Structural Context

Just In Time is a **free-form container**. Its final substep must be an `"End"` terminator captioned `"End Just In Time"`.

Constraints (per Method JSON Structure):

- `subSteps` must be present and must end with the `"End"` terminator as the last element.
- Any number of legal operational steps may be placed before the terminator.

- The terminator's serialized `stepType` is `"End"` ; its caption is `"End Just In Time"` .

Canonical Example

Defer an aspirate so the reagent is not drawn until its dispense is ready:

```
{
  "stepType": "Just In Time",
  "parameters": { "slack": 0 },
  "subSteps": [
    {
      "stepType": "Fixed-8 Aspirate",
      "parameters": {
        "pod": "Pod1",
        "where": "P3",
        "operation": "Aspirate",
        "amount": "100",
        "firstWell": 1,
        "liquidType": "Well Contents",
        "autoSelectPrototype": false,
        "prototype": "F8 Medium"
      }
    },
    {
      "stepType": "Fixed-8 Dispense",
      "parameters": {
        "pod": "Pod1",
        "where": "P4",
        "operation": "Dispense",
        "amount": "100",
        "firstWell": 1,
        "liquidType": "Tip Contents",
        "autoSelectPrototype": false,
        "prototype": "F8 Medium"
      }
    },
    { "stepType": "End", "parameters": { "caption": "End Just In Time" } }
  ]
}
```

The wrapped children keep their own required keys — see Fixed-8 Aspirate and Fixed-8 Dispense. Load and unload tips outside the block; any legal operational steps may be wrapped.

Common Mistakes

- **Serializing the terminator as `"stepType": "End Just In Time"`** : `"End Just In Time"` is only the display caption; the registered step type is `"End"` . Emitting the caption as a step type fails import.
- **Omitting `slack` entirely**: the key is required at enqueue, so a missing key fails at Run. Author `0` .
- **Giving empty parameters to the wrapped child steps**: Just In Time does not relax its children's requirements — each child still needs its own required keys, or enqueue fails on that child.
- **Substituting `Group`** : `Group` only organizes steps; it does not defer them. Wrapping the same children in `Group` changes the run schedule.

Worklist

Property	Value
stepType	"Worklist"
Category	Free-form container
Terminator	"End" (editor caption "End Worklist" is display text only — the step type is "End")
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Structure/envelope: see Method JSON Structure.

Behavior Summary

The Worklist step reads a CSV worklist file and repeats its child steps once per data row, binding that row's columns as scoped variables for the iteration. It combines the iteration of a `Loop` with the per-iteration variable scoping of a `Let`, driven by the rows of a file. At enqueue it parses the file, then for each data row in the selected range it binds that row's `column` → `value` pairs and enqueues the child steps inside that scope — the same scoping mechanism a `Let` step provides. Every column value is bound as a **string**. If `loopVariable` is set, that row's 1-based data-row number is also bound under that name, as a number. Iterate the whole file (`whole?` = true) or a `start .. end` data-row range. A child step reads a column by writing `=ColumnName` in any expression-capable field (a column named `AmountA` is read as `"=AmountA"`); the `loopVariable` name is referenced the same way. The step also registers the file as a run attachment on validated runs.

Parameters Reference Table

Step-Level Keys

Every editor-produced step carries all five keys — emit all five, using `""` for the ones you are not using.

Key	Type	Units	Required	Default	Description
filename	expression-capable string	—	Yes	"" (constructor)	Path to the worklist CSV file. Evaluated as an expression at run time, so an <code>=</code> -prefixed value is resolved before the file is opened. If the expression cannot be resolved, the run fails with the expression-evaluation error for that value.
loopVariable	string	—	No	""	Name of an extra variable bound in each iteration to that iteration's 1-based data-row number. With a partial range this is the absolute row number, not a counter restarting at 1 (with <code>start: "3"</code> the first iteration binds <code>3</code>). Bound as a number, not a string. Must not match any column name in the file (see rule 1). May be empty — treat it as genuinely optional.
start	expression-capable string	data row # (1-based)	Conditional	""	First data row to process. Read/validated only when <code>whole?</code> is false. Write it as a string (<code>"2"</code> , <code>"=(Co1*8)-7"</code>), not a JSON number.
end	expression-capable string	data row # (1-based)	Conditional	""	Last data row to process. Read/validated only when <code>whole?</code> is false. Write it as a string (<code>"5"</code> , <code>"=(Co1*8)"</code>), not a JSON number.
whole?	boolean	—	No	true	When true, iterate the entire worklist and ignore <code>start</code> / <code>end</code> . When false, iterate the <code>start .. end</code> data-row range. Enqueue reads it tolerantly, so omission is safe. Note the trailing <code>?</code> is preserved in the serialized key.

Universal base keys (`caption` , `isPreconfigured` , `dynamic?` , ...) — see Base Keys & Serialization Conventions.

Do not author `let` / `weak` / `prompt` on a Worklist node. Unlike `Let` and the other scope steps, Worklist does not persist these keys — its per-row scope is created internally, one binding set per data row. See Base Keys & Serialization Conventions §4.

Enumerated / Constrained Values

No enumerated string values. `whole?` is a plain boolean; `start` / `end` are strings that evaluate to data-row numbers.

Worklist file format

- Comma-separated values. The **first** non-comment line is the header row; its fields become the column/variable names. A blank first line fails parsing.
- Every line after the header that is neither blank nor a comment is a **data row**. Row numbering for `start` / `end` and for `loopVariable` counts data rows only: the header, `#` comment lines, and blank lines are not counted, and the first data row is row 1.
- Lines whose first non-whitespace character is `#` are comments and are skipped.
- Lines must end with CRLF (`\r\n`) — a Windows CSV. A file saved with LF-only line endings is read as a single line: the whole file becomes the header row, so no data rows are found and the step iterates zero times. If the merged header contains an empty field, the run fails on the empty-variable-name check; otherwise it completes with no error.
- Fields may be double-quoted (`"..."`); surrounding quotes and surrounding whitespace are stripped.
- A quoted field may contain commas — a comma inside quotes does not split the field.
- An unclosed double quote fails parsing.
- Every column value is stored as a **string** in the per-row bindings.
- Each data row must have exactly as many values as there are header columns — otherwise parsing fails.
- A file with a header but no data rows is not an error: the step simply runs zero iterations.

Example worklist file:

```
Plates, AmountA
Plate1, 10
Plate3, 5
```

Two data rows. Iteration 1 binds `Plates` = `"Plate1"` and `AmountA` = `"10"` (plus the `loopVariable` , if set, to `1`); iteration 2 binds `Plates` = `"Plate3"` , `AmountA` = `"5"` , counter `2` . A child step reads them as `"=Plates"` and `"=AmountA"` in any expression-capable field.

Cross-Field Validation Rules

- `loopVariable` **must not** equal any column name in the file.
- When `whole?` is false, `start` **must** be non-empty.
- When `whole?` is false, `end` **must** be non-empty.
- `start` must be within 1..(number of data rows).
- `end` must be within 1..(number of data rows).
- `start` **must not** exceed `end` .
- `filename` must resolve to a readable file: an empty path or a path that cannot be opened fails at enqueue.
- The file must have a header row — no columns is an error, and an empty column name is an error.
- Each data row must have the same field count as the header.
- A child step that fails during a pass is re-raised (N is the 1-based data-row number).

Structural Context

The Worklist step is a **free-form container**. It requires one child:

- Trailing anchor (index 0):** `"End"` with caption `"End Worklist"` .

Constraints (same shape as `Loop / Group`, per `Method-JSON-Structure.md`):

- `subSteps` must be present and end with the `"End"` terminator as the last element.
- Any number of operational steps may precede the terminator.
- A Worklist containing only `"End"` is structurally valid but iterates over no child work.

Worklist does not participate in the `Break` protocol. Like `If`, `Group`, and `Let`, it does not consume a `Break` level — only `Loop` does. A `Break` inside a Worklist aborts the current pass and propagates outward as an error rather than ending the row iteration cleanly: with an enclosing `Loop`, that `Loop` consumes the level and terminates (which ends the Worklist with it); with no enclosing `Loop`, the method fails at run time. To end a Worklist early, guard the child steps with an `If` instead.

Canonical Example

Iterate the whole worklist shown above, binding the data-row number as `Row`; the child `Pause` reads the `Plates` column of the current row:

```
{
  "stepType": "Worklist",
  "parameters": {
    "filename": "C:\\Methods\\WorkListInput.txt",
    "loopVariable": "Row",
    "start": "",
    "end": "",
    "whole?": true
  },
  "subSteps": [
    { "stepType": "Pause", "parameters": { "mode": "PromptedGlobal", "message": "=Plates" } },
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Worklist" } }
  ]
}
```

Iterate only data rows 2–5:

```
{
  "stepType": "Worklist",
  "parameters": {
    "filename": "=WorklistPath",
    "loopVariable": "Row",
    "start": "2",
    "end": "5",
    "whole?": false
  },
  "subSteps": [
    { "stepType": "Move Labware", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Worklist" } }
  ]
}
```

Common Mistakes

- **`loopVariable` collides with a column name:** fails at enqueue. Pick a name not used as a CSV header.
- **Setting `start` / `end` but leaving `whole? true`:** the range is silently ignored. Set `whole?` to `false` to activate the range.
- **Assuming numeric CSV values stay numeric:** every value read from a *column* is bound as a **string**. Convert in the child step if arithmetic is needed. The `loopVariable` row number is the exception — it is bound as a number.
- **Off-by-one on rows:** `start`, `end`, and the `loopVariable` value are 1-based **data-row** numbers — the header line, `#` comment lines, and blank lines are not counted, and the first data row is row 1.
- **Emitting `start` / `end` as JSON numbers:** write `"2"`, not `2`. The editor writes both as strings and expression evaluation depends on the string form.
- **Confusing `end` (last data row) with `"End"` (terminator step):** `end` is a parameter inside `parameters`; `"End"` is the step type that must be the last element of `subSteps`.
- **Writing `"stepType": "End Worklist"`:** "End Worklist" is only the editor caption. The step type is `"End"`.
- **Saving the worklist with Unix (LF) line endings:** the parser expects CRLF. An LF-only file yields no data rows and the step silently does nothing.

- **Dropping the trailing `?` from `whole?`** : the key is literally `whole?` in the serialized form — a `whole` key will not be read.
- **Expecting `Break` to end the Worklist**: it does not — see `$Structural Context`. Guard the child steps with an `If` instead.

[↑ Table of contents](#)

Let

Property	Value
stepType	"Let"
Category	Container (free-form)
Terminator	"End" (caption: "End Let")
Compatible Hardware	All

Behavior Summary

The Let step binds scoped variables for its child steps. At run time it evaluates each entry in `let`, applies the `weak` and `prompt` flags for any matching names, extends the environment with the resulting bindings, and enqueues its children in that extended environment. A `let` entry is a **strong** binding — it shadows any outer definition of the same name. Listing that name in `weak` makes the binding defer to an existing outer definition; listing it in `prompt` asks the operator for the value at run time, using the `let` value as the dialog default. Only `let` holds values: `weak` and `prompt` are per-name flag sets whose keys must also appear in `let`. When execution leaves the Let container, the bindings go out of scope.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
let	object	—	Yes	{}	Dictionary of variable definitions — the only one of the three keys that holds values. Each property name becomes a scoped variable; each property value is a literal or an <code>=</code> -prefixed expression, evaluated at run time. Author values as JSON strings (e.g. <code>"50"</code>). JSON numbers and booleans are accepted, but only a string value goes through the expression evaluator — a non-string is passed through unchanged, so an <code>=</code> -expression only works when the value is a string. Must be present (empty <code>{}</code> is fine); omitting it fails at Run on the missing-key lookup.
weak	object	—	Yes	{}	Dictionary of weak-binding flags — not an independent variable map. A variable listed here binds only if the name is not already defined in an enclosing scope; use it for defaults a caller can override. Keys are variable names that must also appear in let (the value lives in <code>let</code>); values should be <code>null</code> — presence of the key is the flag, and the value stored here is never read. A <code>weak</code> key with no matching <code>let</code> key has no effect. Must be present (empty <code>{}</code> is fine); omitting it fails at Run on the missing-key lookup.
prompt	object	—	Yes	{}	Dictionary of runtime-prompt flags . A variable listed here raises a dialog at run time letting the operator confirm or change its value. Keys are variable names that must also appear in let (the <code>let</code> value supplies the dialog default); values should be <code>null</code> — presence of the key is the flag. The dialog message and title are generated automatically; the value you store here is not used. A <code>prompt</code> key with no matching <code>let</code> key produces no dialog. The operator's answer is evaluated exactly like a <code>let</code> value (a leading <code>=</code> makes it an expression) and is then converted to a number whenever it is numeric-looking — so an answer meant as text but written as digits (a leading-zero barcode, for instance) comes back as a number. A name flagged in both <code>weak</code> and <code>prompt</code> is not prompted when an outer scope already defines it — the weak check runs first. Must be present (empty <code>{}</code> is fine); omitting it fails at Run on the missing-key lookup.

Each of the three keys is a plain JSON object: the property name is the variable name; the property value is the value or `=`-expression (for `let`) or `null` (for `weak` / `prompt`). Every editor-produced Let step carries all three keys; author new JSON the same way.

Universal base keys (`caption`, `isPreconfigured`, `dynamic?`, `stepUI`, `disabled`, ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

See also Concept Guide: The Variable Environment (`let` / `weak` / `prompt`) for how variable scopes nest across all scope steps.

Enumerated / Constrained Values

No enumerated value constraints. `let` values are literals or `=`-prefixed expressions; `weak` and `prompt` values are unused flags and should be `null`.

Cross-Field Validation Rules

1. `let` **must** contain at least one entry for the step to have any effect — entries in `weak` or `prompt` alone do nothing. An empty `Let` (all three `{}`) is structurally valid and simply executes its children unchanged.
2. Every key in `weak` **must** have a corresponding key in `let`. A `weak` key with no `let` entry never binds.
3. Every key in `prompt` **must** have a corresponding key in `let`. A `prompt` key with no `let` entry produces no dialog.
4. Variable names **should** be valid script identifiers — no spaces, no leading digits (*not enforced at enqueue but will cause failures in child steps that reference invalid names*).
5. Expression values (prefixed with `=`) **must** be syntactically valid VBScript/JScript expressions. Use `=` for VBScript and `==` for JScript.

Structural Context

The `Let` step is a **free-form container**. It requires one child:

- **Trailing anchor:** an `"End"` step (caption `"End Let"`) is required as the container's last child. Author it as the last element of `subSteps`.

Constraints:

- `subSteps` must be present and contain at least the `"End"` terminator as the last element.
- Any number of operational steps may be placed before the `"End"` terminator.
- Child steps reference the variables defined in `let` via `=VariableName` syntax.

Canonical Examples

Simple `Let` defining two variables:

```
{
  "stepType": "Let",
  "parameters": {
    "let": {
      "Volume": "100",
      "TipType": "P200"
    },
    "weak": {},
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Let" } }
  ]
}
```

`Let` with expression values:

```
{
  "stepType": "Let",
  "parameters": {
    "let": {
      "HalfVolume": "=TotalVolume / 2",
      "DestColumn": "=SourceColumn + 1"
    },
    "weak": {},
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Span-8 Aspirate", "parameters": {} },
  ]
}
```

```

    { "stepType": "Span-8 Dispense", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Let" } }
  ]
}

```

Let with weak bindings (defaults that an outer scope can override). The values live in `let`; `weak` carries only the flags:

```

{
  "stepType": "Let",
  "parameters": {
    "let": {
      "DefaultVolume": "50",
      "MixCycles": "3"
    },
    "weak": {
      "DefaultVolume": null,
      "MixCycles": null
    },
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Let" } }
  ]
}

```

Let that prompts the operator. `SampleCount` is defined in `let` (that value is the dialog default) and flagged in `prompt`:

```

{
  "stepType": "Let",
  "parameters": {
    "let": {
      "SampleCount": "96"
    },
    "weak": {},
    "prompt": {
      "SampleCount": null
    }
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Let" } }
  ]
}

```

Common Mistakes

- **Forgetting the "End" terminator:** Let is a free-form container and must end with `"End"`.
- **Using `let` as a flat key-value instead of a nested object:** Write `"let": { "Var": "value" }`, not `"let": "Var=value"`.
- **Putting values in `weak` or `prompt`:** only `let` holds values. A name that appears in `weak` or `prompt` but not in `let` never binds and never prompts — the value stored under that key is never read.
- **Confusing `let` with `weak`:** `let` is the strong binding that always takes effect; adding the same name to `weak` makes it defer to an outer-scope definition.
- **Literal vs. expression values:** Bare strings are literals; expressions must be prefixed with `=` (e.g., `"=TotalVolume / 2"`).

Loop

Property	Value
stepType	"Loop"
Category	Container (free-form)
Terminator	"End" (caption: "End Loop")
Compatible Hardware	All

Behavior Summary

The Loop step repeats its child steps a computed number of times based on a start value, end value, and increment. At runtime it evaluates the `start`, `end`, and `increment` parameters (which may be expressions), calculates the iteration count as $\text{floor}((\text{end} - \text{start}) / \text{increment}) + 1$, and executes its children once per iteration. If a `variable` name is provided, each iteration binds that variable to the current loop value ($\text{start} + \text{increment} \times \text{index}$) for use by child steps. If no variable is provided, iterations are labeled "Loop pass #N". The Loop step validates that the increment direction is consistent with the start-to-end direction — a positive increment with $\text{start} < \text{end}$, or a negative increment with $\text{start} > \text{end}$. When start equals end, exactly one iteration executes regardless of increment. The loop variable is scoped to the child steps. A `Break` step inside the loop ends iteration early; `Break`'s `levels` counts enclosing **Loop** steps only, not intervening containers. If a child step raises an error, the Loop re-reports the error tagged with the 1-based iteration number.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>variable</code>	string	—	No	""	Name of the loop variable. When non-empty, each iteration binds this name to the current loop value, so child steps can read it as <code>=Name</code> in any expression-capable field. When empty, child steps cannot reference the loop variable. Use a valid script identifier — no spaces, no punctuation such as <code>.</code> , <code>,</code> , <code>&</code> , no leading digit. An empty <code>variable</code> is valid. Omitting the key entirely is equally valid.
<code>start</code>	expression-capable string	—	Yes	"1" (editor default only — JSON import does not apply it; always author the key)	Starting value of the loop counter. Evaluated as a double. If prefixed with <code>=</code> , evaluated as a script expression.
<code>end</code>	expression-capable string	—	Yes	"1" (editor default only — JSON import does not apply it; always author the key)	Ending value of the loop counter. The loop iterates while the counter has not passed this value. Evaluated as a double. If prefixed with <code>=</code> , evaluated as a script expression.
<code>increment</code>	expression-capable string	—	Yes	"1" (editor default only — JSON import does not apply it; always author the key)	Step value added to the counter each iteration. Evaluated as a double. If prefixed with <code>=</code> , evaluated as a script expression.

Internal Keys

Universal base keys (`caption`, `dynamic?`, `stepUI`, `disabled`, ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Enumerated / Constrained Values

No enumerated value constraints. The three counter parameters (`start` , `end` , `increment`) accept numeric literals or expression strings prefixed with `=` .

Cross-Field Validation Rules

1. When `start` < `end` , `increment` **must** be positive (> 0); otherwise enqueue fails.
2. When `start` > `end` , `increment` **must** be negative (< 0); otherwise enqueue fails.
3. When `start` == `end` , exactly one iteration executes regardless of `increment` value (including 0).
4. `start` , `end` , and `increment` **must** all be present and non-empty. An omitted key fails at Run on the missing-key lookup; an empty value cannot be converted to a number and so fails at enqueue; a printed method also flags this in place of the loop line.
5. `variable` , when supplied, must be a valid script identifier. An invalid name is not rejected at enqueue, but fails when a child step references it.

Structural Context

The Loop step is a **free-form container**. It requires one child:

- **Trailing anchor:** an "End" step (caption "End Loop") is required as the container's last child. Author it as the last element of `subSteps` .

Constraints:

- `subSteps` must be present and contain at least the "End" terminator as the last element.
- Any number of operational steps may be placed before the "End" terminator.
- A Loop with only "End" (no operational steps) is structurally valid but does nothing.
- Child steps reference the loop variable via `=VariableName` syntax.

Canonical Examples

Simple loop repeating 3 times with a named variable:

```
{
  "stepType": "Loop",
  "parameters": {
    "variable": "Count",
    "start": "1",
    "end": "3",
    "increment": "1"
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Loop" } }
  ]
}
```

Loop with an expression bound, whose child consumes the loop variable — the child aspirates from a first well that advances one column per pass, starting at well 2. Only the variable-consuming keys are shown; the aspirate's other required keys are omitted for brevity (see Fixed-8 Aspirate):

```
{
  "stepType": "Loop",
  "parameters": {
    "variable": "column",
    "start": "2",
    "end": "=NumColumns",
    "increment": "1"
  },
  "subSteps": [
    {
      "stepType": "Fixed-8 Aspirate",
      "parameters": {
        "useWellExpression": true,

```

```

    "firstWellExpression": "=column"
  },
  { "stepType": "End", "parameters": { "caption": "End Loop" } }
]
}

```

Countdown loop (negative increment):

```

{
  "stepType": "Loop",
  "parameters": {
    "variable": "Step",
    "start": "10",
    "end": "1",
    "increment": "-1"
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Loop" } }
  ]
}

```

Common Mistakes

- **Omitting start , end , or increment** : the editor default of "1" does *not* survive JSON import — import replaces the step's dictionary with exactly the keys you author. Enqueue reads each key with a throwing accessor, so an omitted key imports fine and then fails at Run. Always emit all three.
- **Emitting start / end / increment as JSON numbers**: the editor always writes these as strings, and expressions ("=NumCols") can only be strings. Author "1" , not 1 , so the method round-trips identically to an editor-saved export.
- **Forgetting the "End" terminator**: The Loop is a free-form container and must end with "End" .
- **Confusing end (loop bound) with "End" (terminator step)**: The end parameter is the loop counter bound; "End" is the structural terminator step type that must be the last child.
- **Assuming iteration count equals end value**: Iteration count is $\text{floor}((\text{end} - \text{start}) / \text{increment}) + 1$, not end . For example, start=1, end=100, increment=25 gives 4 iterations (1, 26, 51, 76), not 100.
- **Multiplying the loop variable by the probe count in a well expression**: when a Loop sweeps plate columns for a Fixed-8 or Span-8 step, pass the loop variable straight into firstWellExpression — "=column" , as in the second example above — not "=1+(column-1)*8" . Well numbering is row-major, so the top well of column c is simply well 3; multiplying by 8 assumes column-major numbering and lands the first tip mid-plate — the wrong wells, and a run-time tip-fit failure as soon as fewer than eight rows remain below it. See Probe & Mandrel Selection.

Named Container

Property	Value
stepType	"Named Container"
Category	Free-form container
Terminator	"End"
Compatible Hardware	All

Behavior Summary

The Named Container step groups child steps under a caption and enqueues them in order. It adds no control flow — no condition evaluation, no looping, no variable binding — and it performs no enqueue-time validation or configuration.

Every Named Container is a branch of an "If" step. "If" is the only step that uses Named Container. Do not add Named Container anywhere other than in an "If" step's substeps.

Parameters Reference Table

The Named Container step has no step-specific parameters of its own. Its one meaningful key is the base key `caption` — and unlike most base keys, you should author it, because JSON import does not supply it (see the note below the table).

Key	Type	Units	Required	Default	Description
<code>caption</code>	<code>string</code>	—	No, but author it in practice	<code>""</code>	Branch label shown in the step tree and in printed methods. Not expression-evaluated. Use "Then" / "Else" for a plain If branch, or a descriptive label such as "Wash the Tips". Always author this key.

Why author `caption` on a Named Container. The "Then" / "Else" captions are written by the If step only when the *editor* creates the If. JSON import replaces the step's parameters wholesale, so `caption` is not filled in for you. A branch authored without `caption` therefore imports with no caption at all, and the editor falls back to the built-in label "Named Container" for both branches.

Preserve `helpContextID: 13` when round-tripping an existing method.

Other universal base keys (`dynamic?`, `stepUI`, `disabled`, ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#).

Enumerated / Constrained Values

No enumerated constraints.

Cross-Field Validation Rules

No cross-field validation rules. The Named Container performs no runtime validation; it enqueues its children in order.

Structural Context

The Named Container is a **free-form container**. It requires one terminal child:

- **Trailing anchor:** "End" terminator.

Constraints:

- `subSteps` must be present and contain at least the `"End"` terminator as the last element. Omitting `subSteps` fails import.
- The last element must be `"End"`. Any other type fails import.
- Any number of operational steps may be placed before the `"End"` terminator.
- A Named Container with only `"End"` (no operational steps) is valid — it represents an empty branch.

Canonical Examples

Empty Named Container (an empty Else branch):

```
{
  "stepType": "Named Container",
  "parameters": { "caption": "Else" },
  "subSteps": [
    { "stepType": "End", "parameters": {} }
  ]
}
```

Named Container with operational steps (a Then branch):

```
{
  "stepType": "Named Container",
  "parameters": { "caption": "Then" },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "Move Labware", "parameters": {} },
    { "stepType": "End", "parameters": {} }
  ]
}
```

Named Container with a descriptive branch label (common in real methods) and a nested container:

```
{
  "stepType": "Named Container",
  "parameters": { "caption": "Wash the Tips" },
  "subSteps": [
    {
      "stepType": "Group",
      "parameters": { "description": "Wash cycle" },
      "subSteps": [
        { "stepType": "Transfer", "parameters": {} },
        { "stepType": "End", "parameters": {} }
      ]
    },
    { "stepType": "End", "parameters": {} }
  ]
}
```

Common Mistakes

- **Forgetting the `"End"` terminator:** Named Container is a free-form container and must end with `"End"`.
- **Omitting `caption` on an If branch:** JSON import replaces the step's parameters wholesale, so the parent If does not supply `"Then"` / `"Else"`. Author `caption` on each branch — `"Then"` / `"Else"`, or a descriptive label such as `"Wash the Tips"` (both are common in real methods). With no caption the editor labels the branch `"Named Container"`.
- **Using Named Container outside an `"If"`:** structural validation does not reject a Named Container at method root level between Start and Finish, but the editor never creates one there and the imported node is read-only, so it cannot be deleted afterwards. Use `"Group"` when you want a plain captioned container.

Next Item

Property	Value
stepType	"Next Item"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Structure/envelope: see Method JSON Structure.

Behavior Summary

The Next Item step advances a global variable to the next value in a list. At enqueue it evaluates `variable` to a variable name, reads that global's current value, evaluates `values` to an array (a bare string is wrapped via `=Array(...)`), finds the current value in the list (case-insensitively), and assigns the following value to the global. When the current value is the last one in the list — or the list itself is empty — behavior is governed by `onDone`: `"Break"` breaks out of one enclosing loop, `"SetTo"` assigns the evaluated `setTo` expression, and `"StartOver"` wraps back to the first value.

Two cases bypass `onDone` entirely and assign the **first** value in the list: the global being unset or empty, and the global's current value not appearing in the list at all. The second case is silent — a typo in the value, or a value written by another step, restarts the sequence instead of raising.

It is the list-driven counterpart to a `Loop`: place it inside a `Loop` whose body consumes `variable`, and use `onDone = "Break"` to end the loop when the list is exhausted. This is a **leaf** step; it takes no child steps.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>variable</code>	expression-capable string	—	Yes	<code>""</code>	Name of the global variable to advance. Evaluated as an expression to a variable name.
<code>values</code>	expression-capable string	—	Yes	<code>""</code>	The value list. Without a leading <code>=</code> the string is taken literally and then wrapped as <code>=Array(<literal>)</code> , so <code>"P1","P2","P3"</code> and <code>=Array("P1","P2","P3")</code> are equivalent — the bare comma-separated form is the standard authored form. With a leading <code>=</code> it must evaluate to an array (or to a single string that <code>Array(...)</code> can wrap).
<code>onDone</code>	string (enum)	—	Yes	<code>"Break"</code>	What to do when the current value is the last one in the list (or the list is empty). One of <code>"Break"</code> , <code>"SetTo"</code> , <code>"StartOver"</code> . Read only when the list runs out, so omitting it fails at that moment rather than on every pass; always emit it.
<code>setTo</code>	expression-capable string	—	Conditional	<code>""</code>	Value assigned to <code>variable</code> when <code>onDone = "SetTo"</code> and the list is exhausted. Evaluated as an expression. Ignored for other <code>onDone</code> values.
<code>name</code>	string	—	No	—	Optional display alias. When bound, the step caption becomes <code>"Next <Name>"</code> instead of <code>"Next Item"</code> . Cosmetic only — no runtime effect. Not required; omitted in normal authored steps.

Universal base keys (`caption` , `dynamic?` , ...) — see Base Keys & Serialization Conventions. Emit `variable` , `values` , `onDone` and `setTo` on every Next Item step; `caption` is optional.

Enumerated / Constrained Values

`onDone`

Value	Meaning when the list is exhausted
"Break"	Break out of one enclosing <code>Loop</code> . The global is not reassigned on this path — it keeps the last list value.
"SetTo"	Set <code>variable</code> to the evaluated <code>setTo</code> expression.
"StartOver"	Wrap back to the first value in the list. Requires at least two values (see rule 5).

`onDone` is matched **case-sensitively**. Be sure to use the given correct capitalization.

On the "Break" path the step breaks before assigning, so `variable` is left holding the last value in the list. Re-entering the loop with that value still set breaks again immediately — reset the global with a `Set Global` step before the loop if the loop can run more than once.

Cross-Field Validation Rules

- `variable` **must** be bound.
- `values` **must** evaluate to an array, or to a single string that `Array(...)` can wrap.
- Every element of `values` **must** be convertible to a string. Numeric elements are converted (`1` becomes `"1"`); an element that cannot be converted fails at enqueue (N is 1-based within the list).
- The current value **must** match at most one entry — comparison is case-insensitive. If it matches two (e.g. `"A"` and `"a"`), the step fails at enqueue. Duplicates elsewhere in the list are not detected, so keep the whole list unique anyway.
- `onDone = "StartOver"` requires at least **two** values. With an empty list — or a single-value list — the step fails at enqueue (misleading in the single-value case).

Matching the current value against the list is case-insensitive, and it is a **string** comparison — `1` and `01` are different values. If the current global value is empty/unset, or is not in the list at all, the step assigns the **first** list value without consulting `onDone` .

Structural Context

This step is a **leaf** and does not support `subSteps` . Include no `subSteps` (or an empty array).

Valid placement. Next Item is not itself a container and its variable-advance works anywhere, but `onDone = "Break"` requires an enclosing `Loop` — only `Loop` consumes a break level. Other containers — `Worklist` , `If` , `Group` , `Let` , `Just In Time` — do **not** absorb the break; a Next Item "Break" inside a `Worklist` with no outer `Loop` aborts the `Worklist` with a pass error rather than ending it cleanly.

With no enclosing `Loop` at all, the break is uncaught and surfaces as a run-time error. See also Break for the break protocol, and Next Labware for the labware-group analog of this step.

Canonical Examples

Bare comma-separated list — the standard authored form. Quote string values; numbers need no quotes:

```
{
  "stepType": "Next Item",
  "parameters": {
    "variable": "CurrentPlate",
    "values": "\"plate1\\",\"plate2\\",\"plate3\\",\"plate4\\\"",
    "onDone": "Break",
    "setTo": ""
  }
}
```

```
}
}
```

The same thing written as an explicit expression — equivalent, because a bare `values` string is wrapped as `=Array(<literal>)` :

```
{
  "stepType": "Next Item",
  "parameters": {
    "variable": "CurrentPlate",
    "values": "=Array(\"plate1\", \"plate2\", \"plate3\", \"plate4\")",
    "onDone": "Break",
    "setTo": ""
  }
}
```

Cycle and reset to a sentinel value when exhausted:

```
{
  "stepType": "Next Item",
  "parameters": {
    "variable": "Tube",
    "values": "=Tubelist",
    "onDone": "SetTo",
    "setTo": "=\"DONE\""
  }
}
```

Round-robin forever (wrap to the first value — two values minimum):

```
{
  "stepType": "Next Item",
  "parameters": {
    "variable": "Reagent",
    "values": "=Array(\"A\", \"B\")",
    "onDone": "StartOver",
    "setTo": ""
  }
}
```

Typical loop wiring — the `Loop` supplies the iteration, Next Item supplies the values and ends the loop when they run out:

```
{
  "stepType": "Loop",
  "parameters": { "variable": "", "start": "1", "end": "999", "increment": "1" },
  "subSteps": [
    {
      "stepType": "Next Item",
      "parameters": {
        "variable": "CurrentPlate",
        "values": "\"plate1\", \"plate2\", \"plate3\", \"plate4\"",
        "onDone": "Break",
        "setTo": ""
      }
    },
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": {} }
  ]
}
```

Common Mistakes

- **Assuming list elements must be quoted strings:** they need not be — `"1,2,13,14"` is a valid `values` list and numbers are converted to strings. But matching is string-based, so `1` and `01` are different values, and a global holding `1` will not match a list entry of `01` .
- **Relying on the "appears twice" error to catch duplicates:** it only fires when the *current* value matches two entries. Duplicates the sequence never lands on go undetected — keep the list unique yourself.
- **onDone = "StartOver" with a one-value list:** fails at enqueue even though the list is not empty. Use two or more values.

- **Misspelling** `onDone` : matching is case-sensitive and unrecognized values silently behave as `"StartOver"` — `"break"` will not break.
- **Expecting** `Break` **outside a** `Loop` : `onDone = "Break"` calls the break primitive; with no enclosing `Loop` it is uncaught and surfaces as a run-time break error. A `Worklist` does not consume the break — see `Break`.
- **Re-entering a loop without resetting the global**: after a `"Break"` the global still holds the last list value, so the next entry breaks immediately.
- **Confusing** `name` **with** `variable` : `name` is only a caption alias; `variable` is the global that gets advanced.

[†] Table of contents

Pause

Property	Value
stepType	"Pause"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions.

Behavior Summary

The Pause step halts method execution either until the user acknowledges a prompt or for a timed duration on a specific resource.

In "PromptedGlobal" mode the step saves the pod positions and pauses the interlock/light curtain, displays message and waits for the user to acknowledge it, then resumes the interlock/light curtain and restores the pod positions.

In "TimedResource" mode the step pauses a single resource — the whole system, a pod, or a deck position — for time seconds, which lets other resources keep working in parallel while that resource waits. No prompt is shown, and pod positions and the interlock/light curtain are not touched.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
mode	string	—	Yes	"User" — an internal default the run time rejects; always overwrite it	Pause mode: "PromptedGlobal" or "TimedResource" — see Mode Values below. Not expression-evaluated: the literal value is compared (case-insensitively) against those two names, so "=someVariable" is never resolved and always fails. Emit the canonical spelling.
message	expression-capable string	—	Conditional	"Paused" (internal default — see note below)	Message displayed to the user. Read only when mode is "PromptedGlobal", and required then. Evaluated as an expression.
time	expression-capable string	seconds	Conditional	"0"	Duration to pause, in seconds. Read only when mode is "TimedResource", and required then. Emit it quoted — either a constant ("30") or an expression ("=IncubationTime").
location	expression-capable string	—	Conditional	(no default)	The resource to pause. Read only when mode is "TimedResource", and required then. Evaluated as an expression, so a literal name is written plain ("P4") and a computed one is written with a leading = ("=\AMPure plate\""). See Location Values below.

Emit all four keys. Every Pause step carries mode, message, time, and location: "PromptedGlobal" steps carry the unused location (""), and "TimedResource" steps carry the unused message ("Paused"). These defaults do **not** survive a JSON import — the imported parameter set replaces the step's property dictionary wholesale, so a key you omit is simply absent, and omitting a key the chosen mode reads fails at run time. Emitting all four keys avoids that failure.

Enumerated / Constrained Values

mode Values

Value	Meaning
"PromptedGlobal"	Pause everything and display <code>message</code> ; wait for user acknowledgment.
"TimedResource"	Pause a single resource for <code>time</code> seconds.

These are the only two values the run time accepts. Anything else — including the internal default `"User"` — fails at run time; see Common Mistakes.

location Values (when mode is "TimedResource")

Value	Meaning
"the whole system"	Universal pause — pauses every resource
"Pod1"	Pauses Pod 1 only (on a single-pod instrument, the only pod)
"Pod2"	Pauses Pod 2 only (dual-pod instruments only)
<i>(deck position name)</i>	Pauses that deck position
<i>(labware name)</i>	Resolved to the deck position that currently holds labware of that name; that position is paused

Notes on `location` :

- The three special names above are matched case-insensitively; everything else is treated as a labware or deck-position name.
- `"Pod1"` and `"Pod2"` are valid only if that pod exists on the configured instrument. Naming a pod that is not installed fails at run time. Single-pod instruments — including the i3 Fixed-8 — address their pod as `"Pod1"` .
- A name that is neither a special name, nor labware currently on the deck, nor a bound deck position fails at run time as a position-not-found error.
- A labware name (e.g. `"=\\\"AMPureXP plate\\\""`) resolves to whichever deck position currently holds that labware.

Cross-Field Validation Rules

1. `mode` **must** be either `"PromptedGlobal"` or `"TimedResource"` . Any other value fails at run time as a not-configured error.
2. When `mode` is `"TimedResource"` , `location` **must** be present and name a resource that exists. An empty `location` fails at enqueue as a location-not-specified error; a name that resolves to no deck position fails as a position-not-found error.
3. When `mode` is `"TimedResource"` , `time` should be set to a positive value (*a time of 0 is technically valid but produces a zero-duration pause*).
4. When `mode` is `"PromptedGlobal"` , `message` **must** be present. It is read unconditionally in that branch, and the default does not survive JSON import.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

User pause with a message:

```
{
  "stepType": "Pause",
  "parameters": {
    "mode": "PromptedGlobal",
    "message": "Please add reagent to the reservoir and click OK to continue.",
    "time": "0",
    "location": ""
  }
}
```

```
}  
}
```

Timed pause on the whole system for 30 seconds:

```
{  
  "stepType": "Pause",  
  "parameters": {  
    "mode": "TimedResource",  
    "message": "Paused",  
    "time": "30",  
    "location": "the whole system"  
  }  
}
```

Timed pause on a specific deck position (incubation):

```
{  
  "stepType": "Pause",  
  "parameters": {  
    "mode": "TimedResource",  
    "message": "Paused",  
    "time": "300",  
    "location": "P4"  
  }  
}
```

Timed pause on the position currently holding a named plate, using an expression for duration:

```
{  
  "stepType": "Pause",  
  "parameters": {  
    "mode": "TimedResource",  
    "message": "Paused",  
    "time": "=IncubationTime",  
    "location": "=\AMPureXP plate\ ""  
  }  
}
```

Common Mistakes

- **Emitting "User" as mode** — the run time recognizes only "PromptedGlobal" and "TimedResource" and fails as not-configured on anything else. Always overwrite the internal default.
- **Treating time as milliseconds** — time is in **seconds**. A five-minute incubation is "300", not "300000".
- **Authoring time as a JSON number** — the editor always writes time as a string, whether it is a constant ("30") or an expression ("=IncubationTime"). Quote it in both cases to preserve round-trip fidelity.
- **Putting an expression in mode** — mode is the one key on this step that is not expression-evaluated. "=modeVar" is compared literally against the two accepted names, does not match, and fails as "not properly configured".
- **Omitting the keys the chosen mode does not read** — the imported parameter set replaces the step's dictionary outright, so omitted keys are absent rather than defaulted. Emit all four.

Run Method

Property	Value
stepType	"Run Method"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Run Method step runs another method from the current project inline, as part of the current method. At enqueue time it resolves `filename` against the project's method collection, rejects a `filename` that matches the full project path of the method currently open in the editor, evaluates the `let` / `weak` / `prompt` dictionaries into a fresh environment, attaches that environment to this step's own call node — which becomes the enclosing scope for the called method's steps — and enqueues those steps into it. The `let` entries are the argument values handed to the called method. Run Method is a **leaf** in the JSON tree — no `subSteps` .

Parameter-passing contract. A `let` value reaches the called method only if that method's Start step marks the same variable overridable — that is, lists the same name in its own Start `weak` dictionary. If it does not, the called method's Start step rebinds the name to its own `let` value and the caller's value is silently discarded; nothing is reported and the method simply runs with the wrong value. The names must match.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>filename</code>	string (expression-capable)	—	Yes	<code>""</code>	Project method name. For a method inside a folder, use the backslash-delimited project path — <code>Subfolder\MethodName</code> — which in JSON must be written with an escaped backslash: <code>"Subfolder\\MethodName"</code> . Matching is case-insensitive. Do not include a drive letter, directory, or a <code>.bmf</code> extension (the on-disk Biomek method file): the method is looked up in the current project, not on disk. Omitting the key raises a missing-key SYSTEM ERROR at run time, and an empty string fails to resolve as a method-not-found error.
<code>let</code>	object	—	Yes	<code>{}</code>	Variable bindings passed into the called method. Keys are variable names. Author values as JSON strings: a number as a quoted string (<code>"4"</code>), an expression as an <code>=</code> -prefixed string (<code>"=SampleVolume * 2"</code>). A bare JSON number is accepted on import, but the editor normalizes it back to a string, so string form preserves round-trip fidelity. Omitting the key raises a missing-key SYSTEM ERROR. See §4 of Base Keys & Serialization Conventions.
<code>weak</code>	object	—	Yes	<code>{}</code>	Marks <code>let</code> entries as weak. Keys are variable names that must also appear in <code>let</code> ; a key present only here has no effect. Values are flags — author <code>null</code> . A weak entry is skipped when the variable is already bound in the calling scope, so the caller's existing value wins and the <code>let</code> value acts as a default. Omitting the key entirely raises a missing-key SYSTEM ERROR.
<code>prompt</code>	object	—	Yes	<code>{}</code>	Marks <code>let</code> entries that the operator is asked to confirm at run time. Keys are variable names that must also appear in <code>let</code> ; a key present only here produces no prompt and no variable. Values are flags — author <code>null</code> . At run time the operator sees a dialog

Key	Type	Units	Required	Default	Description
					titled <i>Enter Value</i> reading Enter a value to use for '<name>', pre-filled with the <code>let</code> value; the response is evaluated and, when it converts to a number, stored as a number. The prompt text is not configurable. Omitting the key entirely raises a missing-key SYSTEM ERROR.

Universal base keys — see Base Keys & Serialization Conventions.

This step does **not** carry a `dynamic?` key (unlike Run Procedure). No other step-specific keys are expected on Run Method — emit only `filename`, `let`, `weak`, and `prompt`.

Enumerated / Constrained Values

None. `filename` is a free-form project method name; `let`, `weak`, and `prompt` accept any user-defined variable names as keys.

Cross-Field Validation Rules

1. **Self-reference is rejected.** If `filename` (case-insensitively) equals the full project path of the currently open method, enqueue fails as a self-reference error. This is a single-level check against the method open in the editor — it is not a general recursion detector, and mutual recursion between two other methods is not caught.
2. **Method must exist.** If the name does not resolve in the project method collection, enqueue fails as a method-not-found error. An empty `filename` fails here like any other unresolvable name.

Structural Context

Run Method is a **leaf** — it never has child steps of its own. Do **not** include a `subSteps` array with elements (an omitted array or `[]` is fine). The step does extend the variable environment, but the environment it extends is the scope the *called* method's steps run in, not a scope for children of this node.

Canonical Example

A method stored at the root of the project, with two argument values:

```
{
  "stepType": "Run Method",
  "parameters": {
    "filename": "Reagent Prep",
    "let": {
      "PlateCount": "4",
      "Volume": "=SampleVolume * 2"
    },
    "weak": {},
    "prompt": {}
  }
}
```

Foldered method, one weak entry and one prompted entry

`Assays\Serial Dilution` lives in the project folder `Assays`; the backslash is escaped for JSON. `PlateCount` yields to a same-named variable already bound in the caller's scope, and the operator is asked to confirm `Volume` (pre-filled with `50`) when the step is enqueued:

```
{
  "stepType": "Run Method",
  "parameters": {
    "filename": "Assays\\Serial Dilution",
    "let": {
      "PlateCount": "4",
      "Volume": "50"
    },
    "weak": {
      "PlateCount": null
    },
    "prompt": {
```



```
    "Volume": null
  }
}
```

Common Mistakes

- **Treating filename as a file path:** it is a project method name, not a location on disk. "C:\\Methods\\Reagent Prep.bmf" fails as a method-not-found error. Use "Reagent Prep", or "Subfolder\\Reagent Prep" for a foldered method.
- **Omitting filename :** it is required — a missing key raises a SYSTEM ERROR at run time, and an empty string fails to resolve.
- **Omitting let / weak / prompt :** all three are required at run time — a missing key raises a SYSTEM ERROR. Write {} even when empty.
- **Putting prompt text in prompt , or values in weak :** both are flag sets over let keys. Their values are never read and their keys do nothing unless the same name is in let .
- **Assuming a let value always wins:** it is discarded unless the called method's Start step marks that variable overridable in its own weak dictionary.
- **Confusing with Run Procedure:** Run Method runs a method stored in the current project; Run Procedure inlines a procedure defined earlier in the same method by Define Procedure.

† Table of contents

Define Procedure

Property	Value
stepType	"Define Procedure"
Category	Free-form container
Terminator	"End" (caption: "End Procedure")
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Define Procedure step declares a **named, reusable procedure** made up of its own child steps. It does not execute its children where it sits. At enqueue it evaluates and validates the procedure name, captures its child steps and its `let` bindings as the procedure definition, and binds that definition to a global variable with that name. A later `Run Procedure` step resolves that name and inlines the captured body, applying its own `let` values over the procedure's `let` defaults. Only `let` crosses that boundary — see the `weak` and `prompt` rows below. Define Procedure is a **free-form container** whose body (the child steps) is the procedure definition.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
procedure	string (identifier, expression-capable)	—	Yes	""	Name of the procedure being defined. The value is expression-evaluated first, then checked as an identifier; it must be non-empty and a valid identifier. This name becomes the global variable other steps reference.
let	object	—	Yes (include, even as {})	{}	Default (strong) argument bindings for the procedure: keys are variable names, values are literals or <code>=</code> -prefixed expressions. These are the values every <code>Run Procedure</code> call starts from before applying its own <code>let</code> overrides. Author it explicitly — JSON import replaces the step's parameters wholesale, so the step's own default does not fill it in. See also §4 of Base Keys & Serialization Conventions.
weak	object	—	No (include as {} to match exports)	{}	Weak (fill-in) bindings: name → value used only when that name is not already bound in an enclosing scope. Persisted for round-trip fidelity, but not applied when the procedure is invoked — <code>Run Procedure</code> builds a fresh scope from the procedure's <code>let</code> plus the call's own <code>let</code> only, so <code>weak</code> authored here has no runtime effect. Put weak defaults on a <code>Let</code> step inside the procedure body instead.
prompt	object	—	No (include as {} to match exports)	{}	Names to prompt the operator for. Only the key matters: author the value as <code>null</code> , because the dialog text is generated from the variable name (<code>Enter a value to use for '<name>'</code>) and the stored value is never displayed; a prompted name must also appear in <code>let</code> , which supplies the dialog's default. Not applied when the procedure is invoked — a prompt declared here produces no dialog; author prompts on a <code>Let</code> step inside the procedure body.

Always author all four keys on a Define Procedure step.

Universal base keys — see Base Keys & Serialization Conventions. A Define Procedure step may also carry `isPreconfigured` and `caption`; neither affects procedure configuration. Define Procedure does not carry a `dynamic?` key — do not author one. (Contrast `Run Procedure`, which carries `dynamic?: true`.)

Enumerated / Constrained Values

None enumerated. `procedure` must be a valid identifier: it must start with an ASCII letter and contain only ASCII letters, digits, and underscores, up to 255 characters. Spaces, leading digits, leading underscores, and punctuation are rejected.

Cross-Field Validation Rules

1. **Procedure name required and must be a valid identifier.** If the evaluated `procedure` value is empty or is not a valid identifier, enqueue raises an OLE error naming the offending value as an invalid procedure name.
2. **Name must not be shadowed by a `let` variable.** If a `let` / `weak` binding with the same name is in scope where the Define Procedure runs, enqueue fails as a shadowing error. Redefining the same procedure name is allowed; the later definition replaces the earlier one when execution reaches it.
3. **Placement must precede every caller.** The name is bound when the Define Procedure step is *enqueued*, so it must sit ahead of every `Run Procedure` that calls it. A Define Procedure inside an `If` branch or a loop body binds the name only if and when that body runs; otherwise the call fails as a procedure-not-found error.

Structural Context

Define Procedure is a **free-form container**.

Constraints (per Method JSON Structure):

- `subSteps` must be present and must end with the `"End"` terminator as the last element.
- The steps between the start of the container and the terminator constitute the procedure body.
- The terminator's `stepType` is `"End"`. The terminator's `caption` should be `"End Procedure"`.

Canonical Example

```
{
  "stepType": "Define Procedure",
  "parameters": {
    "procedure": "AddReagent",
    "let": {
      "Volume": "100"
    },
    "weak": {},
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": { "caption": "End Procedure" } }
  ]
}
```

Common Mistakes

- **Expecting inline execution:** Define Procedure does not run its body — it registers it in a global for `Run Procedure` to invoke later. A method with only a Define Procedure and no matching `Run Procedure` does nothing at runtime.
- **Redefining a name:** two Define Procedures with the same `procedure` follow global-set semantics — the later one supersedes when execution reaches it, so which definition a given call sees depends on enqueue order.
- **Reusing a `let` variable's name:** if the procedure name is already bound as a `let` / `weak` variable in scope, enqueue fails as a shadowing error. Pick a name that no enclosing scope binds.
- **Omitting `let`:** this step imports and enqueues without it, but every `Run Procedure` that invokes the procedure then raises a missing-key SYSTEM ERROR. Always author `let`, even as `{}`.

- **Expecting weak / prompt here to do something:** they round-trip but are not applied when the procedure is invoked — only `let` is. Put weak defaults and operator prompts on a `Let` step inside the procedure body.
- **Non-identifier name via expression:** `procedure` is expression-evaluated and only then validated, so an expression producing `"add reagent"` throws just like the literal would.

Run Procedure

Property	Value
stepType	"Run Procedure"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Envelope / structural rules: see Method JSON Structure.

Behavior Summary

The Run Procedure step invokes a procedure previously declared by a `Define Procedure` step. At enqueue time it evaluates `procedure` to a name, looks it up as a bound global variable (the `Let` step stored by `Define Procedure`), then builds a fresh `Let` step, seeds it with the procedure's own default `let` bindings, overrides those with this step's `let` bindings, copies in the procedure's substeps, and enqueues the result. It is a **leaf** in the JSON tree: it owns no child steps and inlines the referenced procedure's body at runtime.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>procedure</code>	string (identifier, expression-capable)	—	Yes	<code>""</code>	Name of the procedure to invoke.
<code>let</code>	object	—	Yes (include, even as {})	<code>{}</code>	Override argument bindings for this call — include it even when empty (<code>{}</code>). Applied <i>after</i> the procedure's defaults, so these win. Keys are variable names; values are literals or <code>=</code> -prefixed expressions, and a string constant inside an expression must itself be quoted (<code>"=\\"Empty\\""</code>). See the <code>let</code> row of <code>Let</code> for the value-encoding rules. Omitting <code>let</code> fails when the step enqueues — see Common Mistakes.
<code>weak</code>	object	—	No	<code>{}</code>	Weak bindings. Persisted and round-tripped, but not consumed by this step — see the note below. A non-empty <code>weak</code> is legal and is preserved verbatim, but it is still never read.
<code>prompt</code>	object	—	No	<code>{}</code>	Prompt definitions. Persisted and round-tripped, but not consumed by this step — see the note below.
<code>runProcedure</code>	boolean	—	No	<code>true</code>	Inactive — not read at runtime (always <code>true</code>). It is never read as configuration — it exists only to distinguish this step from <code>Define Procedure</code> , which the <code>stepType</code> already does. Do not rely on it; emit <code>true</code> to match exports.

Every editor-produced Run Procedure step carries all five keys above; author them the same way.

Universal base keys — see Base Keys & Serialization Conventions. A Run Procedure step may also carry `isPreconfigured` or `caption`, and — where a generic HTML editor panel has been attached — the `html` / `html_*` panel family and `stepUI`. All of these are inert editor state, not Run Procedure configuration.

`dynamic?` is a framework-set base key (§2 of the base-keys doc). Every editor-produced Run Procedure step carries `dynamic?: true`. Emit it as `true`, exactly as the canonical example does; do not compute a value for it.

Note on `weak / prompt`: although the shared convention (§4 of the base-keys doc) lists `let / weak / prompt` as genuine per-step keys on procedure steps, this step's enqueue transfers only the `let` dictionary into the generated Let step. `weak` and `prompt` are created and round-tripped but never read here. Author override values in `let`. (Define Procedure, by contrast, deep-copies all three into the procedure definition.)

Enumerated / Constrained Values

None enumerated. `procedure` must name a variable bound to a procedure defined by an earlier Define Procedure step.

Cross-Field Validation Rules

Both errors below are raised while the step enqueues, and both go through the same enqueue-failure prompt: depending on the answer, the run either stops with the error or defers this call and continues with the rest of the method.

1. **Procedure must be bound.** If the name is not bound in the current environment when the step enqueues, it fails as a procedure-not-found error.
2. **Bound value must be a procedure.** If the name is bound to something that is not a procedure (for example a number or string set by Set Global), it fails as a not-a-procedure error.

Structural Context

Run Procedure is a **leaf** and has no populated children. Do **not** include a `subSteps` array with elements; the executed steps come from the referenced Define Procedure's body, inlined at enqueue.

Canonical Example

```
{
  "stepType": "Run Procedure",
  "parameters": {
    "procedure": "AddReagent",
    "let": {
      "Volume": "200",
      "SourcePlate": "=\"P1\""
    },
    "weak": {},
    "prompt": {},
    "runProcedure": true,
    "dynamic?": true
  }
}
```

`"200"` is a literal; `"=\"P1\""` is an expression whose result is the string `P1` — a string constant used inside an `=` expression must carry its own quotes.

Common Mistakes

- **Calling before defining:** the procedure global must be bound before this step runs (Define Procedure earlier in enqueue order), else the step fails as a procedure-not-found error.
- **Putting overrides in `weak / prompt`:** only `let` overrides are applied by this step; use `let`.
- **Omitting `let` entirely:** enqueue fails as a missing-key error. Include `let` even if empty.
- **Unquoted string constants in an `= expression`:** `"=P1"` reads the *variable* `P1` and fails if nothing binds it; write `"=\"P1\""` for the literal text, or drop the `=` entirely.

Scripted Let

Property	Value
stepType	"Scripted Let"
Category	Container (free-form)
Terminator	"End" (caption: "End Scripted Let")
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions.

Scripting is an advanced capability. Prefer a dedicated step wherever one exists. A Scripted Let runs a script whose only supported purpose is defining variables for its child steps. For anything beyond that, contact Beckman Coulter for applications support.

Behavior Summary

The Scripted Let step runs a VBScript or JScript block, then enqueues its child steps in a scope containing the variables the script bound. Children read those variables with `=VariableName`.

The script binds a variable by calling the global command `Extend "name", value` (VBScript) or `Extend("name", value)` (JScript); `WeakExtend` binds the same way but defers to an existing outer definition of that name. Both are **bare** global commands, not members of a named object.

Inside the script, any variable that is in scope at this step is readable by its **bare name** (not via `VARIABLES(...)`).

The bindings are visible only to the step's descendants; they go out of scope when execution leaves the container, so a step placed after the "End" terminator cannot reference them.

Do not author any other script commands besides `Extend` or `WeakExtend` in a Scripted Let step. If other commands exist, preserve them for round-tripping.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
code	string	—	Yes	""	Script code to execute. The script runs before child steps are enqueued. Always author this key.
description	string	—	Yes	""	Human-readable description; the editor uses it as the step caption when no <code>caption</code> override is set.
language	string	—	No	"VBScript"	Script language ProgID — see Language Values below. If omitted, the step also falls back to "VBScript" at enqueue. Author it explicitly.
let	object	—	No	(none)	Inactive — overwritten at runtime. Never author.
weak	object	—	No	(none)	Inactive — overwritten at runtime. Never author.
prompt	object	—	Yes	{}	Names to prompt the operator for at run time. Only the key is used — the stored value is never read, so author it as <code>null</code> . The dialog message and title are generated automatically (Enter a value to use for '<name>', title Enter Value) and the default offered is the value the script bound for that name. A prompted name therefore has an effect only if the script also binds it with <code>Extend / WeakExtend</code> ; a <code>prompt</code> entry with no matching binding produces no dialog. Unlike <code>let / weak</code> , this

Key	Type	Units	Required	Default	Description
					key is not reset at enqueue, so omitting it fails at Run as a missing-key SYSTEM ERROR.

Universal base keys (`caption` , `stepUI` , `disabled` , ...) — see Base Keys & Serialization Conventions.

If any additional keys appear on a `Scripted Let` step in an exported method, they do not impact the behavior of the `Scripted Let` step. Preserve them verbatim for round-tripping.

Enumerated / Constrained Values

language Values

Value	Meaning
"VBScript"	Microsoft VBScript
"JScript"	Microsoft JScript (JavaScript-like)

Cross-Field Validation Rules

- `code` must be valid syntax in the selected `language` ; parse/runtime errors surface at execution. Newlines may be authored as `\r\n` or `\n` , but `\r\n` is preferred as that is the format generated by the UI.
- Add variables with the **bare** global commands `Extend / WeakExtend` (never `EXTENDED_COMMANDS.Extend`). Read in-scope method variables by their bare names.
- A name listed in `prompt` produces an operator dialog only if the script also binds that same name with `Extend / WeakExtend` .

Structural Context

The `Scripted Let` step is a **free-form container**. It requires one child:

- Trailing anchor:** `"End"` with caption `"End Scripted Let"` .

Constraints:

- `subSteps` must be present and contain at least the `"End"` terminator as the last element.
- Any number of operational steps may be placed before the `"End"` terminator.
- Child steps can reference variables defined by the script via `=VariableName` syntax.

Canonical Examples

Define one variable in VBScript — the default form (the editor's language selection defaults to VBScript, so that is what a step carries unless the author picks JScript):

```
{
  "stepType": "Scripted Let",
  "parameters": {
    "code": "Extend \"mixVolume\", 42",
    "description": "Define mixVolume",
    "language": "VBScript",
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Comment", "parameters": { "description": "child steps use =mixVolume" } },
    { "stepType": "End", "parameters": { "caption": "End Scripted Let" } }
  ]
}
```

A child step consumes the binding by writing `"=mixVolume"` in any expression-capable field — for example a `Transfer` step's volume. After the `"End"` terminator the name is out of scope again.

The same in JScript (parentheses + semicolon).

```
{
  "stepType": "Scripted Let",
  "parameters": {
    "code": "Extend('mixVolume', 42);",
    "description": "Define mixVolume",
    "language": "JScript",
    "prompt": {}
  },
  "subSteps": [
    { "stepType": "Comment", "parameters": { "description": "child steps use =mixVolume" } },
    { "stepType": "End", "parameters": { "caption": "End Scripted Let" } }
  ]
}
```

Common Mistakes

- **Forgetting the "End" terminator:** Scripted Let is a free-form container and must end with "End" .
- **Omitting prompt :** unlike `let / weak` , `prompt` is not reset, so a missing key raises a missing-key SYSTEM ERROR. Include `{}` .
- **Authoring `let` or `weak` :** the step overwrites both with empty dictionaries before the script runs and removes them afterwards, so any authored value is discarded. Bind variables from the script instead.
- **Expecting a `prompt` value to be the message:** only the key matters. The message is generated from the variable name, and the name must also be bound by the script or no dialog appears.
- **Prefixing the command with `EXTENDED_COMMANDS` :** call `Extend / WeakExtend` as **bare** global commands. `EXTENDED_COMMANDS.Extend` ... fails at run as an object-required error — the name is not a referenceable object.
- **Referencing a script variable after "End" :** the binding is out of scope; the method fails validation as a not-defined-variable error.
- **Wrong language syntax:** JScript — `Extend('name', value);` . VBScript — `Extend "name", value ;` do not wrap the whole argument list in parentheses (`Extend("name", value)` is a VBScript syntax error). `Extend("name"), value` parenthesizes only the first argument and is also valid VBScript — leave it alone when round-tripping.
- **Confusing with Script step:** Script is a leaf step that just executes code. Scripted Let is a container that also defines scoped variables for its children.

Script

Property	Value
stepType	"Script"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions.

Scripting is an advanced capability — use it as little as possible. Prefer a dedicated step over a script wherever one exists: to persist a value, use the **Set Global** step, not a script. For anything beyond a trivial calculation (or the HTML editor panel described under Advanced), contact Beckman Coulter for applications support. The examples here are intentionally minimal.

Behavior Summary

The Script step executes a VBScript or JScript `code` block. Code is executed both at enqueue (validation) time and at runtime. The step is a **leaf**: no child steps, no scoped variables.

Scope:

- To define variables for child steps, use Let or Scripted Let. `Extend` / `WeakExtend` are not supported in a Script step (see Common Mistakes).
- To persist a value, use the Set Global step rather than scripting it.
- Method variables are read by their **bare name**.

Keep any script trivial: a small, self-contained calculation.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
code	string	—	Yes	""	Script code to execute. Always author this key, even if it is empty.
description	string	—	Yes	""	Human-readable description displayed in the step caption. Omission raises a missing-key SYSTEM ERROR.
language	string	—	No	"VBScript"	Script language ProgID. Author it explicitly — "VBScript" or "JScript" — rather than relying on either default.
let	object	—	No	(none)	Free-form dictionary that some Script steps carry. The step's enqueue does not read it: it is neither a child scope (Script is a leaf) nor a step-parameter override. Where it is populated it is authored data that the step's own script reads (<code>StepDictionary.Let</code>). Preserve verbatim when round-tripping; do not author it for a new script.

Universal base keys (`caption` , `stepUI` , `disabled` , ...) — see Base Keys & Serialization Conventions.

If any additional keys appear on a `Script` step in an exported method, they do not impact the behavior of the Script step. Preserve them verbatim for round-tripping.

Enumerated / Constrained Values

language Values

Value	Meaning
"VBScript"	Microsoft VBScript
"JScript"	Microsoft JScript (JavaScript-like)

Cross-Field Validation Rules

1. `code` must be valid syntax in the selected `language` ; parse/runtime errors surface at execution. Newlines may be authored as `\r\n` or `\n`, but `\r\n` is preferred as that is the format generated by the UI.
2. Read a method variable by its **bare name** (not `VARIABLES(...)`). Using workspace variables is an advanced capability and is not covered here. Do not manipulate Biomek globals from a script — use the Set Global step.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Minimal VBScript that executes cleanly (no external object refs — the safest baseline):

```
{
  "stepType": "Script",
  "parameters": {
    "code": "Dim n\r\nn = 1",
    "description": "Minimal no-op VBScript",
    "language": "VBScript"
  }
}
```

Minimal JScript equivalent:

```
{
  "stepType": "Script",
  "parameters": {
    "code": "var n = 1;",
    "description": "Minimal no-op JScript",
    "language": "JScript"
  }
}
```

`code` is a single JSON string; embed line breaks as `\r\n` (CRLF) to match Biomek's Windows-native line endings.

Common Mistakes

- **Using `MsgBox`** : not available in the Biomek script host — it errors. Use a Pause step for an operator message.
- **Calling `Extend` / `WeakExtend`** : unsupported in a Script step — the calls fail as an unsupported-command error. Use Scripted Let to bind variables for child steps.
- **Setting a global from a script**: don't — use the Set Global step. Manipulating globals or the workspace from a script is an advanced capability, not covered here.
- **Reading a variable with `VARIABLES("x")`** : fails as an object-expected error — read the variable by its bare name instead.
- **Expecting the code to run when the step is reached**: Biomek steps run as soon as their required resources are available. If the script does not utilize any Biomek workspace resources, it will run ahead of the surrounding physical steps, in addition to running at enqueue (validate) time. Don't use a script to react to something that happens during the run.
- **Confusing with Scripted Let**: Script is a leaf — it doesn't define scoped variables for child steps. Use Scripted Let to bind variables for a group of children.

Set Global

Property	Value
stepType	"Set Global"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions. Structure/envelope: see Method JSON Structure.

Behavior Summary

The Set Global step defines or updates a **global** variable. Unlike Let, whose bindings are visible only inside that container, a global is not tied to any container's scope. At **enqueue** (validate) the step evaluates `name` to a variable name and `value` to a value, then stores the pair in the run's global variable table. The binding is established during the enqueue pass, so any step enqueued after this one — at any nesting depth — can reference it, and re-running Set Global with the same `name` simply overwrites the value.

Globals outlive the run: they are cleared by a Finish step with `clearGlobals` set to `true` (its default), which is what prevents carry-over into the next method.

If `name` is already bound by a **scope** binding in effect at this point — a Start step variable, a `let` / `weak` / `prompt` binding on an enclosing container, a Loop counter, or a Run Method / Run Procedure argument — the step raises the shadowing error (Cross-Field rule 2). An existing **global** of the same name is not shadowing; it is simply updated.

Parameters Reference Table

Key	Type	Units	Required	Default	Description
<code>name</code>	expression-capable string	—	Yes	<i>(none — must be authored)</i>	The global variable name. Passed through the expression evaluator: a bare string is used verbatim (leading/trailing whitespace trimmed) as the name; a <code>=</code> -prefixed string is evaluated and its <i>result</i> becomes the name (indirection — see Common Mistakes). Must be a valid identifier — see Cross-Field rule 3.
<code>value</code>	expression-capable string	—	No	<code>""</code>	The value to assign. A bare string is stored verbatim (whitespace-trimmed) as a string ; a <code>=</code> -prefixed string is evaluated and the result type is stored. So <code>"25"</code> stores the string <code>"25"</code> while <code>"=25"</code> stores the number <code>25</code> — use the <code>=</code> form when later steps do arithmetic or numeric comparison on the global. The engine tolerates omission — an omitted <code>value</code> behaves as <code>""</code> .

Enumerated / Constrained Values

No enumerated constraints on `value`. `name` is constrained to identifier syntax (Cross-Field rule 3). Both keys accept literal strings or `=`-prefixed expressions.

Cross-Field Validation Rules

- `name` **must** be present in `parameters` — otherwise the step fails as a not-configured error.
- `name` **must not** be shadowed by a scope binding in effect at this point — otherwise the step fails as a shadowing error. The check covers **any** name currently bound in the variable environment: a Start step variable, a `let` / `weak` / `prompt` binding, a Loop counter, or a Run Method / Run Procedure argument. An already-existing global of the same name does **not** trigger it.

3. `name` **must** evaluate to a valid identifier — it begins with a letter, then letters, digits, or underscores only, 1 to 255 characters. Otherwise the step fails as an invalid-variable-name error. An empty `name` fails this same check.

Structural Context

This step is a **leaf** and does not support `subSteps`. Include no `subSteps` (or an empty array).

Canonical Examples

Set a numeric global (the `=` prefix makes the stored value the **number** 4, not the string `"4"`):

```
{
  "stepType": "Set Global",
  "parameters": {
    "name": "PlateCount",
    "value": "=4"
  }
}
```

Set a literal text global (stored as the string `"Plate A"`):

```
{
  "stepType": "Set Global",
  "parameters": {
    "name": "CurrentPlateLabel",
    "value": "Plate A"
  }
}
```

Set a global using an expression:

```
{
  "stepType": "Set Global",
  "parameters": {
    "name": "RemainingVolume",
    "value": "=TotalVolume - UsedVolume"
  }
}
```

Clear a global's value. The variable stays defined and holds an empty value — it is not un-defined. Next Item treats an empty value as "start from the first item":

```
{
  "stepType": "Set Global",
  "parameters": {
    "name": "Status",
    "value": ""
  }
}
```

Common Mistakes

- **String-typed numbers:** `"value": "4"` stores the string `"4"`, not the number 4. Prefix numeric values with `=` (`"=4"`) when downstream steps do arithmetic or numeric comparison on the global. Both spellings occur in practice, and some methods work around the string form with an explicit conversion such as `=cdbl(SomeGlobal)"`.
- **Literal-vs-expression value:** an unprefixed value is a literal string; prefix with `=` to evaluate (`"=Count + 1"`, not `"Count + 1"`).
- **Prefixing name with `=`:** `"name": "=PlateCount"` sets a global whose *name* is the current value of `PlateCount` (indirection), not the global `PlateCount`. Author the name bare: `"name": "PlateCount"`.
- **Invalid name characters:** no spaces, hyphens, dots, or leading digits in `name` — the step hard-fails at enqueue (Cross-Field rule 3).
- **Shadowed variable:** if any binding in scope already uses the name — a Start step variable, a `let / weak / prompt` binding, a Loop counter, or a Run Method / Run Procedure argument — Set Global fails with the shadowing error (Cross-Field rule 2). Reusing a Start-step variable name is the most common way to hit this. Rename the global, or drop the Start variable and set it here instead.

- **Colliding with a procedure name:** Define Procedure registers its procedure as a global of the same name. Setting a global with a procedure's name overwrites it, and the matching Run Procedure then fails as a not-a-procedure error. Keep global names and procedure names distinct.
- **Confusing with Let:** Let creates scoped variables for child steps. Set Global creates run-wide globals. Use Set Global when you need a variable accessible outside the current container.

† Table of contents

Run Program

Property	Value
stepType	"Run Program"
Category	Leaf
Terminator	N/A
Compatible Hardware	All

Base keys + casing: see Base Keys & Serialization Conventions.

Behavior Summary

The Run Program step launches an external program during method execution. `executableName` names the program, `parameters` supplies its command line, `runPath` sets its working directory, `visibility` sets the window show-mode, and `resource` names a resource that is reserved before the program starts.

`waitForRuntime` selects one of three wait modes (see Enumerated Values): continue immediately, hold the resource until the program exits, or block all method activity including look-ahead. When `captureOut` is `true`, the program's standard output is written to the global variable named by `globalName`. When `lightCurtain` is `true`, the light curtain/interlock on the system is paused — so it can be broken — while the program runs, and resumed when the program returns.

Every string-valued key is expression-evaluated at enqueue; `lightCurtain`, `isInteractive` and `captureOut` are read as booleans.

Parameters Reference Table

Step-Level Keys

The step defaults all 11 keys, so exported methods always carry all of them.

Every key below except `timeEstimate` is read at enqueue without a default: omitting one raises a missing-key SYSTEM ERROR at Run time (see Base Keys & Serialization Conventions §0). Unprefixed string values are literal; prefix with `=` to evaluate (e.g. `"=\bsf shake \" & shakeSpeed"`).

Key	Type	Units	Required	Default	Description
<code>executableName</code>	expression-capable string	—	Yes	""	Path/name of the program to run. If it evaluates to empty, enqueue fails as a file-required error.
<code>parameters</code>	expression-capable string	—	Yes	""	Command-line arguments passed to the program. (<i>This parameter key literally spells <code>parameters</code>, distinct from the step node's <code>parameters</code> object.</i>)
<code>runPath</code>	expression-capable string	—	Yes	""	Working directory the program starts in.
<code>resource</code>	expression-capable string	—	Yes	"<Everything>"	Resource to reserve before running — see Enumerated Values for the three forms the editor writes. A value that does not evaluate to a resource object reserves the universal resource instead.

Key	Type	Units	Required	Default	Description
visibility	expression-capable string	—	Yes	"Invisible"	Window show-mode; mapped to a Win32 <code>SW_*</code> code. The evaluated value must be one of the tokens in Enumerated Values.
waitForRuntime	expression-capable string	—	Yes	"Hold the resource until the program completes"	How the method waits. Values are full English sentences — see Enumerated Values; the constraint applies to the evaluated value.
timeEstimate	expression-capable string	seconds	No	"1"	Estimated run time, used for scheduling. The only key with a fallback, so omission is safe. The evaluated value is converted to a whole number of seconds — author an integer string (e.g. "1" , "30"), or an expression that evaluates to a number ("=Incubation_Timer"); a value that cannot be converted raises a translation error (see Cross-Field Validation).
lightCurtain	boolean	—	Yes	false	If <code>true</code> , the light curtain/interlock is paused — so it can be broken — while the program runs. See Runtime notes.
isInteractive	boolean	—	Yes	false	Marks the launched action as requiring user interaction.
globalName	expression-capable string	—	Yes	""	Name of the global variable that receives captured standard output. Required (non-empty after evaluation) whenever <code>captureOut</code> is <code>true</code> , in every wait mode — otherwise enqueue fails as a global-name-required error. The global is created if it does not already exist, and later steps read it as an expression ("=BarcodeResult"). The name must be a valid identifier: begin with a letter, then letters, digits or underscores only. See Set Global for the global-variable model.
captureOut	boolean	—	Yes	false	Capture the program's standard output into the global named by <code>globalName</code> . The <code>globalName</code> requirement is enforced in every wait mode, but real output is only delivered in the block-all mode — see Runtime notes.

Universal base keys (`caption` , `disabled` , ...) — see Base Keys & Serialization Conventions. Do not author them.

Exception — `dynamic?` is authored on Run Program. Unlike most steps, the editor writes `dynamic?` explicitly on this step: `true` when `captureOut` is `true` OR `waitForRuntime` is the block-all sentence, else `false` . Author it to match the same rule when writing a Run Program step from scratch, so the method round-trips identically and the error-handling behavior matches an editor-saved step.

Enumerated / Constrained Values

waitForRuntime Values

Full English sentences used as enum keys. Only these three select a wait behavior:

Value	Effect
"Allow the method to continue independently"	Fire-and-forget: the method continues immediately and nothing is held.
"Hold the resource until the program completes" (<i>default</i>)	Resource held until the program exits. Subsequent method steps that do not depend on the given resource continue running.
"Block all method activity including look-ahead until the program completes"	Blocks method activity; also the only mode in which <code>captureOut</code> delivers real output.

An unrecognized value is **not** rejected: it selects neither wait nor block, so the step behaves exactly like "Allow the method to continue independently" .

resource Values

The editor writes one of three forms; author the same forms in JSON.

Value	Meaning
"<Everything>" (default)	Magic string — the angle brackets are a literal value, not a placeholder. Reserves the universal resource, so nothing else runs concurrently.
"=Positions.<DeckPositionName>"	Reserve a single deck position, e.g. "=Positions.BSF" . This is the common case.
"=World.Devices.Pipettor1.Pod1" / "=World.Devices.Pipettor1.Pod2"	Reserve a pod.

A bare position name such as "P4" is **not** a resource reference — unprefixed values are literal strings, and a value that does not evaluate to a resource object silently falls back to reserving the universal resource. Write "=Positions.P4" instead. If the expression itself fails to evaluate, the step fails at enqueue — see Cross-Field Validation.

visibility Values

"Invisible" (default, → SW_HIDE), "Visible" (→ SW_SHOWNORMAL), or a raw ShowWindow constant: "SW_HIDE" , "SW_MAXIMIZE" , "SW_RESTORE" , "SW_SHOW" , "SW_SHOWDEFAULT" , "SW_SHOWMAXIMIZED" , "SW_SHOWMINIMIZED" , "SW_SHOWMINNOACTIVE" , "SW_SHOWNA" , "SW_SHOWNOACTIVATE" , "SW_SHOWNORMAL" . Any other value raises a visibility-flag error.

Cross-Field Validation Rules

The rules the enqueue-time validation enforces:

- 1. `executableName` **required**. If it evaluates to empty, enqueue fails as a file-required error.
- 2. `timeEstimate` **must parse as an integer**. On expression-eval failure, enqueue fails as a translation error.
- 3. `captureOut` **requires** `globalName` . If `captureOut` is `true` and `globalName` evaluates to empty, enqueue fails as a global-name-required error. This check runs in **every** wait mode, not just the block-all mode.
- 4. `resource` **must evaluate**. A `resource` expression that fails to evaluate fails the step at enqueue as an invalid-resource error. Literal values (`<Everything>` , `""` , or a plain string) never reach this check, because a literal always evaluates successfully.
- 5. `visibility` **must be a known flag**. An unrecognized value fails as an invalid-visibility-flag error.
- 6. `globalName` **must be a valid identifier**. When `captureOut` is `true` , writing the global rejects a name that does not begin with a letter or that contains anything but letters, digits and underscores.

Runtime notes:

- `lightCurtain: true` reserves the universal resource — everything else must be idle — then pauses the light curtain/interlock for the duration of the program and resumes it when the program returns, including on error.
- When `captureOut` is `true` , the program's standard output is written to the global named by `globalName` . While simulating, that global is set to the literal "SIMULATING" instead. Real output is only delivered when `waitForRuntime` is the block-all sentence; in the other two wait modes the step still writes the global, but it receives an empty string.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

Fire-and-forget launch, invisible window:

```
{
  "stepType": "Run Program",
  "parameters": {
    "executableName": "C:\\Tools\\notify.exe",
    "parameters": "--plate done",
    "runPath": "",
    "resource": "<Everything>",
    "visibility": "Invisible",
    "waitForRuntime": "Allow the method to continue independently",
  }
}
```

```

    "timeEstimate": "1",
    "lightCurtain": false,
    "isInteractive": false,
    "globalName": "",
    "captureOut": false,
    "dynamic?": true
  }
}

```

Blocking run that captures stdout into a global:

```

{
  "stepType": "Run Program",
  "parameters": {
    "executableName": "C:\\Tools\\readbarcode.exe",
    "parameters": "",
    "runPath": "C:\\Tools",
    "resource": "<Everything>",
    "visibility": "Invisible",
    "waitForRuntime": "Block all method activity including look-ahead until the program completes",
    "timeEstimate": "10",
    "lightCurtain": false,
    "isInteractive": false,
    "globalName": "BarcodeResult",
    "captureOut": true,
    "dynamic?": true
  }
}

```

Expression-driven call reserving one deck position:

```

{
  "stepType": "Run Program",
  "parameters": {
    "executableName": "C:\\Biomek\\shaker.py",
    "parameters": "=\\bsf shake \" & shakeSpeed",
    "runPath": "",
    "resource": "=Positions.BSF",
    "visibility": "Invisible",
    "waitForRuntime": "Hold the resource until the program completes",
    "timeEstimate": "=Incubation_Timer",
    "lightCurtain": false,
    "isInteractive": false,
    "globalName": "",
    "captureOut": false,
    "dynamic?": true
  }
}

```

Common Mistakes

- **Wrong `waitForRuntime` value.** Use one of the three sentences verbatim. An unrecognized value is not rejected — the step silently falls through to fire-and-forget (no wait, no block).
- **`timeEstimate` as a JSON number.** Source stores it as a string and re-evaluates it as an expression; keep it a string ("5").
- **Treating `<Everything>` as a placeholder.** It is a literal magic value; leaving it as-is reserves the universal resource.
- **A bare position name in `resource`.** "P4" is a literal string, not a resource reference, and quietly reserves the universal resource. Write "=Positions.P4" .
- **Expecting captured output outside the block-all mode.** `globalName` is required whenever `captureOut` is `true` — in every wait mode — but the global only receives real output when `waitForRuntime` is the block-all sentence.
- **`lightCurtain: true` misread as "enforce".** It *pauses* the curtain so it may be broken during the program — the opposite of what the name suggests.

SILAS

Property	Value
stepType	"SILAS"
Category	Leaf
Terminator	N/A
Compatible Hardware	All (requires configured SILAS modules on deck)

Base keys + casing: see Base Keys & Serialization Conventions.

Behavior Summary

The SILAS step sends a command to an external SILAS-controlled device module (for example a plate reader, incubator, centrifuge, or barcode reader). It names the target module and hands it a pre-configured binary message carrying the command and its parameters. Set `dynamic?` to make the step wait for the action to finish; leave it off and the step is fire-and-forget.

Around that command the step also handles labware give/take between the deck and the device's internal storage, and manages light curtain/interlock access for pod clearance. It can create data sets from the data the device sends back.

The step is generic — a single step type handles all SILAS device types. Each configured SILAS module appears as a separate palette entry in the Biomek UI, but every one of them serializes to the same "SILAS" step type.

Parameters Reference Table

Core Parameters

Key	Type	Units	Required	Default	Description
<code>module</code>	<code>string</code>	—	Yes	—	SILAS module name. Must name a SILAS module configured on the instrument. See Deck Layouts Format (in <i>Biomek Instrument & Project Reference</i>) §"The <code>device</code> block" for where configured module names come from.
<code>message</code>	<code>object</code>	—	Yes	—	SILAS command message. In JSON, serialized as a structured object (see Message Structure below).
<code>dynamic?</code>	<code>boolean</code>	—	No	<code>false</code>	If <code>true</code> , blocks until device action completes and enables RDD/DataSet features.
<code>lightCurtainAccess</code>	<code>boolean</code>	—	No	<i>(module default)</i>	If <code>true</code> , pauses pipettor light curtain/interlock and clears pod zones during action. Falls back to module's default if not bound.

Data Set Parameters (conditional: `dynamic?` must be `true`)

Key	Type	Units	Required	Default	Description
<code>makeDataSet</code>	<code>boolean</code>	—	No	<code>false</code>	If <code>true</code> , create data set(s) from device response.
<code>dataSetName</code>	<code>string</code>	—	Conditional	<code>""</code>	Name for the created data set. The step has no built-in fallback name: when <code>dynamic?</code> and <code>makeDataSet</code> are both <code>true</code> and this is empty, enqueue fails as a data-set-name-required error. If the device returns more than one data message, the created data sets are named <code><dataSetName>0</code> , <code><dataSetName>1</code> , ...

Key	Type	Units	Required	Default	Description
copyOnTransfer	boolean	—	No	true	If <code>true</code> , the created data sets travel with liquid transfers to the destination labware.
translate	string	—	No	(key absent)	COM ProgID of a custom translator object that interprets SILAS response messages. When present, the translator replaces the built-in data-set extraction (and <code>copyOnTransfer</code> is not applied). Omit the key entirely when no translator is in use; if the key is present but empty, enqueue fails with an error requiring the translator's ProgID.

Storage & Override Parameters

Key	Type	Units	Required	Default	Description
override Index	string	—	No	—	Overrides the storage slot/stack at enqueue time; applied only when <code>Give Or Take</code> is not <code>"None"</code> . Writes the value into <code>Storage Model Info.Locations</code> under key <code>"1"</code> and injects it into <code>Command.Parameters.Index</code> (Array storage) or <code>Command.Parameters.Stack</code> (Stacks storage), according to the module's storage model.
rdd	object	—	No	(auto-created)	Runtime Data Dictionary. Injected into action context. Can contain <code>OtherProperties</code> list for simulation.

Display Parameters (design-time only)

Key	Type	Units	Required	Default	Description
defaultBitmap	string	—	No	""	Bitmap path resolved from SILAS bitmap directory.
defaultTooltip	string	—	No	""	Tooltip from the SILAS message 'Action Description' field.

Universal base keys (`caption`, `defaultCaption`, `dynamic?`, `stepUI`, ...) apply to this step but are omitted here — see [Base Keys & Serialization Conventions](#). Note: the step-specific `dynamic?` row above documents the SILAS blocking flag, which name-collides with the base key but has step-functional semantics on SILAS.

Message Structure

In JSON serialization, the `message` parameter is converted from its binary format to a structured object. The logical structure is:

message:	
_biomekType: "SILASMessage"	– Type discriminator; must be exactly this value
Command:	– sub-message
Command Name: string	– The SILAS command (device lifecycle "Open"/"Close"/"Initialize", or a module action)
Parameters:	– sub-message; open-ended / command-specific
Index: string	– (Array storage) slot index
Stack: string	– (Stacks storage) stack name
Get?: string	– "True"/"False"; written by the step on the auto-Open/Close messages
Workflow: string	– "Transport" "Pipette"
Time: string	– Estimated time in MILLISECONDS; injected at enqueue from top-level Time x 1000 (do not author)
...	– Other command-specific parameters (copied verbatim from the caller)
Transports:	– sub-message injected at enqueue (do not author); keys are 1-based deck-position slot numbers as strings
"1": string	– Slot number → labware UniqueID
Time: string	– Total estimated time in SECONDS (this is the one you author)
Success?: boolean	– Device-reported success flag (captured, not authored); some devices emit the strings "True"/"False"
Action Description: string	– Human-readable action tooltip
Storage Model Info:	– sub-message; present for Give/Take
Give Or Take: string	– "Give", "Take", or "None" (see the table below – the wording is from the module's point of view)
Locations:	– sub-message; keys are 1-based deck-position slot numbers as strings
"1": string	– Value is a single string of the form <code>'<storage location>,<count>'</code> , where the storage location is the module-side slot index (Array) or stack name (Stacks) – e.g. <code>'1',1`</code> for slot 1, count 1

Messages captured from the SILAS device include additional device bookkeeping keys alongside the ones above (for example `%Address`, `%MessageID`, `%Time Stamp`, `Changed?`, `Touch Count`, and `Command.Action Summary` / `Command.Extensions`). Preserve them verbatim —

round-tripping the containing Biomek method through Method JSON export/import keeps whatever the device supplied. Values are not restricted to strings: booleans and numbers round-trip as-is. A JSON array is only legal in one place — a two-element array [value, submessage] when the same key carries both a value and a sub-message.

Every key above is emitted **verbatim** and **case-sensitively**. Beyond Time / Index / Stack / Get? / Workflow , the contents of Parameters are command-specific and open-ended — do not assume a fixed schema.

Note: The exact JSON shape of message depends on the SILAS module and action. The structure above shows the common fields.

Enumerated / Constrained Values

Give Or Take Values (inside Storage Model Info)

Give and Take are worded from the **module's** point of view, not the deck's — the module gives labware out, or takes labware in. In the step editor these are the "Retrieving From Module" and "Sending To Module" choices.

Value	Editor wording	Meaning
"Give"	Retrieving From Module	The module gives labware out: transfer FROM the module's storage TO the deck position
"Take"	Sending To Module	The module takes labware in: transfer FROM the deck position TO the module's storage
"None"	No Change	No labware transfer

Storage Model Types (module property)

Model	Access Pattern	Key in Message
Array	Random access by integer index	Command.Parameters.Index
Stacks	LIFO access by stack name	Command.Parameters.Stack

Light Curtain AccessSide (module property, not a step key)

AccessSide and AccessExtent are configured on the SILAS module itself, **not** the step. They govern the pod-clearing behavior when lightCurtainAccess is true :

AccessSide value	Pod Clearing Behavior
"Left"	Clears Pod1 from Min.X to Min.X + AccessExtent
"Right"	Clears the right-hand pod — Pod2 when the instrument has one, otherwise Pod1 — from Max.X – AccessExtent to Max.X
"Center"	Fully retracts Pod1 to its Min.X, and Pod2 (if present) to its Max.X

The asymmetry is real: "Left" always clears Pod1, while "Right" picks Pod2 when the instrument has one. Min.X and Max.X are the **selected** pod's own travel limits, so on a two-pod instrument the "Right" range is measured against Pod2's maximum, not a deck-wide one.

AccessSide must be exactly Left , Right , or Center (matched case-insensitively). Failures name the property and point at the Device Editor:

Access Side has not been defined. Check the settings in the Device Editor for this device.
Access Side value is invalid. Check the settings in the Device Editor for this device.
Access Extent has not been defined. Check the settings in the Device Editor for this device.
Negative values for "Access Extent" are not allowed. Check the settings in the Device Editor for this device.
An invalid value was found. Check the settings in the Device Editor for this device.

The last covers a non-numeric AccessExtent . Note that AccessExtent is validated **even when AccessSide is Center** , which does not use it — the Device Editor grays the field out for Center but still stores it, so this only bites a device definition edited outside the editor.

Set these in the SILAS module's configuration UI; do not add them to the step's `parameters`.

Cross-Field Validation Rules

1. `module` **must** name a SILAS module configured on the instrument.
2. `message` **must not** be null and must be parseable (raises error with reason if malformed).
3. `message` **must** contain a `Command` submessage.
4. If `lightCurtainAccess` is `true`: module must have `AccessSide` and `AccessExtent` properties; `AccessExtent` must be ≥ 0 .
5. If `makeDataSet` is `true`: `dataSetName` must not be empty.
6. For "Give" (module $\rightarrow$ deck): the module-side slot or stack must contain labware. With Array storage the destination deck position must also be empty.
7. For "Take" (deck $\rightarrow$ module): the source deck position must contain labware, and the module-side destination must have room — an empty slot for Array storage, or remaining capacity in the named stack for Stacks storage.
8. For Array storage: index must be in range `1..Items.Count`.
9. For Stacks storage: the named stack must exist.

Structural Context

This step is a leaf and does not support subSteps.

Canonical Examples

The `module` name, `Command Name`, and the keys and values inside `Parameters` in the examples below are illustrative placeholders — substitute the actual names captured from the target SILAS device's UI or from a SILAS device message export. A real captured SILAS message also carries device bookkeeping keys (`%Address`, `%MessageID`, and so on) that are omitted here for readability; keep them when round-tripping the containing Biomek method through Method JSON export/import.

Basic SILAS action (fire-and-forget):

```
{
  "stepType": "SILAS",
  "parameters": {
    "module": "PlateReader",
    "message": {
      "_biomekType": "SILASMessage",
      "Command": {
        "Command Name": "RunProtocol",
        "Parameters": {
          "ProtocolName": "Absorbance96"
        }
      },
      "Storage Model Info": {
        "Give Or Take": "None"
      },
      "Time": "30",
      "Action Description": "Run plate read protocol"
    },
    "dynamic?": false
  }
}
```

SILAS action that sends labware into the module and creates a data set. "Take" is correct here: the module takes the plate in from deck position slot 1, into module slot 1 (Array storage).

```
{
  "stepType": "SILAS",
  "parameters": {
    "module": "PlateReader",
    "message": {
      "_biomekType": "SILASMessage",
      "Command": {
        "Command Name": "ReadPlate",
        "Parameters": {
```

```

    "Index": "1"
  },
  "Storage Model Info": {
    "Give Or Take": "Take",
    "Locations": {
      "1": "'1',1"
    }
  },
  "Time": "60",
  "Action Description": "Read plate with absorbance"
},
"dynamic?": true,
"makeDataSet": true,
"dataSetName": "AbsorbanceData",
"lightCurtainAccess": true
}
}

```

Common Mistakes

- **Missing `dynamic?` when expecting data:** Data sets are only created when `dynamic?` is `true` — `makeDataSet` alone is silently ignored.
- **Fabricating message content:** SILAS messages are device-specific and should be captured from the SILAS UI editor. Do not invent message structures without knowledge of the target device's protocol.
- **Reversing Give / Take :** the words describe what the *module* does. "Take" sends labware from the deck into the module; "Give" retrieves labware from the module onto the deck. Picking the wrong one produces a labware error (empty source / occupied destination) rather than a silent mistake.
- **Authoring `Command.Parameters.Time` :** set the top-level `Time` (seconds) only. The step derives `Command.Parameters.Time` (milliseconds) from it at enqueue, so a hand-written value there is redundant and does not survive.
- **Empty `dataSetName` with `makeDataSet` :** there is no default name — enqueue fails as a data-set-name-required error.
- **Empty `translate` :** leave the key out when you have no translator object. Present-but-empty is treated as a misconfiguration and fails the step.

Create Labware Group

Reference manual: this step appears as *Create Group* (Chapter 30); the JSON `stepType` is `"Create Labware Group"`.

Property	Value
stepType	"Create Labware Group"
Category	Leaf
Terminator	N/A
Compatible Hardware	i7, i5 — not declared for i3

Base keys + casing: see Base Keys & Serialization Conventions. Structure/envelope: see Method JSON Structure.

Behavior Summary

The Create Labware Group step tags a set of labware as a named, ordered group so a later `Next Labware` step can cycle through them. The current labware can then be referenced using an expression `=group` where `group` is a value defined in `groupName`. If `moveFirstLW` is true it immediately fetches the first member to `location` using `pod` and `gripSide`. This is a **leaf** step and is the create half of the Create-Labware-Group / Next-Labware pair; it is meant to be followed by a `Loop` containing a `Next Labware`.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>positions</code>	array	—	Yes	<i>(none — must be bound)</i>	Ordered list of group members. Each element is either a string deck-position name (expression-capable — <code>"=Pos"</code> resolves through a method variable), or a nested dictionary with <code>device</code> / <code>stack</code> / <code>endStack</code> for a storage-device stack (see below). The step fails as a not-configured error if this key is absent.
<code>groupName</code>	expression-capable string	—	Yes	<code>""</code>	Name of the group being created. Trimmed, then evaluated as an expression; must be a valid identifier (see rule 3). Paired with <code>Next Labware</code> , which reads the same group under the <code>name</code> key (not <code>groupName</code>).
<code>moveFirstLW</code>	boolean	—	No	<code>false</code>	When true, immediately move the first group member to <code>location</code> .
<code>location</code>	expression-capable string	—	Conditional	<code>""</code>	Destination deck position for the first labware when <code>moveFirstLW</code> is true. Ignored otherwise.
<code>pod</code>	string	—	Conditional	<code>""</code>	Pod/gripper used for the initial move when <code>moveFirstLW</code> is true (e.g. <code>"Pod1"</code>).
<code>gripSide</code>	string (enum)	—	Conditional	<code>""</code>	Grip side for the initial move when <code>moveFirstLW</code> is true. Must be <code>"A1 near"</code> or <code>"A1 away from"</code> . See enum below. Same values and semantics as the <code>gripSide</code> key on the other labware-handling steps (<code>Move Labware</code> , <code>Hold Labware</code> , <code>Next Labware</code>).

Universal base keys (`caption`, `dynamic?`, ...) — see Base Keys & Serialization Conventions.

`positions` member element forms

Form	Shape	Meaning
On-deck position	JSON string, e.g. "P1"	Name of a deck position; the labware currently there joins the group. A name that is not a deck position, or a position holding no labware, is skipped silently — it still counts as a member of the group, but nothing is tagged.
Storage-device stack	nested dictionary { "device": "...", "stack": "...", "endStack": "..." }	A stack on a SILAS storage device. Each of <code>device</code> / <code>stack</code> / <code>endStack</code> is an expression string; every non-null labware in <code>stack</code> joins the group and is given <code>GoToIndex = endStack</code> .

Enumerated / Constrained Values

gripSide

Value	Meaning
"A1 near"	A1 corner held near the pod.
"A1 away from"	A1 corner held away from the pod.

Cross-Field Validation Rules

Enqueue-time validation rules:

- `positions` **must** be bound — otherwise the step fails as a not-configured error.
- `groupName` (trimmed, evaluated) **must** be non-empty — otherwise the step fails as an empty-group-name error.
- `groupName` **must** be a valid identifier — otherwise the step fails as an invalid-group-name error.
- A storage-device member's `device` **must** exist — otherwise the step fails as a device-not-found error.
- The member device **must** use the `Stacks` storage model — otherwise the step fails as an unsupported-device error.
- The named `stack` and `endStack` **must** exist on the device — otherwise the step fails as a stack-not-present error.
- A member stack **must not** contain nested labware stacks — otherwise the step fails as a nested-stacks-not-supported error.
- Every expression-capable field — `groupName`, each `positions` entry, and a member's `device` / `stack` / `endStack` — **must** evaluate; an unresolvable expression fails as an evaluation error naming the field.

If `moveFirstLW` is true, the initial fetch runs the `Next Labware` engine and can additionally raise any of that step's errors (see `Next Labware`).

Structural Context

This step is a **leaf** and does not support `subSteps`. Include no `subSteps`.

Canonical Examples

Create a group from four deck positions (no immediate move):

```
{
  "stepType": "Create Labware Group",
  "parameters": {
    "groupName": "SourcePlates",
    "positions": ["P1", "P2", "P3", "P4"],
    "moveFirstLW": false
  }
}
```

Create a group and bring the first member onto the working position:

```
{
  "stepType": "Create Labware Group",
```

```

"parameters": {
  "groupName": "SourcePlates",
  "positions": ["P1", "P2", "P3"],
  "moveFirstLW": true,
  "location": "P5",
  "pod": "Pod1",
  "gripSide": "A1 near"
}
}

```

Create a group from a storage-device stack:

```

{
  "stepType": "Create Labware Group",
  "parameters": {
    "groupName": "StackedPlates",
    "positions": [
      { "device": "Stacker1", "stack": "In", "endStack": "Out" }
    ],
    "moveFirstLW": false
  }
}

```

Common Mistakes

- **Use `groupName` here but `name` in `Next Labware`**: the group-name key differs between the two steps — `groupName` to create, `name` to cycle. Keep the string value the same across both.
- **`moveFirstLW` without a valid `pod`**: the initial fetch resolves `pod` before it does anything else, so a blank or unknown pod fails as an unknown-pod error rather than a parameter-name error. Problems with `location` / `gripSide` surface as the paired `Next Labware` errors when the fetch actually has to move labware.
- **Nested labware stacks in a storage-device member**: not supported (rule 7). Flatten first or the group creation aborts partway.
- **Assuming i3 support**: this step is declared for i5 and i7 only. A method containing it still opens on an i3, but enqueueing fails as a step-not-compatible error.

Next Labware

Property	Value
stepType	"Next Labware"
Category	Leaf
Terminator	N/A
Compatible Hardware	i7, i5 — not declared for i3

Base keys + casing: see Base Keys & Serialization Conventions. Structure/envelope: see Method JSON Structure.

Behavior Summary

The Next Labware step cycles a named labware group (created by `Create Labware Group`) to its next member. Each time it runs it removes the current labware from the working `location` — returning it to its home position or off-deck device — brings the next unused member of the group to `location` using `pod / gripSide` for gripper moves, and updates the group's tracking global to the new position.

After a Next Labware step for a group with name `MyGroup`, other steps can reference the current labware in the group's position using the expression `=MyGroup`.

Three booleans tune it:

- `checkUsed` — only cycle when the current labware is actually "used".
- `leaveOnDeck` — if the next member is already on deck, repoint to it instead of physically moving it.
- `breakOnFail` — when no unused member remains, signal the enclosing loop to break instead of erroring.

This is a **leaf** step and the cycle half of the Create-Labware-Group / Next-Labware pair — it is meant to be placed inside a `Loop`, with `breakOnFail` `true` so the loop ends when the group is exhausted.

The Next Labware step must come after a Create Group step.

Parameters Reference Table

Step-Level Keys

Key	Type	Units	Required	Default	Description
<code>name</code>	expression-capable string	—	Yes	""	Name of the labware group to cycle. Note the key is <code>name</code>, not <code>groupName</code> — <code>Create Labware Group</code> uses <code>groupName</code> for the same value. Trimmed and evaluated; empty or unknown group is an error. Group must be previously created with a <code>Create Group</code> step.
<code>location</code>	expression-capable string	—	Yes	""	Working deck position: the current labware is removed from here and the next member brought here.
<code>pod</code>	string	—	Yes	""	Name of the pod whose gripper performs physical moves (e.g. <code>"Pod1"</code>). Always required.
<code>gripSide</code>	string (enum)	—	Conditional	"" (no runtime default; the <code>A1 near</code> fallback is print-only)	Grip side for gripper moves. Required whenever a gripper move occurs; in practice, always author it.

Key	Type	Units	Required	Default	Description
<code>checkUsed</code>	<code>boolean</code>	—	No	<code>true</code>	When true, only cycle if the current labware has been "used" (acted upon in a pipetting step); when false, always replace it — and in that case the search for the next member also skips members that were already retrieved once, so the group still advances.
<code>leaveOnDeck</code>	<code>boolean</code>	—	No	<code>true</code>	When true, if the next member is already on deck leave it in place to be used where it is (no gripper move); when false, move it to <code>location</code> anyway. It also decides what happens when the outgoing labware's home position is occupied: true leaves that labware where it is and points the group at the next member in its own position; false raises the rule 6 error.
<code>breakOnFail</code>	<code>boolean</code>	—	No	<code>true</code>	When true and no unused member remains, an enclosing loop swallows the failure and breaks; when false, continue (the raised error propagates).

Universal base keys (`caption` , `dynamic?` , ...) — see Base Keys & Serialization Conventions.

Enumerated / Constrained Values

`gripSide`

Value	Meaning
<code>"A1 near"</code>	A1 corner near the pod.
<code>"A1 away from"</code>	A1 corner away from the pod.

Cross-Field Validation Rules

Enqueue-time validation rules:

1. `pod` **must** name a pod that exists on the instrument — otherwise the step fails as an unknown-pod error.
2. `name` (the group) **must** be non-empty — otherwise the step fails as an empty-group-name error.
3. The group **must** exist in the deck's `LabwareGroups` — otherwise the step fails as a group-not-found error. Create a group with the `Create Group` step before using `Next Labware`.
4. Labware sitting at `location` **must** belong to the group — otherwise the step fails as a labware-not-in-group error.
5. The outgoing labware **must** have a recorded home — otherwise the step fails as a no-home error.
6. When the home position is occupied and `leaveOnDeck` is false, the move is blocked as a return-blocked error.
7. Bringing in the next member with a missing destination fails as a destination-not-found error.
8. When no unused member remains, the step fails as a no-unused-member error. If `breakOnFail` is true, an enclosing loop breaks instead of failing.
9. If the outgoing labware's home is not a deck position (it came from an off-deck storage device) and no suitable off-deck destination is free, the step fails as a no-off-deck-destination error.
10. If the outgoing labware had to be left in place (its home was occupied and `leaveOnDeck` is true) and the next member lives on a storage device, the step fails as a cannot-move-next-labware error.

Structural Context

This step is a **leaf** and does not support `subSteps` . Include no `subSteps` .

Pairing. `Next Labware` is meaningless on its own — it requires a group created earlier by `Create Labware Group` (see `Create Labware Group`) and is placed inside a `Loop` with `breakOnFail = true` so the loop terminates when the group is exhausted (the exhaustion error is absorbed and the loop exits cleanly).

Canonical Examples

Cycle a group inside a loop, moving each member to the working position:

```
{
  "stepType": "Next Labware",
  "parameters": {
    "name": "SourcePlates",
    "location": "P5",
    "pod": "Pod1",
    "gripSide": "A1 near",
    "checkUsed": false,
    "leaveOnDeck": false,
    "breakOnFail": true
  }
}
```

Repoint at the next member instead of fetching it (`leaveOnDeck` `true`). This suppresses only the *incoming* move — returning the labware currently at `location` to its home can still emit a gripper move. `pod` is still supplied — it is resolved on every execution:

```
{
  "stepType": "Next Labware",
  "parameters": {
    "name": "SourcePlates",
    "location": "P5",
    "pod": "Pod1",
    "gripSide": "A1 near",
    "checkUsed": false,
    "leaveOnDeck": true,
    "breakOnFail": true
  }
}
```

Typical loop wiring (paired with Create Labware Group, not shown):

```
{
  "stepType": "Loop",
  "parameters": { "variable": "", "start": "1", "end": "999", "increment": "1" },
  "subSteps": [
    {
      "stepType": "Next Labware",
      "parameters": { "name": "SourcePlates", "location": "P5", "pod": "Pod1", "gripSide": "A1 near", "breakOnFail": true }
    },
    { "stepType": "Transfer", "parameters": {} },
    { "stepType": "End", "parameters": {} }
  ]
}
```

Common Mistakes

- **Using `groupName` instead of `name`** : this step reads the group under the `name` key, unlike `Create Labware Group` which uses `groupName` . Using `groupName` here silently leaves the group unspecified.
- **No enclosing loop with `breakOnFail`** : without the loop, group exhaustion surfaces as a runtime error; with `breakOnFail` = `true` inside a `Loop` it's absorbed as a clean exit.
- **`checkUsed` = `true` on labware never marked used**: if the current labware isn't flagged "used" by a preceding pipetting step, the step re-pins the group's tracking global to the current `location` and returns without selecting a new member — producing a loop that never advances.
- **Omitting `pod`** : the pod is resolved on every execution, before the move-vs-no-move decision, so an absent or unknown `pod` fails as an unknown-pod error even in a repoint-only configuration.
- **Expecting `leaveOnDeck` to suppress all movement**: it governs only the *incoming* member. Returning the labware currently at `location` to its home still emits a gripper move.
- **Assuming i3 support**: this step is declared for i5 and i7 only. A method containing it still opens on an i3, but enqueueing fails as a step-not-compatible error.

Concept Guides

Cross-cutting Biomek Software concepts the per-step docs assume you are already familiar with. Each guide is self-contained and usable without Biomek source-code knowledge. The step docs stand alone — these guides can be removed without breaking them.

#	Guide	Description
01	Expressions	Expression syntax, how a field decides literal vs. expression, and which fields accept expressions.
02	Labware & Position Resolution	Deck position vs. labware class, and the per-step-family key spellings for each.
03	Probe & Mandrel Selection	Which pod probes/mandrels are active — Span-8 <code>spacing</code> / <code>useProbes</code> / <code>mandrelExpression</code> , Fixed-8 <code>numberOfTips</code> , and the Multichannel Select Tips container rules.
04	The Variable Environment	How <code>let</code> , <code>weak</code> , and <code>prompt</code> bring named variables into scope for expressions.
05	Well Volume Tracking	The enqueue-time per-well volume ledger — what seeds it, what debits it, and how to read the "well only has 0.000 uL" and "volume out of allowed range" errors.

Devices — how an installed peripheral is bound to a deck position, and which steps address one by position vs. by device name — are covered where the binding is read, in Deck Layouts Format (in *Biomek Instrument & Project Reference*) §"The `device` block".

The instrument/pod model, techniques (auto-select, family prefixes, height override), liquid & volume types, base-key conventions, and 1-based row-major well numbering are covered in the top-level Foundations documents:

- Instrument variants, pod types, pod-naming rule, and which pod family each step accepts — Introduction to Biomek and Liquid Handling Method Writing.
- Techniques, template-vs-technique split, auto-select algorithm, `F8` / `S8` / `MC` family prefixes, Fixed-8 no-auto-select rule, liquid types, volume types, and height override — Pipetting Techniques, Templates, and Auto-Selection.
- Base keys, key-name casing, and the `disabled` property — Base Keys & Serialization Conventions.
- Well numbering math (`firstWell` / `+wellsX` stride, `=Col` sweep) — Introduction to Pipetting Steps §"Well Numbering".

Read alongside the format specification (`../Method-JSON-Structure.md`) and the per-step reference (`../step-documentation/`).

¹ Table of contents

Concept Guide: Expressions

Many string-valued fields in a method can hold either a **literal value** or an **expression** that is evaluated at run time. This guide defines the expression syntax, how a field decides which one it got, and which fields accept expressions.

1. The `=` prefix

An expression is a string whose **first character is** `=`. Everything after the `=` is evaluated at run time; the result is coerced to whatever the field needs (a number, a string, a well index, a boolean array, ...).

```
"amount": "10"           // literal: aspirate 10 µL
"amount": "=dose * 2"    // expression: evaluate dose*2 at run time
```

A string **without** a leading `=` is taken verbatim as a literal. **Condition-valued** fields are the exception: the `If` step's `condition`, for example, **auto-prefixes** the `=`, so a bare string like `"Volume > 50"` is evaluated as an expression (a leading `=` is optional but recommended for clarity). This is why almost every expression-capable field is typed as a **string in the JSON**, even when it carries a number — the leading `=` is how the engine tells "the literal number 10" apart from "the formula that happens to start with a digit." Write numeric values destined for these fields as quoted strings (`"10"` , not `10`).

2. What you can write after the `=`

The expression language supports the usual building blocks:

- **Arithmetic** — `+` `-` `*` `/` , parentheses, e.g. `"=(col - 1) * 2 + 1"` .
- **Variable references** — any name in scope: loop counters, `let` / `weak` bindings, globals, prompted values. E.g. `"=col"` , `"=aspirateVolume"` , `"=podID"` .
- **Member access** — dotted names where the runtime exposes them, e.g. `"=thisSpanPod.name"` .
- **Array literals** — square-bracket lists, used where a field expects a list (e.g. a probe-selection expression: `"=[1,2,3,4,5,6,7,8]"`).
- **Pipetting context variables** (`C_...`) — engine-injected names that appear inside pipetting technique/template payloads, e.g. `"=C_Volume * 0.5"` . A pipetting step populates them from its own properties and the selected technique, and the actions in the technique's template read them at run time; a few also surface in method-level conditions when an enclosing pipetting/technique scope injects them (e.g. an `If` condition on `=C_...`). You do **not** hand-author them, and you rarely reference them in ordinary method-level fields. See Pipetting Techniques, Templates, and Auto-Selection.

Where a variable comes from is governed by scope — see Concept Guide: The Variable Environment (`let` / `weak` / `prompt`) for `let` / `weak` / `prompt` and globals.

3. Fields gated by an explicit "use expression" flag

Some fields have a *paired boolean* that selects between two different keys: a literal key and an expression key. When the flag is **off**, the literal key is read and the expression key is ignored; when **on**, the reverse. Setting the `=` string alone is not enough — you must also flip the flag.

For example, Span-8 Aspirate has a flag key `useWellExpression` , which, when set to `true` , tells the engine to read `firstWellExpression` and ignore `firstWell` to determine well selection. It also has flag key `useExpression` , which, when set to `true` , tells the engine to read `mandrelExpression` and ignore `useProbes` to determine probe/mandrel selection.

```
// Iterate plate columns via a loop counter Col:
"useWellExpression": true,
"firstWellExpression": "=Col",    // read
"firstWell": 1                   // ignored while the flag is true
```

Refer to individual step documentation for the keys available to the step and their interactions, as they are not all the same.

4. Fields that take an expression directly (no flag)

Most expression-capable fields have **no** gating flag — they simply accept either a literal string or a `=`-prefixed string in the same key. Common examples across the pipetting steps:

- Volumes: `amount`, `amounts` (per-probe array of strings), item-level `volume`.
- Counts: `repeats`, `replicates`, `numberOfTips`.
- Height override: `height`, `heightFrom`.
- Layout: `spacing`.
- Wash: `washVolume`, `washCycles`, `span8WashVolume`, `span8WasteVolume`.
- Technique name: `prototype` (may be a literal technique name *or* an expression that resolves to one at run time).
- Pod name: `pod` (a literal `"Pod1"` / `"Pod2"` or an expression resolving to one).
- Labware position: `where` / `location` (a literal deck position or an expression).

Per-step docs mark each key "expression-capable" where it applies; when in doubt, check the step's parameter table.

5. What is *not* expression-capable

- **Structural / envelope fields** — `stepType`, `format`, `formatVersion`, and the step-node fields (`disabled`, the shape of `parameters` / `subSteps`). These are read as literals only.
- **Enumerated discriminators** read as fixed tokens — e.g. `stop` (`"Sources"` / `"Destinations"` / `"Either"`). The `liquidType` sentinels (`"Well Contents"` / `"Tip Contents"`) are also fixed tokens on the steps that use them; a few step families (Fixed-8 and Multichannel Select Tips, for example) do mark `liquidType` as expression-capable in their parameter tables — check the step's own table.
- `_biomekType` and the comArray shape keys (`arraySubtype`, `dimensionCount`, `lowerBound`).

If a field is not documented as expression-capable, author a literal.

6. Common mistakes

- **Forgetting the `=`**. `"dose * 2"` is a literal string, not a formula — it will be coerced (often to 0 for a numeric field) instead of evaluated.
- **Writing a bare number where an expression field wants a string**. `"amount": 10` may round-trip oddly; author `"amount": "10"`.
- **Setting the expression key but not the flag** (or vice versa). If `useWellExpression` is `false`, your `firstWellExpression` is dead weight; the engine reads `firstWell`.
- **Referencing a variable that is not in scope**. The name must be defined by an enclosing `let` / `weak` / `prompt`, a global, or a loop/worklist counter, or evaluation fails at run time.
- **Assuming a stride matches the probe count**. `"=1+(Col-1)*8"` is a classic error — see Concept Guide: Probe & Mandrel Selection. To sweep plate columns the correct expression is simply `"=Col"` (no stride multiplier at all), because with row-major numbering the top well of column `c` is well `3`. A `Wellsx` stride is for iterating *rows*, not columns, and even then it is never the pod's probe count.

Concept Guide: Labware & Position Resolution

Almost every physical step names *where* it acts (a **deck position**) and *what kind of labware* is there (a **labware class**). These are two different things, and the **JSON key names for them vary by step family**. This guide pins down the distinction and the per-family key spellings so you address the right thing with the right key.

1. Two distinct concepts

- Deck position** — a named slot on the instrument's deck (e.g. "P9", "P13", a wash station like "WS1", or a trash like "TR1"). It identifies a *physical location*. Position names come from the target instrument's deck layout (P1 ... P30 , WS1 , TR1 , TR2 , W1 , tip-load positions, ...); do not invent them.
- Labware class** — the *type/geometry* of the container that sits there (e.g. "BCFlat96", "CostarFlat384Square"). A class defines well count, spacing, depth, per-well capacity (maximum volume), and dead volume. It does **not** say where the labware is.

A step generally needs **both**: the position tells the pod where to go; the class tells it the well geometry to pipette into. At enqueue the software checks that the class you named matches the labware actually placed at that position — a mismatch fails with a class-mismatch error.

A position field resolves its value **case-insensitively** as **either** a literal deck-position name ("P13"), a labware **instance** name (the labware's `properties.name`), **or** a labware **class** name — if a name maps to several positions, the first is used. All three forms resolve to the same physical labware. Not all steps support all options. Refer to the individual step's documentation.

Prefer using labware **instance names** when possible. If the instance name is used, then changes to the labware's position on deck in the Instrument Setup step do not require updates to downstream steps.

Note — position-name casing. The **keys** under `deckItems` and the deck-layout's `positions` object are the position name **camelCased** — only the first letter is lowercased; internal capitals are kept: P1 → p1 , TR1 → tR1 , WS1 → wS1 , TL1 → tL1 . Pipetting-step position **values** (`where` , `location` , `position`) use the display form (P1 , TR1 , ...). Because position matching is case-insensitive, either form resolves — follow each step's own examples.

2. Key-name variance by step family

The serialized key names are **not** uniform. Use the spelling the target step family expects:

Step family	Position key	Labware-class key
Span-8 / Fixed-8 (Aspirate, Dispense, Mix, ...)	<code>where</code>	<code>what</code>
Multichannel (Aspirate, Dispense, Mix, ...)	<code>location</code>	<code>labwareClass</code>
Transfer / Combine (per item in <code>items</code>)	<code>position</code>	<code>labwareClass</code>
Instrument Setup (per labware in <code>deckItems</code>)	the <code>deckItems</code> key is the deck position	<code>class</code> (on the labware object) — path-prefixed, see note below

So "the source plate class" is `what` on a Span-8 or Fixed-8 step, `labwareClass` on a Multichannel or Transfer-item, and `class` on an Instrument Setup labware object — the *same concept*, three key spellings. Author whichever the step's own parameter table lists; do not carry a Span-8/Fixed-8 `what` / `where` into a Multichannel step.

Note — class value form. Mixing the two class-name forms is a common hard enqueue failure: using the `LabwareClasses\`-prefixed form in a pipetting step, or the bare short name in `deckItems`, both fail. The rule: on Instrument Setup labware objects, `class` is serialized **path-prefixed** — e.g. "LabwareClasses\BCFlat96"; every pipetting step's `what` / `labwareClass` field takes the

short class name only (e.g. "BCF1at96"). The `LabwareClasses\` prefix belongs **only** in Instrument Setup's `deckItems` `class` field. Author each key in the form its own step's parameter table shows.

3. Positions can be expressions

On the pipetting steps the position field is expression-capable: `Span-8` `where` and `Multichannel` `location` both take either a literal deck position ("P13") or a `=`-prefixed expression that resolves to one at run time (e.g. `"=sourcePos"`). A `Transfer/Combine` item's `position` takes a literal deck position or labware name — check the step's own parameter table before assuming expression support. The `labware-class` field is normally a literal class name. See Concept Guide: Expressions.

4. Labware must be declared before it is used

A pipetting step does not *create* labware — it references labware already placed on the deck by an **Instrument Setup** step earlier in the method. That setup step is where a position gets its labware class, its `volumeType`, and any `evalAmounts` / `evalLiquids` fill (see Pipetting Techniques, Templates, and Auto-Selection §"Liquid Types" and §"Volume types and locatable surfaces"). Both `evalAmounts` and `evalLiquids` arrays must be full-length — exactly equal to the labware's well count (or section count for a reservoir); see Instrument Setup § `evalAmounts` / `evalLiquids`. Downstream steps must name a class **consistent with** what setup placed at that position.

Consequence for edits: if you change the labware class at a position in Instrument Setup, every downstream field that names that class (`what` / `labwareClass`, and tip-box class references such as a `Load-Tips` step's `tips` *when it holds a class* or a `Select-Tips Load`'s `tipType`) must be updated to match, or the step fails at enqueue. A `Select Tips Load`'s `tipsLocation` is a *deck-position* reference, not a class name — it needs updating only if the tip box is moved to a different position, not when its class changes.

5. Tips are labware too

Tip boxes are labware on the deck (e.g. class `"BC230"`; see Catalog: Labware Classes (in *Biomek Instrument & Project Reference*)), and tip-loading steps address them — but the **key names differ from the pipetting steps above**, and the plain `Load-Tips` steps have no separate position key.

`Multichannel Load Tips`, `Span-8 Load Tips`, and `Fixed-8 Load Tips` all use the `tips` key only, which may be the box class, box name, **or** deck position.

Only `Select Tips Load` splits class (`tipType`) from position (`tipsLocation`). On that step `tipType` is a **bare** tip-box class name (`"BC230"`) — do not prefix it. The identically-named key on an Instrument Setup labware object is a different value: the tip class *inside* the box, written as a `TipClasses\` path (`"TipClasses\T230"`). See Tip-class grammar.

So the "tip class to load" is `tips` on a `Multichannel/Span-8/Fixed-8 Load Tips` step but `tipType` on a `Select Tips Load` step; an explicit tip-box position exists only as `tipsLocation` (`Select Tips Load`) or, on any `Load Tips` step, as a value passed through `tips`. A named tip class or position that does not resolve to an available box on the deck fails.

Author the key the target step's own parameter table lists — do not invent a tip-source key on a `Load Tips` step (its tips come from `tips` / `tipType`). Note `tipLabwareClass` is a real key on the lower-level pipetting steps (`Fixed-8` / `Span-8` / `Multichannel Aspirate`, `Dispense`, `Mix` — paired with `refreshTips` — and on `Hold Labware`), so this warning is about not copying it onto steps that don't list it.

6. Reachability

The auto-locate above ("a **reachable** box") uses the same envelope constraint that governs every pipetting step. Each pod can only service deck positions within its physical travel envelope, and on a multi-pod instrument the two pods' envelopes overlap only in part. A step that names a `position` (or `where` / `location`, per family) **outside the chosen pod's reach** fails at enqueue or run even though the position and labware class resolve fine. When you switch a method from one pod to another (e.g. `MC` → `Span-8`), re-check every position field — layouts valid for one pod are not automatically reachable by the other. To find which positions a pod can reach, consult the target instrument's deck configuration rather than assuming; the Biomek Editor's own deck view is the authoritative source.

A tip trash resolves the same way, and is a common surprise: tip-unload steps name neither a labware class nor a device, but search for a position carrying the `can Discard Tips` characteristic — on `Span-8` and `Multichannel`, one the unloading pod can reach. The three failure

messages and the per-family remedies are in Reachability and Access (in *Biomek Instrument & Project Reference*) §"Symptom to likely cause".

7. Quick checklist

- Am I using the position/class **key names for this step family** (what / where vs labwareClass / location vs labwareClass / position vs class)?
- Does the class I name match what Instrument Setup placed at that position?
- Are deck-position names real ones from the target deck layout, not invented?
- After a labware-class change in setup, did I update every downstream class reference and tip-box reference?
- If the method unloads tips, does the target deck carry a Can Discard Tips position that the unloading pod can reach? (Check per pod on a two-pod instrument.)

[†] Table of contents

Concept Guide: Probe & Mandrel Selection

Which probes/mandrels of a pod are active for a step are defined in Span-8 `spacing`, `useProbes`, and `mandrelExpression`; Fixed-8 `numberOfTips`; and the Multichannel Select Tips container family for partial-head Multichannel work.

1. Well spacing (`spacing`)

`spacing` applies to **Span-8 only**. Fixed-8 spacing is implicit (the arm pitch is fixed) and is not a JSON-settable key.

`spacing` is the interval between the wells that adjacent probes address: `1` = every well (adjacent rows), `2` = every other well, and so on. Omit `spacing` or set it to `"1"` for the standard interval (every well on a 96-well plate). Set it to `"2"` on a 384-well plate (a Span-8 arm is pitched for 9 mm rows; 384-well plates have 4.5 mm rows, so the minimum reachable interval is 2). A spacing smaller than the labware allows (e.g. `1` on a 384) is rejected.

2. Probe / mandrel selection

Not every probe has to be active. A pod's active probes are chosen one of two ways, selected by a flag (see Concept Guide: Expressions for the flag/expression pairing):

- **Static** — `useProbes`, an **8-element boolean comArray**, one entry per probe (1–8), `true` = active. At least one must be `true`. This is the **Span-8** form (Fixed-8 uses a count via `numberOfTips` instead — see below).

```
jsonc "useProbes": { "_biomekType": "comArray", "arraySubtype": "boolean", "values": [true, true, false, false, false, false, false, false] // probes 1-2 only }
```

- **Expression** — set `useExpression: true` and give `mandrelExpression` an expression that resolves to the probe list at run time, e.g. `"=[1,2,3,4,5,6,7,8]"`.

For Span-8 note the physical constraint from Introduction to Biomek and Liquid Handling Method Writing §"Pod Types": tip type is configured per group of four probes (1–4, 5–8), and **all selected probes must share tip type and syringe size** — you cannot co-select a fixed-tip probe and a disposable-tip probe.

For **Fixed-8**, the count of active tips is given as `numberOfTips` (a string, `"1"` – `"8"`, expression-capable). Fixed-8 does not have a per-probe boolean mask like Span-8's `useProbes`. For simple pipetting steps (aspirate/dispense/mix), the Fixed-8 pod must first load tips with a Load Tips step, which is where `numberOfTips` is set. The loaded tips are then used for the pipetting step.

For the Transfer step, `numberOfTips` is set directly on the Transfer step's dictionary.

3. Span-8 probe-selection keys by step type

Span-8 probe-selection keys are **not interchangeable across step types**. Carrying a key onto a step that does not read it is wrong -- it is silently ignored, and the intended probe selection does not take effect. Check the scope note on each key below.

Simple Span-8 steps (Aspirate, Dispense, Load Tips, Unload Tips, Wash Tips)

Key	Type	Role
<code>useProbes</code>	boolean comArray[8]	Static probe mask (one entry per probe, at least one <code>true</code>)
<code>useExpression</code>	boolean	Authoritative flag. When <code>false</code> , the engine reads <code>useProbes</code> ; when <code>true</code> , it reads <code>mandrelExpression</code> instead.
<code>mandrelExpression</code>	string	Run-time expression resolving to the probe list (only read when <code>useExpression</code> is <code>true</code>)

These three keys are the complete probe-selection interface for simple Span-8 steps.

Tip-mode keys: Transfer, Combine, Span-8 Transfer From File, Span-8 Serial Dilution

These four steps carry the three keys above **plus** a tip-mode group. The tip-mode keys are a different way of choosing probes: instead of naming probes, they say "use whichever probes carry this kind of tip", and when either is `true` the `useProbes` mask is not consulted.

Key	Type	Role
<code>useFixedTips</code>	boolean	Use the pod's fixed-tip probes. Read on <code>Transfer</code> , <code>Combine</code> , <code>Span-8 Transfer From File</code> , and <code>Span-8 Serial Dilution</code> . Not read on the simple Span-8 steps.
<code>useDisposableTips</code>	boolean	Use the pod's disposable-tip probes. Same four steps as <code>useFixedTips</code> .
<code>useMandrelSelection</code>	boolean	Inert (UI-state) -- written alongside the two above, but the engine does not read this key on any step

Set at most one of `useFixedTips` / `useDisposableTips` . With both `false` , probe selection falls back to the expression/mask keys.

Span-8 Serial Dilution only

Serial Dilution additionally carries two flags that exist on no other step:

Key	Type	Role
<code>useMandrelExpression</code>	boolean	Authoritative probe-source flag (replaces the role of <code>useExpression</code> for probes on this step)
<code>useSectionExpression</code>	boolean	Authoritative well-source flag for section/well expressions
<code>useExpression</code>	boolean	Non-authoritative on Serial Dilution -- acts as UI scratch state only; the engine reads <code>useMandrelExpression</code> and <code>useSectionExpression</code> instead

Key difference: on Serial Dilution, `useExpression` does **not** control whether `mandrelExpression` is read -- `useMandrelExpression` does. On simple steps, `useExpression` is the authoritative flag.

`useMandrelSelection` is inert UI-state on every step that carries it. The engine never reads it. Do not author it expecting runtime behavior.

4. Partial-head Multichannel: the Select Tips family

A Multichannel head normally loads/pipettes a full grid. To use only *some* mandrels of the head, wrap the operations in a **Multichannel Select Tips** container (terminator: "`Multichannel Select Tips End`") and use its child steps (`... Select Tips Load` , `... Aspirate` , `... Dispense` , `... Unload` , etc.). Rules that matter:

- The pod must have **no tips** when the container begins, and **no tips** when the terminator runs — a Select Tips group that loads tips must also unload them before the `End` .
- Complex partial patterns that need multiple load/unload cycles use an optional `rearrangePos` (a tip-box position for temporary storage); it becomes required only when a child actually needs it.

This container is the sanctioned way to do partial-mandrel work on a Multichannel pod. Span-8 selects probes directly with `useProbes` ; Fixed-8 selects a count with `numberOfTips` .

5. Quick checklist

- `spacing` (Span-8 only) = interval between probes: `1` = every well (96-well plate), `2` = every other well (384-well plate).
- Static probe mask = 8-boolean `useProbes` `comArray` (≥ 1 true); or `useExpression` + `mandrelExpression` .
- Fixed-8: active-tip count is `numberOfTips` ("`1`" – "`8`"), no per-probe mask.
- Partial Multichannel work goes inside a Multichannel Select Tips container, and must unload all tips before its terminator.

- Well numbering / firstWell math — see Introduction to Pipetting Steps §"Well Numbering".

[↑ Table of contents](#)

Concept Guide: The Variable Environment (`let` / `weak` / `prompt`)

A Biomek method runs against a **scope** of named variables. Any expression (`=name`) resolves the name against the scope in effect at that step. Three JSON keys — `let` , `weak` , and `prompt` — control how variables come into scope. This guide gathers the rules that govern them in one place. Cross-link: `01-expressions.md` for the expression language itself.

1. The three binding kinds

Each of the three keys is a JSON object (a dictionary) whose keys are variable names and whose values are either literal strings/numbers or `=`-prefixed expressions:

```
"let": { "SourceCol": "1",      "Volume": "=BaseVolume * 2" },
"weak": { "TipType": "P200" },
"prompt": { "OperatorName": null }
```

- `let` — a *strong* binding. Defines or overrides the variable for this step's descendants, shadowing any outer definition of the same name.
- `weak` — a *weak* binding. Takes effect only if the variable is **not** already defined in an enclosing scope. Use this for defaults that a caller can override.
- `prompt` — the variable's value is asked of the operator at run time. Only the **key** (the variable name) matters; the value stored in the `prompt` object is **not used** and should be authored as `null` . The dialog message is generated automatically from the variable name (Enter a value to use for '`<name>`'). Every prompted variable must **also** appear in `let` — that `let` entry supplies the default shown in the dialog. A `prompt` entry with no matching `let` key has no effect and produces no dialog.

Variable names should be valid script identifiers — no spaces, no leading digit.

2. Scoping

Bindings introduced by a scope step are visible to that step's descendants and go **out of scope** when execution leaves the container. Strong (`let`) shadows any outer definition of the same name; weak fills in only when the name is otherwise unset.

3. Which steps carry the environment keys

All three keys are present, and **required at enqueue**, on the container steps that open a new scope: **Start, Let, Run Method, Define Procedure, Run Procedure**. Include them even when empty — a bare `"let": {}, "weak": {}, "prompt": {}` is fine; omitting one of these keys on a scope step hard-fails at Run.

Scripted Let is a partial exception. It requires `prompt` to be present (empty `{}` is fine) — an omitted `prompt` hard-fails at Run just like on the other scope steps — but its `let` and `weak` keys are reset internally before the script runs, so omitting `let` and `weak` is safe on Scripted Let. See Scripted Let for the exact per-key behavior.

Some exported **Script** steps carry a `let` (and `weak`) key left over from editor state. It has **no runtime effect** on a Script step — the Script step ignores these keys entirely — so do not author them there.

Worklist does not persist these keys. A Worklist step builds its per-row scope internally; do not author `let` / `weak` / `prompt` on a Worklist step.

4. Globals and other name sources

Scoped `let` / `weak` / `prompt` are only one of several ways a name becomes visible to an expression. Distinguish them from:

- **Globals** — created by Set Global, Next Item, Create Labware Group, and Next Labware. Globals live for the whole method run and beyond rather than being tied to a container's scope.
- **Loop / Worklist counters** — a Loop or Worklist step exposes its counter variable to its children.
- **Pipetting C___ context variables** — a separate, engine-injected scope used inside a technique's template. Not authored here; see Pipetting Techniques, Templates, and Auto-Selection.

All of these are names an expression can reference, but their lifetimes and authoring mechanisms differ.

5. Author's short list

- On Start — and on every scope step that requires them — include `let` / `weak` / `prompt` even when their objects are empty.
- Use `let` for values you set directly, `weak` for overridable defaults, and `prompt` for values you want the operator to type at Run.
- For every `prompt` entry, add a matching `let` entry (its value is the dialog default) and author the `prompt` value as `null`.
- Reference the bound names from child steps with `=Name`.

[†] Table of contents

Concept Guide: Well Volume Tracking

Biomek keeps a running per-well volume for every labware whose contents it knows about, and it maintains that ledger **while the method is being enqueued** — before anything moves. Steps debit it when they aspirate and credit it when they dispense: a dispense that would push a well above the labware's capacity fails at enqueue, and an aspirate that would drive a tracked well below empty fails at enqueue.

This is the mechanism behind two of the most common enqueue errors:

```
Cannot pipette 50.000 uL; the well only has 0.000 uL in it.  
Well volume out of allowed range: 3000.000 uL.
```

Neither is a run-time warning and neither can be suppressed. Both are reported against a position, not against the step that caused the imbalance, so the step named in the error is often the first one to notice a shortfall created several steps earlier.

1. What seeds the ledger

The ledger starts at whatever Instrument Setup declares. Two keys on the labware object do it:

- `volumeType` — "Known" or "Nominal" puts the labware under tracking. "Unknown" leaves it untracked.
- `evalAmounts` — the per-well starting volumes, row-major, in μL .

A tracked labware with no `evalAmounts` starts every well at **0**. That is the single most common cause of the "well only has 0.000 uL" error: the plate was filled by an operator off-deck, the method never says so, and the first aspirate is drawing from a well the engine believes is empty. **If liquid arrives on the deck by any route other than a pipetting step in this method, you must seed it here** — an off-deck pre-fill, a plate carried over from a previous method, or a reagent the protocol assumes is already in the reservoir.

The array length must equal the labware's well count exactly — there is no shorthand or replication. See § `evalAmounts` / `evalLiquids` in Instrument Setup for the per-array error messages and the row-major indexing convention.

2. What moves it

Every pipetting step that moves liquid updates the ledger: Transfer, Combine, the Fixed-8, Span-8 and Multichannel Aspirate/Dispense/Mix steps, Serial Dilution, and Select Tips Aspirate/Dispense/Mix steps nested inside a Select Tips container. Aspirates debit the source; dispenses credit the destination.

Two consequences catch people out:

- The engine debits what the technique actually draws, not the volume you wrote.** The ledger sees the larger number, so a source well sized exactly to the sum of the item volumes can still run short. Two technique settings add to the draw, and `splitVolume` transfers carry the overhead once per split cycle — see "Sizing a source well" below.
- Destinations become tracked whether or not you declared them.** Once liquid has been dispensed into a plate, its wells carry tracked volumes from that point on, even if it was placed "Unknown". A later aspirate from that plate is then subject to the same below-empty check.

Sizing a source well against the real draw

Exactly two technique settings inflate the source-well debit above the volume you asked for:

Setting (technique General tab)	Effect on the draw
Pre-wet + Pre-wet overage	When pre-wet is on, the pre-wet cycle draws <code>volume + overage</code> from the source. It is returned to the same well immediately afterward, but the check runs on the draw, so the well must hold the larger amount at that moment.
Conditioning excess, Conditioning section count, Conditioning section	Every aspirate draws <code>volume + (count × section volume) + excess</code> . The excess may be an absolute μL figure or a percentage of the transfer volume (e.g. 15%), so it does not always scale the same way

Setting (technique General tab)	Effect on the draw
volume	across techniques.

The projected draw for one aspirate is the **larger** of those two expressions or the specified volume. If the technique's **Override liquid type settings** is off, the liquid type supplies the pre-wet overage instead; no other liquid-type setting reaches this calculation.

Air gaps are not part of it. A leading or trailing air gap and the blowout are drawn from air, not from the well, and never debit the ledger. Sizing a source well for them wastes reagent.

Worked example — a 25 µL aspirate under a technique with pre-wet on, a pre-wet overage of 1.0, and no conditioning, from a well seeded at exactly 25.0:

```
projected draw = 25 + 1 = 26 µL
Cannot pipette 26.000 µL; the well only has 25.000 µL in it.
```

Seeding that well at 26.0 or higher clears it.

One caveat this arithmetic does not cover: a technique's **calibration slope** and **calibration offset** scale what the pump physically pulls, but they do not enter the projection above. A technique with a slope above 1.0 draws more than the ledger believes, so the well can run genuinely low at run time without any enqueue error. Leave a margin rather than seeding to the exact projected figure.

A mix is bounded by what the well holds, not by what it can hold

Each mix cycle is a real aspirate from the well followed by a real dispense back into it, so the aspirate half is checked like any other:

```
Cannot pipette 200.000 uL; the well only has 150.000 uL in it.
```

The per-cycle mix volume must be no larger than what the well contains **at that point in the method** — not what the labware class can hold. A 200 µL mix in a 300 µL well is fine only if 200 µL is actually present when the mix runs.

- **Cycle count is free.** Every cycle returns its draw to the same well, so the net ledger change across a mix is zero. Ten cycles of 150 µL need 150 µL present, not 1500 µL.
- **No technique overhead is added.** The pre-wet and conditioning terms from "Sizing a source well" above do not inflate a mix cycle.
- **Where the mix sits decides its budget.** A technique that mixes on the *aspirate* side mixes the source before the transfer draws from it, so its budget is the source's full contents. A technique that mixes on the *dispense* side mixes the destination after the delivery lands, so its budget includes the volume just delivered — into a well that started empty, that is the transfer volume. A 100 µL transfer cannot be followed by a 150 µL mix at the destination.

Separately, a mix requires the tips to be empty of liquid before it starts:

```
The mix operation cannot begin until tips are empty.
```

Air gaps do not trip this; anything classified as a liquid does. Most often seen when a step reuses tips that still hold contents — an aspirate followed by a standalone Mix step on the same tips, with nothing in between to discharge them.

3. Where the ceiling comes from

The upper bound is the labware class's per-well maximum — per **section** for a reservoir. It is checked both when you seed the labware and every time a dispense credits a well. The capacities are in the per-type tables in Catalog: Labware Classes (in *Biomek Instrument & Project Reference*).

Reservoirs are addressed by section rather than by well throughout: a one-section reservoir has one tracked volume, not 96.

4. Diagnosing a below-empty failure

The error names a position but not a well, so the first job is to work out *which* well it means. Resist the urge to raise every source volume in the method — that is the most common wasted attempt, and it cannot succeed against the case in step 2.

1. **Was the well ever seeded?** If the liquid gets there off-deck, add `volumeType` and `evalAmounts` in Instrument Setup. This is the most common answer.
2. **Is it a well you never seeded at all?** Every well that has received a dispense is tracked from that point on, so the shortfall is often in an **intermediate** well — a dilution plate, a mastermix well, a pooling target — whose entire contents came from an earlier step in this method. No amount of over-provisioning your *sources* reaches it; the fix is upstream, in whatever dispensed too little. **The reported figure is the clue.** A message naming an odd amount, `7.500` rather than something you seeded, points at a computed intermediate: search the method for the step that would leave exactly that much behind.
3. **Does the arithmetic close?** Add up every draw from that well across the whole method, including technique overhead and every split cycle, and compare against the seeded volume.
4. **Is a residual or floor eating the difference?** If the method leaves a deliberate residual in the source, the seeded volume has to cover the draws *plus* the residual.
5. **Is it a reservoir being addressed as a plate?** A 96-element seed on a one-section reservoir is a length error, not a volume error, and reports differently.
6. **Did an earlier step drain it?** The ledger is cumulative across the method. A well topped up by an earlier dispense is only as full as that dispense left it.

5. Quick checklist

- Seed any labware that starts with liquid in it — tracked labware defaults to empty, not to unknown.
- Seed destinations too, with `0.0`, when the technique follows the liquid level.
- Size sources against the technique's real draw, not the nominal item volume.
- Size mix volumes against what the well holds when the mix runs, not against the labware's capacity.
- Check reservoir seeds are section-length.
- Treat a below-empty error as an accounting question about the whole method, not about the step that reported it.